Puppeteer: Journaled Programs and the Dissolution of Infrastructure

Alvaro Rivera

2026-07-13

Preface

What this is

Seven analytic theory papers (Gregor, 2006, Type I) on journaled-program substrates. Each names a structural property — porosity, separability, partition, continuity, substrate, symptom, accidental category — and traces its consequences.

Single-volume reading

For a continuous reading — preface plus the seven papers as numbered chapters, with table of contents — see the unified monograph:

Puppeteer: Journaled Programs and the Dissolution of Infrastructure (PDF)

Individual papers below are the canonical citable artifacts; the monograph is the recommended entry point for a first read.

The papers

#	Title	Version	DOI	One-line summary
1	Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems	0.1-draft	10.5281/zenodo.20404863	Names *porosity* — the representational sparsity defect that database theory, domain-driven design, REST API design, and Event Sourcing each document under a different name — and the conditions under which it is rejected.
2	Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime	0.1-draft	10.5281/zenodo.20740697	Names the structural property of DSL programs that makes compilation, cross-invocation caching, and dense journaling possible at all; what is commonly read as three runtime optimizations is one consequence.

#	Title	Version	DOI	One-line summary
3	Reactions and the partition: opt-in eventual consistency in actor-native systems	0.1-draft	10.5281/zenodo.20792156	Names the developer-controlled partition of an event's implied work into *now* and *deferred*, exercised through *Reactions*; opt-in eventual consistency falls out as a downstream consequence, not a design choice.
4	Preserving semantic continuity across actors: a tell-based approach without orchestration	0.1-draft	10.5281/zenodo.11207062	Identifies the assumption naturalized across fifty years of actor-systems literature — that causation between actors is operational, not programmatic — and shows that it is a contingent historical choice rather than a necessity of the actor model.

#	Title	Version	DOI	One-line summary
5	The Journal as Substrate: Unifying Deployment, Replication, Backup, and Offline Operation in Distributed Systems	0.1-draft	10.5281/zenodo.21349146	Identifies four operational disciplines documented separately in the distributed-systems literature as instances of a single structural condition, and names the property under which they dissolve into one mechanism.
6	Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks	0.1-draft	10.5281/zenodo.20317450	Names the property by which a layer of infrastructure can be discriminated as compensating for a deficiency of the persistence model rather than serving a requirement of the problem domain. Redis is the canonical worked example.

#	Title	Version	DOI	One-line summary
7	After the substrate: building software without a datacenter	0.1-draft	10.5281/zenodo.20398998	Observes the joint operational consequence of the prior six papers: under a journaled-program substrate, cloud and microservice ecosystems become libraries the program invokes rather than habitats the program inhabits, and the datacenter ceases to be a structural requirement of running production software. The server-role becomes ephemeral as a side observation; the practical consequence is the headline.

How they relate

The papers form a chain of preconditions: each construct names a property that becomes available once the prior papers' properties hold.

Paper 1 establishes the vocabulary of *porosity* and *anti-porosity*. Paper 2 names *program–value separability* as the structural property that compilation, caching, and dense journaling all depend on. Paper 3 names the *now/deferred partition* exercised through *Reactions*; the partition is exercisable because the journal is dense (Paper 1) and the Reactions are programs (Paper 2). Paper 4 extends semantic continuity across actor boundaries through *tell*, which sits inside the Reactions surface of Paper 3. Paper 5 reframes deployment, replication, backup, and offline operation as instances of a single substrate property. Paper 6 names *infrastructural symptom* as the property by which compensatory layers can be diagnosed in any architecture. Paper 7 observes the joint operational consequence of the prior six: under a journaled-program substrate the datacenter ceases to be a structural requirement of running production software, and software construction passes back into the hands of those who model the domain.

What is Puppeteer, and why it appears here

Puppeteer is a runtime that combines CQRS, the Actor Model, and Event Sourcing with a domain-specific language whose programs are journaled as the unit of persistence. Across the seven papers it appears as the instantiation — the existence proof that the conditions each construct defines can be realized in a working system. It is neither the subject of the papers nor required by them. Each construct could in principle be instantiated by other systems: an extension of Akka or Microsoft Orleans, a fresh runtime built around a journaled DSL, or a framework that has not yet been written. The role of the instantiation is to demonstrate realizability; the contribution lies in the constructs themselves and the conditions they name.

How to read

If your question is specific, an entry point may answer it without reading the whole series:

- *"Which of the layers in my production stack are actually structural, and which are compensating for something?"* → Paper 6.
- *"What becomes operationally possible when production software no longer needs a datacenter?"* → Paper 7. (For the auxiliary question — *"is the server-role in my architecture a necessity, or a contingent artifact?"* — Paper 7 §7 offers a diagnostic lens.)
- *"How does a journaled program actually run — across actor boundaries, across machines, across failures, across deployments?"* → Papers 4 and 5.

- *"What are the structural preconditions on which the rest of the series rests — vocabulary, compilation, consistency model?"* → Papers 1 through 3.

For the full argument, read in order from Paper 1 through Paper 7. The papers are cumulative — each relies on vocabulary and conditions established by its predecessors. Paper 1 is the entry point; Paper 7 is the closing claim.

All seven are working drafts (versions 0.1 through 0.4) and are open to feedback. Issues are welcome at https://github.com/alvaroNCubo/puppeteer-papers/issues.

How to cite

All papers in this series are self-deposited preprints on Zenodo and have not undergone peer review.

The following papers are archived in Zenodo with citable DOIs:

Paper 1 — *Anti-porous architecture*:

> Rivera, A. (2026). *Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems* (v0.1) [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20404863

Paper 2 — *Program–value separability*:

> Rivera, A. (2026). *Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime* (v0.1) [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20740697

Paper 3 — *Reactions and the partition*:

> Rivera, A. (2026). *Reactions and the partition: opt-in eventual consistency in actor-native systems* (v0.1) [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20792156

Paper 4 — *Preserving semantic continuity across actors*:

> Rivera, A. (2026). *Preserving semantic continuity across actors: a tell-based approach without orchestration* (v0.1) [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.21207062

Paper 6 — *Most infrastructure layers are symptoms of the persistence model*:

> Rivera, A. (2026). *Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks* (v0.1) [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20317450

Paper 7 — *After the substrate*:

Rivera, A. (2026). *After the substrate: building software without a datacenter* (v0.1) [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20398998

The remaining paper (5) can be cited by its GitHub URL while its Zenodo deposition is pending.

License

This repository contains two kinds of content, licensed separately.

Papers and accompanying assets — the seven `0X-*.md` files, this README, and the asciinema recordings and GIFs under `paper7-assets/` — are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). You may copy, redistribute, adapt, and build upon them, including for commercial purposes, provided you give appropriate attribution to the author.

Code — the C# projects under `labs/`, used as reproducibility artifacts for the papers' empirical sections — is licensed under the Apache License 2.0.

The two licenses apply independently. Citing or quoting from the papers is governed by CC BY 4.0; using or modifying the code is governed by Apache 2.0.

ewpage

Anti-porous architecture

TL;DR

State-oriented persistence and operation-oriented persistence are not merely different storage strategies; they encode fundamentally different representational primaries. The same encoding choice — *what the representation is shaped to record* — recurs at the domain layer (the in-memory type hierarchy) and at the endpoint contract (the wire DTO). When the choice is structure, the system pays the cost in sparsity; when the choice is operation, the system pays it nowhere because the form is sized to the instance that produced it.

Architectures that shape their domain models to fit a storage substrate produce *porous* designs under structural pressure: rich object graphs — recursive, heterogeneous, with different node and edge types — do not survive a relational schema, so domain classes either fragment across joined tables that return empty cells or collapse into wide tables of redundant primitives; endpoint DTOs accumulate optional fields to serve heterogeneous use groups behind a single contract; and persistence layers record *what state exists* without recording *how it came to be*.

This paper argues that these are surface manifestations of a single architectural decision — *structure-centric encoding*, the choice to make the representation primary the type-space the form must serve rather than the operation that determined the specific case. Four bodies of literature (database theory, domain-driven design, REST API design, Event Sourcing) have each diagnosed a face of this decision locally, in their own surface vocabulary, without naming the encoding decision behind it. We name the defect *porosity* and its principled inversion (operation-centric encoding cross-layer) *anti-porosity*, and we identify three simultaneous conditions a CQRS + Actor Model + Event Sourcing combination must satisfy to sustain it. A system satisfying these conditions exists; it is presented in §4 as one realization, not as the foundation of the claim.

Claims this paper makes

1. **Porosity has three distinct surface manifestations, all instances of structure-centric encoding.**

    - *Domain*: the type hierarchy is recorded by its structural envelope — one table covering the universe of variants — rather than by the operation that produced the specific variant in each row. Two visible patterns: over-normalized schemas, where classes fragment across many tables joined by type and queries return rows of mostly-empty cells; and under-normalized schemas, where classes collapse into wide tables of optional fields conditioned on a `status` column.
    - *Endpoint*: a single API contract serving heterogeneous use groups records the union of fields any use case might require; each invocation expresses one case and populates only its slots. The contract is shaped by the type-space of use cases, not by the case the invocation actually expressed.
    - *Persistence*: representations capture *what state exists* (horizontal: the structural envelope of values) without capturing *how it came to be* (vertical: the operations that produced them). Three substrate-specific flavors: state-only recording in relational stores (vertical sparsity — operations discarded); fabricated edges in document stores when graph shape exceeds tree shape; and command-DTO-bloat in conventionally serialized event sourcing, where the event mirrors the input payload received rather than the script the execution actually ran.

2. **The three are not independent problems.** Each is a downstream consequence of the same upstream choice: structure as the representational primary at each layer. Once that choice is in place, every local fix recreates the pressure in a different layer.

3. **A combined CQRS + Actor Model + Event Sourcing implementation can reject all three porosities by making operation the representational primary at each layer.**

    - *Domain*: one class per role — one variant per operation outcome — with sum-type discrimination supported natively. The form a row takes is determined by the operation that produced it, not by the union of possible statuses.
    - *Endpoint*: DTOs with filterable optional parameters; the same contract serves heterogeneous use groups but each invocation selects only the slots its case expresses.

- *Persistence*: dense journals of *verbs* rather than state. Each entry records the script that executed and exactly the inputs that script consumed — not the type-space of inputs the command DTO could accept, not the structural envelope of state at a point in time. The state, however rich, is reconstructed deterministically by replay.
- The three mechanisms project the same encoding decision (*operation, not envelope*) to three layers.

4. **The inverse encoding is realizable in production; existence is proof of construction, not of principle.** A system whose architecture jointly satisfies the three conditions of Claim 3 has run continuously in production since 2018. It descends from a 2005 wrapper through a sequence of internal projects, was ported from Java to .NET, and currently operates in the core of eCommerce and payments platforms — payment hubs, account-balance ledgers, KYC pipelines, customer-facing storefronts, and payment-processor integrations. The system is the Puppeteer framework; §4 describes how it realizes each condition (with §4.0 covering the historical genealogy). Because the construct *porosity* was named retrospectively, the system's continued operation demonstrates that the three conditions are jointly satisfiable in production — not that the analytic frame explains the system's success. Appendix B applies the diagnostic to a public schema constructed independently of the present authors. Code references throughout (Appendix A) point to the public repositories; comprehensive quantitative results are deferred to a companion case-study paper.

5. **The contribution is identifying the unified encoding decision behind four locally-named defects.** Four bodies of work — relational database theory, domain-driven design, REST API contract design, and Event Sourcing — each documented surface manifestations of structure-centric encoding in their own vocabulary, without naming the encoding decision itself. CQRS, the Actor Model, and Event Sourcing are individually well-known constituent patterns; what we claim as novel is neither the patterns nor the detection of any single porosity, but the observation that the four traditions converge on a single architectural choice (structure as the representational primary), that *anti-porosity* names its principled inversion (operation as the representational primary), and that the inversion is implementable rather than merely conceivable. We note explicitly that other actor frameworks (Akka, Microsoft Orleans, Proto.Actor, EventStore) could *in principle* be extended to satisfy the three conditions, but none currently does (§7); the analytic frame's independence from any specific realization is therefore *conjectural*, not established. Puppeteer is currently the only realization that jointly satisfies the three conditions, and the open invitation in §5 is the explicit mechanism by which the conjecture of broader realizability would be tested.

6. **CQRS + Actor + Event Sourcing is interesting here for operation-centric encoding cross-layer, not for separation of con-**

cerns. These three patterns are conventionally introduced as separations: reads from writes (CQRS), concurrent units (Actor Model), events from projections (Event Sourcing). When applied together as a single discipline, they constitute a mechanism that projects operation-centric encoding to all three layers in which porosity manifests. This interpretation is available only once the encoding decision is named — porosity as the structure-centric default, anti-porosity as its inversion — rather than treating each layer's symptom as a local problem.

When NOT to use this approach

The principle of anti-porosity has limits, and the implementation pattern that realizes it has narrower limits still. We name two regimes in which the implementation is not the right fit. In each case, anti-porosity may continue to apply as a diagnostic — porosity remains a real pathology — but the CQRS + Actor + Event Sourcing realization, as instantiated in Puppeteer, imposes costs that may not be repaid.

A. Domains where the actor cannot achieve state locality

The actor model with journal-based persistence assumes that the working set of events required to rehydrate the actor's current state can be bounded. This holds when the actor's automaton passes through cyclic states — created, modifiable, finalized, archived — such that, at some point in the cycle, prior events become safely evictable without altering the current state. Domains where the actor's lifecycle has no such evictable point produce *unbounded rehydration*: every load replays a history that grows without ceiling.

Consider an airline flight. From the moment a schedule is published, an actor we may call `FlightOperation` accumulates events: the first booking, subsequent bookings, seat assignments, gate changes, boarding events, departure, landing. Throughout this sequence the state is genuinely active — answering *"is the flight bookable?"*, *"who is aboard?"*, *"has it departed?"* requires recent history. But once the flight has landed and reconciliation completes, the active state collapses: the flight no longer changes, and the remaining consumers are analytical (capacity-utilization reports, for instance) and can be served from periodic projections. At that point the entire history of bookings and gate changes can be marked evicted without compromising correctness. The actor has reached locality: its working set is bounded.

Compare this to an `Airline` actor that consolidates every flight, every passenger, every booking the airline has ever handled. Such an actor never reaches a "done" state. Its working set grows with operational history, and each rehydration replays material that has no bearing on any decision currently being made. The two designs share the same business domain; they differ in whether the actor's responsibility has a natural end.

The structural analogy with virtual memory is exact. In an operating system, a process exhibits *locality of reference* when its accessed pages stay within a bounded working set; the OS evicts pages outside that set without affecting correctness. When locality is lost — when references span the full address space uniformly — the system enters *thrashing*: pages are reloaded as fast as they are evicted, and useful work approaches zero. The actor model with a journal exhibits the same structural property: events take the role of pages, the actor's required history takes the role of the working set, and skips (event eviction) take the role of page replacement.

Virtual Memory	Actor + Journal
Working set bounded	Non-skipped events bounded
Locality of reference	Locality of automaton state
Page eviction	Skip / event eviction
Page size	Actor granularity
Thrashing	Unbounded rehydration

The locality property of an actor is, to a substantial degree, downstream of its naming. The word *actor* evokes *action*: a `Packer`, a `Dispatcher`, a `FlightOperation` — names that denote bounded responsibility with a natural end. Names that denote aggregations (*Inventory*, *Catalog*, *Customer*) imply unbounded responsibility, with no point at which prior events become evictable. A noun-aggregator actor is the structural equivalent of an OOP god class: state accumulates because nothing about the name suggests when it should stop. The same observation applies one level further down, to the partitioning of instances: an actor scoped per-country rather than per-shipment reproduces the problem even when the type is well-named. Granularity and partitioning are not separate decisions from naming; they are the same decision expressed at different levels.

Microsoft Orleans makes this granularity choice explicit in the very name of its activations — *grains* — emphasizing each as a small, self-contained unit. The naming heuristic developed here is consistent with that view and offers a verifiable test: if the proposed actor name is a noun-aggregator rather than a verb-doer, the design will likely fail at locality.

In typical production deployments, the cost of locality failure is partly amortized. Continuous-running datacenters confine rehydration to rare events — process restart, hardware failover — rather than steady-state operation; zero-downtime release patterns (developed in Paper 3) hide cold-start latency behind warm replicas during deployment; and journal storage is operationally cheap: it is economically negligible at relevant scales, and its access pattern — sequential append, with occasional sequential reads at rehydration — does not impose the random-I/O performance demands that drive RDBMS disk specifications. A

Puppeteer deployment is largely indifferent to disk speed, where a comparable RDBMS workload is bound by it. Locality is therefore best understood as a structural assumption of the model rather than a property whose violation is immediately catastrophic. The "when not to use" case bites when locality fails **and** the amortizing properties above do not hold — for instance, single-instance systems with frequent restarts, or workloads where actor cardinality is so high that eviction-and-reload occurs as part of normal operation.

The mechanism by which Puppeteer evicts journal events — the *skip* — presupposes locality: the framework can implement skips, but it cannot manufacture locality where the actor's responsibility is unbounded. Locality is a property of the design, not of the framework.

B. Domains where the actor's verbs cannot be kept fast

Each actor processes its mailbox serially: only one verb executes at a time on a given actor instance. The framework sustains thousands of requests per second per actor, at peak, on the assumption that every verb completes within a few milliseconds. A verb that blocks for hundreds of milliseconds — synchronous I/O to a slow service, an expensive query, a costly computation — does not merely slow itself; it blocks every subsequent verb in the mailbox, and ingest throughput collapses. The single-thread invariant of the actor turns from a correctness guarantee into a throughput cap.

Two patterns sustain the fast-verb invariant in practice. **I/O hoisting**: the caller resolves external dependencies — third-party calls, slow lookups, expensive joins — before invoking `perform`, and passes the resolved values as parameters. The actor receives data; it does not fetch. **Saga-phased I/O**: when the workflow genuinely requires interleaved external calls, the work is decomposed into a Saga. The actor advances one phase, releases, an external coordinator performs the I/O, and the response re-enters the actor as the next event. The actor never waits inside a single verb.

Where neither pattern applies — synchronous I/O is intrinsic to the verb, or the underlying algorithm is unavoidably slow — the actor's serial processing is structurally incompatible with the workload. In those cases the framework's claim of high per-actor throughput cannot be honored.

In both regimes, the principle of anti-porosity may continue to apply as a diagnostic; what fails is the realization, not the principle.

---

1. Introduction: the invisible cost of porous designs

This paper makes an analytic theory contribution. It identifies and names an encoding decision that prior literature has documented separately in surface vocabulary across four traditions (the construct, *porosity*); derives the design principle required to invert the decision across the three layers in which it manifests (*anti-porosity*); and presents an instantiation — a system whose underlying observations date to a 2005 prototype and whose current form has been in continuous production use since 2018 — as confirmation that the inversion is realizable. The contribution is the analytic frame; the instantiation is an existence proof of realizability. The genre is *theory for analyzing* (Gregor, 2006, Type I): analytic constructs that allow phenomena to be described, classified, and unified, with empirical evaluation supplementary. Hevner-style design science evaluation (Hevner, March, Park, & Ram, 2004) — which would require bundling artifact evaluation with construct introduction — is deferred to a companion case-study paper; the Gregor Type I frame separates the two cleanly. The paper is self-contained as a Gregor Type I analytic contribution: the frame stands on independent grounds and can be evaluated as such. Its position relative to companion papers in a larger series is additional context, not a precondition for its evaluation.

A defect recurs in mainstream software architectures: representations that carry more structure than the operations they describe ever populate. Relational schemas accumulate columns whose values depend on the row's state — required for some, irrelevant for others. API contracts grow optional fields that serve different use groups under one route, with no contractual signal of which fields belong to which case. Event payloads serialize whatever the command DTO carried, regardless of which inputs the operation consumed. Each looks local, and each has a literature that diagnoses it locally: database theory names normalization anomalies and NULL semantics; domain-driven design names anemic and persistence-driven models; REST API design names overloaded contracts and field bloat; Event Sourcing names payload bloat and command-event coupling. Four traditions, four vocabularies, four sets of remedies. A partial cross-tradition unification has long existed under the name *object-relational impedance mismatch* (Ireland, Bowers, Newton, & Waugh, 2009), but it covers a single boundary — objects against relational stores — and treats the cost as a problem of mapping rather than as a consequence of an encoding decision. To our knowledge, no prior work names the encoding decision underneath the four surface manifestations at the level of generality this paper proposes; §7 reviews the prior unifications and positions the contribution as extension rather than discovery.

We call this defect *porosity*: the representation is shaped to record structure (the type-space the form must accept) rather than the operation that determined the specific case. Its visible symptom is widespread structural emptiness — fields without values, columns whose meaning depends on the row, queries returning rows with cells that the application never reads, plus the absence of any record of how state came to be where state-only persistence is used. The defect emerges most visibly when complex or polymorphic abstractions are forced into relational tables (the historical default of structure-centric encoding), and intensifies as the abstractions become richer; it recurs at every layer where structure is treated as the representational primary, including endpoint contracts and event-sourcing payloads independent of any substrate. The vocabulary that names the encoding decision at the level of generality needed to see it across surfaces — beyond the object/relational boundary that impedance-mismatch literature already covers — has not, to our knowledge, been established in prior work. Developers do not say *this design is porous*; they say *this design is fine; the empty fields are normal.* Decades of structure-first modeling have made the trade-offs that produce porosity (over-normalization versus under-normalization, joins versus wide rows, NULLs versus duplicate tables, command-DTO-as-event versus per-call script) feel like inherent properties of software construction. They are not. They are the consequence of one architectural decision — structure as the representational primary — repeated millions of times across surfaces. Naming the decision is the first step toward seeing it.

This paper argues that porosity is not inevitable, that the four traditions cited above describe surface manifestations of a single encoding decision (structure as the representational primary), and that their independent local remedies are partial solutions to a problem that admits a unified one — the inverse decision applied across surfaces. We name the unified rejection of porosity *anti-porosity*, identify the conditions under which a CQRS + Actor Model + Event Sourcing combination sustains it across all three layers in which it manifests (domain, endpoint, persistence), and present a system that satisfies those conditions as confirmation that the principle is realizable. The historical genealogy of that system — how it came to satisfy the conditions before they were named — is presented in §4.0, alongside the mechanisms it uses to do so.

The paper is organized as follows. §2 enumerates three layers of porosity and argues that each is a face of the same defect. §3 introduces anti-porosity as a unified design principle and states the conditions a system must satisfy to realize it. §4 describes the origins and mechanisms by which one such system, Puppeteer, realizes those conditions; the section is descriptive of an instantiation, not constitutive of the principle. §5 reports operational observations drawn from production deployments. §6 addresses anticipated counter-arguments from the principle. §7 places this work in the context of related literature. §8 concludes.

1.1 A formal characterization

Porosity is not a database problem, nor an API problem, nor an event-sourcing problem. It is a defect of the representational primary — the architectural choice of what the representation is shaped to record. Four formal traditions, each diagnosing the defect from its own surface, converge on the same encoding *decision.*

The defect, stated in one sentence:

> *Porosity is the property of a representation that encodes **structure** (a schema, contract, payload shape, or state form) when encoding the **operation** that determined the specific case would preserve what that case actually contributed. Structure is **type-spaced**: the form covers the universe of cases the representation must serve. Operation is **instance-spaced**: the case that this invocation, this row, or this execution produced. Where the representation is shaped for the type-space, capacity that the instance does not use is paid in sparsity.*

The dual:

> *Anti-porosity, or density preservation, is the choice to make the operation the representational primary, encoded at each layer as an instance-shaped artifact — the* script* at the persistence layer, the sum-type *variant* at the domain layer, the selected *use case* at the endpoint layer — and to derive structural projections only where downstream consumption requires them.*

The defect has two visible forms. **Horizontal sparsity**: slots populated only by some instances of a shared form (NULL columns by status, optional DTO fields by use case, ES payload fields the execution did not consume). **Vertical sparsity**: the operational trace that the operation determined but the form did not capture (state schemas that record terminal values without the operations that produced them). Both are symptoms of the same encoding decision — structure as primary.

The four formalisms that follow each illustrate this defect in their own vocabulary; their convergence is on the encoding decision, not on a single surface.

Graph theory. Domain models can be formalized as *semantic graphs*: nodes typed by entity class, edges typed by relation, with both nodes and edges potentially heterogeneous in type (Diestel, 2017). The graph is itself an operational object — edges express verbs that relate nodes — but conventional relational projection captures only the nodes' structural attributes, discarding the operational character of the edges. The graph-database and Semantic Web literatures (Lassila & Swick, 1999; Robinson et al., 2015) preserve graph shape directly; the relational literature does not.

Information theory. This paper's terminology — *structural capacity* (bits available to encode the type-space of cases the form admits) and *informational content* (bits the specific instance actually contributes) — takes its inspiration from, but is not derived formally from, Shannon's framework (Shannon, 1948). The lineage is explicit; the derivation is not. Shannon's *entropy* and *channel capacity* are defined under assumptions (equiprobable states, channel noise as transmission perturbation) we do not establish here; the terms above are introduced to characterize representational shape, not channel behavior. Structure-centric encoding fixes the form to cover the type-space; the instance's contribution occupies a subset of that capacity, leaving the remainder as structural excess.

Storage engines. Each storage substrate exposes what we will call its *structural alphabet*: the set of shapes it can natively express. The term is introduced here; survey treatments (Hellerstein, Stonebraker, & Hamilton, 2007) catalogue substrates and their internals without isolating this property as an abstraction. Relational stores express rows × columns × types; document stores express nested trees of typed values; graph stores express labeled directed graphs. These alphabets are all *structural* — they capture forms that data can take, not operations that produce data. An operation-centric substrate exposes a different alphabet: the primary unit is verb-instance (an executable script), not row-shape. The journal substrate developed in §4.3 is one such case.

Database normalization theory. Codd's relational model (Codd, 1970), refined through successive normal forms (Codd, 1972; Fagin, 1977), is the canonical historical formalism for representing data in the relational alphabet. Normalization is prescribed to eliminate update, insert, and delete anomalies. The anomalies are themselves diagnostic of structure-centric encoding: they arise because the relational model records state-as-form without the operations that produced it. The procedures that resolve them — splitting wide tables, introducing join columns, accepting NULLs in optional attributes — work within the structure-centric paradigm; they do not question the encoding decision.

Synthesis. Each tradition saw the defect from its own layer and named a local remedy. The unification proposed here is not over four symptoms but over one decision: across substrates and surfaces, the representational primary was chosen as structure rather than operation. Once the encoding decision is named, the four traditions become four windows onto the same defect.

The vocabulary of the definitions above is partly borrowed and partly introduced: *semantic graph* from graph theory; *sparsity* generalizes the information-theoretic notion; *structural alphabet* and *structural capacity / informational content* are this paper's terms; *normal forms* and *anomalies* from database theory. The contribution is the encoding-decision frame — naming the architectural choice (structure-as-primary) that the four traditions each documented in their own surface vocabulary without naming as a single decision.

1.2 Terminology

This paper uses several near-synonyms to refer to aspects of the same conceptual unit. We distinguish them once and use them consistently.

- *Operation*: the conceptual unit — a verb applied to its consumed inputs. This is the artifact treated as the representational primary under operation-centric encoding.
- *Verb*: the action body defined once in the domain library (a method or

procedure with declared parameters). The same verb is invoked many times with different inputs.
- *Invocation*: the act of applying a verb to specific inputs at a specific time.
- *Script*: the textual representation of an invocation — the form persisted in the executable journal (§4.3). A script is the program-form of an operation.
- *Execution*: the runtime evaluation of the script against the current domain library, producing state.
- *Event*: in *conventional event sourcing*, the serialized DTO record produced after an invocation. This paper reserves *event* for the conventional ES usage and uses *script* or *operation* for the operation-centric form; the distinction is foundational (§2.3, §4.3) and not a naming preference.

The terms describe the same lifecycle phenomenon viewed from different vantage points: the verb is defined once, an invocation occurs, the script captures it, the execution produces state. The persisted artifact under operation-centric encoding is the script; the persisted artifact under conventional event sourcing is the event.

2. Three faces of porosity

2.1 Porosity in the domain layer

The domain layer's primary structural artifact is the type hierarchy. Under structure-centric encoding, that hierarchy is projected onto a homogeneous form — a table — whose shape covers the type-space (all possible variants); each row occupies only one variant. The sparsity that follows is the cost of recording structure-of-the-class rather than the operation that produced the row.

In object-oriented modeling, the canonical pattern for representing entities that pass through distinct lifecycle states is inheritance: each state is a subtype, with its own fields, invariants, and methods. A purchase order, for example, naturally decomposes into `Draft`, `Requested`, `Paid`, `Dispatched` — each subtype carrying only the attributes and operations that its state admits. The transition from one state to another is the construction of a new subtype instance, not the mutation of fields on a single class.

The relational model does not support this pattern cleanly. A type hierarchy can be projected onto a relational schema via one of three known idioms — single-table, class-table, or concrete-table inheritance (Fowler, 2002) — each of which produces porosity in a different form. In practice, single-table inheritance dominates: the schema collapses the four subtypes into one wide table with a `status` column and optional fields populated only for some rows. Attributes that belong logically to `Paid` are nullable for rows still in `Draft`; attributes that belong to `Dispatched` are nullable for everything else. The hierarchy is recovered at read time by inspecting the `status` field and conditionally interpreting the rest. The semantic graph — a clean polymorphic structure — has been projected onto a homogeneous vector space, and the gap between them surfaces as NULLs.

Formally, the relational projection encodes a sum type as a product type. The state hierarchy — `Draft`, `Requested`, `Paid`, `Dispatched`, each variant with its own fields and invariants — is naturally a tagged union: a sum type. The relational schema, lacking native sum-type support, encodes it as a product type — a single tuple of fields that must coexist regardless of variant, with the discriminator demoted to a column. The mismatch is structural. In the information-theoretic vocabulary of §1.1, sparsity — structural symbols present in the representation that carry no information for any specific instance — is the inevitable cost of forcing a sum-type structure through a product-type alphabet.

The choice is not driven by ignorance of the alternative. Developers familiar with object-oriented design recognize that inheritance is the structurally cleaner pattern; they adopt the property-field approach because the substrate makes it the path of least resistance. The other two idioms impose joins by type, schema duplication, and migration overhead severe enough to outweigh the modeling benefit. Porosity here is the signature of a forced compromise — not an absence of OOP wisdom, but a substrate that does not admit it.

The downstream effects extend beyond schema shape. State transitions on the porous table become updates against a row that other transactions may also read or modify; lock contention emerges as a structural cost of the design choice, not as an accident of high traffic. The broader observation — that a domain whose lifecycle was never logically concurrent now pays for transactional concurrency it did not request — is treated separately, as a distinct symptom of persistence-as-source-of-truth, in Paper 5.

2.2 Porosity in the endpoint layer

The endpoint layer's primary contractual artifact is the DTO — a fixed-shape tuple of fields defining what flows between client and server. The same structure-vs-operation tension reappears one layer up: the DTO is shaped for the type-space of use cases the endpoint must serve, while each invocation expresses one specific case.

When an endpoint serves a single, homogeneous use case, the DTO is well-defined and dense: each field participates in the operation's semantics. When an endpoint serves heterogeneous use groups behind a single route, the DTO accumulates optional fields covering the union of all cases — the contract is shaped by the universe, not by the case. This mirrors the wide table with `status`-conditioned columns of §2.1, with the encoding decision identical: structure as primary at the contract layer.

From the perspective of type theory, a DTO is typically modeled as a product type: a fixed collection of fields that must coexist. Heterogeneous use groups, by contrast, correspond naturally to a sum type — a tagged union — where each variant carries only the fields relevant to its case. When a product type is forced to encode what is semantically a sum type, sparsity appears as optional fields and conditional validation.

Consider an endpoint `POST /orders` that serves both a customer self-checkout flow and an administrative batch-insertion flow. The customer flow specifies items, a delivery address, and a payment token; the administrative flow specifies override pricing, source attribution, and an internal correlation ID. A unified DTO carries the union of all fields, validates conditionally based on caller identity or feature flags, and provides no contract-level indication of which fields belong to which use group. Documentation shoulders the burden the schema cannot.

Formally, this is again a projection of heterogeneous semantic variants onto a homogeneous vector space. In terms of information theory, the DTO's structural capacity exceeds the information actually conveyed by any single call. The unused fields constitute representational sparsity: structural symbols present in the representation that carry no information for the specific operation being performed.

The defect manifests symmetrically on the response side. The same DTO that returns an order's full detail to a desktop client returns more than a mobile client requires; clients either over-fetch or fragment the operation into multiple round-trips, neither of which the original fixed-shape schema admits. The DTO, as a homogeneous projection of underlying semantic variants, reproduces at the wire contract level the porosity already observed in relational schemas of §2.1.

Costs accumulate at multiple levels. Validation logic becomes conditional. DTO families proliferate as variants are introduced to handle slightly different use groups. Client code absorbs out-of-band knowledge about which fields apply when. The contract — intended to be the disciplined surface between systems — becomes porous: its meaning depends on contextual knowledge that the contract itself does not encode.

2.3 Porosity in the persistence layer

The persistence layer's job is to durably capture what the system has done. Three regimes recur in practice — relational stores, document stores, and conventionally implemented event sourcing — and each manifests porosity differently while sharing a common cause: each chose structure as the representational primary. Relational and document stores record state (the structure of the world at a point in time), discarding the operations that produced it — vertical sparsity. Conventionally implemented event sourcing improves on state-only persistence by recording verbs, but persists the command DTO (the structure of what was received) rather than the script of what executed — horizontal sparsity, of a different shape from §2.1 but the same encoding decision.

Relational persistence captures state, not the operations that produced it. A row records that an order is in `paid` state and stores the amount and payment method, but cannot answer how the order arrived there or what triggered the transition. The information loss is structural and asymmetric: state can be derived from operations by replay, but operations cannot be derived from state — the function from operations to state is many-to-one. Recording only state is therefore strictly less informative than recording operations; under evolved semantics (a bug fix, a richer derivation, a new derived field) or under audit (who triggered what, in what order), the missing direction is the one that matters and the one the schema cannot supply. The state-conditioned fields of §2.1 propagate to persistence: when invariants depend on state — for instance, `amount` is required only when `status = paid` — the schema cannot enforce them. The available alphabet (foreign keys, nullability, primitive `CHECK`) admits only primitive referential integrity, never the conditional, cross-field, cross-object invariants that emerge naturally from a sum-type domain. Invariant enforcement migrates to code by structural necessity, not by stylistic choice.

Document stores partially relieve the schema rigidity by allowing nested heterogeneous shapes. A purchase order can be a document with embedded items, addresses, and payment details, sparing the joins that relational schemas require. But the alphabet admits only a particular kind of structure: the labeled rooted tree. Domain semantic graphs are not constrained to trees — they admit cycles, references shared across documents, and edges of arbitrary type. When the same item is referenced from multiple orders, or when payments link to other documents, the missing edges must be fabricated at the application layer. The substrate handles trees natively and degrades the moment graph shape is required, in the strict graph-theoretic sense of §1.1.

Event sourcing, in its mainstream conception, improves on both by recording operations rather than state: replay reconstructs state by re-applying events. But in its conventional form — where events are serialized data structures mirroring the input commands — the entire input payload is captured regardless of which inputs the operation actually consumed. Consider an operation `RecordPayment` whose interface accepts twenty parameters; internally, the state-changing logic consumes four. The conventional implementation persists all twenty in the resulting event. The event payload is a fixed-shape tuple; the unconsumed fields constitute, in the informal sense established in §1.1, structural excess alongside the consumed content. Event sourcing inherits the porosity it was intended to escape: the journal records what the operation *received*, not what it *consumed*. The distinction is itself foundational and not merely a serialization preference. *Conventional event sourcing stores events (serialized DTOs); operation-centric persistence stores scripts (executable invocations).* They are not interchangeable forms of the same data — they are different representational primaries, and the structure of each (received-DTO shape vs. consumed-input shape) is itself a face of the encoding decision §1.1 names.

The persistence layer thus exhibits porosity in three forms: state without operations in relational stores (vertical sparsity), fabricated edges where graph shape exceeds tree shape in document stores (horizontal sparsity of structural mismatch), and command-DTO-as-event in conventionally serialized event sourcing (horizontal sparsity of received-not-consumed). The encoding decision is identical across the three: structure — state, tree, or command DTO — is treated as the representational primary, and the operation that produced it (or the script that executed) is either discarded or recreated as structural envelope. Three layers, one decision: §3 establishes the principle that addresses them as one.

3. The unified principle: anti-porosity

A system is anti-porous when its representations, at every layer, are *operation-centric*: shaped by the instance the operation determined rather than by the type-space the form might serve. In the vocabulary established in §1.1, this means choosing operation as the representational primary — encoding the script that executed (verb + consumed inputs), the variant the operation produced (sum-type instance), or the use case the invocation expressed (per-call wire selectivity) — rather than the structural envelope into which any instance of the type must fit. In the language of type theory, this means representing heterogeneity as sum types rather than product types; in the language of information theory, it means structural capacity sized to the instance's contribution rather than to the type-space.

A criterion of demarcation. The structure-vs-operation encoding decision is observable wherever a representation is shaped for a universe of cases rather than the specific instance: function signatures with unused optional parameters, Protocol Buffer schemas with many `optional` fields, HTML forms serving heterogeneous use groups, generic-type parameters never specialized to specific types. The present paper draws a narrower line: it analyzes layers that persist artifacts whose lifetime exceeds the operation that wrote them — the in-memory aggregate (the domain), the wire DTO at the contract boundary (the endpoint), and the durable schema or journal (persistence). Representations whose lifetime is bounded by a single operation — a function call's argument list, a generic type parameter at one instantiation, an HTML form rendered for one request — exhibit the same encoding decision but do not pay the cost in *stored* sparsity. The frame may apply to them with appropriate adjustment; the present paper does not claim it across them.

The principle manifests as three simultaneous conditions, each inverting the structure-centric default observed in §2. First, the domain layer admits sum-type structure natively: a state hierarchy is encoded as variants (one per operation outcome), not as a product type with a `status` discriminator and conditionally populated columns. Second, the endpoint layer admits per-call selectivity at the wire: contracts express sum-type discrimination across heterogeneous use groups (one variant per invocation), not a fixed-shape DTO that unions all of them. Third, the persistence layer records operations and only the inputs the operations actually consumed: the journal carries the script that executed (an instance-shaped artifact), not the structural envelope of what was received.

The three conditions are not independent fixes; they are the consequence of a single decision applied at three surfaces. As §2 made structurally evident, each manifestation of porosity arises from the same architectural choice — to treat structure as the representational primary, the type-space as the form, and the instance's contribution as a subset that leaves the remainder sparse. Anti-porosity is the inverse choice — to treat operation as the representational primary — applied as a single discipline across the three layers. A patch applied only at the domain (aggressive denormalization, hand-rolled inheritance) leaves the API and journal demanding structural envelopes; a patch applied only at persistence (compact event encoding) leaves the journal's density at the mercy of upstream layers' choices. Anti-porosity is not the sum of three local choices; it is a single architectural decision projected to three surfaces.

Operationally, anti-porosity is *density preservation*: the representation's structural capacity matches its informational content because the form was sized to the operation's contribution rather than to the type-space — consumed content retained, structural excess discarded. The combination of CQRS, the Actor Model, and Event Sourcing — conventionally presented as three separations of concern — admits a different reading once anti-porosity is named as the principle they jointly satisfy: they constitute, when applied as a single discipline, a mechanism for density preservation across representations. The principle does not prescribe a particular framework. Akka, Microsoft Orleans, Proto.Actor, or any system that combines actors, CQRS, and event sourcing could in principle satisfy the three conditions; doing so requires a substrate decision none of those frameworks has currently made (§7). The next section describes one realization — the Puppeteer framework — selected as the existence proof for the principle, not as its definition. The realization rests on a single architectural choice: that operations themselves — verbs invoked by callers — be recorded as the durable representation. This extends the principle of *code-as-data* (familiar from Lisp at the language layer) to the persistence layer in a narrower form than Lisp's strong homoiconicity. §4.3 develops the property formally as *script-form persistence*, and the resulting artifact as the *executable journal*.

4. An instantiation: how Puppeteer realizes the three conditions

This section describes how the Puppeteer framework realizes the three conditions stated in §3. It is the only section of this paper in which authorial voice ("we") and the specifics of one implementation are concentrated. The realization is presented as an existence proof for the principle, not as its definition; the conditions could be realized differently in another framework that made comparable substrate decisions (§7).

4.0 Origins of the instantiation

A practical encounter with the underlying phenomenon predates the present analysis by nearly two decades. In 2005, work that eventually became the Puppeteer framework began as a wrapper around a DLL that exposed services in the then-emerging vocabulary of *web services*. While building this wrapper, an unexpected property of the construction emerged. By intercepting the parameters of every incoming call and writing them, in order, to a log — what would now be called a *journal* — the full state of the underlying automaton could be recovered without inspecting its code. Starting from any consistent prior state and replaying the log, the automaton arrived at exactly the same final state. Persistence, traditionally treated as a separately designed concern, had emerged as a side effect of the wrapper. The phenomenon was named *autopersistence*, after its observable effect rather than from any theoretical premise. What the wrapper-log approach made visible was something the relational-persistence approach had hidden: the log was *dense* — it contained no empty fields, recording only what each call had actually used. The relational schemas the same applications wrote to, by contrast, were full of NULLs, columns that changed meaning by row, and joins that returned mostly-empty cells. The journal was *dense*; the schema was *porous*. The label and the principle followed; the observation came first. The mechanisms described in §4.1–§4.3 are the present-day form of properties already implicit in the 2005 wrapper: a domain visible to the framework only by reflection, parameters captured exactly as the call carried them, and a journal that records operations rather than state. Between the 2005 prototype and the present design, the framework was used in a sequence of internal projects, ported from Java to .NET, and progressively refined; the form described in §4.1–§4.3 has been in continuous production use since 2018.

4.1 Roles, not concepts: the library

Anti-porosity in the domain layer admits a single thesis: **a sum type, not a table**. The framework partitions the domain by *roles* — operations that act — rather than by *entities* — nouns that exist. Each role is a subtype: a class with its own fields, invariants, and verbs. The purchase order example of §2.1 is encoded directly as a sum type, with `Draft`, `Requested`, `Paid`, and `Dispatched` realized as distinct subtypes of a common base, each carrying only the attributes its role admits. The system's domain is the union of these roles, not a flattened table indexed by a `status` column.

The classes that realize these roles form what the framework calls the *library* — pure conceptual artifacts that do not know which "play" they participate in. A `Packer` is a *class puppet*: it knows how to pack; it does not know whether the packer instance is being persisted, replayed, or audited. The framework discovers domain types reflectively at runtime, scanning the configured library assemblies (`Actor.cs:15` declares the marker base class; `DomainLibraries.cs:117–128` performs the assembly scan; `ActorHandler.cs:61` invokes it at handler construction). The domain need not register itself; the framework inspects what is there.

The continuity is structural, not anecdotal: the 2005 wrapper (§4.0) intercepted parameters without inspecting the DLL it wrapped; reflection-based discovery preserves that property as a structural feature of the current design.

Polymorphism is a first-class concept of the DSL itself, distinct from how it is realized in the host language. Argument-parameter compatibility is computed at parse time via subtype checks (`AstExpression.cs:236–246`, `AreCompatible`); runtime substitution of subtypes is handled by the symbol table (`SymbolTable.cs:134`, `IsSubclassOf` check). Where the host language (C#) optimizes for verbose precision, the DSL optimizes for script readability and for boundary decoupling. Type promotion at the call site is permissive: polymorphic arrays promote to whatever collection type the library declares (`List`, `IEnumerable`, array, and so on); parameters accept any enumeration and promote at parse time to the specific enum the library expects. The DSL writes `Credit`, the DTO carries `Credit` as text, and the library's `PaymentMethod` enum receives the matching constant — no lexical change at any boundary. The contract is structurally decoupled from the domain class: each side may evolve its enum vocabulary independently, with the DSL mediating. **This is not syntactic sugar but a design decision: the DSL is shaped to read as domain narrative, not as a transcription of a programming language.**

Anti-porosity in the domain layer follows from the alignment of three properties: role-oriented partitioning (sum-type variants in the type hierarchy), reflection-based discovery (the framework imposes no schema on the library), and DSL-level polymorphism (sum-type discrimination is honored end-to-end in the operation contract). The remaining two layers — endpoint and persistence — are realized by separate but compatible mechanisms, treated in §4.2 and §4.3.

4.2 Parameter modifiers: `?` and `Eval`

The framework makes the direction of every value flow explicit through four parameter modifiers: `In`, `Out`, `InOut`, and `Eval`. The first three mirror the in/out/ref convention familiar from systems languages, ensuring caller-puppet isolation: the puppet cannot modify the caller's environment outside the declared direction. The fourth, `Eval`, is unique to this realization. The modifier system serves two complementary purposes: explicit isolation at the call contract, and honesty in the operation's journal record.

For `Out` parameters, the journal writes the literal character `?` as a placeholder (`Parameters.cs:755–757`). The reason is structural: an `Out` parameter's value is computed by the puppet, not supplied by the caller — there is no input value to record. At replay, the deserializer reads `?` and produces a default value of the parameter's type (`Parameters.cs:780–782`); the puppet's logic re-executes and fills the slot. The journal thus records exactly what the call carried at the input boundary — output slots appear as honest reservations, not fictitious inputs.

`Eval` parameters address a different problem: values the puppet uses that are not supplied by the caller and are not deterministic across replays. Generated identifiers and game-session state — a random number or a gameboard captured for a game's session — are typical cases: values that pertain to the operation's semantics but cannot be re-derived at replay. The `Eval` modifier directs the puppet to capture the value, on first execution, as a literal-assigning script (`name = (type)(value);`), persisted as part of the operation's journal record (`Parameter.cs:33–36, 163–224, 228–247`). On replay, the script re-executes and assigns the same literal — determinism restored. V1's interpreted DSL had an `eval` command for the same purpose; V2 replaced it with the parameter modifier because the command form could not always be compiled (`EvalStatement.cs:52–55`).

The modifier system ensures the journal carries exactly what the call was:

- Caller inputs recorded as values.
- Puppet outputs recorded as `?` reservations.
- Non-deterministic values recorded as literal-capturing scripts.

Anti-porosity at the parameter level emerges from two complementary disciplines:

- Explicit direction at the call boundary.
- Explicit capture of non-determinism.

`?` and `Eval` are dual mechanisms that operationalize anti-porosity at parameter granularity. Integration into the broader journal — and the script-form property that allows replay against an evolved domain — is treated in §4.3.

Boundary cases of `Eval`. Two cases of the `Eval` mechanism are worth naming explicitly, both of which the production deployments cited in §5 have had to address.

External dependencies. When an `Eval` expression resolves to a value derived from external state — a currency exchange rate at the moment of invocation, a foreign-key lookup against another service — the framework captures the resolved literal but not the dependency. The journal entry is `rate = (decimal)510.0`, not `rate = CurrentExchangeRate("USD","CRC")`. On replay against a future library, the captured literal is reproduced exactly; the external state may have changed. This is correct under *snapshot semantics*: the journal records what the system actually saw at the time of the operation, not what the world is now, and replay reconstructs the historical execution. The framework's contract is historical faithfulness, not present truth. Applications that require present-truth on replay (uncommon, since replay is typically used for state reconstruction, audit, or upgrades against evolved library logic) must re-derive the external value explicitly in the new library rather than relying on the captured literal. The boundary case is worth naming because the contract is sometimes assumed to be present truth and the framework's contract is not that.

Sensitive values. `Eval` captures literal values into the journal, persisted as text. For values that should not appear in plaintext at rest — cryptographic nonces, payment tokens, PII — the framework requires deployment-level mitigation. Three mitigations are available and have been used in the production payment deployments cited in §5: (a) at-rest encryption of the journal storage substrate (available in all four shipping backends); (b) application-level redaction policies applied at write time (specific to the application, e.g., card-number tokenization before `Eval` captures the value); (c) in-script transformation so that the `Eval`'d value is already cryptographically opaque before capture (e.g., the verb captures a hash or token, not the secret). The framework does not provide column-level encryption *inside* the script representation, because an encrypted literal in a script would not parse and execute on replay; opaque encrypted blobs must be decrypted before script evaluation, which is the application's responsibility. The point worth naming here is that `Eval`'s literal-capture property does not exempt journal storage from the standard at-rest-encryption disciplines other persistence substrates require. Conventional ORM-style serialization shares this property and the same mitigation playbook; the framework inherits both the property and the obligation.

4.3 Operations, not state: the executable journal

In the framing developed here, the primary property of event sourcing is not temporal traceability but density preservation: it preserves semantic density that state-based persistence models discard.

By storing operations rather than state projections, the journal maintains an isomorphic representation of the semantic graph that tabular or document projections render sparse. The mechanism that achieves this we call *script-form persistence*: each entry in the journal is simultaneously data — storable, indexable bytes — and a program — a parsable, executable script in the framework's DSL. The journal does not store a serialization of the actor's state; it stores the script that produced it. We call the resulting artifact the *executable journal*: a durable log of executable DSL scripts, replayable against the current library to reconstruct state.

The relationship to *homoiconicity* in its strong sense, familiar from Lisp (McCarthy, 1960; the term itself coined by Mooers and Deutsch, 1965), is one of *lineage, not equivalence*. Lisp's homoiconicity has consequences — macros that operate on S-expressions as primitive data structures of the language, eval-quote symmetry, code-modifying-code at the language layer — that the present realization does not claim. The DSL scripts in the executable journal are parsed by the framework parser and executed by the framework runtime; they are not consumed as primitive data structures by other DSL scripts, which is the property that would justify the strong term. The narrower property the present paper claims, and the only one its argument requires, is that persisted journal entries are textual representations of executable programs in the actor's DSL — retrievable as text, parsable into ASTs, executable against the current library. Bash scripts, SQL stored procedures, and shell command logs share this narrower property; we therefore do not claim homoiconicity in the strong sense.

Executable journal and *script-form persistence* are the precise terms for what the argument requires; the lineage to Lisp is acknowledged without claim of identity.

Conventional event sourcing persists the command DTO; the present realization persists the execution script. The script encodes consumed inputs only — the bloat of received-but-unused parameters does not arise by construction.

The script-form representation enables a capability that state-storing substrates structurally cannot offer: re-execution against new semantics. When the domain library evolves — a logic fix, a richer derivation, a new computation depending on past inputs — the journal replays against the updated library, producing corrected projections without altering the historical record. Storage engines that persist state, by contrast, can only restore the state that was written; the operations that produced it have been discarded.

The density preservation property is structural, not accidental. **A script cannot contain parameters the execution did not use, because the script is generated after parameter direction and evaluation rules have been applied** (§4.2). The journal therefore inherits density from the modifier discipline: caller inputs recorded as values, puppet outputs as `?` reservations, non-deterministic values as literal-capturing scripts. Two operational properties verify this in code. First, density is preserved through an asymmetry between inputs received and inputs consumed: only the latter are serialized (`Parameters.cs:755-757`, `:780-782`; `Lexer.cs:600`, where `?` is tokenised as `TokenType.question`). Second, the journal admits exactly two primitive operations — append and read-forward (`JournalWriter.cs:65`; `JournalReader.cs:35`) — exposed through a unifying interface (`Diary.cs`); the vocabulary of relational stores — SELECT, UPDATE, DELETE, INSERT, and the transactional locking that surrounds them — is absent by construction, not by convention.

A journal of DSL scripts therefore combines three properties — script-form persistence (executable text retrievable and re-runnable), graph preservation, and density preservation — under one representational substrate.

5. Empirical observations

The empirical claims in this paper are minimal by design, consistent with the *theory for analyzing* genre declared in §1 (Gregor, 2006, Type I): the contribution is the analytic frame (the encoding decision underneath four locally-named manifestations), for which an existence proof of realizability is supplementary. Quantitative measurements of the instantiation are deferred to a companion case-study paper; the analytic frame stands without them. The principle predicts that representations satisfying the three conditions of §3 will exhibit higher information density per entry than relational projections of the same operations; the magnitude of the resulting storage-volume gap depends on the workload regime (broadcast vs pointwise vs analytical), characterized below. The worked example illustrates the broadcast regime where the gap is largest, and the deployed instantiation listed at the end of the section confirms the principle survives in production.

A worked example. Consider a generic domain in which buyers issue purchase orders against future scheduled events; an event is *confirmed* once it has occurred; and an event is *settled* with one of several outcomes (award, refund, restatement). The pattern recurs across ticketing, conditional pre-orders, and milestone escrow. Three primitive verbs realize the lifecycle:

Phase	Statement	Scope
Pre-event (× n)	`order = Company.Purchase(buyer, items, events, amount, currency)`	one order per call; ranges over items × events
Event occurs	`event.Confirm(date, operator)`	one event; implicitly all order-items bound to it
Resolution	`event.Settle(outcome, date, operator)`	one event; implicitly all order-items bound to it

The receiver determines scope; no row enumeration is required.

The journal model. The framework's V2 pattern stores each action's verb body once in an action library; the journal records per invocation only the action identifier, the parameter values, and minimal administrative metadata (timestamp, operator, entry ID). The journal grows with parameters, not with action bodies. (V1's raw-script representation, in which the verbatim script appears in each entry, is preserved for backward compatibility.) This split mirrors the separation between definition and invocation in any compiled or interpreted language; in this realization, that separation extends to persistence.

Storage backends. The conditions of §3 concern the representational shape of journal entries, not the substrate that physically stores them. Four backends ship with the framework — filesystem (UTF-8 script entries in append-only files), in-memory (for testing and ephemeral actors), and two relational engines (MySQL and SQL Server) — and the script-form, density, and append-only access patterns are preserved across all four. The relational backends expose the journal through standard SQL interfaces familiar to operations teams; entries remain executable scripts queryable as text columns. Choice of backend is a deployment decision; the structural properties the paper attributes to the journal are invariant across that choice. A commonly raised concern — that a journal is a black box for operations staff who must inspect it under pressure — is therefore largely a deployment-configuration question rather than a property of the principle.

The structural gap, and the workloads under which it appears. A

`Confirm` invocation in the journal carries the parameters of one verb plus its administrative metadata — typically tens of bytes. The same operation expressed as relational mutations must enumerate every order-item bound to the event, copying parent dimensions onto every child row, because SQL's `JOIN` cannot quantify — it materializes Cartesian projections of existing rows, it does not abbreviate them. For an event with many bound items, the relational expansion exceeds the script representation by several orders of magnitude.

The gap is regime-dependent, and presenting only the favorable case would mislead. Three regimes are worth distinguishing.

Broadcast operations over large aggregates (the worked example above): one verb expressed as the framework's script applies to N items via the receiver's implicit quantification; the relational form must enumerate N rows. The write-amplification ratio is roughly N. The structural gap is large — several orders of magnitude when N is in the thousands — and grows with aggregate cardinality. Events bound to many order-items, role permissions broadcast to large user populations, and price changes applied across product catalogues all fall here.

Pointwise operations (one verb, one row affected): the journal grows by one script entry; the relational form mutates one row. The write-amplification ratio is roughly 1. The structural gap on storage volume is absent. The information density per entry is still higher under the operation-centric form (the entry records the script, not the received DTO), but the storage footprint is comparable. Pointwise updates — credit one wallet, increment one counter, mark one item shipped — fall here.

Analytical workload (read-heavy, snapshot queries): the journal does not address the workload directly; analytical queries run against projections, not the journal itself. The journal grows monotonically with operations regardless of read demand; the relational projection sized to the analytical workload occupies a stable footprint determined by current state. On *storage volume*, the journal accumulates without bound while the projection does not — the ratio inverts and favors relational storage. The trade is paid in journal storage (which §4.3 and Paper 5 argue is operationally cheap and write-only) and in projection-replay latency for cold projections; the gain is the ability to rebuild any projection against evolved semantics. Whether the trade is favorable depends on workload composition and on whether projection-rebuild is a valued operational capability.

The structural gap is therefore *strongest under broadcast regimes*, *neutral under pointwise regimes*, and *inverts in storage volume under purely analytical workloads*. Information density per entry remains higher under operation-centric encoding in all three regimes; the storage-volume gap is regime-specific.

Why the gap is structural. The compactness of the script representation rests on three model properties:

- **Quantification.** The receiver implicitly ranges over the affected aggregate; the verb applies to all members at once.

- **Polymorphism.** A single `Settle(outcome, ...)` accepts heterogeneous outcomes as a sum type, sharing representation across branches.
- **Parameters as metadata.** Operator, date, and outcome travel as arguments of one statement; in the relational form they are copied into every affected row, because the row is the unit of identity.

The relational form has none of these. A `JOIN` cannot quantify universally; an `UPDATE … WHERE …` issues the directive, but its result is enumerated rows. The porosity of the relational layer denotes here, in concrete terms, the structural consequence of substituting *"$\forall$ item $\in$ event"* with *"N copies of the antecedent."*

Forensic observation. The diagnostic is not specific to this example. Any mature relational schema can be read for porosity directly: counting a verb's parameters against the columns of the rows its invocation affects, identifying NULL-allowed fields whose values the originating operation never had to supply, and noting cross-product tables that materialize what the verb expressed as quantification. Appendix B applies this procedure in detail to the *Ordering* bounded context of Microsoft's `dotnet-eShop`, a reference architecture constructed independently of the present authors, and reports specific findings against its EF Core schema. Under the regime distinctions above, the eShop case exhibits horizontal sparsity in the domain layer (per-state populated columns) and vertical sparsity at the persistence layer (`Ignore(b => b.DomainEvents)`), both substrate-induced; the write-amplification ratio of the workload is left as an open question, since the analysis is structural rather than measured.

Existence proof in production. A system whose architecture jointly satisfies the three conditions of §3 has been in continuous production use since 2018, after more than a decade of preceding refinement (described in §4.0) traceable to a 2005 prototype. Puppeteer, the framework described in §4, currently runs structurally distinct subsystems within the core of eCommerce and payments platforms: payment hubs, account-balance ledgers, KYC pipelines, customer-facing storefronts and experiences, and payment-processor integrations. Code references throughout this paper (Appendix A) point to verifiable mechanism implementations. A separate case-study paper presents quantitative observations from these deployments — endpoint latency distributions, journal growth rates, replay performance, and developer-velocity comparisons — alongside the operational details (deployment, workload, infrastructure) that fall outside the scope of an analytic paper.

What the existence proof does and does not establish. Three limitations are worth naming. *First, retrospective interpretation.* The construct *porosity* was named after the system existed (§4.0); the system therefore demonstrates that the three conditions are *jointly satisfiable in production*, not that the frame *explains* its operational success. *Second, single-realization risk.* The inverse realization is currently coextensive with Puppeteer. Appendix B reduces this risk by applying the porosity diagnostic to a public reference schema (Microsoft's `dotnet-eShop`) constructed independently of the present authors — but Appendix B analyzes a *porous* system, not an *anti-porous* one constructed by an independent party. *Third, falsifiability of the principle as such.* The analytic frame predicts that operation-centric encoding admits density preservation; it does not currently predict outcomes the system would falsify. Specifically: a system that operates well while violating the principle would show the conditions are not necessary; a system that satisfies the conditions and fails would show they are not sufficient. We have not identified instances of either.

Open invitation. Three forms of external evidence would strengthen the analytic claim and are explicitly invited. *(a)* Analyses of independently-constructed systems under the porosity diagnostic (analogous to Appendix B), particularly systems built before 2024 that satisfy operation-centric encoding through different mechanisms — Akka Persistence, Microsoft Orleans + EventStore, or domain-specific event-sourced systems would be natural candidates. *(b)* Independently-constructed systems that satisfy the three conditions of §3 and fail under operational pressure, which would falsify sufficiency. *(c)* Independently-constructed systems that operate well at scale while explicitly violating the three conditions, which would falsify necessity. The author maintains a companion repository where such analyses are linked alongside the present paper.

Note on evaluation deferral. Deferring quantitative measurements to a companion case-study is a pattern recurrent across the paper series. Under Gregor Type I the deferral is supported — the analytic frame stands on independent grounds — but each analytic paper has so far transferred its empirical debt forward, with no paper yet redeeming the full debt. Three statements bound the position.

What is already published. Companion Paper 5 (*Substrate operations*) includes labs L1–L6 reporting append latency, density-per-event, cross-DC convergence parity, and buffer/RDBMS comparisons — bounded empirical observations scoped to Paper 5's structural claims about substrate primitives, not a comprehensive case-study of Puppeteer's production deployment.

What the present paper does not claim. Claims about Puppeteer's holistic operational performance — endpoint latency distributions, developer-velocity comparisons, replay performance, longitudinal journal growth — are not made here. The present paper invokes the system only for the structural observations §4 lists and the existence claim of §5. Subsequent analytic papers may, like Paper 5, report bounded empirical observations scoped to their own structural claims; the comprehensive case-study is separate.

What is owed. The comprehensive case-study paper is owed; the analytic series will not continue indefinitely deferring it. A reader who finds the analytic series progressing while the case-study remains unwritten is observing the failure mode the present authors name here and commit to avoid.

6. Counter-arguments

This section addresses the strongest objections to the argument advanced in §§1–5. Each objection is presented in its strongest form before its rebuttal; rebuttals draw on the principle articulated in §§1–3, not on the specific realization of §4.

"This is just CQRS done well." CQRS already separates read and write; what does anti-porosity add? Two things, addressed in Claim 5 and §3. First, CQRS is one of four traditions that document a face of structure-centric encoding locally; anti-porosity is the unified diagnosis of structure-centric encoding as the common defect. CQRS implementations vary widely in this respect — some recreate porous DTOs and porous event payloads despite separating reads from writes. Second, the contribution of this paper is not any pattern (CQRS or otherwise) but the recognition that the four traditions converge on a single architectural choice — structure as the representational primary — which prior work has not named with sufficient generality to see across surfaces. **CQRS addresses separation of concerns; anti-porosity addresses the encoding decision (what the representation is shaped for). The two operate at different conceptual layers.**

"In small systems, porosity doesn't hurt." True. The operational costs of porosity scale with system size, longevity, and rate of change. The conceptual claim, however, does not depend on these. Porosity is a structural property visible at any scale; whether one chooses to address it depends on the regime in which the system operates. This paper does not advocate adopting any specific framework for small or short-lived systems; *When NOT to use this approach* explicitly enumerates regimes where the realization pattern is not the right fit, and the principle itself remains diagnostic in those regimes — porosity remains a real pathology, even where its correction would not repay its cost. **The claim of this paper is therefore diagnostic rather than prescriptive: it names a defect whose relevance depends on scale, not one whose correction is universally mandatory.**

"Joins are sometimes necessary." Anti-porosity does not prohibit joins. **As illustrated in §5, joins migrate to the read side, where enumeration serves analytical needs rather than polluting the representation of operations.** The unified principle (§3) constrains only how state is *recorded* — not how it is queried for derivative purposes. Read projections may be denormalized, joined, indexed, and aggregated however a downstream consumer requires; the journal remains dense, but analytical and reporting use cases retain the full vocabulary of relational queries against derived views.

How read projections are constructed under anti-porosity. Two mechanisms are available, both operation-centric. First, *replay-on-demand*: a consumer requests a projection by replaying the journal against a projection-specific reducer. This is correct, evolution-tolerant (the reducer can be updated and the projection rebuilt), and cheap for projections that aggregate small subsets of the journal. Second, *incremental projection via reactions*: a long-lived observer subscribes to the operation stream and updates a projection store as each verb commits. The observer is described as a *Reaction* in the companion Paper 3; it is itself operation-centric — the reaction processes the same operation that the journal records, not a relational diff of state — and therefore does not reintroduce porosity at the source-of-truth boundary. Projections may be relational (typed rows, indexed for query patterns) without polluting the journal, because the relational projection is *derived* from operations rather than *defining* the source of truth. The cost paid for relational reads is conventional (denormalization, indexing, eventual consistency lag), not structural; under anti-porosity the relational projection is one consumer of the source of truth, not its custodian. The full treatment of reactions, their consistency model, and projection rebuild strategies is the subject of Paper 3.

"Business intelligence and analytics require SQL." Conventional BI assumes that the relational store *is* the source of truth. Under anti-porosity, the operation log is the source of truth, and analytical projections are downstream consumers. SQL access remains available — to the projection, not to the source of truth. **Because the operation log stores operations rather than state (§3, §4.3), projections for BI can be recomputed with evolving semantics — a capability unavailable when the relational store is itself the historical record.**

"Our data lake is the source of truth." This is an organizational decision, not a structural property of the system. The data lake's role can be reframed: it remains a shared projection consumed by downstream teams, while the operation log is the source of truth for the producer system. Anti-porosity is preserved internally even when external contracts demand denormalized projections at the system boundary. **The distinction is between organizational source of truth and architectural source of truth. Anti-porosity concerns the latter.**

7. Related work

This paper sits at the intersection of multiple bodies of work. Two prior streams attempted cross-tradition unification — *object-relational impedance mismatch* and the *data abstraction* tradition — and are reviewed first; four further traditions addressed faces of porosity locally and are reviewed after. Each is positioned for its relationship to the present paper, not for completeness; the literature review is not exhaustive (the methodological caveat is stated explicitly in §7.x below).

Prior unification attempts. The diagnosis that database-shaped representations and behavior-shaped objects do not align cleanly has a long history under the name *object-relational impedance mismatch.* Ireland, Bowers, Newton, and Waugh (2009) provide the canonical peer-reviewed treatment: a systematic classification of the mismatch's faces along structural, behavioral, and architectural axes, with explicit comparison of the mapping strategies (ORM, OODB, document-relational) that have been proposed against each face. That paper is the closest prior naming to the present paper's *porosity*: both describe the cost of recording objects under a relational primary, both diagnose the cost (NULLs, joins, fragmented hierarchies) as structural rather than incidental, and both observe that the proposed remedies — work *within* the relational paradigm, do not question it.

The present paper differs from impedance-mismatch literature on three points. First, the impedance-mismatch frame is binary — object vs relational — and does not generalize to surfaces where neither side of the binary applies: an endpoint contract is not "object" or "relational", an event-sourcing payload is neither either, yet both exhibit the same defect under the encoding-decision frame of §1.1. Second, impedance-mismatch literature names the *layer* at which the mismatch occurs (the object/relational boundary) without naming the encoding decision underneath it (structure as the representational primary); once that decision is named, the same defect becomes visible at endpoint and event-sourcing layers where no relational substrate is present. Third, impedance-mismatch remedies historically work *within* the relational paradigm (ORMs, document mappers, polyglot persistence, schema-on-read) rather than questioning structure as primary; the present paper proposes the inverse choice (operation as primary) projected across surfaces.

The *data abstraction* tradition (Parnas, 1972; Liskov & Zilles, 1974) and its successors in module-level encapsulation address a related but distinct concern: hiding implementation behind operations exposed by an interface, so that the representation can change without callers noticing. The principle "the representation should fit the operations that act on it" is older than the present paper's diagnosis, and to that extent the present paper is descended from this tradition rather than novel against it. What this paper adds is the observation that the principle is honored at the module/class boundary but routinely violated at the persistence, contract, and event-payload boundaries — surfaces that the data-abstraction literature did not theorize as needing the same discipline. The contribution is the projection of the data-abstraction principle to surfaces where it had not been previously claimed, plus the unification with impedance mismatch under a single encoding-decision frame.

Algebraic data types and type theory. The sum-type vs product-type framing used throughout this paper (§2.1, §3) is borrowed directly from algebraic data type theory (Hindley, 1969; Milner, 1978), where the distinction has been formally established for decades. The contribution of the present paper is not this framing itself but its application to runtime representation: *which encoding the representational primary adopts at each layer* is the architectural decision the four traditions of §2 each documented in their own vocabulary. Type theory provides the formal language; the paper applies it as an architectural diagnostic across surfaces that compile-time type systems do not reach.

Operation-based CRDTs. In distributed-systems literature, operation-based

CRDTs (Shapiro, Preguiça, Baquero, & Zawirski, 2011) privilege operation over state in their replication semantics: the durable artifact replicated between nodes is the operation, not the resulting state. This is operation-centric encoding at the replication layer, established and well-defined. The present paper's contribution is the cross-surface generalization: the same encoding decision can be made at the persistence primary, the domain primary, and the endpoint primary of a single-node system, not only at the replication boundary between nodes. The unification is across surfaces, not within the replication layer alone.

Append-only architectures. Append-only durable artifacts appear in Datomic, Kafka with log compaction, and the broader event-sourcing tradition. These systems share the structural property — additions only, no in-place mutation — but differ from the present realization in *what is appended*: append-only databases and logs typically append serialized state-deltas or command DTOs, not executable scripts of the operation that consumed specific inputs (§4.3). Append-only is therefore necessary but not sufficient for operation-centric encoding; the latter requires the additional decision that what is appended *is the script*, not the DTO.

Four applied traditions. *Domain-driven design* (Evans, 2003; Vernon, 2013) documents *anemic domain models* and *persistence-driven design* as pathologies and prescribes ubiquitous language and aggregate-rooted modeling as remedies. It diagnoses the symptom — a domain that lacks behavior — without naming the encoding decision behind it (structure-centric domain representation in service of structure-centric persistence). *REST API design* (Fielding, 2000; Richardson and Ruby, 2008) and the GraphQL specification (2015) document overloaded endpoints and fixed-shape DTOs, prescribing resource orientation and field selection respectively. GraphQL field selection moves toward operation-centric encoding at the wire — each invocation expresses what it consumes, not the union — but the discipline lacks corresponding operation-centric expression in the domain or persistence layers. *Event Sourcing* (Young, 2010; Fowler, 2005; Vernon, 2013) documents event-payload bloat and command-event coupling and prescribes command-event separation, upcasting, and versioning; it recognizes that persisting state is insufficient and persists operations instead, but typically represents those operations as fixed-shape command DTOs — the envelope of what was received rather than the script of what executed — a partial inversion that does not complete to operation-centric encoding. *Database theory* (Codd, 1970; Kent, 1983; Fagin, 1977) is the oldest and most formal of the four; it documents normalization anomalies and NULL semantics and prescribes successive normal forms. It names the effects (anomalies) without questioning the underlying decision: the relational model records state as structure, never operations as primary.

These four traditions have rarely engaged with each other on the question of representational form, perhaps because their domains appear to address different problems. This paper argues that they are observing the same encoding decision through different lenses.

Each tradition named a face of structure-centric encoding in its own layer and prescribed remedies that work within the structure-centric paradigm rather than questioning it. Among the cross-tradition unifications already reviewed, impedance-mismatch literature covers one surface (objects/relational) and the data-abstraction tradition addresses representation-fitting-operations at the module level; neither, to our knowledge, names the encoding decision (structure vs operation as the representational primary) at the cross-surface level proposed here.

Four formalisms. The grounding established in §1.1 invokes existing formal vocabulary, with two abstractions introduced here. Graph theory (Diestel, 2017), with extension to typed and labeled graphs in the Semantic Web (Lassila & Swick, 1999) and graph databases (Robinson, Webber, and Eifrem, 2015), provides the language of semantic graphs. Information theory (Shannon, 1948) inspires — without formally underwriting — the framing of representations in terms of *structural capacity* and *informational content*; both terms are this paper's, not Shannon's. Storage-engine literature (Hellerstein, Stonebraker, and Hamilton, 2007) catalogues the relational, document, and log substrates whose properties we abstract under the term *structural alphabet*; that abstraction is introduced here. Database normalization theory (Codd, 1970, 1972; Fagin, 1977) provides the canonical historical formalism for the relational case. The contribution is the observation that these four already-established formalisms — together with the two bridging terms — agree on a single defect that none names with sufficient generality across substrates. **The paper's formal vocabulary is therefore mostly assembled and partly introduced — explicitly so where introduced.**

Aspect-oriented programming and ORM annotations. The aspect-oriented programming tradition (Kiczales et al., 1997; AspectJ; Spring AOP) identified persistence as one of the ugliest cross-cutting concerns and sought to externalize it from domain code. The most widespread practical instantiation is ORM-style annotations (the Java Persistence API, Hibernate, Entity Framework). These approaches share a goal with anti-porosity — keeping the domain class free of persistence concerns — but operate at a different level: they decorate the domain class with framework-specific metadata that describes how to map its fields to a relational substrate. The substrate's structural alphabet is preserved; only the syntactic appearance of pollution is moved from the class body to its annotations. Anti-porosity, by contrast, addresses the substrate itself: by replacing relational projections with an executable journal of operations (§4.3), the question of how to map domain fields to relational columns disappears. **The domain class need not be annotated, because the persistence substrate no longer requires a projection of its fields.**

Other actor-model frameworks. Multiple frameworks implement combinations of CQRS, the Actor Model, and Event Sourcing. Akka (Lightbend) on the JVM, Microsoft Orleans (Bernstein et al., 2014) with its grain abstraction, Proto.Actor across multiple languages, and the dedicated EventStore database all share substantial conceptual infrastructure with the realization presented in §4. They differ from it, and from one another, in a single consistent respect: **each persists events as serialized data structures defined at the command boundary, rather than as executable scripts derived after parameter evaluation**. The underlying storage organization typically retains relational patterns — category-indexed streams, table-backed projections — rather than per-actor append-only structures. None currently articulates anti-porosity as a unified principle, and none persists scripts in the script-form sense developed in §4.3. The differences are not failures of these frameworks against their stated goals — each does what it sets out to do — but they illustrate that anti-porosity requires a deliberate decision at the substrate layer that mainstream actor frameworks have not made. The decision is available to them: any of these frameworks could be extended to record operations as scripts rather than as DTOs, and to migrate the storage organization from category-indexed streams to per-actor append-only journals. **The distinction is not in the use of actors, CQRS, or event sourcing, but in what is chosen as the durable representational unit.**

Scope of the literature review. The review above is positioned rather than exhaustive. We did not conduct a systematic literature review in the PRISMA sense: we did not enumerate ACM DL, IEEE Xplore, or dblp queries with predeclared keywords and inclusion criteria. The traditions reviewed were selected because they are the ones in which the present authors have prior fluency and whose vocabulary the paper engages with directly. Other prior naming attempts may exist that the present authors are unaware of; the falsifiable form of the paper's novelty claim is therefore: *"we are aware of no prior work that names structure-vs-operation as the representational primary at the cross-surface level proposed here, and we engage explicitly with the two cross-tradition unifications we did find (impedance mismatch, data abstraction)".* A reader who knows of a prior naming the paper missed is invited to write to the author; the contribution will be revised accordingly.

What this paper adds. The paper does not propose a new pattern. It identifies the encoding decision (structure vs operation as the representational primary) that the four applied traditions each described in surface vocabulary; positions the contribution as an *extension* of impedance-mismatch literature (covering surfaces beyond the object/relational boundary) and of the data-abstraction tradition (projecting its principle to persistence, contract, and event-payload surfaces); names the defect (*porosity*) and its principled inversion (*anti-porosity*); and demonstrates that the inversion is implementable as a unified discipline across surfaces. The Puppeteer framework serves as proof-of-existence. The analytic frame is *in principle* independent of any specific realization, but in practice the only realization currently known to the authors that jointly satisfies the three conditions is Puppeteer; the §7 review of "Other actor-model frameworks" enumerates frameworks that could be extended to satisfy them but do not currently. Independence of the frame from any specific realization is therefore *conjectural*, not established; the open invitation in §5 is the explicit mechanism by which the conjecture would be tested. **The novelty lies in naming the encoding decision at a level of generality the prior unifications did not reach, and in showing that the inversion is operationally realizable in at least one production system.**

8. Conclusion

This paper has identified the encoding decision — *structure vs operation as the representational primary* — that four applied traditions each documented in surface vocabulary, and that prior cross-tradition unifications (impedance mismatch, data abstraction) named at narrower scope. We have named the structure-centric defect *porosity*, and the operation-centric inversion *anti-porosity*. Anti-porosity requires three simultaneous conditions: a domain whose hierarchy admits sum-type structure natively, an endpoint whose contract permits per-call selectivity, and a persistence layer that records the script that executed rather than the state that resulted or the command that was received. A system that satisfies all three through a single architectural decision — the executable journal — has been in continuous production use since 2018, after more than a decade of preceding refinement traceable to a 2005 prototype (§4.0); that system, Puppeteer, is presented in §4 as the existence proof for the principle, not as its definition. The analytic frame is *in principle* independent of any specific realization, but Puppeteer is currently the only realization we know of that jointly satisfies the three conditions; whether other systems could realize them through different mechanisms is open to test (§5 *Open invitation*). In the vocabulary established in §1.1, anti-porosity aligns structural capacity with informational content across the three representational layers a system exposes.

The implications extend beyond the paper. When porosity is removed at the substrate, much of the infrastructure traditionally introduced to compensate for substrate mismatch becomes optional rather than essential: caches that hid slow joins, ORMs that translated between domain and table, message queues that moved flat tuples between systems. The question of how systems built on porous substrates may adopt the new discipline progressively follows from the autopersistence property the framework inherits from its 2005 origin: a system whose initial state and event sequence are observable can be reconstructed independently of how it persists internally; a system whose only durable artifact is state cannot reconstruct the operations that produced it.

Several lines of work follow. A comprehensive case-study paper will report quantitative measurements from the framework's production use; bounded empirical observations supporting specific structural claims have already appeared in Paper 5 (labs L1–L6) and are referenced in §5 of the present paper. Six companion papers extend the present argument across the series (Papers 2–7), enumerated in the Cross-references section below. Independently, the forensic procedure introduced in §5 invites verification against public corpora — an exercise initiated in Appendix B against `dotnet-eShop` and open to readers for replication in their own domains.

The contribution of this paper is the identification of the encoding decision as the common cause behind four locally-named surface manifestations, the analytic frame that allows the decision to be diagnosed across surfaces (§§1–3), the demonstration that the inversion is realizable in production through one existence proof (§4) and that the diagnostic reproduces in a system the present authors did not build (Appendix B), and the explicit invitation for external falsification (§5 *Open invitation*). Whether the construct survives broader scrutiny depends on what readers find when they apply the diagnostic in their own domains and on whether independently-constructed systems satisfying or violating the three conditions of §3 come to light. Naming the encoding decision is a precondition for these tests, not a substitute for them.

Cross-references

This paper establishes vocabulary that subsequent papers in the series reuse:

- *Paper 2 — Dual compilation*: the anti-porosity principle as it manifests at the language layer (interpreted versus compiled execution paths).
- *Paper 3 — Reactions and the partition: opt-in eventual consistency in actor-native systems*: the consistency model that anti-porosity admits, with deferred derivative behaviors expressed as domain code rather than externalized pipelines; also the mechanism by which read projections are constructed under anti-porosity (§6).
- *Paper 4 — Preserving semantic continuity across actors: a tell-based approach without orchestration*: cross-actor coordination under operation-centric encoding; the journal's density is prerequisite for the handoff mechanism.
- *Paper 5 — The Journal as Substrate: Unifying Deployment, Replication, Backup, and Offline Operation in Distributed Systems*: the substrate operations enabled by the executable journal; reports labs L1–L6 that empirically support its structural claims (also referenced from §5 of the present paper for the evaluation-deferral discussion).
- *Paper 6 — Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks*: extends the anti-porosity argument outward to the broader infrastructure ecosystem, treating compensating layers (caches, ORMs, message queues) as artifacts whose necessity dissolves once the substrate is changed.
- *Paper 7 — After the substrate: building software without a datacenter*: extrapolates the argument forward, examining what is possible once journal-as-substrate is the operational primary rather than a substitute for relational persistence.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `b42d0f7` (2026-05-26), which ports the SimpleX/StageManager/Usher fixes from the active development branch on top of the prior public snapshot. The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:177e2d61486bdbdfd2d5e774fcf392a45406e60d;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:b42d0f76b4278c36a45c221cc1453e3ee8ffb3de
```

A prior snapshot — commit `2f31f96` (2026-05-18) — is also archived in Software Heritage and remains resolvable for reference:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

The line numbers cited in the body have been verified against `b42d0f7` and remain stable across the two snapshots; the SimpleX integration did not move the cited mechanisms.

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:846`) resolve against the `b42d0f7` snapshot. A reader can construct per-file SWHIDs by adding the qualifiers ;path=<path>;lines=<NN> to the `b42d0f7` directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Bernstein, P., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Microsoft Research Technical Report MSR-TR-2014-41). Microsoft Research.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

Codd, E. F. (1972). Further normalization of the data base relational model. In R. Rustin (Ed.), *Data base systems* (Courant Computer Science Symposia 6, pp. 33–64). Prentice-Hall.

Diestel, R. (2017). *Graph theory* (5th ed.). Springer.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems*, 2(3), 262–278.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Fowler, M. (2005). *Event sourcing*. martinfowler.com. https://martinfowler.com/eaaDev/EventSourcing.html

GraphQL Foundation. (2015). *GraphQL specification*. https://spec.graphql.org

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*, 1(2), 141–259.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

Hindley, R. (1969). The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146, 29–60.

Ireland, C., Bowers, D., Newton, M., & Waugh, K. (2009). A classification of object-relational impedance mismatch. In *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA '09)* (pp. 36–43). IEEE.

Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*, 26(2), 120–125.

Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C., Loingtier, J.-M., & Irwin, J. (1997). Aspect-oriented programming. In M. Akşit & S. Matsuoka (Eds.), *ECOOP'97 — Object-oriented programming* (pp. 220–242). Springer.

Lassila, O., & Swick, R. R. (1999). *Resource description framework (RDF) model and syntax specification* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/1999/REC-rdf-syntax-19990222/

Liskov, B., & Zilles, S. (1974). Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4), 50–59.

McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4), 184–195.

Milner, R. (1978). A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3), 348–375.

Mooers, C. N., & Deutsch, L. P. (1965). TRAC, a procedure-describing language for the reactive typewriter. In *Proceedings of the 20th National Conference of the ACM* (pp. 229–246).

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058.

Richardson, L., & Ruby, S. (2008). *RESTful web services.* O'Reilly Media.

Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases* (2nd ed.). O'Reilly Media.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423; 27(4), 623–656.

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)* (pp. 386–400). Springer.

Vernon, V. (2013). *Implementing domain-driven design.* Addison-Wesley.

Young, G. (2010). *CQRS documents.* https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

---

Appendix A: Source-code verification

This appendix consolidates the source-code references cited throughout the paper. All paths are relative to the public Puppeteer repository; line numbers reflect the source at commit `b42d0f7` (see *Code provenance* above). The framework's source was migrated from Spanish to English identifiers during paper development; the line numbers and file names below reflect the English-language source as it currently stands.

A.1 Reflection-based discovery (§4.1)

Claim	File	Lines
`Actor` marker base class for domain entry points	`Puppeteer/Actor.cs`	15

Claim	File	Lines
Reflection-based discovery of domain types via assembly scan	`Puppeteer/EventSourcing/DomainLibraries.cs`	117–128
Domain library loaded at handler construction	`Puppeteer/EventSourcing/ActorHandler.cs`	61
Polymorphic argument-parameter compatibility at parse time (`AreCompatible`)	`Puppeteer/EventSourcing/Interpreter/Libraries/AstExpression.cs`	236–246
Runtime substitution of subtypes via symbol table (`IsSubclassOf` check)	`Puppeteer/EventSourcing/Interpreter/SymbolTable.cs`	134

A.2 Parameter modifiers (§4.2)

Claim	File	Lines
Four parameter modifiers defined (`In`, `Out`, `InOut`, `Eval`)	`Puppeteer/Parameter.cs`	33–36
`Eval` value setter generates literal-assigning script on first invocation	`Puppeteer/Parameter.cs`	163–224
`EvalScript` property (valid only for `Eval` parameters)	`Puppeteer/Parameter.cs`	228–247
Parser recognises `Eval` modifier in parameter declarations	`Puppeteer/Parameters.cs`	65, 110–111
`Eval` parameters' `EvalScript` serialized to journal	`Puppeteer/Parameters.cs`	334–367
`Out` parameters serialize as `?` placeholder	`Puppeteer/Parameters.cs`	755–757
`?` for `Out` parameters deserializes to default value	`Puppeteer/Parameters.cs`	780–782

Claim	File	Lines
`?` tokenised as `TokenType.question` in the lexer	`Puppeteer/EventSourcing/Interpreter/Lexer.cs`	600
V1 `eval` statement preserved (interpreted only); V2 redirects to parameter modifier	`Puppeteer/EventSourcing/Interpreter/Libraries/EvalStatement.cs`	52–55

A.3 Executable journal (§4.3)

Claim	File	Lines
Journal high-level interface (append-style writes, read-forward only)	`Puppeteer/EventSourcing/DB/Diary.cs`	228–337
Append primitive (`AppendRecord`)	`Puppeteer/EventSourcing/DB/FileSystem/JournalWriter.cs`	65
Read-forward primitive (`ReadAll`)	`Puppeteer/EventSourcing/DB/FileSystem/JournalReader.cs`	35
Script entries persisted as UTF-8 bytes (`EncodeScriptEvent`)	`Puppeteer/EventSourcing/DB/FileSystem/BinaryEventCodec.cs`	41–101
`ReplayEvent` re-executes scripts at journal replay	`Puppeteer/EventSourcing/ActorHandler.cs`	846
On-demand parser invocation for type resolution at replay	`Puppeteer/EventSourcing/Follower/Reaction.cs`	1027–1029

Appendix B: Diagnostic application to an independently-constructed schema

B.1 Purpose

This appendix applies the diagnostic procedure of §5 ("Forensic observation") to a schema constructed independently of the present authors. The purpose is to test, in the smallest available form, whether the construct *porosity* is identifiable in a system whose authors had no relationship to the construct's naming. A positive identification reduces — though does not eliminate — the risk named in §5 that the construct is coextensive with the present authors' realization.

The system selected is the *Ordering* bounded context of Microsoft's `dotnet-eShop` (https://github.com/dotnet/eShop), the official .NET reference implementation for cloud-native e-commerce. The Ordering context implements the canonical DDD + CQRS + event-sourcing patterns that §2 and §7 of this paper engage with. References below resolve against commit `9b4f943` (2026-current upstream `main`).

B.2 Method

The diagnostic of §5 asks three questions of any persisted representation:

1. **Horizontal sparsity (§2.1):** are columns of a table populated only by some rows, with the discriminator silently encoded in another column?
2. **Vertical sparsity (§2.3a):** are domain operations recorded as persistent artifacts, or only their terminal effects on state?
3. **Operation vs row (§5):** where the substrate forces enumeration of rows, would an operation-centric representation quantify (one verb over an aggregate) rather than enumerate?

We apply each question to the schema established by the initial EF Core migration plus its 2023–2024 refinements.

B.3 Observations

Schema under analysis. Migration `20230925222426_Initial.cs` (lines 122–162) creates the `ordering.orders` table with eleven columns:

Column	Nullability	Set by
`Id`	NOT NULL	sequence `orderseq`
`Address_Street`, `_City`, `_State`, `_Country`, `_ZipCode`	NULL	Address value object (owned entity flattened to 5 columns)
`OrderStatusId`	NOT NULL	enum discriminator
`Description`	NULL	varies by state transition (see below)
`BuyerId`	NULL	set by `SetPaymentMethodVerified()`, not at construction
`OrderDate`	NOT NULL	constructor
`PaymentMethodId`	NULL	set by `SetPaymentMethodVerified()`

Horizontal sparsity test (§2.1). The `OrderStatus` lifecycle (`Ordering.Domain/AggregatesModel/OrderAggregate/OrderStatus.cs:7–14`) defines six states: `Submitted`, `AwaitingValidation`, `StockConfirmed`, `Paid`, `Shipped`, `Cancelled`. The schema collapses all six into a single discriminator column (`OrderStatusId`). The `Description` field is the clearest horizontal-sparsity case: its meaning is conditioned on `OrderStatusId`, and its values are assigned literally as the lifecycle advances (`Order.cs:115,126,138,151`):

`OrderStatusId`	`Description`
`Submitted`	NULL
`AwaitingValidation`	NULL
`StockConfirmed`	`"All the items were confirmed with available stock."`
`Paid`	`"The payment was performed at a simulated \"American Bank checking bank account ending on XX35071\""`
`Shipped`	`"The order was shipped."`
`Cancelled`	variable per cancel path

The `Description` field is the operation's residue, persisted into a structural column whose meaning the schema does not encode and whose population is conditional on a sibling column. This is the §2.1 defect literally — a sum-type discriminated field encoded as a product-type column. `BuyerId` and `PaymentMethodId` are nullable for the same reason: they are determined later in the lifecycle by `SetPaymentMethodVerified()`; the schema accepts orders that have not yet been verified by populating NULL.

An anecdotal aside, not part of the diagnostic. The domain class `Order` (line 23) carries a private boolean field `_isDraft` accompanied by `#pragma warning disable CS0414 // The field 'Order._isDraft' is assigned but its value is never used`. This is in-memory dead code — plausibly a minor maintenance artifact rather than evidence of structural porosity, and we explicitly do not count it as diagnostic. We mention it because the analogous pattern at the persistence layer would be diagnostic — columns retained across migrations whose values no operation reads — but the present analysis does not enumerate such columns in the schema and the inference from one to the other is suggestive, not probative. Readers should weight the in-memory observation accordingly.

Vertical sparsity test (§2.3a). The Entity Framework configuration (`Ordering.Infrastructure/EntityConfigurations/OrderEntityTypeConfiguration.cs:9`) declares `orderConfiguration.Ignore(b => b.DomainEvents)`. The aggregate's domain operations — `OrderStartedDomainEvent`, `OrderStatusChangedToAwaitingValidationDomainEvent`, `OrderStatusChangedToStockConfirmedDomainEvent`, `OrderStatusChangedToPaidDomainEvent`, `OrderShippedDomainEvent`, `OrderCancelledDomainEvent` — are dispatched in memory and excluded from persistence by construction. The schema captures the terminal state (`OrderStatusId = Shipped`), not the operations that produced it. The transition trace exists only ephemerally, during the request that produces it, and is then dropped. This is the §2.3a defect literally: state-as-form persisted, operation-as-trace discarded.

Outbox as §2.3b instance. Migration `20231026091055_Outbox.cs:14-30` adds an `IntegrationEventLog` table:

Column	Type
`EventId`	uuid
`EventTypeName`	text
`State`	int
`TimesSent`	int
`CreationTime`	timestamp
`Content`	text (JSON-serialized payload)
`TransactionId`	uuid

The `Content` column holds the integration event serialized verbatim — the command DTO as received, not the script of what executed. This is the §2.3b defect at the outbox boundary: the event payload is shaped by the type-space of any integration event the system may emit, not by the specific operation that produced this event. Additionally, the `State` column itself reproduces the §2.1 defect — the outbox event's own lifecycle (`Pending`, `InProgress`, `Published`, etc.) collapsed into a discriminator column.

Operation vs row test. The `SetStockConfirmedStatus()` operation (`Order.cs:108-117`) is, semantically, one verb applied to the aggregate. Under the relational projection, no row in the `orders` table records that this operation occurred; only `OrderStatusId = StockConfirmed` and `Description = "All the items..."` are observable, both attributes of the terminal state. To answer the question *"at what time did this order transition to StockConfirmed?"*, the schema has no answer — the operation's timestamp is not in the schema (only `OrderDate`, the constructor time, is recorded). To recover the transition history one must join against the dispatched (but unpersisted) domain events, which is impossible after the request that produced them returns. The operation existed; the row does not record it. An operation-centric representation would quantify (one verb in the journal: `SetStockConfirmed(orderId, at: timestamp)`); the relational form has no quantifier and no place to put the verb.

B.4 What this demonstrates

The diagnostic reproduces. The Ordering bounded context of `dotnet-eShop` — constructed by Microsoft engineers without reference to the construct *porosity* — exhibits all three of the manifestations enumerated in §2: horizontal sparsity in `orders.Description` (§2.1), vertical sparsity by explicit `Ignore(b => b.DomainEvents)` (§2.3a), and contract-induced sparsity at the `IntegrationEventLog.Content` blob (§2.3b). The construct is therefore not strictly coextensive with the present realization: its diagnostic identifies the defect in a system the construct's authors did not build.

B.5 What this does not establish

Three caveats are worth naming.

First, this is a *forensic* demonstration, not a *normative* one. The dotnet-eShop architects produced a competent reference implementation under the prevailing structure-centric default. The team's choices (single-table inheritance for `Order`'s lifecycle, owned entities for `Address`, outbox for reliable event delivery, explicit `Ignore` for in-memory domain events) are within the standard playbook for DDD + CQRS on a relational substrate. The diagnostic identifies that the system *exhibits* porosity, not that it is *wrong* by some externally-imposed standard. eShop serves its purpose; the porosity is the cost of the encoding decision the system makes, not a defect of execution within that decision.

Second, this demonstration analyzes a *porous* system, not an *anti-porous* one constructed by an independent party. The reduction in single-realization risk is therefore partial: it shows the diagnostic reproduces externally, not that the inverse encoding has been independently constructed and successfully operated.

Third, the analysis is small in scope (one bounded context of one system, one migration sequence). A systematic survey of schemas — across multiple companies and frameworks — would strengthen the claim that porosity is a default rather than a local artifact. We do not undertake such a survey here; the present demonstration is sized to the falsifiability claim of §5 (that the construct is identifiable in independently-constructed systems), not to a comprehensive empirical mapping.

ewpage

Program–value separability

TL;DR

Compilation, caching, and dense journaling are **not** features layered atop interpretation. They **are** downstream consequences of a single structural commitment: values live outside the script, not embedded within it.

The transition often described as "adding compilation" inverts the causal order. A DSL whose programs embed their values has no stable identifier, no template to cache, no separable sub-program to compile ahead of time — interpretation is not a stylistic choice but a structural necessity. Once values move outside the script, the program becomes a function awaiting its arguments — $F(x_1, x_2, \ldots, x_n)$, as the runtime documents itself. And it is now the *same* program on every call, whatever arguments it receives: it has acquired an identity of its own, independent of its values. That identity is the pivot. The runtime decisions below are not consequences of separability directly so much as consequences of a program's having a stable identity — and separability is what confers it.

From that single act of separation, four runtime decisions cohere. Scripts with externalized parameters are eligible for compilation to IL via Expression trees, cache as reusable programs with action identifiers, persist to the journal as compact action entries, and replicate with entropy bounded by argument vectors. Scripts with hardcoded values are not cached, run once, and persist as their literal text. The four are not parallel design choices; they are projections of a single precondition. This paper develops and measures the first three (compilation, caching, dense journaling); the fourth, replication-bounded entropy, follows from the same precondition and is developed in a companion paper — it is named here, not claimed as a result of this one.

This precondition operates on the substrate characterized in the previous paper of this series — Puppeteer's journal as *code-as-data*:

programs persisted in their own executable text. The prior paper develops this as *script-form persistence* (the *executable journal*) and is careful to claim it in a sense narrower than Lisp's strong homoiconicity, which it explicitly disclaims; we adopt the label *code-as-data* from this point on, with that formalism standing as established there. The compactness of these entries is not merely syntactic: each names an operation substantially richer than its surface signature, with the asymmetry scaling with the domain library's depth (§5.5).

This is the principle of program–value separability: a DSL program becomes identifiable, compilable, cacheable, and persistable densely only when it is separable from its values. Externalized parameters are the structural precondition under which compilation, caching, and dense journaling become possible at all.

Claims this paper makes

1. **Causal inversion: program–value separation is a necessary condition for the stable-identity faces.** Compilation, caching, and dense journaling in the runtime described here are not optimizations layered atop interpretation. They are downstream consequences of *program–value separability* — the structural property that values live outside the script body, not embedded within it. A DSL program embedding its values has no stable identifier under which to reuse a specialization across invocations, no template to reference in the journal rather than copy, no bounded argument stream to replicate. The separation is necessary for these faces, not optional — though it need not be *declared* rather than recovered by analysis (§1.2), and it is not claimed that caching of every kind requires it (memoization of results does not). *(Verification: structural, within this class of runtimes — a script embedding its values produces a different cache key per invocation, so cross-invocation reuse, reference-based journaling, and bounded replication collapse together; the argument and its two standard objections are made precise in §1.2. The runtime documents the model itself: "Un script funciona como F(x1,x2,…,xn)"* (`ActorHandler.cs:1085`) *— F is the program; the x_i are its externalized values.)*

2. **DSL programs admit a structural binary taxonomy.** *Parametric scripts* are separable from their values: cacheable as reusable programs with action identifiers, persistable as compact action entries in the journal, and economically compilable. *Literal scripts* are non-separable: never cached, ephemeral, and persisted as their literal text. Whether either class is compiled or interpreted at runtime is governed by a separate per-actor policy that, by default, follows the parametric/literal split. *(Verification:*

ActorHandler.cs:1091-1093 documents the rule verbatim. Implementation: PrepareCommandProgram (ActorHandler.cs:1097-1148) decides via parameters.HasUserParameter() and assigns one of three JournalEntry values — IsScript (literal), IsNewAction (parametric, first invocation), IsExistingAction (parametric, cached invocation). Compilation flag set per program by AdjustCompilationMode (Program.cs:134-150) under the CompilationModePolicy enum.)

3. **Interpretation and compilation are strategies over a shared AST substrate.** They are not separate execution paths over translated representations. Both walk the same AST: interpretation walks it directly; compilation specializes it into IL via Expression trees. Both paths pay the same parsing cost; only the compiled path pays the compilation cost. *(Verification: Program.Execute() (interpretation) and Program.ExecuteExpression() → ProgramExpression().Compile() (compilation) operate over the same Statement tree produced by Parser.Parse().)*

4. **Journal density emerges from separability, not from a compression scheme.** A stable $\mathrm{F}(\mathrm{x}_1, \ldots, \mathrm{x}_\mathrm{n})$ collapses repeated invocations into (actionId, values) tuples, with the script definition persisted once and referenced thereafter. The compactness is structural, not designed. *(Verification: Diary.WriteNewActionEntry (definition + first invocation values) vs Diary.WriteActionEntry (actionId + values only on subsequent invocations).)*

5. **Verb richness: surface vs depth.** A single DSL verb may invoke orchestrations richer than its surface signature suggests. This is not the result of translation between layers; the DSL is the invocation language for domain operations implemented directly in the host language. Journal density is therefore the asymmetry between small persisted tokens and the live-domain operations they represent. *(Verification: §4 develops the construct and §5.5 measures it on two open-source DDD aggregates — eShop's `Order` purchase verb dispatches $\alpha = 9$ host-language invocations with a 24-method static closure ($\beta/\alpha \approx 2.7\times$); Grzybek's `SubscriptionPayment` verb dispatches $\alpha = 2$ with a 73-method static closure ($\beta/\alpha \approx 36\times$). α is exact; β is a crude static lower bound on reachable host surface (§5.5), and at n=2 the 2.7×-vs-36× gap is not attributable to any single cause. The supported claim is the qualitative one — the persisted token is far smaller than the behavior it names — carried by α and by journal density (§5.4), not by the β/α magnitude.)*

6. **Hot-loaded DSL programs over a stably-loaded domain.** The runtime supports hot-loaded DSL programs against a stably-loaded domain library: the assemblies configured as the actor's domain libraries are reflectively cached at first use; new DSL scripts can combine the types they carry in new ways without assembly reload. This enables ad-hoc procedure invocations against a live, stateful actor — without redeploying endpoints or altering the domain library. *(Verification: DomainLi-*

braries.GetOrLoad(params Assembly[]) caches public types statically per deduplicated assembly set; ActorV2.Using(scriptForChk, scriptForCmd) introduces new scripts per invocation against that cached surface area.)

When NOT to use this approach

The principle of program–value separability has limits, and the implementation pattern that realizes it has narrower limits still. We name six regimes in which the analysis presented here either does not apply, applies only partially, or invites overreach.

A. This paper is not advocating for compiling all DSL programs unconditionally

The structural taxonomy described here distinguishes parametric from literal scripts; both are first-class. Literal scripts — ad-hoc oracle invocations against a live actor, exploratory queries against a stateful runtime, configuration-time experiments — pay zero amortization cost because they incur no compilation. Treating them as legacy or transient would discard a structurally valid mode of interaction.

B. This paper does not address consistency under concurrent actors

The runtime described operates with strict per-actor ordering and isolation. Multi-actor consistency, reaction propagation across actor boundaries, and the contract under which observable state remains coherent are treated formally in a companion paper of this series.

C. This paper does not claim program–value separability is sufficient for the four-fold coherence

Separability is necessary but not sufficient. SQL prepared statements achieve compilation and caching while exhibiting only two of the four faces — the persisted artifact is row data, not the parameterized statement itself, so the journal is not code-as-data (script-form persistence does not emerge). Other realizations may exhibit further subsets. Puppeteer demonstrates one realization where all four faces emerge by structural commitment; the converse is not entailed by separability alone.

D. This paper does not extend separability to general-purpose programming languages

The principle is formulated for DSL runtimes — domain-specific languages whose programs are short, structured, and amenable to identification by their textual form. General-purpose programming languages introduce variables captured from enclosing scope, side-effecting state in the host, and identity that exceeds the parameter contract. Whether separability admits a meaningful formulation in that broader setting is a separate question, and we do not take it up here.

E. This paper does not claim that hot-loaded DSL programs replace traditional deployment

Claim 6 articulates a runtime capability, not a deployment recommendation. Hot-loaded DSL programs against a stably-loaded domain enable ad-hoc invocation; redeployment of the domain library remains the appropriate move when the implementation itself changes. The two are complementary, not alternatives.

F. This paper does not address security, sandboxing, or resource bounds for hot-loaded scripts

Claim 6 grants a running actor the ability to accept and execute DSL programs formed at call time. The structural argument made here is silent on the operational concerns that capability raises in production: sandboxing of the DSL evaluator, per-script resource limits (CPU, memory, wall-clock), authentication and authorization over which callers may submit ad-hoc invocations, and the trust model under which the supplied script is admitted at all. These concerns require separate machinery — out of scope for the conceptual contribution offered here, but unavoidable for any deployment that exposes hot-loaded invocation beyond a closed boundary.

---

1. Introduction

This paper makes an analytic theory contribution. It identifies a structural property of DSL programs that prior literature has touched but not isolated as one — the construct: *program-value separability*; derives the runtime consequences that follow when the property is held — the principles: compilation, caching, and dense journaling, developed and measured here, together with a fourth, replication-bounded entropy, that follows from the same precondition and is developed in a companion paper; and presents an instantiation — a system in which those principles have been realized in production — as confirmation that the construct is realizable. The contribution is conceptual; the instantiation is the existence proof of realizability, not the substance of the claim. The genre is the one Gregor (2006) names *theory for analyzing* (Type I): it introduces constructs and a taxonomy that let the phenomenon be described and classified, and states a necessary-condition claim about their consequences, with empirical evaluation supplementary. The Hevner-style design-science *evaluation* of the artifact (Hevner, March, Park, & Ram, 2004) — bundling artifact assessment with construct introduction — is deferred to a companion case-study paper; the Type I frame separates the two cleanly, and the instantiation here serves only as an existence proof of realizability.

The structural property at the center of this paper is not, in itself, new, and §7 documents its lineage. Partial evaluation, lambda lifting, closure conversion, and template instantiation all turn on a separation between a program and the values it will receive; each predates this work by decades. What is new is the dependent variable. In those traditions, separability is instrumental to *execution*: a binding-time or free-variable analysis recovers the static/dynamic split, a transformation consumes it to emit a faster residual program, and the split is discarded once the specialized code exists. This paper repositions the same property as the precondition for *persistence and identity*. The separated program is not consumed by a transformation but persisted as the durable, replayable, replicable unit of state — written once as a definition, referenced thereafter by a stable identifier and an argument vector, and reconstructed by deterministic replay. Compilation becomes one face among four, and not the load-bearing one. The claim this paper defends is that compilation, caching, dense journaling, and replication-bounded entropy are not four independent optimizations but four projections of this single precondition once the persisted artifact is the program rather than its effects — a connection the execution-specialization literature has no occasion to draw, because it never treats the program as state.

1.1 Calibrating the surface

You already understand this taxonomy. A SQL prepared statement is parameter-separable: the database parses it once, builds a query plan once, and reuses both for thousands of invocations with different bind values. An ad-hoc query offers no such surface — each is a fresh string, parsed and planned afresh. The same structural distinction governs DSL programs in any runtime that aspires to compilation, caching, and dense persistence. What changes between SQL and the runtime described here is not the principle, but the surface to which it applies. The principle, in other words, is neither new nor Puppeteer's: it is one every database programmer already relies on locally, each time a prepared statement is reused. What is new is the recognition that the same condition, taken as general rather than local, is what underwrites compilation, caching, dense journaling, and replication alike — and that a runtime can be built to honor it everywhere, not only at the query boundary.

That surface tends to be unfamiliar in a specific way. The professional reader of this paper has very likely spent a career persisting application state in relational tables. Where event sourcing has appeared in their work, it has typically appeared in tabular form: events as rows, schemas as projections, queries as joins against materialized views. The implicit assumption — that every representation of state ultimately resolves into something table-shaped — is not a failure of imagination but the horizon that industrial practice has produced over several decades. A runtime in which the persisted artifact is the program itself, in which density emerges from separability rather than from compression of rows, has had no canonical place in that landscape.

Calibration begins by separating two layers that the dominant paradigm tends to collapse into one. The first layer is the implementation: domain types, methods, business rules — written in the host language, indistinguishable in form from any well-designed object-oriented or functional library. The second layer is the invocation: the language by which a running actor is commanded and queried. In runtimes of the kind described here — of which Puppeteer, the framework introduced in the prior paper of this series, is one — that invocation language is a small DSL whose programs read as scripts, persist as journal entries, and execute as either compiled lambdas or interpreted walks over their own AST. The two layers are not translations of one another. The host language carries the implementation; the DSL carries the contract by which the implementation is reached. Conflating them — assuming the DSL must duplicate, transcribe, or shadow the host code — is where most readings of the architecture begin to diverge from its actual structure.

Three readings should be pre-empted at the outset: this is not double programming, not a translation layer, not a mapping specification. The domain implementation lives once, in the host language. The DSL is the language by which that implementation is invoked. The relationship is closer to that between SQL and a relational engine than to that between a model and a generated DTO: SQL does not duplicate the engine's logic; it names operations that the engine performs. A short SQL statement can trigger joins, locks, and execution plans far heavier than its surface text. The DSL of an actor runtime carries the same kind of leverage — small surface, deep behavior — and the persisted journal records not the unfolding of that behavior but the invocations that produced it. With the surface calibrated, the formal question can now be asked: what does it take, structurally, for a DSL program to be compilable, cacheable, and amenable to dense persistence?

1.2 A formal characterization

The question can be sharpened. Compilation requires a stable artifact to specialize. Caching requires a stable identifier under which to file the result of that specialization. Dense journaling — persistence that does not balloon with the size of every invocation — requires a stable description of the operation that can be referenced rather than copied. Each of these three demands the same property of the program: that it admit, at the moment it is named, a separation between the program itself and the values it will receive.

This property is **Program–Value Separability**. A DSL program is separable when its textual form names an operation parameterized by values supplied externally at invocation, rather than carrying those values within its body. Separability is structural, not stylistic: it is a property the program either has or does not have. It is decidable syntactically at preparation time, by inspection of the parameter declarations on the program's surface. The principle that follows admits a compact statement: program–value separation is a necessary condition for the *stable-identity* faces — caching a specialization under a key reused across invocations, journaling the program by reference rather than by copy, and replicating it as a bounded argument stream. Two scopings keep this from collapsing into a definition. The necessity is of the separation itself, not of any particular way of obtaining it: a runtime may have the programmer declare parameters, as the one described here does, or recover the separation by analysis. And the necessity is of *these* faces, not of caching in general — caching of other objects, such as memoized results, requires no separation at all. Necessity is the strong claim; its scope and the two natural objections to it are made precise shortly.

What separability buys, in a word, is *identity*. A program whose values are supplied from outside has a textual form that is stable across invocations — it is the same program the second time and the thousandth — and so it can be named; a program that embeds its values is a different string on every call, and no two invocations are the same program at all. Identity is the pivot on which the rest turns. Once a program has an identity independent of its values, caching has something to file under, compilation has a stable artifact to specialize, the journal has a reference to record in place of a copy, and replication has a unit to ship. The faces traced in this paper are best read not as four properties of separability but as consequences of the *program identity* that separability confers — and it is the absence of that identity, not the presence of embedded values as such, that makes a value-embedding script uncacheable, uncompilable-for-reuse, and undense in the journal.

Separability admits a binary structural taxonomy of DSL programs. A **parametric script** is separable from its values: it is filed under a stable identifier in a runtime cache and persisted to the journal as a compact reference plus an argument vector. A **literal script** is non-separable: it is not cached, runs once as written, and persists as its literal text. Whether either class is compiled or interpreted at runtime is governed by a separate per-actor policy that, by default, follows the parametric/literal split — but admits override. Under that default policy, the runtime documents the joint regime in its own comments: scripts without user parameters are interpreted, uncached, and persisted as Script entries; scripts with user parameters are compiled, cached with an ActionId, and persisted as Action entries. The two strict properties — caching and journal entry format — are encoded as values of a single enum decided at the moment a program enters preparation. The taxonomy is not analytical but operational: the runtime acts on it.

Necessity is not sufficiency. A program that admits separability gains the structural possibility of being cached, persisted as a compact reference, and amortizing its compilation — but does not by that fact alone exhibit dense journaling.

SQL prepared statements illustrate the gap: they are separable, they cache, they compile — yet the persisted artifact in the database is row data, not the parameterized statement itself. The third face — a journal in which the named operation is the persisted entry, not its downstream effects — does not emerge from separability alone. It requires the further commitment, code-as-data, that the runtime treat its programs as the persisted artifact: what is recorded in the journal is the program parameterized and the values it received, not the state changes those values produced. The sections that follow trace how separability, joined to code-as-data, yields the three faces in turn — and where a runtime that holds separability without the second commitment falls short.

Necessity, made precise. Two objections test the claim, and locating where each lands sharpens it rather than defeats it. The first is *memoization*: a runtime can cache the result of an invocation keyed on its inputs with no parameters declared anywhere. This is genuine caching, but of a different object — it caches outputs, not the program. A memoized value-embedding script still hits only on identical re-invocation, never across distinct argument vectors, and yields neither a reusable specialization, nor a reference-based journal entry, nor a bounded replication stream. Memoization caches effects; the faces at issue cache the operation. The second objection is stronger: a runtime can *recover* a separation the programmer never declared — canonicalizing literal scripts and abstracting their literals into a template after the fact, the inverse of constant folding. This does not refute the claim; it relocates it. What such a runtime recovers *is* program–value separation, obtained by analysis instead of by declaration. The separation remains the precondition; canonicalization is merely another route to it, one that pays an analysis cost and yields a separation that is provisional and runtime-derived rather than stable and surface-given. So the claim is not that a programmer must declare parameters — it is that the four-fold coherence requires the separation to exist, however reached, and that a *stable, syntactically decidable* separation is what makes the separated program serviceable as the durable identity under which journaling and replication operate, where an analysis-recovered one is not. Throughout, the definition of separability is syntactic — a property of program text — while the claim concerns that property's consequences; the two are kept apart precisely so the claim is not true by definition.

The DSL program characterized above — the textual artifact with declared parameters — is the unit of separability. When the term *program* appears at the substrate level in subsequent papers of this series (the unit that is recorded, replicated between nodes, and replayed against state), it denotes the pair: a **domain library** (the compiled classes and verbs against which the DSL is interpreted, loaded statically and identical across replicas) and a **journal of invocations** (the ordered sequence of action references with parameter bindings that the runtime has applied). Replay is the deterministic re-execution of the journal against the library; what persists, replicates, and reconstructs state is this pair. A DSL program in the sense of §1.2 is the unit; the substrate-level program is the cumulative state-producing artifact built from many DSL invocations executed against a stable library. The two readings are not in tension: separability of the unit is what makes the cumulative artifact dense; the cumulative artifact is what makes the unit's separability operationally consequential.

The central claim can now be stated without hedging. It is not that separability improves compilation — compilation is merely its most visible consequence, and the paper would stand were it removed, on caching, dense journaling, and replication-bounded entropy alone. The claim is that compilation, caching, dense journaling, and replication-bounded entropy are four operational projections of one structural condition: a program's having an identity independent of its values. Nor is the condition peculiar to this runtime or this DSL. No system can give a program a *reusable* identity — and therefore none of the consequences that depend on one, a cached specialization, a reference recorded in place of a copy, a replayable unit — while its values remain embedded in its representation. Separability is the general precondition; the runtime of §3 is one case study in what follows once it is met.

2. Consequences of separability

2.1 Compilation as specialization

The first consequence of separability is that the program admits ahead-of-time specialization. A separable program is, in form, a function awaiting its arguments — its body refers to values that will arrive at invocation, not to literals fixed at write time. That structural shape is precisely what a specializer requires. Given the program once, the runtime can resolve its references, lower its abstract syntax tree into a typed expression tree, and emit the executable form in advance of any specific call. The values are bound later; the work of preparing the program is done once.

The pipeline that produces this specialized form proceeds in three stages. The parser yields an abstract syntax tree in which user parameters appear not as values but as named slots — placeholders to be supplied at invocation. The runtime then traverses that tree and emits a typed expression: each named slot is bound to a parameter expression of the host runtime, and each operation of the script is bound to the corresponding host-language method or property access. Compilation of that expression yields an executable delegate over the parameter list — a function that takes the values supplied at invocation and runs the program against them.

Specialization, considered alone, is a one-time cost paid against an indefinite number of invocations. The parser runs once for the program; the expression tree is constructed once; the compiler emits the delegate once. Each subsequent invocation supplies new values into the existing delegate and runs through host-language code that has already been resolved, typed, and emitted. The amortization is structural: a separable program admits compilation that pays for itself across repeated invocation, while a non-separable program offers nothing to amortize against. The work, however, is only economical if the specialized form persists between invocations.

2.2 Caching as identification

Persistence between invocations requires an identifier under which to file the work that has been done. Separability supplies that identifier directly: the script string of a separable program is stable across invocations because its values are external to it. The same parameterized program text, encountered a second time, is recognizable as the same program — and the cached specialization belongs to it. A non-separable program has no such anchor; every invocation produces a different string, and no two invocations are the same program at all. Caching is not an addition; it is what becomes possible once the program has a stable name.

The mechanism is direct. On first encounter, the runtime parses the script, prepares its specialized form, and stores both under an action identifier — a stable handle assigned to the script when it is first encountered. The script string serves as the lookup key; the action identifier serves as the persistent reference under which the program's specialization lives. On subsequent encounters with the same script, the lookup succeeds, and the runtime retrieves the specialization without repeating the work that produced it. The cache is not a memoization of values; it is a registry of programs.

On a cache hit, the work that has been preserved is the program itself: the resolved tree, the typed expression, the compiled delegate. The values arriving at this invocation are bound into the delegate's parameter slots and the program runs against them. From the program's standpoint, nothing has changed between the first invocation and the thousandth — the script names the same operation, parameterized by the same slots, executed against the same domain. What differs across invocations is only the values supplied; what persists is the program.

2.3 Dense journaling as reference

The third consequence of separability is that persistence becomes reference rather than duplication. The program, having earned a stable identifier in the cache, can be written to the journal once — as a definition, with the script's text and its parameter declarations recorded in full. Each subsequent invocation is recorded not by duplicating the program text but by referring back to that definition under its identifier, accompanied only by the values supplied at that invocation. Where a non-separable runtime must record every invocation as a complete script — body, values, and all — a separable runtime records the program once and its invocations as compact references against it.

The mechanism distinguishes three kinds of journal entries. When a parametric program is encountered for the first time, the runtime writes a definition entry:

the script's text, its parameter declarations, and the values supplied at this first invocation, all recorded together under the program's newly assigned identifier. Subsequent invocations of the same program write only a reference entry — the identifier of the definition, plus the values for that call. A non-separable program, having no stable identifier, is written each time as a script entry: its text in full, since it cannot be referred back to anything that came before. Three entry kinds, decided mechanically at preparation time from the program's separability, encode the program's relation to its identity. The distinction is, at bottom, ontological: the entry type records whether the thing persisted *is* a program with an identity of its own — an Action, referable and replayable — or merely a script that ran once and has none. The journal is, in this sense, a ledger of which operations possess operational identity and which do not; separability is the test it applies, and the entry type is the verdict it records.

The density follows. For a parametric program invoked many times, the journal's storage scales with the cumulative size of the argument vectors, not with the cumulative size of the script's body — the body has been recorded once, and every later invocation costs only the bytes of its values. This is what makes the journal dense: nothing is copied that has been preserved by reference, and what survives is the operation that took place, not the state that resulted. The persisted artifact is the program — the script as text, parameterized, with its invocation values — rather than the downstream effects of running it. In code-as-data terms, the program lives in the journal as itself, not as a record of what it produced. Compilation, caching, and dense journaling are thus not independent optimizations but successive consequences of the same structural property: once a program is separable from its values, it becomes nameable; once nameable, it becomes cacheable; once cacheable, it becomes referable in persistence. The mechanism by which the runtime accomplishes this — the pipeline that compiles, the cache that retains, the journal that references — is the subject of §3.

3. Realization in a concrete runtime

3.0 Origins of the instantiation

The realization described in this section was not engineered to demonstrate the principle articulated above. The principle was identified by inspecting a runtime that had, over years of independent development for production use, come to satisfy it. The genealogy is structural rather than programmatic — first the artifact, then the principle that explains why the artifact cohered. What follows describes the runtime as it stands; the relationship between its mechanisms and the principle of §1.2 is a recognition, not a derivation.

The artifact is open to inspection, and the claims that follow are meant to be checked rather than taken on trust. The runtime is public at `github.com/alvaroNCubo/puppeteer`, archived in Software Heritage at the commit cited under *Code provenance*; every `file.cs:NN` reference in this section resolves against that public source. The benchmark harness of §5 is likewise public (this paper repository's `labs/` and the runtime's `tests-local/`), and the host aggregates it exercises are public MIT projects. The existence proof offered here is therefore verifiable in the sense that matters for the claim: a reader can read the cited code, build it, and re-run the measurements. The earlier, internal lineage of the runtime is not part of the proof — only the public snapshot is — and where the public snapshot differs from the internal development line, the public snapshot governs every reference and figure in this paper.

3.1 Compilation pipeline

The mechanisms described here are not independent engineering decisions but direct realizations of the separability principle established in the preceding sections. In the runtime described, the compilation pipeline runs from text to executable in three well-defined stages. A `Parser.Parse()` call lowers the script into an abstract syntax tree of `Statement` and `AstExpression` nodes. Each `Statement` carries an `ExecuteExpression` method that, when traversed, emits a `System.Linq.Expressions` node bound to the host parameters and the host-language operations the script names. The runtime composes these into a single `Expression.Lambda` and calls `.Compile()` to produce an executable delegate. Three stages — parsing, lowering, and compilation — produce the artifacts on which the runtime operates: the AST, the expression tree, and the compiled delegate.

The mechanism is straightforward to follow in the source. `PrepareCommandProgram` (`ActorHandler.cs:1097-1148`) orchestrates the first encounter: it rents a parser from a pool, parses the script, and decides — by parameter presence — whether to enter the parametric path. On the parametric path, the program's `ProgramExpression` method (`Program.cs:182-244`) walks the AST: for each Statement, it calls the Statement's `ExecuteExpression`, which returns an `Expression` node bound to the runtime's parameter and output expressions. These nodes accumulate into a `BlockExpression`, which `ProgramExpression` wraps in `Expression.Lambda<Func<Parameters, Output, string>>` (`Program.cs:244`). The compilation itself is one line: `_executable = programExpression.Compile()` (`Program.cs:163`). The compiled delegate lives in the program's `_executable` field; subsequent invocations skip the compile step and call the delegate directly with the values supplied at the call site.

The same mechanism applies one level deeper, to a feature the runtime calls *Eval parameters*. A parameter declared with the `Eval` modifier carries its own sub-script, which the runtime treats as a separable program in miniature: parsed once, compiled to its own delegate, and cached against the parameter's name. The cache structure is a dictionary keyed by parameter name to a tuple of the eval script and its compiled executable (`Program.cs:305`). On each invocation, the runtime compares the parameter's current script to the cached one; if they differ, the entry is invalidated and the sub-program is re-parsed and re-compiled (`Program.cs:323-331`). The principle scales: any region of the program that has a stable textual identity admits the same treatment as the whole.

3.2 Interpretation retained

The runtime preserves interpretation as a first-class execution mode, not as a legacy fallback. When a script presents itself without user parameters, or when a per-actor policy directs the runtime to interpret regardless of parameter presence, the program executes by walking the AST directly: each Statement carries an `Execute` method that runs against the program's current state, with no expression tree built and no delegate compiled. The interpreted path costs zero compilation but pays the cost of tree traversal at every invocation. Its place in the runtime is precisely the inverse of the compiled path's: useful where invocation is one-shot or unanticipated, useless where the program will be reused.

The decision is governed by a per-actor `CompilationModePolicy` enum with three values — `Automatic`, `AlwaysCompiled`, and `AlwaysInterpreted` — declared on the actor and defaulted to `Automatic` (`Actor.cs:8-13`). On the parametric path, `PrepareCommandProgram` calls `AdjustCompilationMode` with `useInterpretedMode: false`; on the literal path, with `useInterpretedMode: true` (`ActorHandler.cs:1117-1131`). Under `Automatic`, the policy honors that hint: parametric programs compile, literal ones interpret. Under `AlwaysCompiled` or `AlwaysInterpreted`, the hint is overridden in favor of the policy's name (`Program.cs:134-150`). Execution itself is dispatched by another switch on the same policy: `Perform` calls `ExecuteExpression` if the program is in compiled mode and `Execute` otherwise (`ActorHandler.cs:1955-1968`).

Caching applies asymmetrically to the two principal kinds of DSL invocation. For commands — programs that mutate actor state and persist to the journal — the runtime caches only when the script declares user parameters (`ActorHandler.cs:1117`). For queries, checks, and emit invocations — programs that read state without persistence — the runtime caches when the script declares user parameters or when no parameters are supplied at all (`ActorHandler.cs:1405`). The asymmetry is operational. Commands run under a write lock and serialize, so caching pays only when the same parametric program is invoked many times. Queries run under a read lock and parallelize, so a cache entry amortizes regardless of parametric reuse — a frequently-emitted query gains from caching even when its parameter set is fixed. One final observation: cosmetic variation in the script's text across releases generates distinct cache identifiers, but the runtime's reaction mechanism — treated in a companion paper — matches behavior against semantic patterns rather than identifier equality, so coherence is preserved across textual variants.

3.3 Hot-loaded DSL programs over a stably-loaded domain

The third element of realization is the runtime's support for hot-loaded DSL programs against a stably-loaded domain library. The phrase deserves care, because it is easy to misread. The hot element is the DSL: at any time during the actor's life, a new script — never seen before by this runtime instance — can be supplied, parsed, prepared, and executed without restart, redeployment, or reflection over a re-loaded assembly. The stable element is the domain library: the host-language types and methods that the DSL invokes are loaded once, by reflection over the assemblies configured as the actor's domain libraries, and held in a cache for the lifetime of the process. Hot-loaded programs combine stable domain elements in new ways; they do not bring new domain elements into being. The runtime is hot at one layer and stable at the layer beneath it.

The mechanism is brief in the source. When an actor is constructed, the public types of its configured library assemblies are walked once by reflection and cached against a deduplicated key: `DomainLibraries.GetOrLoad(params Assembly[])` (`DomainLibraries.cs:77-115`, with both a single-assembly and a multi-assembly overload), invoked from `ActorHandler` (`ActorHandler.cs:61`) with the actor's `LibraryAssemblies`. The cache survives the lifetime of the process; the same assembly set, loaded by a second actor, reuses the same domain dictionary. Against that fixed surface, new DSL scripts arrive through the actor's fluent interface — `ActorV2.Using(scriptForChk, scriptForCmd)` (`ActorV2.cs:32`) — at any time the actor is running. There is no AppDomain reload, no `AssemblyLoadContext` unload, no file-watcher over the assembly: the surface area is fixed at first load and dynamics live entirely above it.

The operational consequence is direct: a running actor can be queried or commanded with a script formed at call time, against any combination of the domain operations the configured library assemblies carry. A configuration adjustment, a one-off audit query, a derivation that the published endpoints do not anticipate — each can be expressed as a fresh DSL program and executed against live actor state without intermediate deployment. The relationship resembles gRPC in its directness — a procedure call against the live system rather than a navigation of REST resources — but without a pre-declared interface contract: the script itself is the contract, formed at call time. What has been shown in this section is that separability is not merely a formal property of DSL programs but one that a concrete runtime can recognize, act upon, and encode into its execution, caching, and persistence behavior. The realization characterized here is one of an extensible family: other actor runtimes — Akka, Orleans, Erlang/OTP — admit, in principle, the same construction, since a parameterized DSL surface above a host-language domain library would yield the same four-fold coherence under the same separability commitment. The expressive reach of such on-the-fly programs depends on the richness of the verbs the domain library exposes, which the next section takes up.

4. Verb richness: surface vs depth

Separability makes the program nameable; verb richness determines how much domain meaning that name carries. The verbs that a DSL invokes are not method calls on data structures. A verb in this setting names an operation against a live, stateful actor — an orchestration that the host language has been written to perform on behalf of the domain. A single verb may invoke many internal methods, traverse domain relationships, evaluate derived properties, trigger downstream behaviors, and write to multiple regions of state, all in the course of executing one DSL statement. Where a setter assigns a field, an actor verb concludes a transaction — moving the actor from one consistent state to another. The asymmetry between the surface — the few characters that name the verb in the script — and the depth — the volume of domain computation that runs when it is called — is structural, not incidental.

Consider a verb that any reader from an e-commerce or supply-chain background will recognize: the `Confirm` of a purchase order. The script that names this verb at the DSL surface is short — only a handful of tokens. What runs when the script is invoked is substantially larger: a verification that the order's reservations are still valid, a hold-to-allocation conversion across one or more inventory ledgers, a tax computation that depends on the order's line items and their fiscal contexts, a posting to the financial ledger, and a set of reaction triggers that propagate the confirmation to subscribed views and downstream actors. Each of these steps is itself a tree of host-language method calls; the leaf operations include arithmetic, predicate checks, persistence writes, and outbound messages. The runtime gives the consistency boundary that separates this depth from its surface first-class support: a check script and a command script can be supplied together via `ActorV2.Using(scriptForChk, scriptForCmd)` and dispatched as a single invocation through `PerformCheckThenCommand` (`ActorV2Invocation.cs:69`). The check runs against the actor's current state under a read lock; if its assertions hold, the command runs under a write lock and is persisted to the journal. If the check fails, no state change is applied and no journal entry is written — the actor's consistent state is preserved by construction.

Confirm is one example among many. The same asymmetry obtains for any verb that names a domain transition rather than a field assignment — Settle, Allocate, Reconcile, Release, and a hundred others that a well-designed domain library will expose. Verb richness is not an artifact of this particular runtime; it is a property of how domain operations are conceived and named. The implication folds back into the journal density argument of §2.3: when each persisted entry refers to a domain operation that may run dozens to hundreds of internal method calls, the journal's compactness in bytes is matched by an enormous compactness in semantic information per byte. The journal does not record state changes; it records named transactions, each of which carries its full operational meaning by reference to the domain library that knows how to run it. The value of separability would be modest if the verbs it named were shallow. It becomes profound when each name stands for a full domain transaction.

Under porous substrates, the asymmetry inverts entirely. There, every domain element must persist into a relational schema, and deep domain logic becomes a serialization burden — each derived state, each accumulator update, each new field propagates as schema complexity, ETL transforms, and projection overhead. Under a code-as-data substrate, the same depth is rewarded, not penalized: the domain library can be made arbitrarily expressive without expanding what the journal must record. Complex abstractions compose without friction: no DTOs, ORM annotations, or projection layers stand between the domain operation and its persistence. Porosity makes deep domains expensive to represent; anti-porosity makes them economical to compose. The two directions reinforce one another: the richer the domain, the more meaning each Action carries per persisted byte; and because the journal pushes no persistence concern back into the domain, the domain is free to grow richer still. Density and domain depth are not in tension — each makes room for the other, the plain structural consequence of recording operations instead of their effects. This asymmetry is why journal density matters at all. The journal is not compressing syntax — shaving bytes from a line of text; it is preserving a reference to an operation whose semantic volume exceeds its textual surface. An ActionId stands not for a line but for a domain transaction: were a verb a mere setter, density would be a trivial saving, but because a verb can name a whole transaction, the compact entry preserves something large by reference rather than copying it. This is the point the prior paper makes from the other side — the substrate stays dense because what it records are operations, not their unfolded effects. The empirical magnitude of these numbers — how many operations a typical verb dispatches, how many bytes a typical journal entry occupies, how the ratio behaves at scale — is the subject of §5.

5. Empirical results

5.1 Methodology

The measurements that follow span two complementary axes. The first axis is *program complexity*: a synthetic straight-line arithmetic kernel parametric on integer inputs (depth 5 to 100 statements); a DSL-rich kernel exercising control flow (for, if), arithmetic, and parameter binding (~500 dispatched operations per invocation); and a multi-item purchase command operating against an external rich-domain aggregate. The first two tiers isolate runtime overhead independent of host-language work; the third locates that overhead within a representative end-to-end transaction.

The second axis is *host codebase*. The production-verb measurements are replicated against two independent open-source MIT-licensed DDD aggregates that share a structural shape (rich behavioral methods on aggregate roots) but represent disjoint business domains: `dotnet/eShop`'s `Order` aggregate (e-commerce ordering with multi-item cart and a four-step state machine), and `kgrzybek/modular-monolith-with-ddd`'s `SubscriptionPayment` aggregate (subscription billing with a payment-lifecycle verb). The dual-codebase design is deliberate: if the measured properties of compilation, caching, and journaling are structural consequences of the runtime rather than artifacts of a particular domain, the same property should appear on both codebases. The selection criterion was structural similarity (DDD aggregates with rich behavior methods rather than CRUD entities) over an unrelated business domain. This criterion selects on the dependent variable, and we are explicit about what that does and does not buy: choosing two rich-behavior aggregates tests whether the measured properties survive a change of domain *within* that class — it does not probe where the mechanism stops paying off. To probe the floor rather than the favourable case, §5.2 and §5.5 add an adversarial *flat-CRUD* verb (a single field setter), and §5.6 reads the result: the CRUD verb is itself separable, so all four faces still obtain. What varies with the domain is not whether they hold but the *magnitude* of the speedup (which persists even on a trivial verb) and the separate, non-face property of *verb richness* (which collapses to ~1× when the domain is shallow). Both aggregates are reachable as public source for reproduction; Appendix A lists the per-section datasets and the commit SHA.

Two hosts are not a statistical sample, and no claim of generality is rested on them. The structural claims of this paper are carried by the argument of §1–§4; the two codebases serve a narrower, falsificationist purpose — to test whether the predicted properties are artifacts of one domain. Were they domain-specific, they would not appear on a second, independently-authored aggregate from a disjoint business domain. That they appear on both, with the same structure and differing only in magnitude, removes domain-specificity as an explanation; it does not, and is not offered to, estimate a population parameter. The two hosts are a robustness replication, not a survey.

Measurements were produced against the public runtime commit `b42d0f7` (Appendix A), built in **Release**, on a 13th-generation Intel Core i9-13900 (24 physical / 32 logical cores), 64 GB RAM, Windows 11 (build 26200), .NET 9.0.14 (RyuJIT, AVX2). Two classes of measurement are reported and they differ in kind. *Deterministic* measurements — journal entry counts and payload bytes (§5.4), DSL dispatch counts and static call-graph closure (§5.5) — are exact, independent of build configuration and of any timing instrument, and reproduce bit-identically across runs; they carry the structural claims and are verified identical between Debug and Release. *Timing* measurements — the compiled-versus-interpreted speedup (§5.2), cold compile cost (§5.2), and eval-cache hit/miss (§5.3) — use the instrument each regime requires. Steady-state throughput (the speedup) uses **BenchmarkDotNet** (v0.14): 8 warm-up iterations and 15 measured iterations of 20,000 invocations each, with tiered compilation disabled (`DOTNET_TieredCompilation=0`) so the dynamically-emitted compiled delegate is fully optimized from the first measured invocation rather than averaged across a Tier-0→Tier-1 transition; means are reported with their 99.9% confidence half-interval. One-shot costs (a program's cold compile, an eval sub-program's recompile) are isolated at their exact call site by a runtime instrumentation hook and timed with Stopwatch — the appropriate instrument for a single hundreds-of-microseconds event, not a tight steady-state loop. Cache and pool hit rates use Interlocked counters; journal-density measurements parse the runtime's binary journal format directly. The pool-hit-rate figures (§5.3) and the cache-footprint figures (§6.3) are such counter and GC measurements — insensitive to build configuration and to the exact commit — taken on the same public runtime line; they are reported as measured rather than separately repinned to `b42d0f7`, whose pool and cache code paths they exercise unchanged. The bench follows a bootstrap-then-measurement pattern: a non-parametric bootstrap script establishes the facade in the actor's persistent symbol table once, and subsequent invocations bind per-iteration values via the runtime's parameter API while the script string stays constant, so the compiled-program cache hits on every call — the regime the amortization argument names. Raw per-iteration data, the BenchmarkDotNet environment reports, and the commit SHA are in the companion `data/` directory (Appendix A).

5.2 Compilation amortization

Compiled execution shows a steady-state speedup over interpreted execution that varies with where the program's work resides. For a synthetic arithmetic kernel of depth 100 — wholly DSL-bound — the speedup is ≈3.1× (interpreted 2.99 µs, compiled 0.97 µs). For a DSL-rich kernel of ~500 dispatched operations exercising for-loops, conditionals, and parameter binding, it is ≈2.2× (interpreted 16.0 µs, compiled 7.3 µs). For a multi-item purchase verb against the eShop `Order` aggregate — one DSL dispatch cascading through the `Order` constructor, three `AddOrderItem` calls, and a four-step state-machine walk to Shipped — it is ≈1.5× (interpreted 1.97 µs, compiled 1.31 µs). These are BenchmarkDotNet means from a single run with tiered compilation disabled; the within-run 99.9% confidence half-interval is a few percent, but the ratios carry process-to-process variation of order ±0.2 (the variation caveat in §5.6 applies), so they are read to about one significant figure. The pattern is monotonic: the more of the per-invocation work is DSL-bound, the larger the speedup; the more it is domain-bound, the smaller. The runtime amortizes only the AST-traversal overhead — the host-language code the verb dispatches runs identically in both modes, so a verb dominated by host work shows the compiler's contribution as a smaller fraction. (Under Debug builds, with neither path optimized by the JIT, the same curve spans 1.49×–4.10×; the Release figures here are the defensible ones and the curve's direction is unchanged.)

A deliberately adversarial datapoint sharpens what the curve measures. A flat-CRUD verb — a single field setter invoked as `o = c.SetValue(v)`, with $\alpha = 1$ host dispatch — does *not* collapse to a 1× speedup: it measures ≈1.9× (interpreted 0.63 µs, compiled 0.34 µs). The speedup does not track domain richness; it tracks the fraction of per-invocation time spent in the DSL dispatch that compilation removes — reflection-based method resolution and parameter binding — and that cost is present even when the host method is trivial. What lowers the speedup is host work, not domain simplicity: the production verb, one dispatch into a heavy aggregate, sits lowest (≈1.5×) precisely because its host work dilutes the fixed DSL overhead, and an I/O-bound verb would approach 1× from below. The flat-CRUD case is genuinely adversarial only for a *separate* property — verb richness, which collapses to $\beta/\alpha = 1\times$ there (§5.5) and which is not one of the four faces — and for none of the faces themselves: the CRUD verb is separable, so it still caches, journals densely, replicates with bounded entropy, and compiles with amortization. The adversarial case maps the *magnitude* of the speedup and the *depth* a verb names; it does not turn any face off.

Specialization is paid once, at first encounter, not at every invocation. Compiling a single program scales about linearly with its statement count: for straight-line arithmetic kernels the cold compile cost, isolated at the `Compile()` call site, runs from 0.56 ms (p50, depth 5) to 4.30 ms (p50, depth 100) — a slope of roughly 39 µs per statement. The eShop purchase verb is a single dispatch and therefore a small expression tree; it compiles cold in 380 µs at p50 (516 µs at p95; N=100 distinct script variants forcing cache misses). Against the 0.70 µs per-invocation steady-state saving for that verb, the cold compile recovers itself after roughly 540 invocations — within the first moments of any actor that outlives its warm-up. These numbers are not performance claims but empirical confirmation of the structural amortization predicted by separability; the comparison is internal (compiled vs. interpreted on the same runtime), not a competitive benchmark against external systems (§5.6).

5.3 Cache amortization

The same amortization pattern applies recursively to sub-programs. A parameter declared with the Eval modifier carries its own sub-script that the runtime treats as a separable program in miniature; on a cache hit, the sub-program pays only the cost of invoking its cached delegate, while on a cache miss — when the sub-script's text differs from the cached entry — it re-parses, rebuilds, and re-compiles. Across three sub-program complexities (a two-term arithmetic expression, a fifty-term arithmetic kernel, and a method call against a facade-bound counter on the eShop side), the cache-hit cost is 0.5–0.6 µs at p50 while the cache-miss cost is 214–243 µs at p50. The miss-to-hit ratio is roughly 400× regardless of sub-program complexity — a separation of nearly three orders of magnitude that confirms the principle extends to any region of the program with a stable textual identity.

Allocation pressure is amortized at a finer granularity by parser and parameter pools. Three workloads measured under AlwaysCompiled policy: 1,000 single-thread invocations against a stable parametric script recorded a 100% parameter-pool hit rate; 1,000 single-thread invocations against distinct cold-cache scripts recorded a 100% hit rate on both pools; 5,000 invocations across 8 parallel threads against distinct scripts recorded 99.90% on parsers and 99.86% on parameters — twelve total misses across the parallel workload's ~10,000 pool rents. Allocation of new Parser and Parameters instances is incurred at startup and under brief thread-fanout transients only; under steady state, both pools service the rent without allocation.

5.4 Journal density

In a parametric purchase workload against the eShop `Order` aggregate — a nine-line DSL script cascading through order construction, four `AddOrderItem` calls, and a four-step state-machine walk — 99.8% of journal entries land as compact action references: 1,001 invocations produce 1,001 action entries plus a single Define entry that carries the script body once. Each action entry's payload averages 115 bytes — the argument vector for seventeen parameters (user identity plus four products' details plus shared modifiers). The Define entry's payload is 642 bytes — the script body itself, persisted once. Had each invocation stored the literal script text instead, the action payload would be 5.6× larger. The runtime's choice to persist named operations rather than their textual content is what makes this density possible. This density does not arise from compression techniques but from reference instead of duplication.

The same compaction structure appears on the second host. Against Grzybek's `SubscriptionPayment` — a two-statement DSL surface (`NewWaitingPayment` followed by `MarkAsPaid`) over a payment-lifecycle aggregate — N=1,000 parametric invocations produce 1,001 Action entries (99.8% of the journal) plus a single Define entry of 207 bytes carrying the script body. Action payloads average 60 bytes. The literal-script storage would be 3.5× larger. The ratio is more modest than eShop's because both the script body and the argument vector are smaller on a narrower verb; the structural property — arguments scaling with invocations while the body is persisted once — is identical.

The magnitude of the ratio is a function of how the host's domain API surfaces its data, not of the compaction mechanism. eShop's `AddOrderItem` carries the full product specification per item — pid, name, price, discount, picUrl, units — so per-iteration argument vectors are dense (115 B). Grzybek's `NewWaitingPayment` accepts five primitive parameters and `MarkAsPaid` takes none, so the argument vector is short (60 B). Hosts whose verbs identify catalog entries by short stable references compound the ratio further; hosts whose verbs string together longer parametric sequences also compound it. Either way the structural property holds: arguments scale with invocations; the script body does not.

5.5 Verb richness

For the eShop purchase verb the DSL script dispatches $\boldsymbol{\alpha = 9}$ host-language invocations per DSL invocation — an exact, deterministic count, reproduced without variance across 1,000 runs. To gauge the host-language surface those 9 entry points reach, a syntactic Roslyn walker computes the *static forward closure* of the call graph, excluding trivial accessors and cutting off at the project boundary: $\boldsymbol{\beta = 24}$ methods within `dotnet-eShop/src/Ordering.Domain`. For the Grzybek `SubscriptionPayment` verb, $\alpha = 2$ (its facade folds value-object construction into one host call) and $\beta = 73$ within `Payments.Domain` plus the shared `BuildingBlocks/Domain` (275 declarations indexed, 105 trivial accessors filtered). The β/α ratios are 2.7× (eShop) and ~36× (Grzybek).

What β does and does not measure must be stated plainly. β is a *static lower bound on reachable declared surface*, not a count of operations executed: it over-counts (branches never taken at runtime remain in the closure) and under-counts (it does not follow virtual dispatch, delegates, or reflection). It is also sensitive to assembly partition — moving code across `.csproj` boundaries moves the cutoff and changes β — so it measures reachable surface *under a chosen boundary*, not an invariant of domain depth. The exact, faithful quantity is α: one DSL token dispatches several host operations. β corroborates only the coarse direction the surface-vs-depth claim of §4 needs — that the host neighborhood a single verb names is much larger than the verb's textual surface — and nothing finer.

The 2.7×-versus-36× gap, in particular, should not be read as the measured effect of a single cause. With two hosts, author coding style, aggregate granularity, and persistence-driven design pressure are fully confounded, and the data cannot apportion the difference among them. Both `Ordering.Domain` and `Payments.Domain` are RDBMS-anchored DDD aggregates — `private set` properties, EF-Core owned-entity annotations, `int?` foreign keys instead of object references, little polymorphism on the roots — and it is *plausible* that this representational pressure flattens a graph a richer domain would deepen. But that is a hypothesis these two points illustrate, not a relationship they establish; and the very features a richer domain would add — polymorphism, delegates — are exactly what the static walker cannot follow, so the conjecture that β would grow by orders of magnitude on such a domain is not one this instrument can confirm. What the measurement does support is the modest, exact claim of §4: a single persisted DSL token names a host-language operation substantially larger than itself, so the journal records far less than the behavior it commands. That observation rests on α and on the journal-density result (§5.4), both exact — not on the β/α magnitude or any causal reading of it.

An adversarial floor case makes the dependence explicit. A flat-CRUD verb — a single field setter that calls nothing — has $\alpha = 1$ and $\beta = 1$, so $\beta/\alpha = 1\times$ and the surface-vs-depth asymmetry vanishes entirely. Verb richness is therefore a property of the host domain's depth, not of separability or the runtime, and it is not one of the four faces: where the domain is shallow there is simply no depth to name. Its collapse disables none of the faces — the flat-CRUD verb, being separable, still caches, journals densely, replicates with bounded entropy, and compiles with amortization; the journal entry is simply a compact reference to a verb that happens to name little.

5.6 Limitations and threats to validity

Several boundaries qualify the magnitudes above. *Sample of hosts.* Two aggregates are a robustness replication against domain-specificity (§5.1), not a sample from which a population magnitude can be estimated; the generality this paper claims is structural (§1–§4), and the empirical role of eShop and Grzybek is to show the predicted properties are not artifacts of one domain. *Single environment.* All timings come from one machine and one runtime build, so absolute microsecond figures are machine-specific. The portable quantities are the ratios — speedup, miss-to-hit, density — each a comparison of two paths under identical conditions, where machine-constant factors cancel. The speedup ratios additionally vary by order ± 0.2 across process launches (ordinary microbenchmark reality), so they are read to about one significant figure of confidence — their ordering and direction, not their third digit. The journal-density and dispatch-count figures, being deterministic counts, do not carry this variation. *Steady state versus production regime.* The speedup is measured with tiered compilation disabled, which isolates the fully-optimized steady state of both paths and removes the Tier-0→Tier-1 transition that otherwise makes the dynamically-emitted compiled delegate bimodal; under the default tiered regime a long-lived actor reaches the same steady state, with wider variance during warm-up. *Synthetic kernels.* The arithmetic and DSL-rich kernels isolate runtime overhead independent of host work; they are not stand-ins for whole applications, and the production verb is included precisely to locate that overhead inside a representative end-to-end transaction.

What none of these qualifications touch is the structural result, because it rests on the *deterministic* measurements rather than the timings. The journal-density and verb-richness figures (§5.4–§5.5) are exact counts and bytes, identical across builds and machines, and they carry the paper's central empirical observation: that the journal's expressiveness per byte rises as the host domain grows richer — each persisted action names a deeper orchestration — while the persisted footprint per invocation does not. The relationship runs both ways. A richer domain library makes each named action carry more meaning per stored byte; and the DSL, carrying no persistence concern of its own, lets the domain be modeled as richly as its designers wish without expanding what the journal must record. The timing results confirm only the *direction* of the amortization argument — that compilation pays for itself, within a few hundred invocations — not a universal constant; the structural claim does not depend on their magnitude.

No external baseline, by design. The speedup of §5.2 is an internal, controlled comparison — compiled versus interpreted execution of the same DSL program, against the same host domain code, under the same journaling, on the same runtime — so the only variable is the one separability is about: the AST-traversal overhead that compilation removes. It is deliberately not a competitive benchmark against Orleans, a hand-written compiled expression tree, or a SQL stored procedure. Those alternatives differ from the runtime here on several axes at once — language, persistence model, domain encoding, transport — so a latency comparison would measure a confounded mixture rather than the separability effect, and this paper makes no claim that the runtime is faster than any of them. The external comparison the thesis does require is structural rather than chronometric: whether a system exhibits the four-fold coherence at all. On that axis SQL prepared statements (§6.4, §7.6) and Truffle/GraalVM (§7.2) are compared explicitly — they share separability and its execution-side consequences but, persisting rows or volatile machine code rather than the named operation, exhibit only a subset of the four faces. The speedup confirms that compilation amortizes as the principle predicts; it is not offered as evidence of competitive performance, and "compiling beats interpreting one's own AST" is indeed its expected, modest content — the load-bearing empirical results are the deterministic density and dispatch measurements, not the timings.

Selection of hosts, and the adversarial case. The two production hosts were chosen for structural similarity (rich-behavior DDD aggregates), which is selection on the dependent variable: on its own it can only show the mechanism surviving a change of domain within that class, not where it stops paying off. The flat-CRUD verb of §5.2 and §5.5 is the corrective, and it locates the limits precisely — but the limits are not where a first reading might place them. The CRUD verb is itself separable, so all four faces still obtain: it caches, journals densely, replicates with bounded entropy, and compiles with amortization. What the adversarial case varies is not *whether* the faces hold but two quantities that are not faces at all: the *magnitude* of the compiled speedup (a gradient set by how much per-invocation time is DSL dispatch versus host work — $\approx 1.9\times$ even for the trivial verb, falling toward $1\times$ only as host or I/O work dominates) and *verb richness* ($\beta/\alpha \to 1\times$ when the verb is a bare setter), a property of the host domain's depth that the journal-density argument uses but that separability does not entail. So this does not qualify the title's claim. Separability remains the precondition for the four faces, and they obtain wherever a program is separable; the adversarial case maps how much each consequence delivers across domains, not a boundary where the consequences disappear. The boundary where separability holds but a face does *not* is a different case entirely — a separable runtime lacking code-as-data, exhibiting caching and compilation but not dense journaling — and it is SQL prepared statements (§6.4), the worked example of necessity without sufficiency.

6. Counter-arguments

6.1 *"This is just JIT compilation"*

A reviewer familiar with managed runtimes might object that the runtime described here is just-in-time compilation in disguise. The objection conflates two distinct compilation events. The .NET CLR's JIT translates IL bytecode into native machine code at first invocation of any method; this happens identically whether the runtime executes its DSL by walking an AST or by invoking a compiled delegate. What the runtime adds is a separate, prior compilation: from DSL script to typed Expression tree to IL — the work that produces the delegate the JIT will eventually translate. The speedup measured in §5.2 is a function of this DSL→IL stage, not of the IL→native stage. Both modes deliver IL to the JIT in the same way, so JIT effects cancel; what remains is the AST traversal overhead that the DSL→IL stage amortizes away. Calling this "just JIT" would erase the layer of specialization that separability makes possible.

6.2 *"Why retain interpretation? Just compile everything"*

A second objection: if compilation is so much faster, why retain interpretation at all? The answer is that compilation is economical only when the program will be reused. A script seen once and never again — an ad-hoc query against a live actor, a one-off configuration adjustment, an exploratory invocation formed at call time — provides no future invocations against which to amortize the compilation cost. Forcing such scripts through the compilation pipeline pays a cold compile cost (§5.2) — hundreds of microseconds to a few milliseconds, depending on program size — and discards the result; the compiled delegate is never invoked a second time, and the journal records the script literally rather than as a referenced action. The runtime's `AlwaysCompiled` policy permits this on demand, but the default policy correctly recognizes that not every script is amortizable. Separability is necessary for compilation to make sense; reuse is the condition under which compilation is economical.

6.3 *"Dual paths add memory cost"*

A third objection: maintaining a compiled delegate per parametric program inflates memory at scale, and the dual-path arrangement compounds the cost. The measurement disagrees. In a single-actor cache holding 100, 1,000, 10,000, and 100,000 distinct parametric programs under `AlwaysCompiled` policy, the per-entry footprint stabilizes at approximately 6 KB across all four scales: 6,039 bytes per entry at 100 programs, 5,981 at 1,000, 5,977 at 10,000, and 5,930 at 100,000 — no super-linear overhead, no hash-table degradation. The marginal memory of an invocation against an already-cached program is effectively zero: 100,000 invocations against a single cached program retain 104 bytes total. A production actor caching dozens of distinct parametric programs and invoking them at scale incurs a memory cost on the order of hundreds of kilobytes. The cache scales linearly with distinct programs, not with invocations — well below the threshold at which the dual-path arrangement would be a structural concern.

6.4 *"This is just SQL prepared statements"*

A final objection: this is just SQL prepared statements rebranded — the same parametric-template-plus-bind-values pattern that relational databases have used for decades. The pattern is indeed the same; what differs is what the runtime persists. A SQL prepared statement is parameter-separable and admits compilation and caching: the database parses it once, builds a query plan once, and reuses both for many invocations with different bind values. But what the database persists is row data — the projections produced by executing the statement against the underlying tables. The parameterized statement itself is not the persisted artifact; the rows it touches are. The runtime described here persists the operation rather than its row-level effects. Two of the four faces of separability — compilation and caching — are present in both systems; the journal density that follows from persisting the named operation is what SQL prepared statements do not achieve. Separability is necessary; pairing it with code-as-data is what produces the dense journal.

6.5 *"A verb dispatching dozens or hundreds of host methods produces an opaque runtime"*

A reviewer might object that a verb whose static call-graph closure reaches dozens or hundreds of host-language methods (§5.5) produces an opaque runtime, and that debugging across such depth is intractable. The objection misreads the architecture. The depth lives in the host language: domain classes, methods, business rules — debuggable with the standard tools the host already provides. The DSL surface only names what the host invokes; it does not introduce a separate execution layer that the host debugger fails to penetrate. Standard practice in the runtime described here proceeds by test-driven development with end-to-end test cases, with breakpoints and step-through performed in the host language; once a script is moved to a runtime endpoint, the same breakpoints continue to apply against the domain library it invokes. More consequentially, the journal makes post-mortem debugging tractable in a way porous substrates do not allow. When a production failure surfaces at journal entry id N, that entry refers — by name — to the exact script that produced the defect, parameterized by the exact values the script consumed. A breakpoint at the journal entry, a step-through of the named operation, and the defect reproduces; the failure is then replicated in a small end-to-end test case, fixed, and released. The dense journal is not the opacity it might appear to be on a superficial reading; it is among the runtime's most powerful debugging artifacts, because the persisted artifact is the operation that ran, not a downstream trace of its effects.

Each objection treats compilation, caching, and journaling as independent engineering choices. The argument of this paper is that they are consequences of a single structural property; the objections dissolve when that property is made explicit.

7. Related work

7.1 Partial evaluation

The structural condition this paper formalizes has a close ancestor in **partial evaluation**, as developed by Jones, Gomard, and Sestoft (1993) and the Futamura projections (Futamura, 1971/1999). Partial evaluation specializes a program with respect to a subset of inputs known in advance, producing a residual program that depends only on the dynamic arguments — and it requires, structurally, that the program's static and dynamic inputs be separable. Where the partial-evaluation literature applies the technique to general-purpose host languages, this paper applies the same separability principle to a domain-specific language whose programs are short, parametric by construction, and stored as journal entries; in this setting, separability is syntactic, decidable from the program's surface rather than from static analysis. Two differences are load-bearing. First, partial evaluation *recovers* the static/dynamic split by binding-time analysis and *consumes* it: the residual program is the artifact of interest, and the separation is discarded once specialization is complete. Here the split is declared on the program's surface and *persisted* — it survives into the journal as an (identifier, argument-vector) pair and into replication as the unit that replays to state. Second, the further consequences traced in §2 — caching as identity, dense journaling, replication-bounded entropy — fall outside the partial-evaluation tradition's concern, which is execution speed rather than persistence; they follow from pairing separability with code-as-data persistence. Partial evaluation is the closest ancestor of the compilation face alone; on the other three it is silent.

7.2 Self-optimizing AST interpreters (Truffle/GraalVM)

The most direct comparison is not a classical technique but a production system: the Truffle framework and the GraalVM compiler (Würthinger et al., 2012, 2017). Truffle hosts a language as a self-optimizing abstract syntax tree whose interpreter nodes rewrite themselves under runtime profiling, and GraalVM applies partial evaluation to the interpreter specialized to a given AST — realizing the first Futamura projection in practice, from interpreter-plus-program to machine code, cached per call target and guarded by deoptimization. On the compilation face (§2.1, §3.1), Truffle is more sophisticated than the runtime characterized here: it profiles observed types and values, speculates aggressively, inlines across a polyglot boundary, and deoptimizes when a speculation fails. A reviewer is right to ask what program–value separability adds once Truffle is on the table. The first half of the answer is that Truffle already turns on it: its specialization holds the AST constant while frame arguments vary, and its compilation amortizes only when the same AST recurs across invocations — so it *presupposes* program–value separation. A program embedding its values would present a fresh AST per call and defeat Truffle's reuse exactly as §2.2 describes it defeating the cache. Truffle is thus not a counterexample to the necessity claim but an independent witness to it, operating on the same precondition.

The second half is that Truffle never leaves the execution axis. Its specialization is volatile execution machinery — machine code keyed to an in-process call target, provisional under deoptimization, discarded at process exit, bound to no stable name, replicated to no other node. There is no journal in which the specialized program is the persisted entry, no identifier under which it is referenced across time, no replication of the program as the unit of state. The separation Truffle exploits is operational and implicit, recovered and re-validated by the runtime; the separation this paper formalizes is declared on the program's surface, decided once at preparation time without profiling or speculation, and elevated to a durable identity — a content-derived action reference that is at once the cache key, the journal entry, and the replication unit. This makes the necessity claim of §1.2 precise: declared separability is not necessary for fast execution — Truffle shows a profile-driven runtime can compile and cache without it — but it is necessary for the persistence faces, because a provisional, in-memory, profile-derived boundary cannot serve as a stable journal reference or a replication anchor. The mechanism follows the same divide: Truffle partially evaluates a host-language interpreter against the AST; the runtime here lowers the DSL AST directly into a typed expression tree and a delegate over the declared parameter vector, with no interpreter to specialize and no speculative guard to maintain.

7.3 Lambda lifting

Lambda lifting (Johnsson, 1985) is a program transformation that rewrites locally-defined functions whose free variables capture an enclosing scope into top-level functions that receive those variables as explicit parameters. The transformation makes the function's dependencies syntactically visible — and, by doing so, makes the function amenable to ahead-of-time compilation, separate compilation, and uncluttered call-graph analysis. The structural insight is the one this paper generalizes: dependencies that remain implicit in the program's body cannot be specialized over, but the same dependencies promoted to the surface can. Where lambda lifting transforms programs that were already written and externalizes their captured environment, the runtime described here requires programs to declare their values as externalized parameters from the start. Lambda lifting opens compilation; this paper extends the same principle to caching, dense journaling, and replication-bounded entropy by joining separability with code-as-data persistence.

7.4 Closure conversion

Closure conversion (Appel, 1992) is a compilation technique that translates a function with captured lexical environment into an explicit closure record — a pair of function pointer and environment data — that the runtime carries as a single value. Closure conversion makes implicit lexical dependencies explicit, but preserves them as runtime data: the closure travels with its environment record, retained in memory, and re-bound on each invocation. Lambda lifting and the principle of this paper sit on the other side of the same choice: rather than carry the environment, eliminate it by promoting the variables that would have been captured into top-level parameters supplied at the call site. The runtime described here makes that choice mandatory and declarative — values must be supplied at invocation, not captured from a surrounding scope — which is what permits the journal to record the program by name without any captured-environment record traveling alongside it.

7.5 Template instantiation

Template instantiation in general-purpose languages — exemplified by C++ templates (Stroustrup, 1994; Vandevoorde, Josuttis, & Gregor, 2017) and the broader tradition of generic programming — applies the separability principle in another setting: a template body names operations parameterized by types or compile-time constants, and the compiler instantiates each template with concrete parameters at compile time, producing specialized code per instantiation. The structural pattern is the same as the one described here: a program that admits parametric reuse must be written so its parameters are separable from its body. Template instantiation operates over types and compile-time values; the runtime described here applies the same principle over runtime values supplied at invocation. The distinction is operational: instead of monomorphizing the program once per parameter tuple, the runtime compiles the program once and rebinds its arguments on every call. Both arrangements presuppose what this paper formalizes.

7.6 Prepared statements

Prepared statements in relational database systems (SQL standard ISO/IEC 9075:2023; Hellerstein, Stonebraker, & Hamilton, 2007) are the closest operational analog to the runtime described here. A prepared statement carries a parametric template (with placeholders for bind values), the database parses and plans it once, caches the plan, and reuses both for many invocations with different argument vectors — the same parametric-template-plus-bind-values pattern §1.1 used as pedagogical anchor for the principle this paper formalizes. The two systems share separability and its first two consequences: compilation (the query plan) and caching (the plan cache). They diverge on what the runtime persists. A relational database persists row data; the prepared statement itself is not the persisted artifact. The runtime described here persists the named operation rather than its row-level effects, completing the four-fold coherence with journal density and replication-bounded entropy. The principle this paper formalizes generalizes beyond any one runtime — it characterizes the structural condition any DSL runtime must satisfy to admit compilation, caching, and dense persistence at all.

These traditions arise in different domains — compilers, functional languages, generic programming, database engines — and they share one structural observation: programs whose dependencies are syntactically separable from their bodies admit forms of specialization that programs with embedded values cannot. What they do not share is what this paper adds. In every case above, separability is instrumental to execution and the separation is consumed by a transformation or a query planner; in none is the separated program the persisted, replayable, replicable unit of state. The contribution here is not to observe separability again but to relocate it — from an enabler of execution specialization to the precondition of a persistence regime — and to show that compilation, caching, dense journaling, and replication-bounded entropy are one precondition's four projections rather than four independent techniques. Prepared statements make the gap concrete: they supply the closest operational analog and still exhibit only two of the four faces, because they persist rows rather than the named operation. Separability is necessary for the four-fold coherence; code-as-data persistence is what carries it past the two faces the prior art already reaches.

8. Conclusion

The argument of this paper can be stated compactly. Program–value separability — the syntactically decidable property that a DSL program declares user parameters rather than embedding values in its body — is the structural precondition under which compilation, caching, and dense journaling become economically meaningful. The four runtime faces traced here are not parallel design choices: they are downstream consequences of separability, paired with code-as-data persistence in the case of dense journaling. The runtime characterized in §3 realizes the principle concretely; the magnitudes reported in §5 confirm the predicted structural amortization across two open-source DDD aggregates — a compiled-versus-interpreted speedup that scales monotonically with DSL-bound work (roughly 1.5× on the eShop purchase verb to 3.1× on a synthetic arithmetic kernel), a cold compile cost (≈380 µs for the eShop purchase verb) that recovers itself within several hundred invocations, and a journal in which ≈99.8% of parametric entries are compact action references producing a footprint several-fold denser than the equivalent literal-script storage. These magnitudes are drawn from two RDBMS-anchored DDD aggregates; whether host domains without that representational pressure would widen the surface-vs-depth gap further is a plausible conjecture, not a result that two data points — or a static call-graph walker blind to polymorphism and delegates — can establish.

The principle takes its place alongside the analysis of the prior paper of this series, *Anti-porous Architecture*, which characterized porosity as a representational sparsity problem and density preservation as the operational consequence of an event-sourced DSL journal. Where the prior paper named the defect that domain representations on tabular substrates accumulate, this paper names the condition under which that defect can be avoided.

Prior paper	This paper
Problem: porosity	Condition for avoiding it
Density preserved	Program–value separability
Executable journal	Stable program identifier
Operations vs state	Functions vs instances

The two papers describe the same phenomenon from opposite sides: density preservation is what is achieved; separability is what makes it achievable. Together, they argue that density in domain representation is not an optimization but a structural property that follows from how programs relate to their values.

Stated as a design space, the two papers occupy a cell that little else does. Two properties are at work, and each is common on its own. *Separability* — values external to the program — is everywhere: every prepared statement, every parameterized query has it. *Persisting the operation* rather than its effects is also well known: event sourcing is built on it. What is rare is their conjunction. SQL is separable but persists rows, not the statement; event-sourced logs persist operations but their events are command-shaped *data*, not separable programs with an identity of their own; ordinary CRUD has neither. A runtime that is *both* separable *and* persists the operation produces what neither tradition has alone: a persistent operation carrying a stable identity independent of its values — an *Action*. Separability (this paper) supplies the identity; code-as-data persistence (Paper 1) makes the identified operation the durable artifact. The Action is precisely what falls out when both hold at once, and it is the corner of the design space — separable *and* operation-persisting — that this series occupies and that the surrounding traditions, each holding one property without the other, do not reach.

The principle is not bound to the runtime characterized in §3. In an Orleans- or Akka-style actor framework with a parameterized command DSL above a host-language domain library, the same precondition would predict the same four-fold coherence; in a service that uses prepared statements over an event-sourced log of named operations, three of the four faces are already available, and the fourth — replication-bounded entropy — follows once the persisted artifact is the operation rather than its row-level effects. The realization in §3 is one example of a construction the principle admits, not the only path to it.

Two extensions of the principle remain to be developed in subsequent work. The fourfold coherence described here does not exhaust the operational consequences of an externalized-parameter, locality-bound substrate: a persistent local write buffer with asynchronous remote replication, for instance, is structurally enabled by the same commitments — the actor's isolation guarantees that locally-buffered, not-yet-replicated entries are invisible to other contexts by construction, not by convention. The persistent buffer and its zero-downtime implications are treated in a companion paper. A second companion paper takes up the reaction mechanism whose semantic-pattern-matching has been mentioned in passing throughout this argument, and the consistency contract under which observable state remains coherent across actors. In each case the same structural observation continues to apply — what makes the journal compact, what makes deep domains compose without friction, is the separation of the program from its values. Porosity makes deep domains expensive to represent; anti-porosity makes them economical to compose.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `b42d0f7` (2026-05-26) — the same public snapshot cited by the prior paper of this series. The empirical measurements of §5 were produced against this commit, built in Release (Appendix A, §5.1). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:177e2d61486bdbdfd2d5e774fcf392a45406e60d;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:b42d0f76b4278c36a45c221cc1453e3ee8ffb3de
```

The core files cited inline (`ActorHandler.cs`, `Program.cs`, `Actor.cs`, `ActorV2.cs`, `ActorV2Invocation.cs`, `DomainLibraries.cs`) are byte-identical between this commit and the prior public snapshot `2f31f96` (2026-05-18); the line numbers below resolve against either.

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers ;path=<path>;lines=<NN> to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Appel, A. W. (1992). *Compiling with continuations*. Cambridge University Press.

Futamura, Y. (1999). Partial evaluation of computation process — an approach to a compiler-compiler. *Higher-Order and Symbolic Computation*, *12*(4), 381–391. https://doi.org/10.1023/A:1010095604496 (Original work published 1971 in *Systems, Computers, Controls*, *2*(5), 45–50.)

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, *30*(3), 611–642.

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*, *1*(2), 141–259.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

International Organization for Standardization. (2023). *ISO/IEC 9075:2023 Information technology — Database languages — SQL*.

Johnsson, T. (1985). Lambda lifting: Transforming programs to recursive equations. In J.-P. Jouannaud (Ed.), *Functional programming languages and computer architecture* (pp. 190–203). Springer-Verlag. (Lecture Notes in Computer Science, Vol. 201)

Jones, N. D., Gomard, C. K., & Sestoft, P. (1993). *Partial evaluation and automatic program generation*. Prentice Hall.

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1. Zenodo. https://doi.org/10.5281/zenodo.20404863

Stroustrup, B. (1994). *The design and evolution of C++*. Addison-Wesley.

Vandevoorde, D., Josuttis, N. M., & Gregor, D. (2017). *C++ templates: The complete guide* (2nd ed.). Addison-Wesley.

Würthinger, T., Wöß, A., Stadler, L., Duboscq, G., Simon, D., & Wimmer, C. (2012). Self-optimizing AST interpreters. In *Proceedings of the 8th Symposium on Dynamic Languages (DLS '12)* (pp. 73–82). ACM. https://doi.org/10.1145/2384577.2384587

Würthinger, T., Wimmer, C., Humer, C., Wöß, A., Stadler, L., Seaton, C., Duboscq, G., Simon, D., & Grimmer, M. (2017). Practical partial evaluation for high-performance dynamic language runtimes. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)* (pp. 662–676). ACM. https://doi.org/10.1145/3062341.3062381

Appendix A — Code references

The references below cite source locations in the Puppeteer codebase as `file:line` pairs against the public commit `b42d0f7` (see *Code provenance*); the cited core files are byte-identical at the prior public snapshot `2f31f96`. The benchmark harness that produced §5 is public: the MSTest cold-compile, eval, journal-density, and dispatch-count benches live in the runtime repository under `tests-local/` at commit `b42d0f7`, while the BenchmarkDotNet speedup harness, the synthetic compile/eval sweeps, and the Roslyn call-graph walkers live in this paper repository's `labs/` directory (`lab01-bdn-speedup/`, `lab05-eshop-roslyn/`, `lab05-grzybek-roslyn/`). Datasets cited in §5 — raw per-iteration CSVs, the BenchmarkDotNet environment reports, and per-lab summaries — are stored in the companion `data/` directory of this paper repository, each stamped with the commit SHA. The measurement environment is recorded in §5.1.

§1.2 — Formal characterization

Reference	Location	What it shows
Runtime self-documentation as `F(x1,...,xn)`	`ActorHandler.cs:1085`	Comment articulating the parametric program model
Parametric/literal regime documented in code	`ActorHandler.cs:1091-1093`	Comment encoding the binary taxonomy: scripts without user parameters are interpreted, uncached, persisted as Script entries; scripts with user parameters are compiled, cached with an ActionId, persisted as Action entries

§3.1 — Compilation pipeline

Reference	Location	What it shows
Orchestration on first encounter	`ActorHandler.cs:1097-1148`	`PrepareCommandProgram` rents a parser, parses the script, decides parametric vs literal path
AST traversal	`Program.cs:182-244`	`ProgramExpression` walks `Statements`, accumulates `Expression` nodes into a `BlockExpression`
Lambda composition	`Program.cs:244`	`Expression.Lambda<Func<Parameters, Output, string>>` wraps the `BlockExpression`
Compilation step	`Program.cs:163`	`_executable = programExpression.Compile()`
Eval parameter cache structure	`Program.cs:305`	Dictionary keyed by parameter name to `(EvalScript, Compiled)` tuple
Eval recompile on script change	`Program.cs:323-331`	Cache lookup, invalidation, re-parse, re-compile

§3.2 — Interpretation retained

Reference	Location	What it shows
`CompilationModePolicy` enum	`Actor.cs:8-13`	Three policy values (`Automatic`, `AlwaysCompiled`, `AlwaysInterpreted`); default is `Automatic`
Policy hint passed at call site	`ActorHandler.cs:1117-1131`	`AdjustCompilationMode(useInterpretedMode, policy)` — `useInterpretedMode: true` for literal path, `false` for parametric
Policy resolution switch	`Program.cs:134-150`	Sets `IsCompiledMode` from policy + hint
Execution dispatch	`ActorHandler.cs:1955-1968`	`Perform` calls `ExecuteExpression` if compiled, `Execute` otherwise
Commands caching rule	`ActorHandler.cs:1117`	Caches only when `parameters.HasUserParameter()` is true
Queries caching rule	`ActorHandler.cs:1405`	Caches when `EMPTY_PARAMETERS` *or* `HasUserParameter()` is true (asymmetry vs commands)

§3.3 — Hot-loaded DSL programs

Reference	Location	What it shows
Domain library loading	`DomainLibraries.cs:77-115`	`GetOrLoad(params Assembly[])` reflects public types and caches them once per deduplicated assembly set; a single-assembly overload remains for the back-compat path
Library binding to actor	`ActorHandler.cs:61`	`libraries = DomainLibraries.GetOrLoad(LibraryAssemblies)`
Fluent script invocation	`ActorV2.cs:32`	`Using(scriptForChk, scriptForCmd)` introduces new scripts at runtime against the cached domain

§4 — Verb richness

Reference	Location	What it shows
Check-then-command fluent dispatch	`ActorV2Invocation.cs:69`	`PerformCheckThenCommand()` — read-lock check + write-lock command + journal write, with rollback on check failure

§5 — Empirical datasets

The datasets that produced the magnitudes reported in §5 are stored in the companion `data/` directory of the paper repository. The production-verb measurements are produced against two independent open-source aggregates:

- **eShop** — `dotnet/eShop` (MIT), `src/Ordering.Domain/AggregatesModel/OrderAggregate/Order.cs`. Multi-item purchase via 10-arg constructor plus four `AddOrderItem` calls plus a four-step state-machine walk to Shipped.
- **Grzybek** — `kgrzybek/modular-monolith-with-ddd` (MIT), `src/Modules/Payments/Domain/SubscriptionPayments/SubscriptionPayment.cs`. Subscription payment lifecycle via `Buy` factory plus `MarkAsPaid`.

The synthetic kernels in §5.2–§5.3 (arithmetic, DSL-rich) carry no external-codebase dependency; their datasets are in `data/paper2-synthetic/`.

Section	Dataset directory	Lab	Host	What it contains
§5.2 (compilation speedup)	`data/lab01-eshop/`	Lab 1	eShop + synthetic	BenchmarkDotNet compiled-vs-interpreted (Release, `DOTNET_TieredCompilation=0`); GitHub-markdown + CSV reports incl. the BDN environment block

Section	Dataset directory	Lab	Host	What it contains
§5.2 (compile cold cost, production verb)	`data/lab02-eshop/`	Lab 2 Tier 3	eShop	Cold compile cost per distinct script variant, N=100, isolated at the `Compile()` call site
§5.2 (compile cost by depth)	`data/paper2-synthetic/`	Lab 2 synthetic	synthetic	Cold compile cost vs statement count (depths 5–100), 50 variants per depth
§5.3 (eval cache hit vs miss)	`data/lab03-eshop/`	Lab 3 Tier C	eShop	Stable vs mutating eval text; per-iteration end-to-end and isolated eval-compile ticks
§5.3 (eval cache by complexity)	`data/paper2-synthetic/`	Lab 3 synthetic	synthetic	Hit vs miss for 2-term and 50-term eval sub-programs
§5.4 (journal density)	`data/lab04-eshop/`	Lab 4	eShop	Action / Script / Define entry counts and payload bytes parsed directly from BinaryEvent-Codec journal_*.bin

Section	Dataset directory	Lab	Host	What it contains
§5.4 (journal density, replication)	`data/lab04-grzybek/`	Lab 4	Grzybek	Same entry-type analysis applied to the SubscriptionPayment lifecycle verb
§5.5 (verb richness)	`data/lab05-eshop/`	Lab 5	eShop	α DSL dispatch counter (runtime) plus β Roslyn forward closure over `Ordering.Domain`
§5.5 (verb richness, replication)	`data/lab05-grzybek/`	Lab 5	Grzybek	$\alpha + \beta$ over `Payments.Domain` plus shared `BuildingBlocks/Domain`

Each dataset directory contains a `headline.md` summary, raw CSVs, and a Git SHA stamping the runtime version against which the lab was run. The Roslyn walker source for the β half of §5.5 lives at `labs/lab05-eshop-roslyn/` and `labs/lab05-grzybek-roslyn/` — both are self-contained .NET 9 console projects that can be re-executed against the public source trees of the respective aggregates.

ewpage

Reactions and the partition

TL;DR

When CQRS introduces eventual consistency, the standard treatment is as a cost: a contract between separated read and write stores that the application must absorb. This paper argues that for actor-native systems the contract is upside-down. The primitive is not eventual consistency at all; it is the developer's pragmatic partition of the work an event implies into *now* (what the verb's contract promises before responding) and *deferred* (placed wherever the developer chose — same thread, another node, the cluster).

Reactions are the artifact through which the partition is exercised — and they are not async events under another name. Each Reaction matches a *trajectory* through the actor's journal, a sequence of one or more stages naming a saga or lifecycle, statically compiled against the domain's libraries with typed identifiers propagated across stages. Reactions serve two purposes simultaneously. The first is **pragmatic deferral** — for latency budget, parallelization, scaling. The second is **categorical segregation** — keeping operational concerns (email, BI, third-party messaging, telemetry) from propagating into domain methods. Reactions are *not* technology-free; they are precisely where that technology lives, **encapsulated** so that other domain verbs do not inherit the dependency, and the encapsulation is structural — the language itself blocks the operational concern from leaking back. The library, in turn, stays free of that propagation, built under its legitimate programming paradigm (currently object orientation). Both purposes share one guardian: **friction**. If Reactions cost the developer more than inline code, the partition is not exercised; verbs degrade and the domain library is contaminated by precipitation. The friction is an architectural responsibility of the framework, not a discipline question for the developer.

Three consequences fall out of the primitive when it is low-friction. The primary one is a **domain library that closes** — that can be declared *task done* because it is structurally free of present and future operational tools. The other two are **fast verbs by construction** and **opt-in eventual consistency** (the developer names each deferred effect by hand, inverting the CQRS-classical assumption). **Eventual consistency is the shadow of the partition, not the design choice.**

Claims this paper makes

This paper makes four core claims. The detailed assertions are folded into them as sub-points; two statements the argument relies on but does not originate are listed afterward as supporting observations.

1. **The now/deferred partition is the primitive — drawn, not deduced, and placed.** Within an event's frame, formal correctness places *everything* in *now*: every domain effect entailed by the verb's contract should, formally, resolve before the event closes. The *now/deferred* split is therefore not a decomposition deduced from correctness but a pragmatic boundary the developer **draws** inside the event's implied work — feature by feature, governed by latency budget, parallelization opportunity, and the categorical-purity argument of claim 2(b), not by the formal contract. The partition also carries a **placement**: at the moment of deferral the developer decides where the deferred work runs, along a proximity/urgency axis (close-to-action ↔ remote-and-deferred). Drawing the cut and placing it are one act, not two.

2. **The artifact that realizes the partition serves a double purpose, structurally enforced, and is declarative trajectory matching — not an event handler.** In the instantiation, the artifact is the Reaction.

   - **(a) Pragmatic deferral** — work the verb cannot afford within its latency budget, or that is better run in parallel, is deferred.
   - **(b) Categorical segregation** — extrinsic operational concerns (email, BI, telemetry, webhooks) land in the artifact, not in domain methods. The segregation is of *propagation*, not of presence: the artifact invokes the technology, encapsulated so other domain verbs do not inherit the dependency. This extends the anti-porosity principle of Paper 1 to a fourth layer — the operational edge — and the encapsulation is *structurally* enforced, not conventional: a Reaction's emit path runs as a query (`PerformEmit`, `isQuery:true`, blocking `expose` and journal writes), so the language refuses to let the operational concern leak back into the actor's state (`ActorHandler.cs:2408-2479`). The legitimate paradigm inside the library (classes, inheritance, invariants, domain methods) is preserved precisely because the encapsulation is non-leaky (§3.1).

- **(c) Mechanism: trajectory matching.** The artifact is a rule the engine applies over the actor's journal — a multi-event *trajectory* of `Seek` stages, statically compiled against the domain's `Libraries` at build time (a stage naming a class the domain does not contain fails to compile), with typed identifiers propagated across stages. The matcher is CEP-style and not claimed as novel (§9.3); what is distinctive is the substrate, the actor's own journaled invocations (`Pattern.cs`, `Reaction.SolveActionReferences()` at `Reaction.cs:1444`, `MatchTree.TryMatchAtLevel()`).
- **(d) Two placement modes, not a slider.** The placement of claim 1 is realized as a choice between `Cue` (push loop on a separate thread, against the actor's live in-memory state) and `Job` (on-demand against the replicated journal, freely placeable), further narrowed by activation scope (`DirectorOnly`/`CastOnly`/`Company`) (`Reaction.cs:20-24`, `Reaction.cs:68-82`).
- **(e) Elision closes the journal-density loop.** When the pattern correlates a trajectory, its constituent events can be marked elided (`Metadata.Elide`, §6.5), so the *skips* of Paper 1 become the natural outcome of a declarative correlation rather than a post-hoc optimization.

3. **Friction is the architectural guardian of the partition.** The artifact must make declaring deferred work no costlier than inlining it, or both purposes collapse together: if deferring costs more lines or cognitive load than inline code, the partition is not exercised — verbs degrade *and* the domain library is contaminated by precipitation (the worse failure, being hard to revert and propagating couplings). Friction is an architectural responsibility of the framework, not a question of developer discipline.

4. **Three consequences follow when the partition is low-friction, ranked.** The primary one is **a domain library that closes** — declarable *task done* by being free of present and future operational tools (§5.1); its operational form is that the domain becomes a single real artifact maintained over years, rather than classes scattered and drifting across services (§5.1). Secondary are **fast verbs by construction** (§5.2) and **opt-in eventual consistency**, in which the developer names each deferred effect by hand, inverting the CQRS opt-out assumption (§5.3). The three are not features or design choices; they fall out of the primitive (claims 1 + 2 + 3) when its exercise is low-friction.

Supporting observations (relied on, not originated here): - *Logical, not physical, CQRS.* Within an actor, reads and writes operate on a single in-memory state, strongly consistent at the actor boundary; the eventual-consistency contract of classical CQRS does not apply there. This is the semantics of a stateful actor (Hewitt, Bishop, & Steiger, 1973; Agha, 1986), recorded as the ground on which §5.3 reframes the consistency vocabulary — not a contribution. *(Verification: `PerformCmd`/`PerformQry` over a shared actor state in §6.)* - *Continua-*

tion across the deployment switch. Deferred work in flight when a node hands off is not lost: a Reaction's cursor is durable and replicated with the journal (`DiaryStorage.GetReactionCheckpoint`), so the successor resumes from the last *confirmed* position — at-least-once, as within a single node (§3.6). The zero-downtime coordination that makes the handoff *seamless* (`Theater.aliveGate`) is a separate companion-paper topic; it governs *when* the successor goes live, not *whether* work survives. (The one common operational exception is a coordinated cut-over of a Kafka producer/consumer pair on a message-format change.)

When NOT to use this approach

The primitive of pragmatic partition is general; the implementation pattern that realizes it has narrower limits. We name three regimes in which the analysis presented here either does not apply, applies only partially, or invites overreach.

A. Domains where every effect must be transactionally atomic with the response

Some domains — core ledgers, certain regulated transaction systems — require that every observable effect of a verb commit in lockstep with the response. This bounds the *deferral*, not the model. The developer still exercises the partition by drawing it at zero deferral (everything in *now*, §2.2): the verb's effects stay on the command path and there is no deferred branch to admit eventual consistency. This is, in fact, where the construct's correctness is *most* verifiable — the operation runs against a single in-memory state and is recorded as one journal entry, so it is atomic and exactly-once by the journal's uniqueness (a journal-internal effect, not the at-least-once external case of §3.6). What the domain forbids is *deferring* an effect to a Reaction whose delivery is at-least-once; such an effect is kept in *now*, or made idempotent at its sink (§3.6). The opt-in framing of claim 4 is then exercised as opt-out-of-nothing.

The standing objection to a journal substrate in these domains is not correctness but *rehydration cost*: reconstructing an aggregate such as a running balance by replaying every operation is O(N). The discipline that answers it is an **in-band state checkpoint** — opening a command with a journaled *observation* of the precondition it depends on. The command reads its own current state into a parameter (an `Eval` over the present value) and records that observation alongside the operation, so the resulting entry is self-contained: it carries both the prior observable state and the effect. Concretely, a counter: a debit command on an account evaluates `account.currentBalance()` as a parameter at the head of the operation — the observation is taken before the debit applies and journaled in evaluated form as part of the same entry — so the entry records both the balance the operation observed and the debit it applied, and no earlier entry is needed to interpret it. (The observation rides the `Eval` parameter path — the precondition the command reads before it operates; it is distinct from `expose`, a script statement the developer places to externalise a computed value into the entry's `ExposeData`, §6.7.) The journaled observation is not state smuggled past the journal — it is a historical fact, as legitimate as the operation that follows, which reads it as context. Because one entry now reconstructs both, predecessor entries can be marked elided (`Metadata.Elide`, §6.5) and rehydration drops toward O(1) per actor — the journal-density mechanism of Paper 1 applied to a high-volume aggregate, trading disk for rehydration time at the extreme of one checkpoint per operation. With this discipline the journal substrate is viable in precisely the atomic, high-throughput domains a bare reading of the partition would exclude — making this the domain where the construct is at once applicable and, by the journal's uniqueness, its correctness verifiable. That answers the question the exclusion would otherwise raise: where correctness matters most, the construct is not absent but at its most checkable.

B. Teams without appetite to think in terms of partition decisions

The primitive shifts a design responsibility onto the developer that some teams do not want. A team that treats Reactions as "fire and forget" — without modeling the contract that follows — loses the very property the primitive was supposed to confer. The framework can lower the friction; it cannot supply the design judgment.

C. Workflows where the deferred branch's failure is structurally inadmissible

A pragmatic partition presupposes that the deferred branch can fail or retry without compromising the verb's already-committed contract. Workflows in which a downstream failure of the deferred work must abort the verb retroactively — sagas with compensating actions are one such pattern, but the right place to discuss them at depth is the companion paper on Choreography. Some such workflows are admissible under the partition; others are not. The honest position is that the partition is not the right primitive for every long-running workflow.

---

1. Introduction: a developer says things

This paper makes an analytic theory contribution. It identifies a structural primitive that frameworks for actor-native systems have left implicit — the developer-controlled partition of an event's implied work into a "now" branch and a "deferred" branch with a placement decision attached — and derives the design principles required to exercise that primitive at scale. The instantiation that demonstrates the construct is realizable is the Reaction primitive of the Puppeteer framework, treated in §6; it is the existence proof of realizability, not the substance of the claim. The genre is the one Gregor (2006) names *theory for analyzing* (Type I): it introduces a construct that lets the phenomenon be described and classified, with empirical evaluation supplementary. The Hevner-style design-science *evaluation* of the artifact (Hevner, March, Park, & Ram, 2004) is deferred to a companion case-study paper; the Type I frame separates construct introduction from artifact assessment cleanly.

A developer is modeling a purchase-order endpoint. They are not thinking about transactions, consistency contracts, or CQRS sides. They are thinking about roles and what each role does next: *"the cashier confirms the purchase and locks the requested items. Then a confirmation goes out, and the rewarder evaluates whether any promotional campaign applies."* They say it at a whiteboard, and a few hours later it is in code — not as a set of architectural decisions translated through a vocabulary, but as the same sentence the developer just spoke. In C# a developer says things — defines classes, writes methods, encodes invariants. What an actor-native runtime adds is that the same act of saying things now extends to the endpoint level: who handles the request, what the response promises, what is handled afterwards.

That extension carries a question the developer has not been asked to answer in industrial practice for some time, and most of this paper is about it. When the developer said *"first this, then that, then the other"*, they drew a partition — between what the verb's response promises *now* and what is *deferred and placed elsewhere*. They drew it without naming it, on pragmatic grounds: latency, parallelization, and the sense that some concerns simply do not belong inside a domain method. The literature has names for the consequences of that partition — domain libraries that close, fast verbs by construction, opt-in eventual consistency — but the developer never reached for those names. They reached for a sentence about who does what, and the runtime was kind enough to let them write the sentence as the program.

The classical actor model — Akka, Orleans, Proto.Actor and their lineage — provides actor isolation and message passing, but it does not provide a declarative, journal-respected mechanism for what the verb commits to as part of its contract. What happens after the verb is, in those frameworks, side effect: a method call into another grain, a `tell` to another actor, a registered timer, a write to a queue. None of those are part of the actor's program in the sense the journal preserves (substrate-level *program* throughout this paper refers to the construct of Paper 2 §1.2: the pair of domain library and journal of invocations). This paper argues that Reactions are that missing mechanism — and that without it, the journal-density property established in Paper 1 and Paper 2 cannot survive contact with operational requirements that span the verb's response.

This paper takes that sentence apart in turn. §2 names the partition the developer drew. §3 examines the artifacts the developer composed to write it: a verb on the actor — a command, a query, or a check-then-command — and the Reactions that handle what the verb deferred. §4 looks at what makes writing a Reaction cheap or expensive, and why the answer matters more than it appears at first. §5 follows what falls out when the writing is cheap: three consequences, ranked. §6 grounds the discussion in code from the public Puppeteer codebase. §7 places the design against Akka and Orleans, §8 addresses the predictable objections, §9 sets the work in the broader literature, and §10 returns to the developer's sentence and observes what changed.

2. The pragmatic partition: now versus deferred

2.1 The first cut is *who*

The first cut the developer makes when modeling is not "what feature is this" — it is *who*. Cashier. Packer. Deliver. Rewarder. Once the actors are named, the work distributes itself: each role naturally absorbs the verbs that belong to it, and the partition between *what I do now* and *what I see go by and react to afterwards* emerges from the modeling, not from a separate engineering decision. The developer never says "this is opt-in eventual consistency"; they say what each role does. That sentence carries both role and feature — the cashier role *and* the priority order, what is settled before responding versus what is handled after — and it is not feature-by-feature; it is role-by-role-with-feature.

2.2 The partition is drawn, not deduced

Formally, the situation is sharper. Every domain effect implied by the event sits, by formal correctness alone, in *now*: nothing in the verb's contract entitles the developer to defer anything. The license to defer is therefore not deduced — it is **drawn**. The developer draws it while modeling, on pragmatic grounds, and the grounds are three. First, a latency budget the verb cannot exceed without breaking the actor's serial-mailbox invariant — the topic of §5.2. Second, an opportunity to parallelize work across roles or across nodes — the placement question of §2.3. Third, a categorical sense that certain concerns (a confirmation going out, an analytics ping, a third-party update) do not belong inside a domain method at all, regardless of timing — the case developed in §3.3. The three grounds are independent of one another, and a single deferral may rest on more than one.

2.3 Placement is part of the same act

At the same act of drawing the now/deferred partition, the developer also decides where the deferred work runs. The axis is not continuous: it has two canonical points named explicitly by the framework. *Cue* runs immediately on a separate thread, against the actor's live in-memory state; *Job* runs on demand against the replicated journal, freely placeable on a follower in another pod or another machine. Section 6.1 develops the two modes in detail. The point relevant here is that partition and placement are one decision, not two — choosing what gets deferred and choosing which of the two structurally distinct modes carries it are the same act, not separable concerns.

This partition — its drawing and its placement, taken as one act — is the primitive of the rest of the paper. Everything that follows is downstream of it. §3 examines the artifacts through which the developer writes the partition (a verb on the actor and the Reactions that handle what the verb deferred), naming the two purposes those Reactions serve. §4 looks at what makes writing them cheap or expensive. §5 follows what falls out when the writing is cheap.

3. The double purpose of Reactions

Before naming what Reactions exclude from the domain library, the paper must name what they do **not** exclude. The domain library is not tool-free. It is built under the legitimate programming paradigm — currently object orientation — that sustains the language of the domain itself. Once that legitimate tool is named (§3.1), the two reasons to defer can be developed without inviting the misreading that "pure" means "tool-free": pragmatic deferral (§3.2) and categorical segregation of operational pollution (§3.3).

3.1 The legitimate tool inside the domain

The rich library is written in a host language under a programming paradigm — currently object orientation. Classes, inheritance, polymorphism, invariants, domain methods, and the composition of conceptual primitives are the *legitimate tool* of the domain. They live inside the library; they are what the library is made of. The categorical segregation developed in §3.3 does not exclude this paradigm — on the contrary, it is what protects it.

The DSL of an actor runtime does not compete with this paradigm. It is not a second implementation of the domain; it is the language by which the domain is invoked. The relation is structurally analogous to that between SQL and a relational engine: SQL does not duplicate the engine's logic; it names operations the engine performs. A short SQL statement can trigger joins, locks, and execution plans far heavier than its surface text. The DSL of an actor runtime carries the same kind of leverage — a few lines invoke domain methods whose implementation lives once in the library and whose behavior may be deep. Two endpoints that share invocation lines are not duplicating knowledge; they are composing distinct calls over the same library, in the sense Hunt and Thomas (1999) named when they defined DRY as the prohibition on duplicate *representation of knowledge*, not on the appearance of similar text. The polymorphism of the domain is a first-class citizen of the DSL (Paper 1 covers the parser and symbol-table treatment of inheritance and interfaces).

A second observation belongs here, anticipating §5.1: the closability argument that follows is about the boundary of the library, not about its exhaustiveness. The library can keep growing in domain concepts under its legitimate paradigm — new classes, new invariants, new relations — and that growth is healthy and expected. Practical incompleteness is not a defect of the closability argument.

What closes is the absorption of *operational* tools that change with the ecosystem; what remains open is the domain's own conceptual development.

3.2 Pragmatic deferral

The first reason a developer defers work is operational. The verb of the actor must complete promptly: an actor sustains its target throughput when its mailbox advances quickly, and a verb that takes too long delays every subsequent verb behind it. The constraint is not that I/O must be absent from the verb — formally permitted, an I/O may sit inside a `PerformCommand` — but that I/O accumulates badly. Many verbs each making a network call, or one verb whose latency is unbounded, both collapse the throughput the actor model is supposed to sustain. The fast verb is structural rather than aspirational, and the developer protects it by deferring.

Three patterns, each at a different scope, sustain the fast verb. *I/O hoisting at the caller* — the caller resolves expensive lookups and passes the resolved values into the `PerformCommand`, so the actor receives data and never fetches. *Saga-phasing between events* — when a workflow genuinely requires interleaved external calls, the actor advances one phase, releases, an external coordinator does the I/O, and the response re-enters as the next event. *Reactions* — work that the verb commits to as part of the contract but does not block on, deferred to a `Cue` or `Job` (§6.1). The three are not alternatives; they compose. What §3.2 names is the third — the case where the deferral is owned by the actor itself, declared as a Reaction.

A practical recommendation belongs to the third pattern. A Reaction's body is fast-verb code, with the same throughput constraint as any verb on the actor, and it is the right home for a side effect (sending a Kafka message, calling an external service): were the same side effect to live inside a `PerformCommand`, it would violate the actor principle that commands compute against in-memory state without external I/O; lifting it into a Reaction keeps the actor's command path pure while keeping the side effect's logic in the domain layer where it belongs. The delivery guarantee that side effect inherits is *at-least-once* — the Reaction's action is confirmed only after it runs (§3.6) — so an external side effect declared here must be idempotent at its sink; a journal-internal effect (`Metadata.Elide`) is the one kind that is exactly-once, by atomic commit (§3.6).

The same discipline governs multi-datacenter replication, but with a distinction the single-datacenter case lets one skip. Under journal replication, what travels between datacenters is the action — its `actionId` and parameters — and each datacenter runs the same domain code, and the same Reaction, against its own locally-replicated journal. So *every datacenter fires the Reaction.* Whether that is symmetry or duplication depends entirely on the effect's target.

When the target is itself **per-datacenter** — a *local* broker whose *local* consumer writes a *local* RDBMS or BI store — N independent firings are exactly the intent: each datacenter maintains its own copy, the topology is symmetric, and a synchronous write to a *shared* external RDBMS (which would make the datacenters contend on every match) is the anti-pattern to avoid. But when the effect has a **single, globally-visible target** — one confirmation email to the customer, one call to a payment processor — N datacenters produce N effects: N emails, N charges. That is not symmetry; it is a duplication bug. Replication does not make such an effect idempotent, and §3.6 does not claim it would. A globally-singular non-idempotent effect must therefore be either made idempotent or deduplicated at its shared target — the per-match idempotency key of §3.6 lets the target collapse the N firings into one — or confined to a single firing, which is what `DirectorOnly` (§6.6) is for: it restricts a Reaction to the primary replica so the effect is not duplicated across the topology (for a globally-singular effect the topology must make that primary unique across datacenters, not per-datacenter). The symmetric-topology argument licenses cross-datacenter replication of an effect only when its target is per-datacenter; it does not license duplicating a user-visible singular effect.

The recommended pattern is therefore to keep slow I/O *outside* the Reaction body altogether: the Reaction emits a message to a broker — Kafka in the canonical case — and a consumer downstream (a BI sink, an analytics writer, a third-party adapter) performs whatever heavy I/O the requirement demands. A typical BI consumer reduces to one insert and several selects against its own store; the actor never sees that load. The framework permits direct RDBMS calls inside a Reaction; the recommendation is not to use them.

3.3 Categorical segregation: encapsulation, not exclusion

Operational concerns demanded by requirements but extrinsic to the domain — email, BI, third-party messaging, telemetry, webhooks — land in Reactions, not in domain methods. The crucial point: the segregation is of *propagation*, not of *presence*. A Reaction can live inside the same library as the domain methods it observes — declared in the same project, next to the classes it cares about. What the framework requires is that the Reaction be its own artifact, with its own scope of effects, rather than collapsed into the body of a `PerformCommand`. The structure is therefore one of co-location without co-mingling. A Reaction whose purpose is to send a confirmation email will, of course, contain a call to an email service; the technology lives there, encapsulated, near the domain methods it serves but distinct from them. What the segregation prevents is that the email concern leak into a domain method that ought to know nothing about email — and that the next domain verb the actor processes inherit a dependency on the email API by virtue of being in the same class. *Reactions are not technology-free; they are the encapsulation that keeps technology from propagating into the domain.*

The function is structurally analogous to an anti-corruption layer in DDD, but architectural rather than stylistic: any complete instantiation treats the deferral artifact as a first-class citizen of the language, and the developer's defaults route operational concerns there. In Puppeteer this artifact is the Reaction.

The legitimate paradigmatic tool inside the library (§3.1) is preserved precisely because the encapsulation is non-leaky.

The non-leak is structural, not stylistic. The Reaction's emit path runs as a query: `PerformEmit` is invoked with `isQuery:true`, which blocks the script from using `expose` or declaring globals, takes a read lock that runs in parallel with queries and other emits, and leaves the journal untouched (§6.5; verification at `ActorHandler.cs:2408-2479`). The technology of the Reaction (a Kafka producer, an HTTP webhook, a BI sink) executes; the journal does not record it as a domain event; the actor's state does not absorb the call. The language refuses to let the operational concern leak back into the actor in the parse phase, before the developer ever has the chance to make a mistake about it.

Section 3.3 connects directly to Paper 1: the anti-porosity transversal, treated there as domain + endpoint + persistence, extends here to a fourth layer — the operational edge of the domain. *Reactions are the anti-porous boundary at the operational edge.*

3.4 Orthogonality of purpose and placement

The two purposes named in §3.2 and §3.3 — pragmatic deferral and categorical segregation — and the two placement modes named in §6.1 — `Cue` and `Job` — are independent decisions. A pragmatic deferral may place close to the action as a `Cue` near-continuation, or far from it as a `Job` running on a journal-only follower; a categorical segregation may do the same. The cells of the resulting two-by-two are all populated in practice: a confirmation email may be a `Cue` (purpose categorical, placement local) or a `Job` on a follower (purpose categorical, placement remote); a heavy projection update may be a `Cue` against the live actor (purpose pragmatic, placement local) or a `Job` distributed across the cluster (purpose pragmatic, placement remote). The developer chooses both dimensions at the moment of declaring the artifact; the two dimensions remain independent in any complete instantiation — there is no "default placement for categorical concerns" or "obligatory mode for pragmatic deferrals." Each declaration states its own combination.

A second observation about the partition belongs here. The partition is not only a *pragmatic* decision (latency, parallelization, categorical separation); it is also a *preservation* mechanism. When the actor replicates across datacenters (§3.2), the journal travels with it and so does the code: each datacenter executes the same Reactions on its own actor against its own locally-replicated journal. Were the verb's consequences side effects bolted onto the actor at runtime — `tell`s, registered timers, callbacks — the replication would either lose them or reproduce them incoherently. Because Reactions are part of the program, declared in the DSL and replicated as code, the actor behaves as its program says, in every datacenter, every time it rehydrates.

3.5 Pattern matching against the actor's trajectory

A Reaction is not an event handler. The framework's primitive is more general — and more difficult to find an analogue for in adjacent runtimes. A Reaction is a rule the engine applies over the actor's journal: a sequence of one or more `Seek` stages that, taken together, describe a *trajectory* through the actor's history. The pattern matches not when one event arrives, but when the actor has, over its lifetime, gone through the stages the rule names. Identifiers captured in one stage propagate to the next, correlating the events: an `order` named in the first `Seek` is the same `order` that must appear in the next, and in the next. The Reaction is part of the actor's program — declared in the DSL, parsed at build time against the domain's libraries, replicated as code together with the journal — not a side effect attached to the actor at runtime.

Three properties are jointly characteristic. *Multi-event*: the smallest match is a saga, not a single message. *Statically compiled*: the pattern is parsed against the domain's `Libraries` in build phase — `[_:Customer].InitiateOrder(order:Order)` resolves to the class `Customer` and to the signature of `InitiateOrder`; if either does not exist in the domain, the Reaction fails to construct. *Trajectorial, not payload-based*: what the matcher inspects is the sequence of *invocations* the actor recorded — its conduct — not the body of any one message. These three properties separate the mechanism from event handlers in Akka, grain reminders in Orleans, and topic consumers in Kafka — all of which fire on a single message rather than a trajectory. They do *not* separate it from complex event processing: correlated multi-event matching with bound, propagated variables is CEP's defining capability, and typed, host-integrated CEP engines already compile patterns against application types (§9.3). The Reaction matcher is best read as CEP over a particular substrate, not as a new matching primitive. The substrate is what differs: the matched events are the actor's own journaled *invocations* — the executable journal of Paper 1 and Paper 2 — so the pattern, the journal, and the domain share one type universe and one persistence substrate rather than a stream fed to an external engine. The contribution this paper claims is the now/deferred partition (§1), not the matcher that serves it.

This is the property that licenses everything else. Without trajectorial matching, *"the cashier's verb is fast and a Reaction handles the rewarder afterwards"* reduces to a callback. With it, what the developer has written is *"when, in the cashier's history, an order was initiated, then confirmed, then paid, react"* — a declaration about how the actor's lifecycle unfolds, not about what to do when a single event arrives. The matching machinery is examined in §6.2; a worked example with the order lifecycle is in §6.3.

3.6 Reactions are journal-indexed, not event-driven

A Reaction is not an event handler. It is a journal-indexed program cursor, and its guarantees are two semantics that must not be conflated: the cursor's *advance* over the journal is exactly-once, while the external *effect* an action performs is at-least-once. Both follow from how the cursor is committed.

Failure model. Processes are crash-stop. The journal and each Reaction's cursor are durable; the cursor — one position per `Seek` of the pattern — is part of the actor's persistent state, advances only forward, and is validated on load so that a non-monotonic checkpoint is refused. Recovery is by replay from the last *confirmed* cursor position. Nothing below assumes a distributed transaction across the journal and any external system; the runtime does not provide one.

The cursor's advance — exactly-once, never silently dropped. The cursor is monotonic and validated on load (a non-monotonic checkpoint is refused, *Failure model* above), so each confirmed position is reached once: a replay re-confirms the same position rather than minting a new one. The cursor advances to *confirmed* only after the Reaction's action has run; if the action throws, or the process fails before the confirmed checkpoint is written, the position is not confirmed and the entry is replayed on recovery (`MatchTree.cs:1565-1575`). The construct therefore never silently skips a match — a failure to execute surfaces as a gap that triggers retry, not a lost effect. The result is *at-least-once execution riding on an exactly-once cursor*: the cursor never double-counts, but the action it gates may run more than once. That at-least-once execution rests on the cursor's monotonicity, not on deduplication or idempotency written inside the Reaction body; those address the *external* edge, below.

What the cursor does not by itself guarantee — no duplicate external effects. Because the action runs before its checkpoint is confirmed, a crash between an external effect (a Kafka send, an HTTP call) and the confirmed-checkpoint write re-runs the action on recovery — the classic dual-write exposure. The runtime binds no external effect to the cursor advance, so end-to-end exactly-once for such an effect is a property of the *sink*, not of the Reaction: it holds when the downstream consumer deduplicates (an idempotency key, an upsert keyed by the entry id or `actionId`) or when the effect is naturally idempotent. The broker-mediated, locally-symmetric topology of §3.2 narrows the exposure — duplicates stay local and dedup is a local concern — but does not remove the requirement.

Where exactly-once does hold structurally. One class of effect is exactly-once by construction: a journal-internal mutation committed atomically with the cursor. `Metadata.Elide` (§6.5) marks the matched entries elided and advances the checkpoint in a single store commit (`MarkEventsAsElidedWithCheckpoint`, `MatchTree.cs:1512`) — effect and cursor are one atomic write, so neither a duplicate nor a gap is possible. Exactly-once is available precisely where the effect and the cursor share one transactional store; for effects outside it, the honest guarantee is at-least-once plus an idempotent sink. A measurement bears out the steady state — a `Cue` activated across several hundred matches fired once per match with no duplicate (§6.9) — but that run induced no crash, so it exercises the common path, not the crash-window duplicate this section names.

The same discipline underwrites cross-actor causation. A `Causation.Continue` (§6.5) records the send in the sender's journal under this same cursor, so the *recording* of the causation inherits the at-least-once-never-dropped guarantee above; the delivery semantics of the send itself are the transport's, and reach exactly-once on the same terms as any external effect — an idempotent receiver. The journal is the boundary of causation because the causation is recorded as program, not because the transport is transactional.

4. Friction as the architectural guardian

The two purposes of §3 share a guardian. If Reactions cost the developer more lines or more cognitive load than inline code, the partition is not exercised. Both purposes collapse simultaneously: verbs degrade *and* the domain library is contaminated by precipitation.

> *Reactions must be at least as ergonomic as inline code. If deferring costs the developer more than not deferring, the partition isn't exercised — verbs degrade AND the domain library is contaminated by precipitation. The contamination is the worse failure: it is hard to revert and propagates couplings. The friction is an architectural responsibility of the framework, not a failure of the developer's discipline.*

The contamination is the worse of the two failures, for two reasons. First, a slow verb is observable and gets fixed: monitoring catches it, profiling diagnoses it, the team rewrites the slow verb on a known schedule. A polluted domain method looks correct and does not — its tests pass, its code reads cleanly, the email it sends is the right email. The dependency on the email API is invisible from the outside, hidden behind the method's signature, until the day the developer needs to revert it. Second, the contamination propagates. Once `sendEmail(...)` lives inside a domain method, every caller of that method inherits the coupling — directly, by transitive call; indirectly, by sharing a class with that method and inheriting its dependencies in test setups, in mocking frameworks, in the surface area of integration. Reverting the design later requires touching every call site, every test, every place the class has appeared as an object of composition.

The architectural responsibility is therefore not to teach the developer discipline. It is to make Reactions, by construction, no more expensive than inline code. The framework's contribution is not the partition itself — the developer always *could* have deferred, in any actor framework. The contribution is friction reduction along two dimensions. The first is language-level cost: declaring a Reaction is a few lines of fluent DSL, comparable to or shorter than an inline implementation of the same effect (§6.1, §6.5). The second is textual proximity: Reactions can be written next to the endpoint they observe — in the same file, a few lines below the `PerformCommand` they continue — so that the developer reading the endpoint sees, on the same screen, what else happens around it:

this confirmation goes out, that loyalty rule fires, this analytics counter is updated. The Reaction's location is itself a documentation aid. When the cost of doing the right thing exceeds the cost of doing the wrong thing, no amount of design review will hold; when the costs are equal or inverted — and when the right thing is also the more legible thing — the partition exercises itself. The framework that takes this seriously is the framework that gets the partition for free.

5. Three consequences (ranked)

The three consequences below fall out of the primitive (§3) when its exercise is low-friction (§4). They are ranked: closability is primary (§5.1); fast verbs by construction (§5.2) and opt-in eventual consistency (§5.3) are secondary. The last of the three is where the paper's central inversion becomes concrete: eventual consistency is not an architectural commitment the system makes up front but the shadow the partition casts — a consequence of a modeling decision, named one deferral at a time (§5.3).

5.1 A domain that closes

The primary consequence of the partition, when its exercise is low-friction, is a domain library that closes. The library is not just kept clean of operational pollution at any one moment; it is structurally protected against the absorption of operational tools that arrive in the future. Email today, Slack tomorrow, a webhook protocol the year after — none of these find their way into a domain method, because the path of entry has been removed at the language level (§3.3). The library that the team is writing now will still be the library the team is reading in three years, and the conceptual primitives it contains will still mean what they meant when they were declared.

> *At some point a domain must be declarable as "task done". A library whose definition must absorb every present and future operational tool — email today, Slack tomorrow, a webhook protocol the year after — is a library that never closes. The only way it closes is if it stays pure: free of operational concerns by construction. Reactions are the artifact that makes that closure possible.*

Two clarifications matter. First, what closes is the boundary, not the conceptual growth of the domain itself. The library can keep absorbing new domain concepts under its legitimate paradigm — new classes, new invariants, new relations — and that growth is healthy and expected. Practical incompleteness is not a defect of the closability argument:

> *If Puppeteer is not exhaustive, that is a practical limitation, not a claim that the domain must be seen as complete.*

Second, "task done" is an organisational sentence, not a software-engineering one. A team that can declare its domain library complete with respect to operational tools can build everything else — refactor, onboarding, upgrade, extension — on a stable foundation. In most enterprises today, "the domain" does not exist as a library at all. It exists as classes scattered across each csproj, copied between services, drifting independently over years until "the customer model" or "the order model" has become a dozen incompatible structures across the codebase. The team speaks of these as if they were one thing; in operational reality they are not. A library that closes its boundary to operational tools while remaining open to conceptual growth is the structural condition under which a single domain artifact, maintained as one over years, becomes economically possible at all.

The closability argument extends the anti-porosity transversal of Paper 1 to a fourth layer. Domain libraries shaped for the operations they describe and the conceptual primitives they admit, not for the operational tools requirements happen to demand this year, are libraries that the team can put down and pick back up without forgetting what they meant.

5.2 Fast verbs by construction

Fast verbs are a construction, not a postulate, and the construction is layered. The actor's own command-path overhead is small enough to vanish into the domain work it carries: a minimal in-memory verb — one field write, no host work — dispatches and persists in ≈0.4 µs on commodity hardware (BenchmarkDotNet, Release, tiered compilation disabled; environment and method in §6.9), so a verb's latency is governed by the domain methods it invokes, not by the runtime around them. The actor's verbs are fast first because the actor's state lives in memory: `PerformCmd` and `PerformQry` operate on it without touching disk or external stores. This is the logical-CQRS observation of this paper — and the broader argument of Paper 1. They are fast also because the domain methods invoked from a verb are themselves rich but local: a single verb can dispatch many calls into the host language and across many levels of the domain library, each operation cheap because each operates on the same in-memory state (Paper 2, in the empirical measurements of the purchase example). The Reaction's contribution is third in this layering, not first: it preserves the speed of the verb against the requirements that would otherwise force I/O into it.

The structural mechanism is, in any instantiation that exercises the partition, that the deferral artifact runs outside the verb's write semaphore. In Puppeteer, a Reaction does not run inside the write semaphore of `PerformCommand`; it runs after the command has persisted, on a separate thread (in the case of `Cue`) or on demand against the journal (in the case of `Job`). The verb returns to the caller without waiting for any deferred step to begin. The actor's serial-mailbox invariant — which would otherwise impose a throughput cap whenever a verb does anything slow — becomes a throughput floor: peak throughput is what verbs are *able* to achieve once the slow work has been hoisted out by I/O hoisting at the caller, by saga-phasing between events, or by Reactions on the deferred branch.

What §5.2 names, then, is the third layer of the explanation. The first two — in-memory state, rich local methods — are already present in any actor framework that respects them. What this paper adds is the condition under which they survive contact with operational requirements: a Reaction primitive cheap enough to absorb every effect that would otherwise be tempted to live inside the verb. Section 6 develops the mechanism in code references; the structural commitment is summarised in `Reaction.cs:1113-1255` and `ActorHandler.cs:2408-2479`.

5.3 Opt-in eventual consistency

The third consequence honors the forward-reference from Paper 1 §6: the eventual consistency that classical CQRS treats as a defining commitment becomes, in actor-native systems, a shadow of the partition rather than its design choice.

Two honesties frame what follows. First, the in-actor half is not new: that a stateful actor reads and writes a single in-memory state, strongly consistent within its own boundary, is the semantics of the actor model itself (Hewitt, Bishop, and Steiger, 1973; Agha, 1986) — the logical-CQRS observation *records* it, it does not discover it. Second, what this section offers is therefore not a mechanism but a *reframing* — an analytic lens, in the Gregor (2006) Type I sense of §1, on where eventual consistency comes from and how it is bounded. The claim is not that opt-in eventual consistency is a new capability (any stateful actor has consistent local reads); it is that, once the deferred branch is named per-Reaction, the eventual-consistency contract stops being a system-wide property the application must absorb and becomes a per-verb list the developer wrote by hand. That is a change in how the consistency surface is *described and reasoned about*, not in what the runtime mechanically does — which is exactly the kind of contribution a theory-for-analyzing makes.

The classical model is opt-out. Read and write stores are physically separated; their synchronisation is asynchronous by default; the application accepts the contract, including its observability gap, as the price of the architecture. Greg Young's *CQRS Documents* make the trade-off explicit: stronger contracts are recoverable, but only by application-level effort layered over the framework's defaults.

The actor-native model is opt-in. The actor itself is the read model — `PerformCmd` and `PerformQry` operate on a single in-memory state (the logical-CQRS observation) — so within the actor's boundary, reads and writes are strongly consistent without any ceremony. The eventual-consistency contract appears only at the deferred branch: when the developer declares a Reaction, what happens after the verb's response is named explicitly. That act of naming is what the contract is. Each deferred effect is documented at the point of deferral, with a name and a placement, and the system makes no further claim about its observability than what the developer has stated.

This inversion is *structural*, and the structural claim is stronger than legibility. Within the actor, reads and writes are strongly consistent (the logical-CQRS observation); eventual consistency arises *only* at a deferred branch, and every deferred branch is a declared Reaction. So the application-level eventual-consistency surface is **co-extensive with the declared deferral set** — it is exactly the Reactions, no more and no less — and the runtime introduces no application-level eventual consistency outside it. That is a closure property of the program, not a documentation convenience: a reader does not merely *find* the windows more easily; the windows *are* the deferrals, enumerable from the source by construction. (The cross-replica dimension — the journal's asynchronous replication across nodes, §3.2 — is a separate consistency parameter, equally explicit as a topology choice, not eventual consistency smuggled in elsewhere.) A classical CQRS system has no such closure: its eventual consistency is a global property of the read/write separation, located in topology, synchronisation policy, and lag — not bounded by any enumerable set in the program. What this paper does *not* claim is a human-factors result (that enumerability lowers defect or comprehension rates — an empirical question for the deferred case-study, §6.10) nor a mechanised proof of the closure (a formalisation in operational semantics is future work); the claim is the structural invariant itself, which is what a theory-for-analyzing contributes.

It is worth saying plainly what naming the partition as the primitive buys, given that this paper concedes the matcher is CEP (§9.3) and logical CQRS the classical pattern read at a different level (§9.1). The partition is not the *outbox pattern*, which is a delivery mechanism for a single deferred effect (record the intent locally, relay it) — a tool one uses *inside* a deferred branch, not the decision of what to defer. It is not *sagas*, which coordinate a multi-step workflow with compensation across events — a different concern, bounded in *When NOT to use* C. And it is not the folklore "defer side-effects", which is advice with no consequences attached to it. The analytic capacity the construct adds is that *one* design act — the developer's now/deferred cut, with placement — is the common upstream cause of three properties the literature treats separately: a domain library that closes (§5.1), fast verbs (§5.2), and opt-in eventual consistency (this section). Naming it as the primitive is what licenses two moves nothing on that list licenses: reasoning that a single lever, friction (§4), governs all three at once; and re-attributing eventual consistency as *derived from the cut* rather than *imposed by the architecture* — the cause-and-effect order classical CQRS assumes, inverted. For a theory-for-analyzing (Gregor Type I) that unification and re-attribution is the contribution. A programming-languages or systems treatment would press further — an operational semantics in which the closure property above is a theorem and the three consequences are derived formally — and that is work a companion paper, not this one, would carry.

6. Implementation in the Puppeteer framework

This section grounds the discussion in code from the public Puppeteer codebase; the path-and-line citations throughout resolve against the commit recorded under Code provenance.

6.1 The two modes: Cue and Job

The `ReactionMode` enum exposes two values, `Cue` and `Job`, declared in `Reaction.cs:20-24`. The two are not points on a continuum chosen by the developer at runtime; they are structurally distinct execution modes, picked when the Reaction is defined and binding the rest of its semantics.

`Cue()` is the live-actor mode. The framework registers the Reaction via `actorHandler.AddRecordWrittenCallback(...)` after an initial catch-up batch (`Reaction.cs:863`); from that point on, every record written to the journal wakes the Reaction's push loop with the new entry, and the catch-up poll itself waits on that same signal rather than on a fixed sleep. Because the Reaction shares the actor's in-memory state with the director — the same rehydration, no separate automaton — its access is inexpensive while the actor is hot. End to end, from a command being issued to the Reaction's action firing, latency was measured at a median of ≈1.3 ms (p99 ≈2 ms) — the signal-driven path, not a polling backoff (§6.9). `ReadForward()` is mandatory: a `Cue` cannot read backward, since its purpose is to react to the actor's ongoing trajectory, not to its past.

`Job()` is the journal-driven mode. There is no push loop. The Reaction runs only when an external scheduler or a call to `Reactions.Execute()` triggers it. It reads the journal directly rather than the actor's memory, which makes the Reaction *portable*: any process with read access to the replicated journal can host the work. The case enabling specialised workers — *"this pod runs the followers for campaigns; that pod runs the heavier analytics followers; another pod activates only periodically to handle older events"* — is built on this primitive. Both `ReadForward()` and `ReadBackward()` are admissible.

`Job` does not rehydrate the actor. When the Reaction needs domain-computed data, the canonical mechanism is `expose` (§6.7) — the command pre-writes the data to the journal alongside the action, and the Reaction reads it during its own rehydration without ever waking the actor. This keeps `Job` cheap to host on a journal-only follower; reaching into the live actor would defeat the cost model that justifies the mode in the first place. The non-rehydration boundary is therefore not a missing feature but a structural commitment that preserves the distinction between the two modes.

The two modes correspond to two structurally distinct points on the proximity/urgency axis introduced in claim 2(d). A comparative table:

Aspect	`Cue`	`Job`
State source	Actor's live memory (same process)	Rehydration from journal
Lifts a separate automaton	No	Yes (follower hydrates)
Trigger	`OnRecordWritten` push callback	Explicit invocation
Latency	Sub-second	Batch / on-demand
Distributable	No	Yes
Direction	`ReadForward` only	Forward or backward

The choice between them is an architectural decision, not a tuning knob. A Reaction modeled as `Cue` cannot transparently become a `Job`: the two assume different things about what is in memory, who hosts the work, and whether the actor must be live for the Reaction to make progress. This is the §2.3 axis materialised — partition and placement are one act because, at the moment of partitioning, the developer is also choosing which of these two modes will carry the deferred work. Both modes operate as journal-indexed cursors per §3.6: the choice between them governs *who walks the journal and when*, not whether the cursor exists.

6.2 Static pattern matching against the domain libraries

The Reaction's binding to the domain is established at build time, not at runtime. Three layers of validation make this concrete.

The first layer is static parsing in `Pattern.cs`. Every reference of the form `[_:Class]`, `[var:Class]`, or `Class(param:Type, ...)` is resolved against the actor's `Libraries`, case-insensitively. If the class or the constructor signature does not exist in the domain library, the Reaction fails to construct — there is no runtime path that can recover from a missing type. The Libraries here are the same domain types that Paper 1 named as the legitimate tool inside the library and that Paper 2 treated as the substrate against which DSL programs name their operations; the Reaction is one more language layer that compiles against that substrate.

The second layer runs per event during rehydration. `Reaction.SolveActionReferences()` (`Reaction.cs:1444`) parses the script of each `ActionEventData` exactly once and caches it under an LRU of 100 entries; identifiers that the pattern captured are marked `IsParameter=true` so subsequent stages can reuse them without re-parsing. The cache amortises the cost of parsing the same script many times across many events; the LRU bound prevents memory growth in workloads with high script diversity.

The third layer is the runtime walk. `MatchTree.TryMatchAtLevel()` traverses the tree of stages by BFS (`WithSharedHydration`, where captures from earlier stages remain visible to later ones) or by DFS (`WithIndependentHydration`, where each branch carries its own hydration context). The choice between the two is part of the Reaction's definition and shapes how identifier propagation behaves across `Seek`, `ThenSeek`, and `ThenFinalSeek` stages.

The parser, importantly, does not execute the script's expressions. It only knows the types. Matches over non-literal values are therefore by structure and type, not by value equality — with two exceptions: literals (a constant in the pattern) and `Id` parameters with `IsParameter==true` (the captured identifiers of §6.3). The composition rules round out the layer: a Reaction has at least one `Seek` and any number of `ThenSeek` stages, optionally terminated by a `ThenFinalSeek`; each `Seek` may carry several `OnMatch` clauses, all of which must match against the same underlying script. The build-time validation stops a Reaction from being defined against a domain it does not fit; the runtime walk only ever evaluates patterns the Libraries have already approved.

6.3 Multi-event patterns: trajectory, not event

The simplest non-trivial Reaction names three stages of an order's life and correlates them by both the order instance and a primitive identifier:

```
actor.Reactions
    .DefineReaction("OrderCompleted")
    .Job().DirectorOnly().ReadForward().WithSharedHydration()
    .Seek("Initiate")
        .OnMatch("[_:Customer].InitiateOrder(order:Order, number:int)")
    .ThenSeek("Confirm")
        .OnMatch("order.Confirm(number)")
    .ThenFinalSeek("Pay")
        .OnMatch("order.Pay(number, [_:PaymentMethod])")
    .Program.Emit("...");
```

Read aloud, the pattern says: *"when, in the actor's history, a customer initiated an order with a given number N, and that same order with that same number N was later confirmed and paid — react."*

Five details of this small example carry the weight of the primitive.

Identifier propagation. The captured `order` named at `Seek("Initiate")` is the same `order` *instance* that must appear at `ThenSeek("Confirm")` and at `ThenFinalSeek("Pay")`; the captured `number` is the same primitive *value*. The matcher correlates by typed identity for object captures and by literal equality for primitive captures — both kinds of identifiers, once captured, are treated as fixed for the remainder of the trajectory. A `Confirm` invoked with a different number, or for a different order instance, would not advance the trajectory; the trajectory is locked to *this* order with *this* number once the first stage matches.

Non-contiguity. Between `Confirm` and `Pay` the actor may have processed any number of unrelated commands; the matcher steps over them. The trajectory is a path through the journal, not a contiguous window.

One match per stage. Each `Seek` and `ThenSeek` matches exactly once by default. For one-to-many correlations — an order with several items being added before payment — the stage is marked `Many()`, valid at any position in the pattern. The surrounding language and its limits are documented in §6.4.

Pattern over conduct. What the matcher inspects is the sequence of invocations the actor recorded — its conduct as a sequence of `PerformCommand` calls — not the body of any one message. The Reaction sees what the actor *did*, not what arrived in its mailbox.

Static binding. `Customer`, `Order`, `PaymentMethod` are types from the actor's domain library, validated at build time against `Libraries`; the symbol table of the actor is available to the matcher, so polymorphism and type relations are respected without further configuration.

Three properties together — *trajectory*, *static binding*, *typed propagation* — set this apart from event handlers and grain reminders, which fire on a single message. Against complex event processing they do not set it apart but align with it (§3.5, §9.3): correlated multi-event matching is CEP's, and what differs is the substrate, not the matching power. The third property is what makes the example above readable as a saga rather than as three loosely-coupled callbacks.

6.4 Filters, time, quantifiers

The matcher accepts three families of refinement on top of the base pattern.

Where clauses. `Where(expression)` (`ReactionEngine.cs:71-83`) chains a boolean condition onto a `Seek`. Inside the expression the developer has access to `@Now` (the event's timestamp, non-nullable), `@EntryId` (the journal entry's identifier, non-nullable), the identifiers captured by the pattern, and the global `time` instance of the framework's `Temporal` type. `time.Days(n)`, `time.Hours(n)`, `time.Minutes(n)` and so on (`Reaction.cs:810`) construct `TimeSpan` values for relative windows. The framework rejects always-false comparisons at build time: comparing `@Now` or `@EntryId` against `null` raises a `LanguageException`, since their types do not admit a null sentinel.

Quantification. The base pattern matches each `Seek` exactly once. A stage marked `Many()` (`ReactionEngine.cs:146`) is the existential modifier for one-to-many correlations: it collapses multiplicity — firing on the first match and tolerating the rest on the same trajectory — and, unlike a positional repeat, it is valid at any stage, not only the first. Two refinements compose with it: `.AtLeast(n)` (`ReactionEngine.cs:166`), a structural quantifier requiring at least n matches before the stage is satisfied, and `.Accumulate(name)` (`ReactionEngine.cs:157`), which hands the raw accumulated set to the next stage under a name — a capture for the domain to compute over, not a computation the matcher performs. Windows bound a stage relative to its parent match: `.Within(entries)` and `.Within(TimeSpan)` (`ReactionEngine.cs:194-220`) close when an event arrives outside the range — no timer, no eviction; the journal closes the window naturally — and `.Aged(TimeSpan)` (`ReactionEngine.cs:225`) is a firing gate that withholds a match until its closing event has settled that span behind the front of the journal. The exact-family quantifiers `None()`, `One()`, and `Exactly(n)` (`ReactionEngine.cs:248-265`) fire when a window closes with the expected count and therefore require a `.Within(...)` — in an open journal there is no other closing point. What the matcher deliberately does *not* admit is any operator that transforms or aggregates the matched data before delivering it: no `GroupBy`, no `Distinct`, no `Sum` or average. That is domain computation and lives in the domain library, not the matcher (§3.3, §6.8) — the matcher correlates a trajectory and hands the raw set over via `Accumulate`. `Within` and `Aged` are not valid on the first `Seek`, which has no parent match to anchor to.

Lifecycle. `SetExpirationDate(DateTime)` (`Reaction.cs:944`) shuts down the Reaction wholesale after a date; `IsExpired` (`Reaction.cs:952`) returns `true` once `DateTime.UtcNow` exceeds the expiration. `SetActive(bool)` (`Reaction.cs:938`) is the manual override. These two differ in scope from `Within(timeout)`: `Within` closes a single window inside a `Many()` stage; `SetExpirationDate` and `SetActive` toggle the entire Reaction. The two scopes are useful for different problems — `Within` lives in the pattern; `SetExpirationDate` lives outside it.

6.5 The three action planes

A Reaction takes exactly one action when its pattern matches. That action is expressed through one of three named *planes*, each describing a different surface of the system the matched trajectory is permitted to affect.

Plane	What the verb touches	What it cannot touch
`Program`	the actor's domain libraries, read-only	the journal, the actor's state, any global
`Causation`	the actor's own journal (a journaled cross-actor send)	the domain libraries or the metadata plane
`Metadata`	the journal's elision register	the domain state or any external effect

A Reaction is therefore not an unbounded scripting surface attached to a pattern. It is a constrained declaration of which surface of the system the matched trajectory is permitted to affect. The builder enforces that only one plane can be configured per Reaction; configuring a second plane after one has already been set is a construction error. The *exactly-one-action* rule is structural, not conventional — the developer cannot accidentally route a single match into two different planes because the surface refuses the second configuration call at build time.

Program — read-only execution against the domain libraries

The `Program` plane executes a script under the same regime as a query. The parser runs in query mode, the script is forbidden from declaring globals, the `expose` keyword is rejected, the journal is not written, and the runtime takes a read lock that runs in parallel with other queries and emits.

The script may invoke arbitrary external technology — an HTTP client, a Kafka producer, a SignalR hub, a BI sink. What the language denies is any path by which those calls could feed back into the actor's state. This is not a discipline rule kept by convention; it is rejected at parse time, before the developer has the chance to write the leak. A Reaction may observe the actor and affect the outside world, but it cannot feed that effect back into the actor.

The optional `when:` guard performs a second read-only check at firing time:

- the pattern asserts that *this trajectory occurred in history*,
- the guard asserts that *it still makes sense to react now*.

The distinction matters under replay, catch-up after downtime, and replica rehydration — the three situations in which the temporal distance between history and the present can be arbitrary. The pattern is bound to the trajectory; the guard is bound to the clock.

A worked example. Consider a Reaction that, after a confirmed purchase, pushes a real-time toast notification to the customer's UI:

```
seller.Reactions.DefineReaction("NotifyPurchaseToast")
    .Cue().Company().ReadForward()
    .Seek("Purchase")
        .OnMatch("[s:Seller].purchase($orderId, $date, $amount,
 ↪  $customer)")
    .Program.Emit(
        "notifier.Push(@customer, @orderId);",
        when: "check (notifier.IsFresh(@date)) WARNING 'toast stale,
 ↪  drop';"
    );
```

The domain verb `Seller.purchase(...)` carries no reference to the notification hub, to UI presence, to toast timing, or to any operational concern of the notification layer. All of that lives inside the Reaction: the call to `notifier.Push(...)` invokes the extrinsic technology, and the guard decides at firing time whether the toast still has an audience. Under the steady path the push fires within seconds of the journal entry; under catch-up or replay, the same match arrives at the matcher hours or days later, the guard returns false, and the emit is suppressed. The freshness policy is plain code in a library class — `ToastNotifier`

— that the actor sees through its `Libraries` but that the domain verb never references. The segregation is guaranteed because there is no syntactic route by which this Reaction could import the notifier back into `Seller.purchase`.

Causation — journaled cross-actor continuation

The `Causation` plane is the journaled cross-actor send. A Reaction on this plane writes exactly one sentence into the sender's own journal instructing another actor; the sender records the causation, the receiver processes the message independently through its own endpoint. Cross-actor coordination therefore appears in the program as a domain fact rather than as infrastructure — neither a saga step nor a tracing span nor a choreography event, but a sentence in the actor's history.

Its full development — the structural conditions under which it preserves cross-actor program continuity, and the comparison against sagas, choreography, and distributed tracing — is out of scope here. In the present paper it is named only as the second of the three surfaces a Reaction may touch.

Metadata — declaring a trajectory closed

The `Metadata` plane marks the events covered by the pattern as elided. After a trajectory has been correlated, its component events no longer participate in any future decision; subsequent rehydrations walk past them.

This is the conceptual bridge to the *skips* introduced in Paper 1. What appeared there as a journal-density optimisation is here the natural outcome of declarative correlation. The developer is not cleaning the journal after the fact. The developer is declaring *"this trajectory is closed; its component events no longer decide anything."* The skip is the consequence of that declaration, not its cause.

Parameter injection

`WithParameters(...)` allows captures from the pattern matching to be modified or augmented before the action runs, regardless of which plane is configured. The parameters are pre-populated with the pattern's captures, so the idiomatic use is to enrich them rather than to construct them from scratch.

Why the planes matter

Without named planes, a Reaction would be an unbounded script runner attached to history — a feature whose semantics would depend on which calls the developer happened to write. With named planes, a Reaction becomes a one-sentence statement about which layer of the system is allowed to change when a historical pattern is recognised. The same primitive that detects the trajectory also declares the surface of effect, and the surface of effect is selected from a closed set of three rather than from the open set of all calls a script could make.

That restriction is what makes the segregation between

- domain verbs,
- cross-actor causation,
- operational side-effects, and
- journal maintenance

reliable across teams and across time. The reliability is not a property of the developer's discipline. It is a property of the surface the language permits the developer to address.

6.6 Activation: where the Reaction runs

After `Cue()` or `Job()`, the developer chooses an activation scope (`Reaction.cs:68-82`). `DirectorOnly()` runs the Reaction only on the primary replica — appropriate when the work must be unique across the topology: orchestrations that should not be duplicated, and the **globally-singular non-idempotent effects of §3.2** (a confirmation email, a charge to a payment processor) that would otherwise fire once per datacenter. Within a cluster, `DirectorOnly` collapses those N firings to one; across datacenters it does so only if the topology elects a *single* primary rather than one per datacenter (§3.2). It is the activation-scope answer to the duplication hazard §3.2 names — complementary to deduplicating at the sink by the per-match idempotency key (§3.6). `CastOnly()` runs only on followers (secondary replicas), enabling the specialised-workers case named in §6.1: followers that hydrate against the journal alone, hosting `Job` Reactions that the director does not need to evaluate. `Company()` runs on the director and on all followers; appropriate when each replica should hold its own evaluation of the trajectory, perhaps for local cache invalidation or for redundant emit paths.

Activation composes with the StageManager — the topology director of the Choreography module. The StageManager decides who is director and who is follower at any moment; the Reaction inherits that identity to decide whether to fire. A Reaction defined `CastOnly` becomes silent on the director and active on whatever process the StageManager has elected as a follower; a director-to-follower handoff therefore changes the population of running Reactions without any further action by the developer. The deployment-time mechanism that keeps such handoffs safe is `Theater.aliveGate` (the continuation observation); its treatment is out of scope for this paper.

6.7 `expose` versus `Program.Emit`

The two mechanisms are easy to confuse and are not the same thing. They are addressed in the same direction — feeding downstream systems data the actor has computed, without forcing those systems to rehydrate the actor — but from opposite sides of the journal.

`expose` is a keyword of the actor's language, valid only inside a `PerformCommand`.

When the verb runs, the script can mark a value as exposed, and the framework persists it into the `ExposeData` of the resulting journal entry alongside the `Action`. It is the *writer's* externalisation point: the cashier confirming an order can expose the order number, the calculated total, the resolved campaign — and downstream systems can consume those values directly from the journal without having to walk the actor's state.

`Program.Emit` is the Reaction's *reader's* externalisation point. A Reaction running over the journal — whether `Cue` or `Job` — has access to the `ExposeData` of the events it traverses (`Reaction.cs:870`, `includeExposeData=true`), and it can use those values in its `Where` clauses, in its captures, and in the parameters of its emit. What it cannot do is *produce* an `expose`: `PerformEmit` blocks the keyword by parsing with `isQuery:true` (§6.5). The asymmetry is structural and intentional. `expose` is the writer's hook into the journal; `Program.Emit` is the reader's hook out of it.

6.8 What does not exist — honest limits

A technical paper that reports what a primitive supports without reporting what it does not is incomplete — but two kinds of absence must be told apart: **deliberate boundaries** that follow from the construct (and would be wrong to add) and **pending gaps** that may simply arrive. The Reaction primitive does not, today, provide:

- **Computation of domain values over the matched events (`Sum`, `Count`, `Average`, arithmetic over payloads) — a deliberate boundary, not a gap.** The matcher's quantifiers shape the *set* of matches — mark a stage `Many`, require `AtLeast(n)`, bound it with a `Within` window, hand the raw set on with `Accumulate` (§6.4) — which is correlation, the matcher's own work. They stop deliberately short not only of computing domain values but even of grouping or de-duplicating the set (no `GroupBy`, no `Distinct`): that shaping, too, is domain work over the accumulated capture. This is not a shortfall against CEP (§9.3): it is the anti-porosity boundary of §3.3 and Paper 1 applied to the matcher — value computation is *domain* work and lives in the domain library under its legitimate paradigm, not in the Reaction DSL. The Reaction recognizes a trajectory of historical facts (the journaled invocations, Paper 2) and defers; a decision that needs an aggregate invokes a domain method or decomposes into two Reactions. The cost is real and owned: you cannot fold a running `Sum` into the pattern itself.
- **Negative or absence patterns.** No `Without`, `NotMatch`, `Negate`, `Unless`, `Missing`. *"X happened but Y did not happen afterwards"* is not directly expressible. The conventional workaround is a periodic `Job` that matches `X` and consults the actor's state via `Program.Emit` with a `when:` check to confirm `Y` has not occurred.
- **Some regex-style quantifiers — partly filled.** The matcher provides `Many()`, `AtLeast(n)`, and the exact family `None()` / `One()` / `Exactly(n)`

(the exact family gated on a `.Within(...)` window, §6.4). Still absent as dedicated forms are `AtMost(n)`, `OneOrMore`, and `Optional`.

- **Absolute time windows** as a dedicated method. The relative `.Within(entries) / .Within(TimeSpan)` anchors a window to the parent match (§6.4); an absolute calendar window (`start`–`end`) is expressed instead via `Where("@Now >= ... && @Now <= ...")`.
- **Optional `ThenSeek`.** A `ThenSeek` cannot be marked optional. Optional branches are written as separate Reactions.
- **`Job`-mode rehydration of the actor.** A `Job` Reaction reads only the journal; it does not wake or rehydrate the actor itself. When domain-computed state is needed by the Reaction, the canonical mechanism is `expose` (§6.7), which pre-writes the data alongside the command. Silent rehydration inside `Job` is intentionally absent — collapsing this boundary would erase the structural distinction between `Cue` and `Job` (§6.1) and reintroduce the cost the design was avoiding.
- **Cross-actor dispatch with journal-recorded causation — once a gap, now resolved.** An earlier draft of this paper named this as a limitation: cross-actor causation was mediated by `ITransport` infrastructure outside the sender's journal, leaving the broker as the seam. Since then, the framework has gained the third action plane — `Causation.Continue` (the Tell primitive) — that records the cross-actor dispatch as a sentence in the sender's own journal while leaving the transport choice to the developer's `ITransport` and the target actor's processing entirely to its own endpoint. The present paper names the plane (§6.5) but does not develop the primitive; its full treatment — design rationale and comparison against sagas, choreography, and distributed tracing — is out of scope here. The bullet is kept as a record of how the gap was named before being filled.

These absences fall into the two kinds named at the top. The value-computation boundary and `Job`-mode non-rehydration are **deliberate**: each follows from a principle the construct rests on — the domain barrier (§3.3) and the `Cue`/`Job` cost distinction (§6.1) — and adding it would erode that principle. The rest — negative patterns, the remaining regex-style quantifiers (`AtMost`, `OneOrMore`, `Optional`), absolute-time sugar, optional `ThenSeek` — are **pending gaps**: matcher features consistent with the construct that simply are not built yet. A reader who expects a CEP engine's aggregation and windowing operators is importing a different boundary: there the engine computes; here the matcher recognizes a trajectory and the domain computes. The framework's scope is what it supports; what it does not support is either a boundary to respect or a gap to model around.

6.9 Measurement

The argument of this paper is structural, and the instantiation is an existence proof of realizability rather than the design-science *evaluation* of an artifact (§6.10). Two measurements nonetheless calibrate the magnitudes the body asserts. Both are taken against the public commit recorded under Code provenance (`6a529c4`), with the harness in `labs/lab03-reactions`. Environment: Intel Core i9-13900, .NET 9.0.14 (X64 RyuJIT), Windows 11, Workstation GC; Release builds; timings via BenchmarkDotNet with tiered compilation disabled (`DOTNET_TieredCompilation=0`). Figures are read to about one significant figure.

Verb overhead. A minimal in-memory verb — one field write, no host work — dispatches and persists in ≈0.4 µs. This is the actor's own command-path overhead, the floor beneath every verb; a verb's observed latency is therefore set by the domain methods it invokes, not by the runtime around them (§5.2). The figure is sub-microsecond, orders of magnitude under any millisecond budget, which is why the fast verb is structural rather than tuned.

Cue reaction latency. End to end — from a command being issued to a `Cue` Reaction's action firing — latency was a median of ≈1.3 ms (p99 ≈2 ms) over 2000 activations, the Reaction firing once per match with no duplicate (exactly-once held, zero double-fires). The run was crash-free, so it exercises the common path, not the at-least-once crash window of §3.6; what it shows is steady-state delivery and latency, not a refutation of that window. The figure is well under the sub-second bar §6.1 claims, and it is governed by the signal-driven push path: a newly journaled entry preempts the catch-up poll because that poll waits on `pushSignal` (`ActorReactions`), the same signal the `RecordWritten` callback sets, rather than waiting out a fixed `Thread.Sleep` backoff. That the latency is a property of scheduling and never a floor of the construct is visible in the contrast: a naive fixed-backoff poll (50 ms → 1 s) measures ≈0.13 s on the identical path — a ≈100× difference from a change that touches scheduling alone, leaving delivery and exactly-once untouched. The construct never bounded the latency.

Threats to validity. The timings are single-environment and machine-specific; only their order of magnitude is portable, and they are reported to one significant figure. No external-system baseline is offered: a latency comparison against another actor runtime would confound language, persistence model, and transport at once, and the comparison this paper rests on is structural, not chronometric. The load-bearing observations are the deterministic ones — the verb-overhead floor and exactly-once-per-match (the monotonic cursor of §3.6, `ActorReactions.cs:186`), build-independent by construction — while the timings only confirm the direction the structural argument predicts.

Adversarial case. The partition does not always pay. When the deferred effect is cheaper than the machinery that defers it — the thread handoff, the pattern match, the separate emit under a read lock — moving it into a Reaction lowers the verb's latency but raises the total work the system does per event. The construct buys a fast verb and a closed library; for a trivial effect on a low-traffic verb it can cost more than it saves, and inlining is then the honest choice. The partition earns its keep where the deferred branch carries real operational weight — the case it was designed for, and the case the *When NOT to use* regimes (A–C) bound from the other side.

6.10 Evaluation deferral

The measurements of §6.9 are bounded observations scoped to the structural claims of §5 and §6, not a comprehensive evaluation of the instantiation. Holistic operational performance — endpoint-latency distributions under production load, developer-velocity comparisons, behaviour across a cluster — is not measured here. Under the analytic frame (§1) the deferral is supported: the construct stands on the structural argument, with empirical evaluation supplementary. The honest qualification is that the series has so far carried this empirical debt forward rather than redeeming it; the comprehensive case-study is owed, and naming the debt here is part of not letting the analytic papers accumulate it indefinitely.

7. Comparison with Akka and Orleans

7.1 Akka actors and Become/Receive

Akka's actors model behavior change with `Become` and `Unbecome`, swapping the receive partial function as the actor moves between modes. A booking actor might `Become(opened)`, then `Become(confirmed)`, then `Become(paid)`; each receive defines a different set of messages the actor accepts and how it handles them. The partition this expresses is across *actor states* — what the actor will accept next changes with the state — and the developer's mental model is a state machine the actor walks through.

Reactions express a different partition. The actor's `PerformCommand` is one verb, not many — it does not `Become` between calls — and the partition that Reactions add is across the *event boundary*: when a command has been processed and persisted, what happens afterwards is the Reaction's domain. Akka's mechanism is suited to actors whose interface changes as state evolves; Puppeteer's mechanism is suited to actors whose interface is stable but whose effects after each verb are rich. Neither subsumes the other; they answer different questions.

7.2 Orleans grains, timers, and reminders

Orleans' grains support timers (in-process, not durable across grain reactivation) and reminders (durable, scheduled). Both share the placement intent of `Job`: deferred work, possibly on another silo, scheduled rather than co-located. The structural differentiator is what gets *persisted*. An Orleans timer or reminder is a runtime registration — scheduled in the grain's activation lifecycle and re-registered on reactivation — that is not itself part of the grain's durable state. A Reaction is part of the actor's journaled program: declared in the DSL, parsed against the domain library (§6.2), and reconstructed from the journal on every rehydration (§3.5). Whether the deferral mechanism is recorded in the persisted program or attached to the runtime is an inspectable difference, and it is the one this section rests on. A separate, *ergonomic* observation — that declaring a Reaction is a few lines of fluent DSL beside the verb (§4) where the Orleans equivalent is procedural — is a design-rationale remark about friction, not a measured result: this paper runs no usability study and makes no quantitative "fewer lines" claim.

7.3 Where the design spaces converge and diverge

Akka, Orleans, and Puppeteer address the same structural pressures: the actor's serial mailbox, the cost of state hydration, the cost of distributing work across nodes, the difficulty of keeping fault isolation honest. The three answer those pressures with different design commitments. Akka commits to a flexible behavior model in which the actor's interface itself can evolve; Orleans commits to virtual actors that materialise on demand, with placement and scheduling as runtime concerns. Puppeteer commits to an externalised DSL with a homoiconic journal (Paper 2) and an anti-porous domain (Paper 1) — and Reactions are the language layer of that commitment: declarative pattern matching over the actor's trajectory (§3.5), with a strict separation between the actor's command path and what runs afterwards (§5.2). The Puppeteer position is one point in a non-trivial design space, not its peak.

Nothing in the present paper claims Reactions are the only realization of the construct. An Akka or Orleans extension that exposed a declarative DSL for trajectory matching with build-time binding to the host language's types would satisfy the same construct; this paper presents one realization, not the realization.

8. Counter-arguments

8.1 "Eventual consistency is a known anti-pattern in business systems"

The objection treats eventual consistency as a uniform property of the system, taken or rejected wholesale, and reads the warning literature on it as applying everywhere it appears. The framing is correct for *opt-out* eventual consistency, the kind classical CQRS introduces by separating read and write stores: every read is potentially stale, every write is potentially unobserved, and the application has to recover stronger contracts wherever the business demands them. Under that framing, the warning is well-earned.

The model developed in this paper inverts the framing (§5.3). Reads and writes inside the actor are strongly consistent — `PerformCmd` and `PerformQry` operate on the same in-memory state. Eventual consistency appears only at the deferred branch, named by the developer one deferral artifact at a time, with a placement attached. There is no system-wide consistency contract that the application must compensate for; there is, instead, a list of named windows the developer has chosen to admit. The literature warning applies to the framework that imposes eventual consistency as a default; a reader of an instantiation that exercises this construct can answer "what is eventually consistent here?" by reading the deferral artifacts declared near the verb and nothing else.

The honest concession (the 20%): some business domains require that every observable effect of a verb commit in lockstep with the response — banking core ledgers, certain regulated transaction systems. The opt-in framing does not soften this; in those domains, the developer cannot extend a license that does not exist, and the partition's primitive is not the right primitive (§When NOT, A). The objection lands where it lands; it does not generalise to every business system that adopts the framework.

8.2 "Reactions are async events with a different name"

The objection holds only if Reactions are read at their surface — as something that runs after a verb, on a different thread, with a callback shape. At that surface they look like async events, and the objection follows.

Read at the structural level of the construct, the artifact realizing the partition is something else. An async event handler responds to a single message; the construct's matching primitive matches a *trajectory* — a sequence of one or more stages naming a saga, a lifecycle, or an anomaly through the actor's history (claim 2(c), §6.3). The pattern is parsed against the domain's libraries at build time, not against text at runtime: a reference to a class the domain does not contain fails to construct (§6.2). Identifiers captured in one stage are typed and propagated to the next, which is what correlates the events into a story — an async event handler has no analogue. The placement axis with two canonical points (in Puppeteer's instantiation, `Cue` and `Job`) carries activation scopes (`DirectorOnly` / `CastOnly` / `Company`) that compose with the StageManager's topology (§6.6); an async event has none of that surface. The action plane choices (`Metadata.Elide`, read-only `Program.Emit` with optional `when:` check, and the journaled cross-actor `Causation.Continue`) are constrained primitives — particularly the read-only `Program.Emit` whose underlying `PerformEmit` structurally blocks `expose` and journal writes (§6.5) — designed to enforce the categorical segregation of §3.3 in the parse phase, not at runtime.

The honest concession (the 20%): the surface mechanism — work happens after the verb returns, on a separate thread or on demand — is shared with async event systems. A reader who looks only at the trigger sees something familiar. The argument of this paper is that the trigger is the least interesting part of the primitive.

8.3 "Even with low-friction Reactions, the developer can pollute the domain"

The objection is correct in spirit: friction reduction is not a substitute for discipline. A determined developer can put `sendEmail(...)` inside a `PerformCommand` even when a Reaction one line below would do the job; the framework cannot prevent it. Were the argument that friction reduction *replaces* developer judgment, the objection would land.

The argument is different. Friction reduction is the lever the framework can pull; developer judgment is the lever it cannot. When friction is high — when the right thing costs more than the wrong thing — no amount of training, code review, or architectural pep-talk holds at scale. When friction is low, training and review have something to lean on; the developer who chooses inline anyway is doing so against an obvious cheaper alternative, and the choice becomes legible to reviewers as a deliberate departure rather than a path of least resistance.

The honest concession is at the seam where friction reduction reaches its limit: a careless team will write polluted code under any framework, and a disciplined team will write clean code under any framework. The framework's contribution is the *gradient* between these — making clean code the default, pollution the exception, and the cost of getting it wrong recoverable rather than irreversible. The objection becomes an argument for friction reduction, not against it: in a world where developer discipline is unreliable, the only thing the architecture can do is shape the cost surface so that the path of least resistance is also the path of least damage.

8.4 "DSL scripts repeat lines across endpoints — this violates DRY"

The objection assumes DRY as a textual prohibition. In its original formulation (Hunt & Thomas, 1999), DRY prohibits the duplicate representation of *knowledge*, not the appearance of similar lines: *"every piece of knowledge must have a single, unambiguous, authoritative representation within a system."* The domain library lives once, in OOP; DSL scripts are the language in which an actor is invoked, structurally analogous to SQL. Two SQL queries that share JOIN patterns are not in violation of DRY — they are compositions over a shared schema. The same is true of two DSL endpoints that share invocation lines: the knowledge is in the library that the lines invoke, not in the lines themselves.

Concession (the 20%): if two DSL endpoints share *domain logic* — not invocation patterns but actual computation — that logic should refactor into a method of the library. The line is between invocation and implementation, not between unique and repeated text.

8.5 "Reactions should be able to rehydrate the actor for richer state queries"

The choice between modes is the structural distinction itself — full state queries belong to the live-actor mode, not to the journal-driven one (§6.1). When the data is anticipated by the developer, the journal pre-write mechanism (`expose` in Puppeteer, §6.7) puts the data where the deferral artifact will consume it during its own rehydration, without ever waking the actor. The journal-only-without-rehydration boundary is what makes the journal-driven mode cheap to host on a remote follower; lifting it would collapse the two modes into one fuzzy mode and reintroduce the cost the original design was trying to avoid.

Concession: there are rare cases of data that no one anticipated would be needed by a future Reaction, and that already exist in many past events without an `expose`. The workaround is operational — add `expose` now, run a migration that replays old events to write the `expose` retroactively, or build an auxiliary `Job` that materialises a side projection. Costly, but solvable, and rare enough that it does not justify reopening the primitive.

9. Related work

9.1 Actor Model and CQRS foundations

The actor model originates with Hewitt, Bishop, and Steiger (1973), which introduces the actor as the universal unit of computation: an entity with private state, a mailbox for incoming messages, and a single thread of behavior responding to them. The model has accumulated decades of refinement — Agha (1986), Hewitt (2010), the *Reactive Manifesto* (Bonér, Farley, Kuhn, & Thompson, 2014) — but the core remains: state isolated, computation message-driven, concurrency through the multiplication of actors rather than the sharing of state. Puppeteer's actor sits in this lineage, with the addition that the journal of received messages is itself the persisted artifact rather than a derived log.

Command Query Responsibility Segregation (CQRS) is named in Young (2010), with intellectual antecedents in Meyer's command-query separation (1988) and Fowler's reflections on enterprise architecture (2002). The classical formulation separates write and read stores at the level of the system architecture, with eventual consistency between them as the design contract. This paper engages CQRS not as the primary frame but as the literature that names the consequences of the partition this paper develops; the "logical CQRS" supporting observation is the classical pattern read at a different level — one in-memory state, two verbs over it, the eventual-consistency contract appearing only at the deferred branch (§5.3, §8.1).

9.2 DDD and the anti-pattern literature

Domain-driven design originates with Evans (2003), where the *anemic domain* and *smart UI* antipatterns are diagnosed: domain types reduced to data containers with behavior emigrated to service layers, and business rules pulled into the presentation/transport edge. Vernon (2013) catalogs the implementational pitfalls of DDD-by-vocabulary-only — bounded contexts and aggregates named but never structurally enforced. Fowler (2003, in *AnemicDomainModel*) gives the antipattern its most cited articulation. These three sources name the industrial reality §5.1 of this paper describes: in most enterprises, "the domain" does not exist as a library — it exists as scattered classes whose names suggest cohesion and whose substance does not.

Fowler's *Patterns of Enterprise Application Architecture* (2002) is the relevant catalog for the DTO antipattern: a Data Transfer Object intended for transport that becomes load-bearing in the domain, contaminating both. The same volume names the *Distributed Object* antipattern — the chatter problem this paper alludes to in §3.1 when it argues for the SQL-analogous use of a DSL as invocation rather than transcription.

Hunt and Thomas (1999) define DRY in *The Pragmatic Programmer* as the prohibition of duplicate *representation of knowledge*, not of repeated text — the formulation §3.1 and §8.4 recover against a more recent industrial reading. Sandi Metz's *The Wrong Abstraction* (2016) provides the natural complement: the cure for repeated text is sometimes more text, not the wrong abstraction.

9.3 CEP and adjacent runtimes

Complex event processing has a substantial literature — Luckham's *The Power of Events* (2002), Etzion and Niblett's *Event Processing in Action* (2010) — concerned with detecting patterns over streams of events, often from heterogeneous sources, often without the type system of the consuming application available. Correlated multi-event pattern matching with bound, propagated variables is CEP's defining capability, and typed, host-integrated engines — such as Esper, Flink CEP, and Drools Fusion — already compile or validate patterns against application types. This paper claims no novelty for the matching mechanism: the Reaction matcher is CEP-style matching, and these engines are its closest prior art. What differs is the substrate. CEP matches over external event streams; a Reaction matches over the actor's own *journaled invocations* (the executable journal of Papers 1 and 2), so the matched units are domain-method calls resolved against the domain library at build time (§6.2) and the matcher, the journal, and the domain share one type universe. That is a difference of setting and integration, not of matching power — and the paper's contribution is the partition construct (§1–§5) the mechanism serves, not the mechanism itself. CEP engines also *compute* over what they match — aggregations, windows; a Reaction's matcher does not. It shapes the match set (group, de-duplicate, bound) but leaves value computation to the domain (§3.3, §6.8) — again a placement of the boundary, not a weaker engine.

Akka and Orleans are the most relevant adjacent runtimes (§7). Their documentation is the canonical reference for Akka's `Become`/`Receive` and Orleans' grain timers, reminders, and stream APIs — the comparators against which §7 contrasts the Reaction primitive without claiming superiority.

9.4 Prior papers in this series

This paper builds on two prior contributions. *Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems* (Paper 1) names the structural defect — porosity — that the present paper extends to a fourth layer (the operational edge of the domain, §3.3) and the legitimate tool — the domain library shaped by its operations — that §3.1 carries forward. *Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime* (Paper 2) names the structural property of DSL programs that makes their compilation, caching, and dense journaling possible at all; the static binding of Reaction patterns against the domain's `Libraries` (§6.2) is one more layer of language built atop that substrate. Both prior papers are self-deposited preprints on Zenodo (Rivera, 2026a, 2026b) and have not undergone peer review, as is the present paper.

10. Conclusion: a developer says things — and now also at the endpoint

In §1 a developer was modeling a purchase-order endpoint by saying things. *"The cashier confirms the purchase and locks the requested items. Then a confirmation goes out, and the rewarder evaluates whether any promotional campaign applies."* That sentence was the program before it was code. Each phrase in it carries one of the technical names this paper has used.

- *"The cashier confirms the purchase and locks the requested items"* — a verb on a single in-memory state shared by reads and writes; the literature calls this *logical CQRS* (a supporting observation of this paper).
- *"Then a confirmation goes out, and the rewarder evaluates…"* — the partition between *now* and *deferred*, with placement chosen at the moment of partitioning (claim 1). Each phrase after the first is itself a Reaction declared next to the verb.
- *"A confirmation"* lands in a Reaction rather than in a domain method because the email concern does not belong inside the verb the cashier exposes — categorical segregation (claim 2(b)), encapsulating the technology where it can be exercised without propagating into the rest of the domain.
- *"The cashier, the order, the rewarder"* — these live inside a domain library that closes its operational boundary while continuing to grow conceptually under its legitimate paradigm (claim 4).

The developer reached none of these technical names while modeling. They reached for a sentence about who does what. The names are downstream — useful for reasoning about the system, for defending the design, for situating it against Akka and Orleans and the broader CQRS literature. The sentence is upstream — useful for getting the work done.

> *At some point a domain must be declarable as "task done". A library*

whose definition must absorb every present and future operational tool — email today, Slack tomorrow, a webhook protocol the year after — is a library that never closes. The only way it closes is if it stays pure: free of operational concerns by construction. Reactions are the artifact that makes that closure possible.

One of those downstream names carries more weight than the others. The CQRS literature treats eventual consistency as the architecture's defining commitment — taken or refused wholesale, a property the application must then absorb wherever the business demands stronger contracts. Read through the partition, it is none of that: it is the shadow the partition casts, present exactly where the developer drew a deferral and nowhere else. That is the inversion this paper turns on — eventual consistency is a consequence of a modeling decision, not a property of the architecture.

In C# a developer says things. In the DSL of an actor-native runtime, a developer says things too — about endpoints, about who does what, about what gets handled afterwards. The act is the same; the surface is bigger. What is new is not how the developer thinks. It is what the language lets them say.

Appendix A. Code references

All references are to the Puppeteer codebase; the files below live under `Puppeteer/EventSourcing/` (the matcher files under `Puppeteer/EventSourcing/Follower/`). Path:line citations were verified against the anchored commit (2026-06-21). Releases after this date may shift line numbers; the symbolic references (method names, class names) are stable and locate the same artifact regardless of line drift.

Reaction primitive — `Reaction.cs`

Range	What it shows	Cited in
20-24	`ReactionMode` enum (`Cue`, `Job`)	§6.1
68-82	Activation builders (`DirectorOnly`, `CastOnly`, `Company`)	§6.6
810	`time` global registered as `Temporal` instance	§6.4
863	`Cue` mode push-callback registration via `AddRecordWrittenCallback`	§6.1
870	`includeExposeData=true` during rehydration	§6.7

Range	What it shows	Cited in
938	`SetActive(bool)` manual override	§6.4
944	`SetExpirationDate(DateTime)` shutdown	§6.4
952	`IsExpired` predicate	§6.4
981-983	The three action planes exposed as properties (`Program`, `Causation`, `Metadata`)	§6.5
1039	`SetMetadataAction()` internal hook for `Metadata.Elide()`	§6.5
1076	`EnsureNoActionConfigured()` build-time guard (exactly-one-action rule)	§6.5
1106	`WithParameters` parameter modifier	§6.5
1113-1255	Action handlers (`ExecuteAction`, `ExecuteProgram`, `ExecuteCausation`)	§5.2, §6.5
1256-1267	`ExecuteMetadata()` with the `MarkEventsAsElided` invocation (Metadata plane)	§6.5
1174	`PerformEmit` invocation from `ExecuteProgram`	§6.5
1444	`SolveActionReferences()` per-event resolution	§6.2

Reaction action planes — `Planes.cs`

Range	What it shows	Cited in
5-13	Header comment naming the three planes (`Program`, `Causation`, `Metadata`) and their verbs	§6.5
44	`Program.Emit(string script)` — read-only emit	§6.5

Range	What it shows	Cited in
53	`Program.Emit(string script, string when)` — read-only emit with check	§6.5
76	`Causation.Continue(string script)` — journaled cross-actor verb (Tell)	§6.5
142	`Metadata.Elide()` — mark matched entries as elided	§6.5

Reaction language layer — `ReactionEngine.cs`

Range	What it shows	Cited in
71-83	`Where(expression)` clause registration with build-time validation	§6.4
146-265	Quantifiers and windows (`Many`, `Accumulate`, `AtLeast`, `Within`, `Aged`, `None`, `One`, `Exactly`)	§6.4

Read-only emit path — `ActorHandler.cs`

Range	What it shows	Cited in
2408-2479	`PerformEmit` method (read-only, `isQuery:true`, read lock)	§3.3, §5.2, §6.5, §6.7
2405	In-source comment marking the query-mode parse: `isQuery:true` blocks `expose` (which would persist to the journal) and global-variable declaration	§6.5

Range	What it shows	Cited in
2423	`parser.Parse(isQuery: true, isCheck: false)`	§6.5
2464	`rwLock.EnterReadLock()` taken before `Perform`	§6.5

Other modules (referenced without line numbers)

File	What it shows	Cited in
`Puppeteer/EventSourcing/Follower/Pattern.cs`	Static parsing of pattern descriptions against `Libraries`	§6.2
`Puppeteer/EventSourcing/Follower/MatchTree.cs`	`TryMatchAtLevel`, BFS/DFS hydration walk	§6.2
`Puppeteer/EventSourcing/Follower/MatchTree.cs:1565-1575`	confirmed checkpoint saved only after `ExecuteAction` succeeds (at-least-once)	§3.6
`Puppeteer/EventSourcing/Follower/MatchTree.cs:1512`	`MarkEventsAsElidedWithCheckpoint` — elision + cursor in one atomic commit	§3.6, §6.5
`Puppeteer/EventSourcing/DB/DiaryStorage.EventElisionStorage`	`MarkEventsAsElided` storage interface	§6.5

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `6a529c4` (2026-06-21). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:463f291ec5e4fa8abef24dc1fe3e46e5a5975739;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:6a529c45367ca2c32a891a067efb5be3fed589ec
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

Appendix B. Bibliography

Agha, G. (1986). *Actors: A model of concurrent computation in distributed systems*. MIT Press.

Akka documentation. (n.d.). https://akka.io/docs/

Apache Flink CEP documentation. (n.d.). https://nightlies.apache.org/flink/flink-docs-stable/docs/libs/cep/

Bonér, J., Farley, D., Kuhn, R., & Thompson, M. (2014). *The reactive manifesto* (v2.0). https://www.reactivemanifesto.org/

Drools Fusion documentation (complex event processing). (n.d.). https://www.drools.org/

Esper reference documentation. (n.d.). https://www.espertech.com/esper/

Etzion, O., & Niblett, P. (2010). *Event processing in action*. Manning Publications.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Fowler, M. (2003). *Anemic domain model*. martinfowler.com. https://martinfowler.com/bliki/AnemicDomainModel.html

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, *30*(3), 611–642.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular actor formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI-73)* (pp. 235–245).

Hewitt, C. (2010). *Actor model of computation: Scalable robust information systems*. arXiv. https://arxiv.org/abs/1008.1459

Hunt, A., & Thomas, D. (1999). *The pragmatic programmer: From journeyman to master*. Addison-Wesley.

Luckham, D. (2002). *The power of events: An introduction to complex event processing in distributed enterprise systems*. Addison-Wesley.

Metz, S. (2016). *The wrong abstraction*. sandimetz.com. https://sandimetz.com/blog/2016/1/20/the-wrong-abstraction

Meyer, B. (1988). *Object-oriented software construction*. Prentice Hall.

Microsoft Orleans documentation. (n.d.). https://learn.microsoft.com/en-us/dotnet/orleans/

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20404863

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20740697

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

ewpage

Preserving semantic continuity across actors

TL;DR

The actor model treats cross-actor causation as operational — message dispatch by a runtime, not a statement in any actor's program. The treatment is not entailed by the model; it is a contingent design decision, adopted as the default frame of the canonical actor lineage and seldom examined as a choice. The cost is structural: an entire ecosystem of compensating patterns — sagas, choreography, distributed tracing, workflow engines — exists to reconstruct cross-actor causal chains that no actor's program records. The treatment even erases a grammatical distinction — one dispatch verb collapsing an assertion of fact (past tense) and a command (present imperative) into a single send — which the programmatic alternative restores.

This paper observes that the assumption can be removed without altering any structural commitment of the actor model. Three conditions describe a system in which the cross-actor send is recorded as a sentence in the sender's program: locality of writes (each journal records only its own actor's activity), causation as program statement (the cross-actor send is a sentence in the sender's journal), and no external coordinator (no party outside the participating actors decides what happens next). Any primitive satisfying these three conditions dissolves the assumption — on the dense journal-as-program substrate the conditions presuppose (the send recorded as a sentence, not a payload). *Tell* — a primitive in the Puppeteer framework — is one such realization.

Three implementations of the same domain — saga, choreography, tell — are exhibited side by side. The reader sees, in journals, that the saga places the joint history in a coordinator's program, that the choreography places it in an external bus log, and that the tell instantiation places it in the sender's own program. Four tests probe these claims under tell. Three show that the journal alone supports replay reconstruction, cross-datacenter replication, and audit query — operations that require external apparatus under the assumption. The fourth shows that after a crash in the dispatch window the journal still records a stranded tell's fate honestly, recovered from the transport's testimony rather than fabricated. Each cross-actor edge is recorded as program in its sender's journal; a multi-hop chain composes these per-edge records — programmatic at every hop, where the alternatives reconstruct the chain from artifacts outside any program (§8.5). **Fifty years of convention are not fifty years of necessity.**

Claims this paper makes

1. **The assumption named.** Cross-actor causation has been treated as operational rather than programmatic across fifty years of canonical actor-systems literature — from Hewitt et al. (1973) through Akka and Orleans. *(Verification: §2 + the genealogy table in §2.4.)*

2. **The assumption is contingent.** No theorem of the actor model entails it. The three structural commitments of the model — autonomy, message-based communication, isolation — define the ontology of actors, not the mechanics of their runtimes. *(Verification: §4.)*

3. **Three conditions resolve the assumption.** Locality of writes (C1), causation as program statement (C2), and no external coordinator (C3) describe a system in which semantic continuity is preserved across actors without violating any of the actor model's commitments. The conditions are not design choices; they are the consequences of reconciling the model's commitments with the construct. *(Verification: §7.1.)*

4. **Any primitive satisfying C1+C2+C3 dissolves the assumption — given a journal-as-program substrate.** The realization is one of many possible, but the conditions are not free-floating: C2 requires the send to be recorded as a *statement of the program* — a dense operation, not a serialized event — which presupposes the anti-porosity of Paper 1 and the externalized parameters of Paper 2. The claim is therefore *tell on such a substrate*, not *tell in any actor framework*; a framework that records sends as opaque payloads would satisfy C2 only by first adopting that substrate, which is the subject of Papers 1–2 and beyond this paper's scope. *(Verification: §7.5.)*

5. **The actor model's commitments survive the reformulation intact.** Autonomy, message-based communication, and isolation each remain unchanged; only the historical interpretation of what each actor's

program is *permitted to say* is removed. *(Verification: §7.3.)*

6. **The compensating ecosystem exists because of the assumption.** Saga orchestrators, event-driven choreography, distributed tracing, and workflow engines each compensate, in their own way, for the absence of cross-actor causation in any actor's program. Their sophistication, maturity, and widespread adoption are evidence of the cost of the assumption, not of its correctness. The scope is the *record* of cross-actor causation: tell relocates that record into the program; it does not provide the transactional recovery — compensation, rollback, timeouts — that sagas and workflow engines also carry, and does not claim to (§6.1). *(Verification: §6 + the side-by-side labs in §8.3.)*
7. **Tell exhibits the conditions in a runnable instantiation.** A Reaction whose `.Causation.Continue(...)` body issues a `tell` records the cross-actor assertion in the sender's journal — as a typed message-action — and the receiver's acknowledgment closes the round-trip. The sender's journal becomes a self-contained record of that cross-actor edge; a multi-hop chain composes these per-edge records — programmatic at every hop, unlike the alternatives' reconstruction from outside any program (§8.5). *(Verification: §8.2 + §8.3 Style 3 + §8.5.)*
8. **Auditing the cross-actor narrative becomes reading the program.** The journal shows what was sent, to whom, with what content, at what time, and with what acknowledgment — without correlation IDs, distributed tracing, or external aggregation. *(Verification: §8.5 G3.)*
9. **Replay reconstructs cross-actor state from the journal alone — and recovers each tell's fate.** A fresh actor with no shared transport and no live receiver, replaying the journal, reconstructs the in-flight tell state; for a tell stranded by a crash in the dispatch window, the rehydrated actor reconstructs the pending tell and the transport testifies its fate, which the journal records as a verdict — acked, not-delivered, or honestly pending — so recovery never leaves the journal asserting a send that did not land. *(Verification: §8.5 G1, G4.)*
10. **Cross-datacenter replication preserves the cross-actor causal chain.** The journal carries the cross-actor causation across data centers because the causation was always recorded in a place replication can carry. *(Verification: §8.5 G2.)*

---

1. Introduction

When one actor causes another to act, where does that causal sentence live? Consider the ordinary case. An actor performs an operation and, as a result, sends a message to another actor. The first actor's operation, state mutation, and emitted events appear in its journal or trace; the second receives the message, and its own operation is likewise recorded. Yet the act of sending — the causal step that connects the two — appears in neither actor's program. It is mediated by the runtime, the dispatcher, or the message broker: the sender's journal does not record that it spoke, and the receiver's journal does not record, in program terms, who spoke to it. Ask later why the second actor did what it did, and the answer has to be assembled from outside every program — partly from the first actor's log, partly from the second's, partly from the broker's offsets, partly from a distributed trace. The causal chain exists. No program contains it.

This is so familiar that it rarely appears noteworthy. But it should. If actors are programs, and a sentence in one actor's program causes an effect in another, then that causal sentence has every reason to appear as part of the first actor's program. That it does not is no logical consequence of the actor model; it is a structural feature of how actor systems have historically been constructed.

This paper names that gap. The construct introduced is *semantic continuity*: the property of a program (in the substrate-level sense established in Paper 2 §1.2: the pair of domain library and journal of invocations) whose causal structure remains recorded as part of the program itself, even when its effects cross boundaries. The defect identified is the absence of semantic continuity at actor boundaries. The principles derived are the conditions under which it can be preserved without violating actor isolation and without introducing orchestration. The instantiation presented is *tell*, a primitive in which the cross-actor send is recorded as a sentence of the sender's journal — a dense program operation, not a serialized payload, which presupposes the anti-porous journal established in Paper 1.

Methodologically, this is an analytic theory contribution: it names a structural assumption that the canonical literature on actor-based systems has documented in different forms without recognizing as a single construct, derives the principles under which the assumption can be rejected, and presents a running system in which those principles are realized as an existence proof — not as the substance of the claim. The weight of the argument is conceptual: the naming of the assumption (§3) and the account of why an entire ecosystem of patterns answers it (§6) carry the claim, while the instantiation (§8) is its existence proof. The genre is the one Gregor (2006) names *theory for analyzing* (Type I): it introduces a construct that lets the phenomenon be described and classified, with empirical evaluation supplementary; the Hevner-style design-science *evaluation* of the artifact (Hevner, March, Park, & Ram, 2004) is a separate undertaking this paper does not attempt. The conditions C1–C3 derived in §7.1 are the necessary conditions the construct entails, not a prescriptive method offered for adoption: deriving what a system must satisfy to preserve the construct is Type I analysis, where prescribing and evaluating an artifact would be a separate design-science (Type V) contribution this paper does not undertake. This is not a systems paper — it presents no performance benchmarks, fault-injection metrics, or latency comparisons against existing actor frameworks; analytic theory measures contribution by the precision of the construct, the validity of the principles, and the realizability of the instantiation. Readers expecting quantitative comparisons against alternatives will find structural comparisons — what each pattern records, where the joint history lives — in §6 and §8.4.

§2 traces the genealogy of the assumption that produces this gap, showing that across five decades and multiple canonical generations of literature the assumption has remained continuous in substance though varied in form. §3 names the assumption explicitly: *causation between actors is treated as operational rather than programmatic*. §4 demonstrates that this assumption is contingent rather than necessary — no theorem of the actor model entails it. §5 examines the architectural and operational consequences of the assumption. §6 shows why existing responses to those consequences — sagas, choreography, distributed tracing, and workflow engines — cannot dissolve the assumption, because they reconstruct cross-actor flow after the fact rather than preserving it as program. §7 reformulates the model: under three explicit conditions, semantic continuity can be preserved across actors. §8 presents the instantiation, *tell*, through an illustrative case study against saga and choreography implementations of the same domain. §9 relates the construct to prior work in this paper series. §10 concludes.

---

2. Genealogy

2.1 The founding decision (1970s)

The actor model was introduced in 1973 by Hewitt, Bishop, and Steiger as a unifying account of concurrent computation in which "all of the modes of behavior can be defined in terms of one kind of behavior: sending messages to actors" (Hewitt et al., 1973, p. 235). Sending messages is positioned as a universal primitive, but the framing goes further: it is positioned as a *machine-level* primitive. "The basic unit of execution on an actor machine is sending a message in much the same way that the basic unit of execution on present day machines is an instruction" (Hewitt et al., 1973, pp. 236–237). The act of sending is therefore parallel to a hardware instruction — it is what the abstract machine performs *between* actors, not content of any actor's program.

A different concurrency tradition emerged in 1978 with Hoare's *Communicating Sequential Processes* (Hoare, 1978). It is worth distinguishing rather than assimilating: CSP's communication is a *symmetric, synchronous* handshake at named ports — sender and receiver rendezvous, each blocking until the other is ready — which is the opposite of the actor model's *asynchronous, fire-and-forget* send. These are different ontologies, and this paper does not fold them together. CSP earns one narrow point of contrast: there too, cross-process communication is a construct of the language, not a statement in either process's program. But the genealogy below tracks the actor lineage specifically; CSP is a foil, not a second witness to the same assumption.

The actor model was formalized in 1986 in Agha's MIT thesis (Agha, 1986), which defined actors as behavior functions over messages and treated cross-actor effects as emitted output messages of the function. The formal account ratified what Hewitt et al. (1973) had stated informally: an actor's program describes how it responds to the messages it receives; what happens *between* actors is the operational semantics of the model, not the content of any actor's program.

By the close of the 1980s, the actor lineage — Hewitt's informal formulation, Agha's formalization — had stabilized a single architectural decision: the boundary between an actor and the message-passing layer was drawn deliberately, and the layer between actors was characterized as operational rather than programmatic. (CSP reached a structurally similar boundary by a different route and under a different ontology; the resemblance is worth a note of contrast, not a claim that one assumption spans both traditions.)

2.2 The pragmatic maturation (1990s–2000s)

The operational frame became productive in Erlang. Armstrong's (2003) PhD thesis argued that fault tolerance becomes tractable precisely because processes do not share programmatic flow: failure is handled across process boundaries by links and exit signals. In Armstrong's own terms, "a process can supervise the existence of another process by setting up a link to it. When a process terminates, it automatically sends exit signals to the process to which it is linked" (Armstrong, 2003, p. 217). Supervision is a structural pattern in which a supervisor observes the failure of a child process and reacts. The supervisor's program is local; the failed process's program does not extend into the supervisor; the cross-process relationship is mediated by VM-level links and exit signals. Cross-process causation is an infrastructure concern. The frame stabilized further by becoming useful: the operational-rather-than-programmatic separation was now what made fault-tolerant systems possible.

2.3 The modern instantiations (2010s–)

Two implementations carried the actor frame into modern production systems. Akka, the canonical JVM actor framework (Lightbend, n.d.), characterizes its fundamental message-send pattern directly:

> *"Tell is asynchronous which means that the method returns right away. After the statement is executed there is no guarantee that the message has been processed by the recipient yet. It also means there is no way to know if the message was received, the processing succeeded or failed."*

The verb is named — `tell` — and so is its absence of program-level effect. The dispatch occurs; the sender's program does not record that it occurred, nor whether the recipient acted on it.

Microsoft's Orleans (Bernstein et al., 2014) introduced *virtual actors*: location-transparent grains whose dispatch is mediated by the Orleans runtime. Each grain has a key; the runtime decides where the grain lives and routes calls; the grain's code observes only local state and method invocation. The runtime maintains the cross-grain dispatch logic as infrastructure; the grain's program never sees nor records cross-grain causation as part of its narrative.

> **Decades apart, one assumption.** In Hewitt et al. (1973): *"Sending a message to an actor makes no presupposition that the actor sent the message will ever send back a message to the continuation"* (p. 241). In Akka 2014–present: *"there is no way to know if the message was received, the processing succeeded or failed."* The two formulations are decades apart and identical in substance. The verb `tell` has been part of actor-systems vocabulary throughout. What this paper observes is that the verb names the dispatch, not the program: a `tell` from actor A to actor B leaves no trace in either actor's program. The verb has existed; the assumption that the verb's effect was extra-programmatic has survived intact alongside it.

2.4 The naturalized assumption

Across fifty years and four canonical generations of literature — Hewitt's foundational paper, Agha's formalization, Armstrong's pragmatic maturation, and the modern instantiations in Akka and Orleans — the question of whether cross-actor flow is a *statement* of any actor's program is not the one the lineage asks. Where cross-entity causation is recorded at all — the message logs and causation identifiers of §6.4 — it is recorded operationally, beneath or beside the program, never as the program's own sentence. The assumption does not appear as a defended thesis; it is the default frame within which the lineage operates.

The forms in which the assumption surfaces vary by generation; the substance is continuous.

Year	Work	Anchor	Form of the assumption
1973	Hewitt et al. — *A Universal Modular ACTOR Formalism* (IJCAI)	"The basic unit of execution on an actor machine is sending a message in much the same way that the basic unit of execution on present day machines is an instruction." (pp. 236–237)	Sending is a machine-level primitive, parallel to a hardware instruction; therefore not content of any actor's program.
1986	Agha — *Actors: A Model of Concurrent Computation* (MIT)	Actors are behavior functions; cross-actor effects are emitted output messages of the function.	The behavior function is local; the cross-actor effect is the *output* of the function, separate from the function's body.
2003	Armstrong — *Making reliable distributed systems…* (KTH)	"A process can supervise the existence of another process by setting up a link to it […] it automatically sends exit signals to the process to which it is linked." (p. 217)	Cross-process flow is infrastructure-level (VM links and exit signals), not part of any process's program.
2010s	Lightbend — Akka *Interaction Patterns*	"There is no way to know if the message was received, the processing succeeded or failed."	The dispatch verb (`tell`) exists; its effect is explicitly outside the sender's program — no acknowledgment record.

Year	Work	Anchor	Form of the assumption
2014	Bernstein et al. — *Orleans: Distributed Virtual Actors*	The Orleans runtime places grains and mediates cross-grain calls; grain code sees only local state.	Cross-grain dispatch is hidden by the runtime; the grain's program never sees nor records cross-grain causation.

The continuity of this assumption across five decades is not the result of oversight, and still less of error. Each canonical contribution was optimizing a different property: Hewitt, Bishop, and Steiger, a uniform account of concurrency; Agha, a formal semantics of actor behavior; Armstrong, fault tolerance through process isolation; Akka and Orleans, distribution and location transparency. None was optimizing the *programmatic preservation of cross-actor causation*, because that was not the property under design. The assumption is the shadow of a different objective function, not a mistake the field failed to notice — which is why questioning it is not a charge against the lineage but a change in what one chooses to optimize. What follows does not dispute the effectiveness of the separation; it questions whether the separation is necessary.

§3 names this assumption explicitly. §4 demonstrates that it is contingent rather than necessary.

---

3. The assumption named

What §2 has shown to be continuous across fifty years admits a single one-line formulation. The structural claim that links the five entries of §2.4 — Hewitt's "machine-level instruction", Agha's emitted output messages, Armstrong's VM-supported links, Akka's `tell`, Orleans' runtime-mediated dispatch — is:

Causation between actors is operational, not programmatic.

The two terms in the formulation are the working categories of the rest of this paper. They also answer, for now, the question that opened it: *where does the causal sentence live?* Under the assumption, it lives in the operational layer — between actors, mediated by the runtime — and not in any actor's program.

Operational designates effects that the system performs around or between actors but that no actor's program records. Message dispatch by a runtime, supervision links between processes, virtual-actor placement decisions, message-broker routing, and distributed-tracing correlation IDs are all operational in this sense: their existence is mediated by infrastructure, and their effect on the system is real, but no program in the system contains the act of mediation as a statement.

Programmatic designates an effect that an actor's program contains as a *statement of the program* — an operation the program executes, observable within the program's own narrative and re-executed when the program is replayed. A method invocation an actor performs on itself is programmatic in this sense; so is a state mutation it writes. The program records what it did, as the doing, and replay re-runs it.

The distinction has a linguistic signature. A *command* is a directive — it asks for what is not yet done, named in the imperative present (`ApplyReward`, `Confirm`); an *assertion* reports a fact already lived, named in the past (`PurchaseConfirmed`). Treating the cross-actor send as operational collapses the two into one runtime dispatch — a single send verb carrying both, as in the canonical frameworks (§2.4); treating it as programmatic keeps them distinct and lets the sender record the one it can truthfully make — the assertion of what it did. The verb tense is the surface mark of which act the sentence performs; the choice this paper argues for is the one under which that mark survives into the program (developed at §8.2).

The boundary is not whether the send is recorded in the actor's own log. A serialized event can be appended there without being a statement of the program. Conventional event sourcing already does this: an aggregate raises a `MessageSent` or integration event when it dispatches to another context, and the event lands in the aggregate's own stream. But that event is *data the program emitted*, not a *sentence the program runs* — on replay it is folded back into state by a reducer, never re-executed as a statement of the program, and the dispatch is relayed by an outbox or broker the program does not contain. By the test used here it is operational-in-effect despite its location. A recorded cross-actor send is *programmatic* only when the send is a dense operation in the program — an executable sentence naming its recipient and message by reference, replayed as program rather than re-applied as state. Replayed *as program* means the recorded statement is re-executed by the interpreter — the program re-runs, reconstructing the state the send produced — not that the external dispatch is repeated: no journaled operation repeats its external effects on replay (replaying a debit reconstructs the balance without re-contacting the bank), and the tell is no exception. The programmatic property is that the send is a re-executable statement the program re-runs, where a serialized event is data a reducer folds without the program ever re-executing a send. That density criterion — a recorded operation, not a serialized payload — is the *anti-porosity* established in Paper 1; the present paper takes it as a precondition and asks what becomes possible at the actor boundary once it holds.

The contrast can be visualized:

```
Under the assumption (§3): causation operational, not programmatic

    A's program                                      B's program
        │
        │      runtime / broker / dispatcher                 │
        │  ─────────────────────────────────────────────> │
        │      (cross-actor causation not                    │
        │       recorded as program)                         │

    A's journal                                      B's journal
    (local effects only)                            (local effects only)
```

```
Under the alternative (§7): causation as program statement

    A's program                                      B's program
        │
        │  ─────────────────  tell  ────────────────────> │
        │                                                    │

    A's journal                                      B's journal
    [tell entry]                                     [receipt entry]
    [ack entry]
```

Both views describe the same flow; they differ in what is recorded as program. Under the assumption, the cross-actor send lives in infrastructure between actors and is invisible from any actor's program. Under the alternative, the cross-actor send is itself a journal entry on the sender's side, with a corresponding receipt entry on the receiver's side.

The naturalized assumption is the claim that, in the ontology of an actor system, the act of one actor causing an effect in another belongs to the *operational* category and not the *programmatic* one. The 1973 framing of message-sending as a hardware-instruction analogue is this claim. The 2014–present framing of `tell` as a fire-and-forget dispatch with no acknowledgment record is this claim. Across formulations, the assumption holds.

§4 demonstrates that the assumption, while continuous in the literature, is contingent rather than necessary: no theorem of the actor model entails it. §5 examines the consequences of the assumption — what is structurally lost when cross-actor causation lives outside any program. §6 shows why the existing repertoire of patterns built atop the actor model — sagas, choreography, distributed tracing, workflow engines — cannot dissolve the assumption, because they reconstruct cross-actor flow *after* the fact rather than preserving it as program. §7 reformulates: the assumption is contingent, the alternative is constructible, and the cross-actor flow can be a sentence of the program.

---

4. Contingency

The assumption named in §3 — that causation between actors is treated as operational rather than programmatic — would be necessary if it followed from a theorem of the actor model, from a foundational property the model formally entails, or from the structural commitments that distinguish actor systems from other concurrency paradigms. It does not. This section traces what the actor model formally requires and what it does not, and shows that the assumption is a contingent feature of how actor systems have historically been built rather than a consequence of what they are.

4.1 What the actor model entails

The actor model, in its canonical formulations from Hewitt et al. (1973) through Agha (1986), formally entails three structural commitments:

1. **Autonomy.** Each actor has its own state and processes messages serially. No actor's behavior is contingent on the simultaneous execution of another's.
2. **Message-based communication.** Actors communicate by passing messages. No actor reads or writes another's state directly.
3. **Isolation.** An actor's state is private to it. The address space of an actor is not shared.

These three commitments are what distinguish actor systems from shared-memory concurrency models. They define the ontology of actors, not the mechanics of their runtimes. They are the substance of "what the actor model is" in any reading of the canonical sources.

4.2 What it does not entail

The three commitments above do not entail that the *act of sending* be invisible to the sender's own program. They do not entail that cross-actor dispatch be mediated by a runtime layer that owns the send. They do not entail that a record of "actor A spoke to actor B at time T" must reside outside actor A's narrative.

In Hewitt et al. (1973), the act of sending is an action the actor performs. The framing of message-sending as a machine-level instruction analogous to a hardware instruction (Hewitt et al., 1973, pp. 236–237) is an interpretive choice that supports the architecture proposed in that paper; it is not a formal consequence of what an actor is. In Agha (1986), the actor's behavior is modelled as a function from (state, message) to (next state, outgoing messages) (Agha, 1986, ch. 4). The outgoing messages are output of the function. The formalization neither requires nor prohibits the function from producing, in addition, a record of the send observable in the actor's own program — the question is not raised. The formalization can be extended, without modifying any of the three commitments above, to include as output not only the outgoing messages but also a record of the send observable within the actor's own program.

What the canonical formulations establish is the *boundary* between actors — autonomy, isolation, message passing as the exclusive cross-actor mechanism. They do not establish that the boundary be invisible from within. The opacity of the cross-actor send to the sender's program is a separate decision that the canonical sources adopted but did not derive.

4.3 The objection from isolation

The most common objection to making cross-actor sends programmatic is that doing so would violate actor isolation. The objection conflates two distinct properties.

Isolation requires that no actor read or write another's state. Recording a cross-actor send in the sender's journal does not require either. The sender writes to its own journal — its own state. The journal entry describes what the sender did: *"I sent a message of type M to actor B at time T."* The receiver, when it processes the message, writes a corresponding entry to its own journal: *"I received a message of type M from actor A at time T."* Both records are local; neither requires shared state. Isolation is preserved.

What changes is not who can read whose state, but what each actor's program is permitted to say about its own activity. That change is local to the actor's own program; it is invisible across the boundary.

4.4 The objection from orchestration

The second common objection is that recording cross-actor causation as program would introduce orchestration. The objection conflates a record of causation with a director of behavior.

Orchestration is the architectural pattern in which an external coordinator decides what each participating actor does next. The coordinator dispatches commands; the participants execute them. Causation flows from the coordinator outward.

A journal entry that records what an actor did is not a coordinator. It does not direct anything. Each actor still processes its own messages autonomously, decides what to do, and emits its own outgoing messages. The journal preserves what happened; an orchestrator would prescribe what should happen. Recording causation does not introduce a director. It introduces narration, not control.

4.5 Closing

The assumption stated in §3 is therefore not entailed by the actor model. It does not follow from Hewitt's formulation, from Agha's formalization, from the requirement of isolation, or from the absence of orchestration. The opacity of the cross-actor send to the sender's program is contingent in exactly this sense — not a theorem of the model — but it is not arbitrary: it is the shadow of a different objective function (§2.4), well suited to location transparency, decoupling, and fault isolation, and so adopted by the canonical sources and left in place. It is not, in any formal sense, a property the model requires.

The consequence is bounded but decisive: the absence of semantic continuity at actor boundaries is not a necessity of the actor model. It persists because it served other objectives, not because the model demands it — so removing it is a change of objective, available without abandoning a single commitment, rather than the correction of an oversight.

---

5. Consequences

When the assumption stated in §3 is in force, cross-actor causation is held outside any actor's program. The consequences of this displacement are not operational inconveniences; they are structural. Four are examined here. Each is a distinct symptom; each is the same absence — the absence of semantic continuity at actor boundaries — observed from a different angle.

5.1 Auditability requires external reconstruction

The question *"why did this happen?"*, asked of an effect that crossed an actor boundary, cannot be answered by reading any single actor's program. The sender's journal records what the sender did; the receiver's journal records what the receiver did; neither records the link. To answer the question, an auditor must consult tools that live outside the programs: correlation IDs threaded through messages by the runtime, distributed tracing systems that observe the runtime from outside, log aggregation pipelines that join records across actor boundaries.

These tools work, but their necessity is evidence that the program does not contain what they reconstruct. They do not extend the program; they substitute for it. Auditability therefore depends on infrastructure that compensates for what the program does not record.

5.2 Replay does not reconstruct cross-actor history

Replaying an actor's journal reconstructs that actor's local history: which messages it received, how it responded, what state it produced. Replay of two actors' journals reconstructs two local histories. It does not reconstruct the joint history. The joint history never existed as a program artifact to be replayed.

The reason is direct. The act of one actor sending a message to another is not in either journal as a program statement. The sender's journal contains the operations leading up to the send; the receiver's journal contains the operations following its receipt. The send itself — the event that joins them — has no representation in either record. Replay can therefore reconstruct each actor in isolation, but not the causal chain that connects them.

This consequence matters wherever event sourcing, time-travel debugging, or post-hoc analysis is intended to span more than one actor. The model that promises replayability delivers it within actors and not across them.

5.3 Cross-datacenter replication loses program-level causation

When journals are replicated across data centers — a standard pattern for disaster recovery and geographic distribution — the per-actor histories travel with them. The cross-actor causation does not, because it was never written anywhere that replication can carry. It lived in the dispatcher, the message broker, the runtime — components that are typically per-cluster or per-DC.

Recovery from a DC failure can therefore reconstruct each actor's local state but cannot reconstruct, from the replicated material alone, the causal chain that led to the system's pre-failure state. The chain is reconstructed by replaying the broker's queue, by re-issuing messages that were in flight, or by reconciliation processes that assume the state and reconstruct backward. None of these steps is part of any actor's program; all of them depend on infrastructure-level apparatus that may or may not have been replicated.

5.4 Debugging across actors is log archaeology

A developer asked to explain why an actor produced a particular state, when that state was caused by a chain of events crossing several actors, cannot read a single program to find the answer. The developer assembles the answer from logs, traces, broker dumps, and timestamps — pieces of evidence whose joining is itself an act of reconstruction. The discipline that emerges is log archaeology: the careful inference of a causal narrative from artifacts that are not narrative themselves.

The discipline is real and, in some organizations, mature. Its maturity is proportional to the absence of programmatic causation — the cost is not that the discipline fails but that it must exist at all, reconstructing from infrastructure what no program records. But its existence is the symptom. A program whose causation is recorded as program admits debugging by reading; a program whose causation is dispersed across infrastructure admits debugging only by inference.

5.5 Closing

The four consequences are not separate problems with separate fixes. They are four manifestations of a single structural absence: causation between actors is held outside any actor's program, so any operation that requires reading the cross-actor causal chain — auditing, replaying, replicating, debugging — must reconstruct it from artifacts that lie outside the program.

Symptom	What is lost	What external apparatus reconstructs it
Auditability	the causal chain as part of any program	distributed tracing, correlation IDs, log aggregation
Replay coherence	the joint history reconstructible from per-actor journals	custom event-sourcing layers that span actors
Cross-DC replication	program-level causation when journals cross DCs	broker-level log replication, reconciliation processes
Debugging	the ability to read the cross-actor flow as a single program	log archaeology — manual correlation of timestamps and IDs

The cost of the assumption is not a list of operational difficulties to be addressed by better tools. It is the systematic displacement of causation from program to infrastructure. The tools that arise — tracing, correlation IDs, reconciliation processes, log archaeology — are not extensions of the program. They are evidence of what the program does not contain.

---

6. Why existing approaches do not dissolve the assumption

The consequences of the assumption named in §3 — auditability through external reconstruction, replay limited to single actors, cross-DC fragility, debugging by log archaeology — are not unrecognized in the actor-systems literature. Each has accumulated a corresponding repertoire of patterns intended to address it. This section examines the principal patterns and shows that they are responses to the consequences, not dissolutions of the assumption that produces them. Each pattern reconstructs cross-actor flow somewhere; none records it as a sentence of any participating actor's program.

This section carries as much of the paper's weight as the instantiation that follows. Once the assumption of §3 is named, sagas, choreography, distributed tracing, and workflow engines stop reading as unrelated tools and resolve into responses to one question — *where does the causal sentence live?* — each answering it the same way: by placing the sentence somewhere outside the participating actors. That so many mature patterns are answers to a single absence is the paper's central claim; §8 is its existence proof.

6.1 Sagas

The saga pattern, in both its orchestrated and choreographed forms, addresses the problem of multi-actor business transactions: an operation that requires several actors to act in sequence, with compensating actions if any step fails.

In the orchestrated form, a saga orchestrator holds a state machine that represents the cross-actor flow. The orchestrator dispatches commands to the participating actors and listens for their responses; if a step fails, the orchestrator emits compensating commands that undo or correct earlier steps. The cross-actor flow is therefore programmatic, but it is the orchestrator's program — not the program of any participating actor. From the perspective of an actor receiving the orchestrator's command, the message is indistinguishable from any other; the actor's program does not know it is in a saga.

In the choreographed form, the orchestrator is dissolved into a protocol. Each participating actor publishes events when it completes a step; the next actor in the flow subscribes to those events and triggers its own step. The cross-actor flow is encoded across the actors as a distributed protocol — but no single actor's program contains the flow. Each actor's program contains its local response to events; the flow itself is the implicit composition of those responses, observable only from outside.

Both forms treat the cross-actor flow as a separate concern from any participating actor's program. The orchestrated form externalizes it to a coordinator; the choreographed form distributes it across event subscriptions. Neither form makes the cross-actor send a sentence of the sender's program. The assumption stated in §3 is preserved by construction. The saga makes cross-actor flow programmatic only by relocating it outside the actors whose behavior it coordinates. A saga also does more than relocate the record: it implements transactional recovery — compensating actions, retries, step boundaries — a concern that is genuinely hard independently of where the causal record lives. tell does not provide that recovery and is not a substitute for it; what tell addresses is narrower and prior — *where the record of the cross-actor send lives.* The two are different instruments, compared here only on that axis. They compose rather than compete: tell's relocation of the record is the substrate a receiver-side saga can sequence over — each asserted fact a step the saga recognizes, a compensating action itself another asserted fact (`SaleReversed`) rather than a coordinator's command.

6.2 Distributed tracing

Distributed tracing — OpenTelemetry, Zipkin, Jaeger, and similar frameworks — addresses the problem of observability across actor boundaries. A trace context is propagated through messages by the runtime; spans are emitted by participating components; a trace storage backend reconstructs the cross-actor causal chain from the spans, presenting it as a tree or timeline.

The reconstruction is real and useful. It also confirms the structural absence it compensates for. The trace is built from artifacts that exist outside the participating actors' programs: the trace context is metadata threaded through messages by middleware; the spans are emitted by instrumentation that observes the runtime; the trace storage is a separate service that joins the spans. None of these artifacts is part of any actor's program. They are the apparatus that makes the absence navigable.

A trace is also lossy in a way that program is not. A trace records that a span with given attributes occurred; a program records what was said and why. The two are not equivalent. The trace is sufficient for observability; it is not the program of the cross-actor flow. It exists precisely because no such program exists.

6.3 Workflow engines

Workflow engines — Temporal, Cadence, AWS Step Functions, Camunda, Argo Workflows — externalize the cross-actor flow into a dedicated programming environment. The workflow is written as a separate program in the engine's own model; the engine dispatches activities to participating services; the engine's program records the flow as it executes.

Workflow engines are unusual among the patterns considered here because they do produce a programmatic record of the cross-actor flow. The flow is a program — an explicit, replayable, durable program. But the program belongs to the workflow engine, not to any participating actor. The actors are activities invoked by the workflow; their programs do not contain the workflow.

The asymmetry matters. From the perspective of a participant, an activity invocation is a remote procedure call from an external system; the participant's program records that it received an invocation and produced a result. The relationship between activities — the flow that connects them — lives in the workflow engine, in a separate codebase, in a separate execution model. Reading the participant's program does not reveal the workflow; reading the workflow does not reveal the participant's local program. Two complementary records that do not compose into one. The workflow and the actors form a bipartite description of what, in a semantically continuous system, would be a single program.

6.4 Causal and message logging

A sharper objection comes not from the business-flow patterns above but from distributed-systems fault tolerance, where recording what one process sent another is decades old. Message logging — in its sender-based, receiver-based, and causal variants, surveyed by Elnozahy, Alvisi, Wang, and Johnson (2002) — records the messages a process sends or receives so that a crashed process can be reconstructed by replaying them (Johnson & Zwaenepoel, 1987). Lamport's happened-before relation (Lamport, 1978) and the vector clocks that refine it (Fidge, 1988) record cross-process causal order directly. These mechanisms predate this paper by decades and unambiguously record cross-entity causation. A distributed-systems reader is entitled to ask: how is a *tell* recorded in the sender's journal different from a sender-based message log, which has existed since the 1980s?

The difference is the operational/programmatic distinction of §3, and message logging falls cleanly on the operational side. A message log is maintained by the recovery protocol beneath the application, for the recovery protocol's purposes: it captures message payloads and delivery metadata so the fault-tolerance layer can resend or replay them after a crash, and it is read by that layer, not by any actor's program. The application's program does not contain the act of sending as a sentence; the log is apparatus the runtime keeps *about* the program — the same category as the distributed trace of §6.2, differing in purpose (recovery rather than observability) but not in kind. A vector clock is narrower still: it records the *order* of causation, not its *content* — that B's state causally followed A's, not what A said to B or why.

Tell records the send as a sentence in the actor's program: a dense DSL statement in the journal that *is* the program — the anti-porosity of Paper 1, the externalized parameters of Paper 2 — read as narrative and replayed as program. The contrast is exact. A sender-based message log answers *"what bytes must I resend to recover this process?"*; the journal answers *"what did this actor say, to whom, and why?"* The first is a recovery artifact, payload-oriented and type-erased; the second is the program. The same holds for event-sourcing causation and correlation identifiers (Young, 2010; Vernon, 2013) and for the process-manager pattern (Hohpe & Woolf, 2003): each threads an identifier or a coordinating state machine through messages so the cross-entity chain can be reassembled downstream — operational reconstruction by the same logic as §6.1–6.3. That cross-entity causation has been captured for thirty years is not in dispute. What no prior mechanism does is record it as a sentence of the sending actor's own program. Message logging is, if anything, the strongest witness for §3's distinction: it is the most mature, most studied apparatus for capturing cross-actor sends, and it lives entirely on the operational side of the line.

6.5 Closing

The patterns examined fall into two groups. Orchestrated sagas, choreographed sagas, distributed tracing, and workflow engines (§6.1–6.3) are responses to the same consequence: cross-actor flow is not in any participant's program, and something must compensate for that absence — a coordinator's state machine, an event-subscription protocol, a trace storage backend, a workflow engine's separate program. Causal and message logging (§6.4) arises for a different purpose — fault-tolerant recovery — but shares the decisive property: the cross-actor send it records lives in a log beneath the program, not as a sentence within it.

Pattern	Where the flow is encoded	Does it preserve the flow as program in the actors?
Saga (orchestrated)	In the orchestrator's state machine	No — externalized to a coordinator
Saga (choreographed)	Across event handlers in each actor	No — only local responses appear
Distributed tracing	In the trace storage, observed by an external system	No — the program is observed, not extended
Workflow engine	In the workflow engine's separate program	No — the workflow is a separate artifact
Causal / message logging	In the recovery layer's message log, beneath the program	No — apparatus for crash replay, not a program statement

The five "No"s share a structure. In every case, cross-actor flow is made programmatic only by placing the program somewhere other than in the actors whose behavior constitutes the flow. The compensation always occurs outside the participants.

These patterns are therefore not solutions to the assumption stated in §3; they are architectural responses that accept the assumption and build systems around it. Their sophistication, maturity, and widespread adoption are not evidence that the assumption is correct. They are evidence that the absence of semantic continuity is costly enough to justify entire subsystems dedicated to reconstructing what the program does not contain.

---

7. Reformulation

The diagnosis is complete. The assumption stated in §3 — that causation between actors is treated as operational rather than programmatic — is not entailed by the actor model (§4); produces a structural absence of semantic continuity at every actor boundary (§5); and survives in every existing pattern that responds to the resulting consequences (§6). The remainder of the paper takes the diagnosis as established and asks what would have to be the case for the assumption to be dissolved. This section answers that question by deriving the conditions under which semantic continuity is preserved across actors, without violating any of the actor model's structural commitments and without introducing orchestration.

7.1 Three conditions

The conditions are derived directly from §4. The actor model entails autonomy, message-based communication, and isolation (§4.1). The construct introduced in §1 requires that the cross-actor causal chain be recorded as part of some actor's program (semantic continuity). Reconciling the two yields three conditions, each of which addresses one structural commitment of the model and one element of the construct. The conditions are not design choices; they are the consequences of reconciling the model's commitments with the construct.

Condition C1 — Locality of writes. When an actor sends a message to another actor, the sender records the act of sending in its own journal, and only there. The receiver, on processing the message — including an acknowledgment returned for an earlier send — records the receipt in its own journal, and only there. No actor writes to another's state. This condition preserves the model's isolation commitment: each journal is local property; recording cross-actor causation does not require shared state.

Condition C2 — Causation as program statement. The cross-actor send appears as a sentence in the sender's program — a journal entry whose content names the recipient and the message. Reading the sender's journal reveals not only what the actor did locally but also the act of speaking that connected its program to another actor's. The cross-actor edge becomes part of the program's narrative rather than an artifact reconstructed from outside it. The construct introduced in §1 — semantic continuity — is realized at the sender's boundary by this condition; symmetrically, the receiver's program records the receipt as a sentence about who spoke to it.

Condition C3 — No external coordinator. No party outside the participating actors authors what each actor does next — what it does, which message it sends, how it responds. The message-passing layer may carry those messages, with whatever reliability it provides; what C3 forbids is an external *author* of the flow — a coordinator whose state machine chooses each actor's next step — not an external *carrier* of it. Each actor processes its own messages autonomously and decides its own response. The journal records what happened; no orchestrator prescribes what should happen, and no participant outside the actors is required to interpret the record. This condition preserves the absence of orchestration that §4.4 identified as a non-negotiable property of the alternative.

The three conditions are independent. C1 alone preserves isolation but does not establish continuity; C2 alone establishes continuity but, without C1, would risk shared-state writes; C3 alone preserves autonomy but does not address the recording question. Together, the three conditions yield a system in which cross-actor causation is recorded as program in each participant's local journal, dispatched through the existing message-passing layer, and coordinated by no external party.

7.2 *tell* as primitive

A primitive that satisfies the three conditions by construction can be defined.

> Define *tell* as a sentence in an actor's program that, in a single act, (a) records an entry in the sender's own journal naming the recipient and the message, (b) dispatches the message through the existing message-passing layer, and (c) requires no coordination with any party outside the sender and the recipient.

The three components correspond to the three conditions. Component (a) satisfies C1 — the journal entry is written only to the sender's journal — and C2 — the entry is the program statement that names the cross-actor causation. Component (b) inherits the existing message-passing semantics of the actor model; nothing about the dispatch mechanism is new. Component (c) satisfies C3 — the act involves only the sender's local write and the dispatch; no orchestrator participates.

The primitive is conceptual. Its naming follows actor-systems convention: in Akka and elsewhere, *tell* designates a fire-and-forget cross-actor send (§2.3). The collision with Akka's verb is not accidental but argumentative: the verb has been part of actor-systems vocabulary for over a decade alongside the assumption that the verb's effect was extra-programmatic. The primitive defined here keeps the verb and changes what the verb records. Where Akka's *tell* leaves no trace in either actor's program (§2.3), this primitive's effect is to make the trace the program. The difference is not in the dispatch semantics but in what the program is allowed to say about the dispatch.

The canonical effect can be stated in one sentence: *tell* turns the journal into the boundary of causation. The boundary between actors does not disappear; it remains the locus of message passing. What changes is that the boundary becomes inscribed in each participating actor's program.

7.3 What changes, what does not

A reader familiar with the actor model may ask whether the conditions above amount to a different model. They do not. Each of the actor model's three commitments survives unchanged.

Autonomy is preserved. Each actor still processes its own messages serially. *tell* introduces no requirement of synchronization with other actors; the sender does not wait for the recipient. The conditions add a write to the sender's local journal; they do not couple the sender's progression to the recipient's.

Message-based communication is preserved. Actors still communicate by passing messages, exclusively. *tell* uses the same message-passing layer that the actor model already specifies; no shared memory, no remote procedure calls, no synchronous read of another actor's state. The dispatch mechanism is unchanged; only the recording at each end is added.

Isolation is preserved. No actor reads or writes another's state. The sender's journal entry is written by the sender. The receiver's journal entry, if any, is written by the receiver. The two records are local; they reference each other by content, not by shared address.

What changes is what each actor's program is permitted to say about its own activity (echoing §4.3). What does not change is what each actor can do, observe, or share. The opacity of the cross-actor send to the sender's program — which §3 named as the assumption and §4 demonstrated to be contingent — is the only thing that the conditions remove. The actor model remains intact; only the historical interpretation of what must remain invisible to the program is removed.

7.4 The shape of the act

The structural shape of *tell* is rendered below. The diagram is included not to introduce new content but to make the relations among the components of §7.2 visible at a glance. The diagram is deliberately mundane: nothing new is introduced except the presence of the journal entries.

```
sequenceDiagram
    participant ProgA as Actor A's program
    participant JA as A's journal
    participant MPL as Message-passing layer
    participant ProgB as Actor B's program
    participant JB as B's journal

    ProgA->>JA: tell msg to B — record the assertion
    ProgA->>MPL: dispatch msg
    MPL->>ProgB: deliver msg
    ProgB->>JB: process and record receipt entry
```

Two journal writes (in JA and JB), one dispatch through the existing message-passing layer, no participant outside the sender and the recipient. The diagram reproduces the three conditions visually: writes are local (C1), the cross-actor flow is recorded as program at both ends (C2), and no coordinator is present (C3).

7.5 Closing

The conditions are not extensions of the actor model; they are the conditions under which the assumption named in §3 can be removed without altering the model's structural commitments. *tell* is one realization of the conditions; any realization that satisfies C1, C2, and C3 would dissolve the assumption identified in §3. But the conditions presuppose a substrate: C2 requires the cross-actor send to be a *statement of the program* — a dense operation, not a serialized event (§3) — which is the anti-porosity of Paper 1 and the externalized parameters of Paper 2. The generality is therefore over primitives *on a journal-as-program substrate*, not over actor frameworks in general; a framework that records sends as opaque payloads would satisfy C2 only by first adopting that substrate, which is the subject of Papers 1–2 and beyond this paper's scope.

The reformulation is conceptual. The remainder of the paper presents the empirical question: can the conditions be realized in a running system? §8 answers by exhibiting an instantiation and demonstrating it through an illustrative case study.

---

8. Instantiation

8.0 Origins of the instantiation

The instantiation discussed below is provided by Puppeteer, an actor-based framework whose journal-as-program substrate predates the analysis presented in this paper. The framework's lineage runs from a 2005 autopersistence prototype — domain classes that persisted themselves through reflection, with no schema decisions in the domain code — through a DSL that emerged as the persistence substrate, to event sourcing as the runtime's primary discipline. By the time the conditions of §7 were articulated, the framework already satisfied them; the present section reads as observation of that alignment, not as derivation of the framework from the conditions. This invites a fair question of circularity — were C1–C3 read off Puppeteer and then presented as model-derived? They are not: §7.1 derives them from the actor model's three commitments and the construct alone, citing no framework feature. That a framework built for other reasons, years earlier, already satisfies them is therefore independent evidence that the model-derived conditions are satisfiable — not a sign they were retrofitted.

8.1 The case study domain

The case study is a simplified loyalty domain modelled on a production scenario from a deployment of the framework. A Seller actor confirms a purchase order via a domain command. A RewardEngine actor holds a registry of campaigns; for each purchase event, the RewardEngine evaluates which campaigns qualify (by date and amount thresholds) and applies the corresponding rewards to the customer.

The flow is intentionally minimal: one actor produces a domain event, another reacts to it, and the relationship between the two needs to cross an actor boundary. The simplicity is deliberate — the case study illustrates a structural property of the cross-actor mechanism, not the complexity of any particular business workflow.

8.2 The instantiation: *tell* in Puppeteer

Puppeteer's Reaction surface exposes three planes — *Program* for in-actor read-only effects, *Metadata* for journal-metadata changes, and *Causation* for cross-actor causation. The three planes name what the verb touches; *tell* lives on the third.

The Seller's Reaction is defined as follows:

```
seller.Reactions.DefineReaction("PurchaseFunnelToRewards")
    .Job().Company()
    .WithSharedHydration()
    .Seek("Purchase")
      .OnMatch("[s:Seller].purchase($order, $date, $amount, $customer)")
    .Causation.Continue(@"
        tell PurchaseConfirmed
            with @order, @date, @amount, @customer
            to RewardEngine('rewards-1')
            once @order;
    ");
```

The `tell` statement is a sentence in the actor's DSL, and the kind of sentence matters: the Seller *asserts a fact it has lived* — `PurchaseConfirmed`, named in the Seller's own vocabulary — addressed to the RewardEngine (`to RewardEngine('rewards-1')`), carrying the values that fact involved (`with @order, @date, @amount, @customer`) under a per-utterance identity taken from the order it carries (`once @order`), so one compiled action issues a distinctly-identified assertion for each purchase. It does not invoke a method on the RewardEngine, and it does not name how the message travels. This follows the single discipline the journal obeys throughout this series: an actor's program may record only what that actor could itself have said. The Seller can say *that a purchase was confirmed*; it cannot say *how the RewardEngine applies rewards* — that is the RewardEngine's verb, recorded in the RewardEngine's own journal — nor *which broker carries the message*, which is deployment, not program. The Reaction is established once and thereafter watches the Seller's journal; when the domain command `s.purchase(...)` lands as an entry, the standing Reaction matches it and its `.Causation.Continue(...)` body fires, journaling the assertion on the Seller's side.

The assertion is journaled as a typed *message-action* — defined once (its signature deduced from the values it carries) and then invoked. This is the same define-then-invoke shape *every* parameterized operation takes on the dense journal-as-program substrate this series builds on (Papers 1–2): the domain purchase, issued with typed parameters (`date`, `amount`), is journaled the same way, so the shape below appears twice — once for the purchase, once for the assertion. After the bridge delivers the assertion to the RewardEngine and the receiver acknowledges, the Seller's journal contains six entries:

```
[0]  s = Seller();
[1]  (define of the message-action s.purchase — its typed signature)
[2]  s.purchase('ord-100', 5/9/2026, 250, 'cust-42');
[3]      (define    of    the    message-action    PurchaseConfirmed    —
its typed signature)
[4]  tell PurchaseConfirmed
        with 'ord-100', 5/9/2026, 250, 'cust-42'
        to RewardEngine('rewards-1')
        once 'ord-100';
[5]  tell ack 'ord-100' from RewardEngine('rewards-1');
```

The values are passed as typed parameters, not interpolated into the script text — the definition records the signature `s.purchase(order, date, amount, customer)` and the invocation records the bound arguments, so the journal stores the operation and its values separately, exactly as the tell does. The tell's identity is the order it carries (`once @order`), which the invocation binds to `'ord-100'` — the ack in entry [5] returns under that same key. The tell's signature is inferred once — at first use, on the live path — and recorded in entry [3]; replay replays that recorded definition rather than re-inferring it, so the message-action is identical and order-independent across replays. This is the define-and-invocation discipline of Paper 2, under which a name is defined once and thereafter invoked; a reused name resolves as any operation does there. The inference is a write-time convenience, never a replay-time computation.

The same separation fixes what crosses the boundary. What the RewardEngine receives is the asserted fact and its values — the bound arguments of entry [4] — and only those; the parameter names, the message definition, and any operational envelope the Seller keeps about the act (the identity or request context under which the purchase was recorded) stay in the Seller's own program and do not travel with the tell. A tell is a domain speech act: a value the receiver's domain needs must be *in the assertion*, carried as one of its values, because the receiver maps the asserted message to a command it owns and can read nothing the assertion did not say. This is the cross-actor face of a discipline the journal obeys throughout — domain values move as arguments; the metadata a program keeps about its own operations is a separate channel that is not part of what it asserts (the *operational*-vs-program distinction of §3).

The three conditions of §7 are realized by these entries.

- **C1 (Locality of writes).** The six entries are in the Seller's journal only. The RewardEngine's journal records its receiving operation independently — the two journals never share storage.
- **C2 (Causation as program statement).** Entries [3]–[4] are the cross-actor assertion rendered as a DSL sentence — defined once as a typed message-action, then invoked. Reading the Seller's journal alone reveals that it asserted `PurchaseConfirmed` to RewardEngine, with what values, at what time. Entry [5] closes the round-trip with the receiver's acknowledgment.
- **C3 (No external coordinator).** C3 forbids an external party that *authors* the flow — one that decides, from its own state, what each actor does next. It does not forbid a *carrier*. The Seller's own program makes the assertion — it records, as a sentence of its program, that it told the RewardEngine and what — and the bridge only carries that assertion onward. The journal names no transport at all: which broker or protocol carries the message is a deployment binding resolved outside the program, not a sentence in it (see below). A saga coordinator is the opposite: its state machine authors the flow, which then lives as a quasi-domain artifact outside every participant's program, and folding it into a participant would dirty that domain with coordination it should not carry. Giving delivery a clean home in the carrier is what lets each actor's journal hold only the causal record — the fact that it spoke. (The reproducibility lab exhibits this edge two ways: a single-process didactic run in which the bridge also stands in for the receiver's own processing, and a separated-receiver run in which a pure in-process broker carries the envelope while the RewardEngine runs its own consumer — mapping the asserted message to a command it owns, journaling that command in its own journal, and acknowledging autonomously. The second instantiates the carrier/receiver split this clause turns on, with no party standing in for the receiver; in a deployment the delivered message is processed by the recipient's own program in the same way.)

A note on entry [5]. The acknowledgment is not the RewardEngine writing into the Seller's journal. The RewardEngine emits an ordinary result in its own journal; the transport routes an acknowledgment envelope back to the Seller; the Seller's own handler processes that inbound envelope and records `tell ack ...` in the Seller's journal. For the ack message the Seller is the *receiver*, so C1's receiver clause applies unchanged — an actor records its own receipt in its own journal. The cross-actor edge is two messages, the assertion outbound and the ack returning, each narrated by the actor that processed it; neither actor writes to the other's journal.

Delivery and correlation are separated by design. The journal is the durable record of what was asserted and what was acknowledged — correlation; redelivery, timeout, dead-lettering, and backoff belong to the transport. Routing, too, lives outside the program: the sentence names the addressee by its logical role (`to RewardEngine('rewards-1')`), and a deployment-level binding — not a clause in the journal — resolves that role to a physical route (which transport, which topic). The journal therefore reads identically no matter which transport that binding selects; the wire never enters the actor's voice. The delivery guarantee is whatever the chosen transport provides; the journal's correlation role is invariant across all of them. If an acknowledgment is lost, the transport's retry redelivers and the assertion's identity (`once @order` — an author-chosen key, here the order the assertion carries — or a content hash when `once` is omitted) is the receiver-side deduplication key, so a redelivered tell does not duplicate the effect; a replayed journal re-executes the assertion — rebuilding the in-flight record — but its dispatch to the transport runs only on live execution, so replay does not re-send (the transport never sees a message from rehydration). A crash in the narrow window between journaling the assertion and dispatching it strands the envelope — journaled as issued, yet never handed to the transport; on recovery the rehydrated sender reconstructs that pending tell from the journal alone and the transport, the sole authority on delivery, testifies its fate, which the sender records as a verdict in its own voice (§8.5 G4) — so a crash no longer leaves the journal asserting a send that never landed: each issued tell is resolved (acknowledged, or marked unacknowledged by its addressee) or honestly left pending for the transport to settle, bounded by what that transport can still testify. The delivery model is thus at-least-once with identity-based deduplication; the consistency machinery it rests on — the now/deferred partition and its deduplication discipline — is developed in Paper 3, on which this construction builds.

The framework rejects `tell` outside `.Causation.Continue(...)`. A direct `tell` from a top-level command throws a runtime exception, with the error message pointing the developer at the correct surface. The cross-actor send is always a sentence in some Reaction's Causation body, never a free-floating call.

One feature of the sentence repays attention: the message is named in the past — `PurchaseConfirmed`, a fact the Seller has already lived — never in the imperative. This is not a style convention but a consequence of the criterion: an actor can assert only what has already happened in its own history, and by the time the assertion is journaled the purchase is a settled entry. A *command* — the directive at a request edge, which may still be refused — is named in the present imperative (`ApplyReward`, `Confirm`); to name a command in the past would be a category error, an order to do what is already done. The tense is the surface mark of which kind of act the sentence is.[1] This is where the construct parts from the canonical actor frameworks: Akka exposes a single send verb (`!`, *tell*) for every message and accordingly advises naming them all in the past tense — folding the report of a fact and the issuing of a command into one form, which is right for the events and wrong for the commands, because one

[1]The surface mark is English verb tense because the DSL is written in English; the distinction it carries — asserting a settled fact versus issuing a directive — is not itself tied to tense, and a language that grammaticalizes aspect rather than tense would mark the same distinction by other means. The claim is about the two kinds of act, not about tense as a linguistic universal.

verb cannot tell them apart. Here the two are kept distinct and the tense marks the difference — the assertion the sender makes about its own past, and the command the receiver runs in its own present, landing in two journals, in two voices.

8.3 Three implementations side by side

The same domain — the Seller confirms a purchase, the RewardEngine applies qualifying campaigns — is exercised in three styles. The code that distinguishes each style and the journals each produces are reproduced below without commentary. The interpretation follows in §8.4.

The saga and choreography implementations are the author's own, written for this illustration and deliberately kept minimal (Appendix A); they are not independent or performance-tuned baselines. The illustration is accordingly structural, not competitive: it shows *where* each style records the joint cross-actor history, not that tell is faster, more resilient, or operationally simpler. Those are measurable dimensions this paper does not test (§1).

Its weight is definitional, not empirical. The alternatives are written in their canonical shape — an orchestrated saga whose participants are commanded by a coordinator that owns the flow, and a choreography whose actors publish to a shared bus — so the reader can see in journals what each pattern's defining feature entails for where the joint history lands. The obvious objection — that a saga could have each participant record its own role — names exactly the tell move: a participant that records its cross-actor participation as a statement of its own program has, by that act, satisfied C2. The contrast is therefore definitional: the alternatives place the joint history outside any participant's program because their defining feature *is* externalized coordination. The journals below illustrate that truth; they are not measured evidence that one pattern outperforms another, and a differently-built saga would not change where externalized coordination, by definition, leaves the record.

Style 1 — Saga (orchestrated)

A SagaCoordinator actor drives the workflow via direct commands to participants:

```
saga.Using("step = 'PurchaseRequested';").PerformCommand();

seller.Using("s = Seller();").PerformCommand();
seller.Using("s.purchase(@order, @date, @amount, @customer);")
      .WithParameters(p => {
          p["order",    typeof(string)]   = "ord-100";
          p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
          p["amount",   typeof(decimal)]  = 250m;
          p["customer", typeof(string)]   = "cust-42";
      })
```

```
      .PerformCommand();

saga.Using("step = 'PurchaseConfirmed';").PerformCommand();
rewards.Using(@"
    foreach (c in loyalty.Campaigns()) {
     if (c.Applies(@date, @amount) == true) { c.Reward(@order, @customer); };
    };")
       .WithParameters(p => {
           p["order",    typeof(string)]   = "ord-100";
           p["customer", typeof(string)]   = "cust-42";
           p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
           p["amount",   typeof(decimal)]  = 250m;
       })
       .PerformCommand();

saga.Using("step = 'RewardsApplied';").PerformCommand();
```

Every value crosses the DSL boundary as a typed parameter — `p["date", typeof(DateTime)]`, `p["amount", typeof(decimal)]`, the string ids too — bound through `.WithParameters(...)`, never string-interpolated into the script text. Each command is issued separately, so each lands as its own journal entry; a parameterized command is journaled as a message-action (define-then-invoke), and two campaigns added by the *same* parameterized command share one definition. The RewardEngine holds two campaigns — `C-newcomer` (min 10, qualifies for the 250 purchase) and `C-highroller` (min 1000, does not) — so the receiver's loop applies one of the two. After the run, the three journals contain:

```
SagaCoordinator's journal (3 entries):
  [0] step = 'PurchaseRequested';
  [1] step = 'PurchaseConfirmed';
  [2] step = 'RewardsApplied';

Seller's journal (3 entries):
  [0] s = Seller();
  [1] (define of the message-action s.purchase)
  [2] s.purchase('ord-100', 5/9/2026, 250, 'cust-42');

RewardEngine's journal (6 entries):
  [0] loyalty = RewardEngine();
  [1] (define of loyalty.AddCampaign)
  [2] loyalty.AddCampaign('C-newcomer', 1/1/2020, 10);
  [3] loyalty.AddCampaign('C-highroller', 1/1/2020, 1000);
  [4] (define of the reward loop)
 [5] foreach (c in loyalty.Campaigns()) { ... c.Reward('ord-100', 'cust-
42'); };
```

Style 2 — Choreography (event-driven, no coordinator)

An event bus mediates the cross-actor handoff:

```
bus.Subscribe(ev => {
    if (ev.StartsWith("PurchaseConfirmed:")) {
        rewards.Using(@"
            foreach (c in loyalty.Campaigns()) {
           if (c.Applies(@date, @amount) == true) { c.Reward(@order, @customer); };
            };")
          .WithParameters(p => {
              p["order",    typeof(string)]   = "ord-100";
              p["customer", typeof(string)]   = "cust-42";
             p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
              p["amount",   typeof(decimal)]  = 250m;
          })
          .PerformCommand();
    }
});

seller.Using("s = Seller();").PerformCommand();
seller.Using("s.purchase(@order, @date, @amount, @customer);")
      .WithParameters(p => {
          p["order",    typeof(string)]   = "ord-100";
          p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
          p["amount",   typeof(decimal)]  = 250m;
          p["customer", typeof(string)]   = "cust-42";
      })
      .PerformCommand();
bus.Publish("PurchaseConfirmed:ord-100");
```

After the run:

```
Seller's journal (3 entries):
  [0] s = Seller();
  [1] (define of the message-action s.purchase)
  [2] s.purchase('ord-100', 5/9/2026, 250, 'cust-42');

RewardEngine's journal (6 entries):
  [0] loyalty = RewardEngine();
  [1] (define of loyalty.AddCampaign)
  [2] loyalty.AddCampaign('C-newcomer', 1/1/2020, 10);
  [3] loyalty.AddCampaign('C-highroller', 1/1/2020, 1000);
  [4] (define of the reward loop)
 [5] foreach (c in loyalty.Campaigns()) { ... c.Reward('ord-100', 'cust-
42'); };
```

```
Bus log (1 entry):
  published: PurchaseConfirmed:ord-100
```

Style 3 — Tell (Puppeteer)

A Reaction on the Seller observes its own purchase and issues a `tell` from its `.Causation.Continue(...)` body:

```
seller.Reactions.DefineReaction("PurchaseFunnelToRewards")
    .Job().Company()
    .WithSharedHydration()
    .Seek("Purchase")
      .OnMatch("[s:Seller].purchase($order, $date, $amount, $customer)")
    .Causation.Continue(@"
        tell PurchaseConfirmed
            with @order, @date, @amount, @customer
            to RewardEngine('rewards-1')
            once @order;
    ");

seller.Reactions.Execute();  // once defined, the Reaction stands over the journal and applies
                                          // to every matching entry —
it is not invoked per command. A Job is
                    // swept on demand (driven here for a deterministic test); a Cue
                         // Reaction runs continuously on its own thread.
seller.Using("s = Seller();").PerformCommand();
seller.Using("s.purchase(@order, @date, @amount, @customer);")
      .WithParameters(p => {
          p["order",    typeof(string)]   = "ord-100";
          p["date",     typeof(DateTime)] = new DateTime(2026, 9, 5);
          p["amount",   typeof(decimal)]  = 250m;
          p["customer", typeof(string)]   = "cust-42";
      })
      .PerformCommand();
// the standing Reaction observes the purchase and asserts PurchaseConfirmed to RewardEngine;
// the bridge delivers and acks back
```

The tell's identity is `once @order` — the per-utterance key *is* the order id, so one compiled action issues a distinctly-identified assertion per purchase, and the ack returns under that same id.

After the run:

```
Seller's journal (6 entries):
  [0] s = Seller();
  [1] (define of the message-action s.purchase)
  [2] s.purchase('ord-100', 5/9/2026, 250, 'cust-42');
  [3] (define of the message-action PurchaseConfirmed)
```

```
  [4] tell PurchaseConfirmed
        with 'ord-100', 5/9/2026, 250, 'cust-42'
        to RewardEngine('rewards-1')
        once 'ord-100';
  [5] tell ack 'ord-100' from RewardEngine('rewards-1');

RewardEngine's journal (6 entries):
  [0] loyalty = RewardEngine();
  [1] (define of loyalty.AddCampaign)
  [2] loyalty.AddCampaign('C-newcomer', 1/1/2020, 10);
  [3] loyalty.AddCampaign('C-highroller', 1/1/2020, 1000);
  [4] (define of the reward loop)
 [5] foreach (c in loyalty.Campaigns()) { ... c.Reward('ord-100', 'cust-
42'); };
```

8.4 Structural reading

The three implementations in §8.3 exercise the same logical flow but record it differently. Three pairs of journals were exhibited.

Saga: the SagaCoordinator's journal contains the workflow narrative — `PurchaseRequested` → `PurchaseConfirmed` → `RewardsApplied`. The Seller's journal records its local purchase only; it does not know it is part of a saga. The RewardEngine's journal records its local reward only; equally unaware. Three journals, three local stories. Only the coordinator's journal contains the joint history.

Choreography: no coordinator exists. The Seller's journal records the local purchase; the publish to the bus is invisible to the actor. The RewardEngine's journal records the local reward; the receipt from the bus is invisible to the actor. The bus's own log records the publish. No actor's journal contains the joint history; the bus log, an external infrastructure artifact, is the only place the cross-actor handoff is recorded.

Tell: the Seller's journal contains the purchase, the assertion (journaled as a typed message-action — defined once, then invoked), and the ack — six entries in all (each parameterized operation is a define plus an invocation) that constitute the joint history as a sequence of DSL sentences. The RewardEngine's journal records its local reward, as in the other styles. Under tell, the sender's journal alone reconstructs this cross-actor edge — the single hop from Seller to RewardEngine; §8.5 examines what happens when the chain is longer.

The three styles produce equivalent business outcomes. They differ structurally in where the cross-actor causal chain is recorded. The difference is not cosmetic. The saga coordinator, the event bus log, the distributed trace, and the workflow engine all exist to compensate for the absence identified in §3. Under tell, that program-level absence is gone: the sender's journal already contains the cross-actor narrative those patterns reconstruct elsewhere, so the apparatus built to recover it has nothing to recover.

The claim is about the *record*, not the wire. Tell does not eliminate the message-passing infrastructure: delivery remains operational, carried by whatever transport a deployment-level binding resolves the addressee to (§8.2), and the bridge that routes envelopes is part of it. What moves into the program is the causal record — the assertion the sender makes, and the acknowledgment of receipt — not the act of delivery. The contribution is the relocation of the narrative into the program, not the removal of transport.

Style	Joint history location	Audit path
Saga (orchestrated)	In the saga coordinator's journal exclusively	Read the coordinator's journal; participants' journals are half-stories
Choreography (event-driven)	In no actor's journal; the bus log is the only joint artifact	Merge participants' journals via correlation id, plus the bus's log
Tell (program-level cross-actor primitive)	In the sender's journal as a sequence of DSL sentences	Read the sender's journal entries [tell] and [ack] verbatim

In the first two styles, an additional architectural element is required to make the cross-actor flow observable as a narrative: a coordinator, a bus log, a trace backend, or a workflow program. In the tell style, no additional element is introduced. The narrative exists where the actor model already records program: the journal.

8.5 Property validation

Four property tests probe the claims of §5 and §6, all reproduced by the harness in `labs/lab04-tell` run against the public commit recorded under Code provenance. The first three (G1–G3) demonstrate that consequences claimed in §5 — auditability through external reconstruction, replay limited to single actors, cross-DC fragility — are reversed under tell, exhibiting properties that would be inaccessible if the assumption named in §3 were in force; they are intentionally chosen to mirror the compensating patterns of §6, each exhibiting a property that, under the assumption of §3, requires an external architectural pattern to achieve. A fourth (G4) turns to a sharper, adversarial question: whether the record stays honest when a tell never crosses.

G1 — Replay coherence (closes §5.2). The first test stages an in-flight tell: the envelope leaves the Seller but the bridge does not deliver it before the test asserts. A fresh actor instance with the same name, no shared transport, no live receiver, and no in-memory state replays the journal alone. Replaying the journal re-executes the tell statement; under replay it rebuilds the in-flight record rather than re-dispatching, so the replayed actor reconstructs the cross-actor state from the journal alone — the program re-ran, with no duplicate send. The joint history exists as a program artifact and replay reaches it.

G2 — Cross-DC replication (closes §5.3). The second test replicates the Seller's journal entry-by-entry to a fresh actor in an independent storage tier — the analogue of moving across data centers. The replicated actor, with no transport connectivity to the original receiver, reconstructs the dedup state from the replicated bytes alone. The cross-actor causal chain travels with the replication because it was always recorded in a place replication can carry.

G3 — Audit query (closes §5.1). The third test asks the audit question — *why did this happen?* — by reading the Seller's journal directly. The cross-actor assertion is entry [4] (the message-action invocation); the acknowledgment is entry [5]. The cause-effect chain is reconstructed without distributed tracing, correlation IDs, or log aggregation.

The audit this closes is the *causal* one — what was said, to whom, when, and with what acknowledgment — because that is what the program records. It is not the *identity* audit — under which user, tenant, or request context an actor acted — which is operational metadata in the sense of §3 and lives, where a deployment keeps it, in a separate envelope keyed to the entries, not in the cross-actor causal record. Claim 8 is a claim about causal auditability; the two audits answer different questions and travel on different channels (Paper 5, §5.2).

Each of these properties is a consequence of cross-actor causation being recorded as program. Under saga, choreography, tracing, or workflow approaches — where it is not — the same properties are reachable only by consulting artifacts outside the participating actors.

G4 — Tell-fate recovery, the honest record under crash. G1–G3 stage an in-flight tell and show the *record* survives a crash, a move across data centers, an audit. A sharper question is what the record *says* when the tell never crosses. The send is journaled before the post-commit dispatch hands its envelope to the transport; a crash in that window strands the envelope — journaled as issued, never delivered, never acknowledged. Left unaddressed this is the one place the "sender's journal alone" property could lie: the journal would assert a send that did not happen. The fourth test stages exactly this crash and rehydrates the sender. Replay reconstructs the set of *pending* tells — issued, neither acknowledged nor settled — from the journal alone; for each, the transport, the sole authority on delivery, testifies its fate and the sender records the verdict *in its own voice* — `tell 'ord-100' unacknowledged by RewardEngine` when the transport reports the envelope failed, the ordinary `tell ack ...` when it reports the envelope was delivered and only the acknowledgment was lost, and nothing while the fate is still in flight (the transport keeps ownership). The verdict names the addressee the Seller did not hear back from, not the broker that carried the message: A may say *"RewardEngine never acknowledged"* — a fact within its own universe — but not *"per the broker that carried it,"* which is infrastructure it has no standing to assert. This is the dual of §6.4's causal and message logging: there, the record of what crossed lives in a recovery layer beneath the program, consulted only by crash-replay machinery; under tell it is a sentence in the sender's program, so recovering it is reading the journal, not excavating infrastructure. After recovery the sender's journal no longer asserts a send that never landed: each issued tell is either resolved — acknowledged, or marked unacknowledged by its addressee — or honestly carried as still pending, for the transport to settle when it can. The guarantee is honesty about the outcome, not omniscience: a transport that cannot testify — one whose own record of a tell's fate did not survive the failure — answers `InFlight`, and the tell stays pending rather than acquiring a fabricated verdict. Delivery stays the transport's; the verdict is the journal's.

The multi-hop limit — an adversarial case. G1–G3 are duals of the consequences named in §5, and so are chosen to show what tell does well. The honest counter-case is a longer chain. The case study is a single hop — the Seller tells the RewardEngine. Suppose instead a chain assembled the way every hop is: a Reaction on A tells B; B carries its own Reaction that observes the entry its receipt produces and, from that Reaction's `.Causation.Continue` body, tells C. Nothing propagates the chain automatically — each actor opts in with its own Reaction (the envelope's causal identifier is per-hop-local, not a threaded chain id), so the hops remain autonomous, as C3 requires. Each hop is recorded as program in the journal of the actor that originated it — A's journal holds the A→B tell and its ack, B's journal holds both its receipt of A and the B→C tell it issued, C's journal holds its receipt of B. No single journal holds the whole A→B→C chain; reconstructing it end to end means composing A's and B's journals (linkable by envelope identifier across them). In this, tell inherits a distributed-history property of the kind §6.1 identified in choreography — but the difference is in kind, not in absence. Under choreography no participant's journal records the cross-actor edge as program at all; the chain lives only in the bus log and is reconstructed from non-programmatic artifacts. Under tell every edge is a programmatic record in its sender's journal, so the multi-hop chain is a composition of programs — each hop locally complete, auditable, and replayable on its own. Tell makes each edge programmatic and local; it does not centralize a multi-hop chain into one journal. The "sender's journal alone" property (§8.4) is therefore a per-edge guarantee: it holds for the hops an actor originates, which is the whole chain only when the chain is a single hop.

8.6 Closing

The case study and the defensive tests together constitute the existence proof. The conditions of §7 are realizable: a system in which the cross-actor send is recorded as a sentence in the sender's program, dispatched through the existing message-passing layer, and coordinated by no external party can be built and exercised. The instantiation in Puppeteer is one such system. The framework's own production deployment, which the case-study domain is modelled on (§8.1), is proprietary and is not exhibited here; the existence proof this paper offers is the runnable instantiation above. Other realizations of the conditions are possible (§7.5); the present section establishes only that at least one is.

---

9. Relation to previous work in this paper series

The journal exhibited in §8.3 — a sequence of DSL sentences in the sender's program, recording the cross-actor send and its acknowledgment — required several structural preconditions to be a viable substrate. The journal had to be dense rather than porous: filled with operations, not with type-erased payloads. It had to record the operations with their parameter references intact, not with values inlined as literals. It had to maintain a discipline that separates immediate from deferred work, with a guardian for the boundary between them. Each precondition has been the subject of prior structural analysis in the present series.

Paper 1 introduces *porosity* — the representational sparsity that arises when domain state is recorded as serialized data structures rather than as programmatic operations. Anti-porosity is the design principle that the journal records what was said, not what was stored. Without that property, the entries the reader saw in §8.3 — `tell PurchaseConfirmed with ... to RewardEngine('rewards-1')` — could not be programmatic at all; they would be opaque payloads.

Paper 2 introduces *externalized parameters* as the structural precondition under which compilation, caching, and dense journaling become possible at all. Without externalized parameters, the journal could not record what was said with parameter references intact — values would be inlined as literals, or the script would lose its connection to the actor's symbol table. The tell sentence in §8.3 carries `@order`, `@date`, `@amount`, `@customer` as references rather than literals; this paper is what makes that representation possible.

Paper 3 introduces the *partition* between immediate and deferred work, with Reactions as the guardian of the boundary. Paper 3 names the *Causation* plane — `.Causation.Continue(...)` — as the third surface a Reaction may touch (Paper 3 §6.5), and records (Paper 3 §6.8) that the cross-actor case an earlier draft had listed as a limitation is resolved by that plane, while leaving the primitive's full treatment out of scope. The Reaction surface that Paper 3 establishes is the surface on which `tell` lives: the `.Causation.Continue(...)` body the reader saw in §8.3 is where the cross-actor send is permitted to appear. The present paper is the treatment Paper 3 deferred.

The series, taken together, defends a single architectural property under different framings:

> *Puppeteer preserves semantic continuity inside an actor. Tell preserves semantic continuity across actors.*

The first sentence is the joint contribution of Papers 1 through 3: the structural conditions under which the journal can serve as a program-level substrate for what an actor does. The second sentence is the contribution of the present paper: the cross-actor extension of that substrate that the reader saw exhibited in §8.3.

Read as a sequence, the series traces a single thread — the operation. Paper 1 establishes that operations precede state; Paper 2, that they carry their own parameters and can author themselves; Paper 3, that they can generate further operations as reactions; and the present paper, that an operation can be addressed, as an assertion, to another actor. *Tell* is the cross-actor extension of the journal-as-program substrate the earlier papers build, not messaging added to it from outside.

Papers 1–3 are self-deposited preprints on Zenodo (Rivera, 2026a, 2026b, 2026c) and have not undergone peer review, as is the present paper. This paper rests structurally on them — C2 presupposes the substrate Papers 1 and 2 establish (§3, §7), and the Reaction surface on which `tell` lives is Paper 3's (§8.2) — so it should be read as the latest in a preprint chain, not as resting on peer-reviewed foundations.

---

10. Conclusion

The actor model has, for fifty years, treated cross-actor causation as an operational concern of the runtime rather than as a construct of the program. The treatment was productive: actor systems became fault-tolerant, scalable, and reliable precisely because the separation between an actor's program and the message-passing layer was operationally effective. Out of that productivity grew the ecosystem of patterns analyzed in §6 and exhibited in §8.3 — saga orchestrators, choreography buses, distributed tracing, workflow engines. Each compensates, in its own way, for the program-level absence the reader saw in the journals.

The present paper observes that the absence is not entailed by the actor model. It is a contingent design decision adopted by the canonical sources and propagated through the lineage as its default frame — productive enough that the question of whether the send could be a statement of the program was seldom raised. The conditions under which the absence can be removed — locality of writes, causation as program statement, no external coordinator — preserve every structural commitment of the actor model. A primitive that satisfies the three conditions, whether named *tell* or otherwise, makes the cross-actor send a sentence in the sender's program; the apparatus that exists to reconstruct that narrative after the fact has nothing left to reconstruct. The message-passing layer itself remains — delivery stays operational — but the causal record no longer lives outside every actor's program.

The contribution of this paper is conceptual. The instantiation is the existence proof; the existence proof is the journal the reader saw in §8.3.

The question that opened the paper has, under tell, a different answer than the one the field gave by default: *where does the causal sentence live?* — in the sender's own program, as a sentence the journal records and replay re-runs.

Fifty years of convention are not fifty years of necessity. The actor model does not need to be replaced. Its commitments — autonomy, message-based communication, isolation — survive the reformulation intact. What changes is the historical interpretation of what must remain invisible to the program. The interpretation, named explicitly in §3, examined for contingency in §4, traced through its consequences in §5, shown to require an entire ecosystem of compensating patterns in §6, and contrasted with the alternative in §8.3, is the only thing the present paper asks the field to revise.

Appendix A. Code references

The labs cited in §8 are publicly available in the Puppeteer codebase. The references below are to the test suite of the framework's repository.

Cross-actor primitive surface — `Puppeteer/EventSourcing/Follower/`

File	What it shows	Cited in
`Reaction.cs`	Reaction class, Action terminator dispatch, replay-safe action execution	§7.2, §8.2
`Planes.cs`	The three plane types (`ProgramPlane`, `CausationPlane`, `MetadataPlane`) and their property accessors	§8.2
`ReactionEngine.cs`	Pattern matching surface, `.OnMatch(...)`, plane passthroughs	§8.2

Reproducibility lab — `labs/lab04-tell/`

The runnable harness that reproduces every journal exhibited in §8 ships with this paper (and in `paper04-data.zip`), built against the public runtime commit recorded under Code provenance. The framework's own end-to-end tests for the same scenarios live in the private fork and are not part of the public clone; this lab is the public, self-contained equivalent.

File	What it shows	Cited in
`Program.cs`	The harness, one method per scenario: the three-style side-by-side case study (orchestrated saga, event-driven choreography, tell — same domain, three journal locations of the joint history); G1 replay coherence; G2 cross-DC replication; G3 audit query; G4 tell-fate recovery across the crash window; and the negative gate for `tell` outside `.Causation.Continue(...)`. Prints each actor's journal and a PASS/FAIL line per assertion.	§8.2, §8.3, §8.4, §8.5
`LoyaltyDomainStubs.cs`	Domain-side stubs for `Seller`, `RewardEngine`, `Campaign` — kept minimal so the focus remains on the cross-actor mechanism	§8.1

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `37ad9cf` (2026-07-05). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:2b3ef9f68cf6e3a7a8e32ec73121122cc2ac74c5;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:37ad9cf44e7749f8fa0d2da8cc0c1093ad5a7e83
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

Appendix B. Bibliography

Agha, G. (1986). *Actors: A model of concurrent computation in distributed systems.* MIT Press.

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors* [Doctoral dissertation, Royal Institute of Technology (KTH)]. https://erlang.org/download/armstrong_thesis_2003.pdf

Bernstein, P. A., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Technical Report MSR-TR-2014-41). Microsoft Research.

Elnozahy, E. N., Alvisi, L., Wang, Y.-M., & Johnson, D. B. (2002). A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, *34*(3), 375–408.

Fidge, C. J. (1988). Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference* (pp. 56–66).

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, *30*(3), 611–642.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular actor formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI-73)* (pp. 235–245).

Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, *21*(8), 666–677.

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Johnson, D. B., & Zwaenepoel, W. (1987). Sender-based message logging. In *Proceedings of the 17th International Symposium on Fault-Tolerant Computing (FTCS-17)* (pp. 14–19).

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, *21*(7), 558–565.

Lightbend. (n.d.). *Akka core: Interaction patterns* [Akka documentation]. https://doc.akka.io/libraries/akka-core/current/typed/interaction-patterns.html (Internet Archive snapshot, 2026-02-16: https://web.archive.org/web/20260216052600/https://doc.akka.io/libraries/akka-core/current/typed/interaction-patterns.html)

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20404863

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20740697

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20792156

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

ewpage

Operations from the substrate

TL;DR

The canonical distributed-systems literature treats four operations — deployment without downtime, cross-datacenter replication, backup, and offline catch-up — as separate disciplines. Each has developed its own mature apparatus: blue-green and rolling updates for deployment; change-data-capture and log shipping for replication; snapshots and point-in-time recovery for backup; message queues and convergence protocols for offline operation. Systems that require all four must implement and operate all four independently.

This paper shows that the separation is contingent. Under the journal-as-substrate condition established in prior papers of this series — where the journal records operations rather than states, is written in the same language as the program, and consists of compact named entries — the four disciplines reduce to a single primitive.

Deployment is replay to the present head. Replication is the same replay at another site. Backup is the corpus held without replay. Offline operation is replay after a delay. The canonical apparatuses become structurally subsumed: each reconstructs, from outside the program, work the substrate already performs by construction.

Six laboratories measure the substrate under conditions where these equivalences hold. A version handover reaches the present head at 532k entries/s; cross-site transfer carries 33–96.7 bytes per event against payloads 3–9× larger; passive consumers converge across heterogeneous backends (FileSystem, MySQL, SQL Server) to identical in-memory state in 18/18 cells; local append latency runs ≈3× below co-located RDBMS engines under matched durability. §7 shows how four runtime primitives — each present for independent structural reasons — compose into the operations the canonical literature

implements as separate subsystems.

For a journaled program, deployment is replay, replication is sharing history, backup is copying the program, and offline operation is delayed replay.

---

1. Introduction

This paper makes an analytic theory contribution. It identifies a structural property whose absence the canonical distributed-systems literature has assumed without naming it as a single construct; derives the equivalences that follow when the property is present; and presents an instantiation showing it is realisable in production. The contribution is conceptual; the instantiation serves as an existence proof rather than as the substance of the claim. The genre is the one Gregor (2006) names *theory for analyzing* (Type I): it introduces a construct that lets the phenomenon be described and classified, with the measurements reported in §5 exhibiting the régime under which the construct holds rather than evaluating an artefact. The Hevner-style design-science *evaluation* of the artefact (Hevner, March, Park, & Ram, 2004) — a comparative systems study of throughput, availability, or fault tolerance against existing stacks — is a separate undertaking this paper does not attempt.

This is neither a systems paper proposing a new mechanism nor a survey of existing apparatuses. It does not introduce a deployment tool, a replication protocol, a backup format, or an offline-operation pattern. It examines a structural property that prior literature has implicitly assumed absent and shows that, under its presence, four operational disciplines of distributed systems reduce to a single primitive. In this genre, contribution is measured by the precision of the construct, the validity of the equivalences derived from it, and the realisability of the instantiation. Sections 5 and 7 exhibit the instantiation and report measurements establishing the régime in which the equivalences are observable.

Distributed systems must address four operational dimensions: release without downtime, cross-datacenter replication, recovery after data loss, and periods of disconnectivity (offline operation). The canonical approach treats these as separate disciplines, each with distinct apparatus and cost semantics: blue-green and rolling updates; change-data-capture and log shipping; snapshots and point-in-time recovery; message-queue buffers and convergence protocols. These bodies of practice are mature and widely adopted, yet they evolved largely independently. Systems that require all four must implement and operate four distinct stacks, each with its own engineering and cost.

The fragmentation is rarely questioned because it is familiar. But the four disciplines can be restated more simply: they vary when and where a program is read — at the present head and in place (deployment), at the present head at another site (replication), held at another site without reading (backup), and read after a delay (offline catch-up). They differ along two axes — temporal position and spatial position — over a single artefact. If that artefact were the program itself, recorded as operations rather than states, the four disciplines would be four readings of the same primitive: reading the program in order.

This paper names that artefact and the equivalences it admits. The construct introduced is the journal as the substrate of the program (in the sense established in Paper 2 §1.2: the pair of domain library and journal of invocations). Under the anti-porosity condition of Paper 1 and the separability condition of Paper 2, the journal records operations rather than states, is written in the same language as the program, and reduces each entry to a compact reference to a parametric verb and its arguments. The journal, so constituted, is not a record kept by the program; it is the program written out over time.

Four equivalences follow. Deployment is replay to the present head. Replication is the same replay at another site. Backup is the corpus held without replay. Offline operation is replay after a delay. The canonical apparatuses above are responses to the absence of the substrate condition; under its presence they become structurally subsumed by the substrate.

§2 traces the genealogy of the four disciplines as parallel bodies of literature. §3 names the assumption that produced the fragmentation. §4 states the substrate theorem and the four equivalences. §5 develops each equivalence and reports measurements exhibiting the régime under which it holds. §6 examines four canonical apparatuses — blue-green deployment, change-data-capture, snapshot backup, and message-queue buffer — and shows that each reconstructs, from outside the program, work the substrate already performs. §7 presents an instantiation in production. §8 relates the construct to prior papers in this series. §9 addresses counter-arguments. §10 concludes.

---

2. Genealogy: four separate operational disciplines

For decades, four operational concerns — deployment downtime, cross-datacenter replication, backup, and offline operation — have evolved as separate bodies of literature and practice. Each developed its own vocabulary, its own apparatus, and its own cost model. The genealogies traced below are not exhaustive; they are sufficient to establish that the canon treats these four as distinct disciplines, each with a dedicated mechanism, and that the resulting fragmentation of the operational stack has not been challenged within any of the four traditions.

2.1 Deployment downtime

The earliest deployment practice in production systems was the cutover: stop the running service, swap binaries, start it again. Downtime was accepted as the cost of release. The first patterned response was articulated by Fowler in *BlueGreenDeployment* (Fowler, 2010), which named a procedure in which "you ensure that you have two production environments, as identical as possible" and traffic is switched from one to the other at release time. The mechanism is environmental: two parallel deployments, one apparatus for routing, one for binary distribution.

The continuous-delivery literature ratified the pattern. Humble and Farley (Humble & Farley, 2010) generalized blue-green into a broader release-engineering discipline in which deployment is a build-pipeline output rather than an operator action, and downtime is a regression to be detected rather than a default to be accepted.

The modern instantiations are canary release (Sato, 2014) and the orchestration-mediated rolling update (Burns et al., 2016; Kubernetes, n.d.). Canary release introduces partial traffic exposure before full cutover, treating the deployed binary as a hypothesis tested against a fraction of production load. The Kubernetes rolling update replaces pods incrementally under a readiness-probe gate, making the deployment apparatus a property of the orchestrator rather than of the application.

A parallel strand of the deployment canon addresses schema and data migration. Ambler and Sadalage (Ambler & Sadalage, 2006) formalized *database refactoring* as a discipline in which every change to the data store is recorded as a forward-and-backward pair of scripts versioned alongside the application code. The pattern is instantiated by migration frameworks such as Rails Migrations, Liquibase (Liquibase, n.d.), and Flyway (Flyway, n.d.), which apply versioned scripts to the data store at release time. The migration apparatus is separate from the traffic-switching apparatus: it runs adjacent to — or before — the cutover, mutates the store in place, and produces a schema or data shape that the next binary version expects.

Across these mechanisms the shape is constant: a separate apparatus is built whose function is to absorb the change — of binary, of schema, or both — without interrupting the service. The apparatus is external to the program; the program is unaware that it is being replaced or migrated.

2.2 Cross-datacenter replication

Replication across datacenters originates in database engines. MySQL replication (MySQL, n.d.) established the canonical primary/replica topology in the 1990s: the primary writes a binary log of statements or row-level events, and replicas tail that log. The replication mechanism lives inside the database engine; the application program is unaware that replication is occurring.

The pragmatic maturation came with change-data-capture (CDC). Debezium (Debezium, n.d.) and Maxwell (Maxwell, n.d.) read the database's binary log and re-emit each change as an event on an external transport, typically Kafka. CDC inserts an apparatus between the database and downstream consumers: a process that watches state changes in one system and reconstructs them as event streams for others. The reconstruction step is necessary precisely because the originating system does not emit events — it emits state changes that CDC observes and translates.

The modern instantiations extend the apparatus topologically. Cassandra's multi-datacenter replication (Lakshman & Malik, 2010) embeds the replication topology in the storage engine itself, allowing writes accepted in any datacenter to propagate to peers under a configurable consistency level. Kafka MirrorMaker (Kreps, Narkhede, & Rao, 2011) generalizes the pattern for log-structured systems: a dedicated process consumes from one cluster and re-produces to another, with offsets tracking progress per topic.

Across these mechanisms the shape is constant: cross-datacenter replication requires a separate apparatus — engine-internal, engine-adjacent, or fully external — whose function is to observe the changes happening in one location and reproduce them elsewhere.

2.3 Backup

The backup canon predates the others. Agent-based backup (Bacula, n.d.; Veeam, n.d.) developed in the 1990s and 2000s as a discipline in which a dedicated process periodically reads the production data store and writes a copy to an archival medium. The agent is external to the application; backup occurs at the storage layer.

The pragmatic maturation introduced snapshot semantics. The WAFL filesystem (Hitz, Lau, & Malcolm, 1994) established the modern primitive: a point-in-time copy taken at the filesystem level, exploiting copy-on-write so that the snapshot is cheap and consistent with respect to the moment of capture. Database engines adopted the same pattern under the name *point-in-time recovery* (PostgreSQL, n.d.), in which continuous archival of the write-ahead log permits reconstruction of any committed state within the archived range.

The modern instantiations move the apparatus to object storage. Cloud-era backup (AWS, n.d.; Azure, n.d.) treats archival as a property of the storage tier rather than of a dedicated agent: lifecycle policies move data to colder tiers automatically, and recovery is a matter of retrieval from the archive.

Across these mechanisms the shape is constant: backup is a separate apparatus that produces a copy of state and is exercised at recovery time to reconstruct that state. The backup procedure observes the data store from the outside; the program being backed up does not participate.

2.4 Offline operation

The offline-operation canon is the most heterogeneous of the four, because it spans network-layer, transport-layer, and application-layer treatments of disconnection. The early precedent is store-and-forward messaging: UUCP and, later, SMTP queue undeliverable messages locally and retry until the remote endpoint is reachable. Disconnection is absorbed by a queue at the boundary of the system.

The pragmatic maturation generalized the boundary queue into a programmable apparatus. RabbitMQ (RabbitMQ, n.d.) introduced dead-letter exchanges to capture messages whose delivery is impeded, deferring them for retry or human inspection. Kafka (Kreps, Narkhede, & Rao, 2011) generalized the pattern further: consumer offsets are explicit, durable, and per-partition, so a consumer that goes offline resumes from its last committed offset when it returns. The queue is no longer a remediation mechanism for failed delivery; it is the substrate over which the consumer's progress is tracked.

The modern instantiations carry the pattern into application-level state. Eventually-consistent stores (DeCandia et al., 2007) treat each replica as a local source of truth that converges with peers when connectivity permits. The Dynamo paper established the design vocabulary — vector clocks, hinted handoff, read-repair — that subsequent systems — Cassandra, Riak, and Redis replication (Redis, n.d.) — instantiate in varying degrees.

Once again, the shape is constant: offline operation requires a separate apparatus — a queue, an offset, a convergence protocol — whose function is to absorb the period during which the system cannot reach its counterpart.

2.5 Where the operational complexity lives

The four disciplines surveyed above can be compared along a single matrix. Each row names a discipline; each column names a dimension of the apparatus the canon developed to handle that discipline.

Discipline	Where the complexity lives	Apparatus	Cost
Deployment	Outside the program, in the release pipeline	Two-environment switch (blue-green); incremental pod replacement under a readiness gate (rolling update); fractional traffic exposure (canary); versioned schema and data migration scripts (Liquibase, Flyway, Rails Migrations)	Duplicate production environment; orchestration layer; manual or automated cutover procedure; forward-and-backward migration pairs
Cross-datacenter replication	Inside the storage engine, or in an adjacent CDC layer	Binary-log shipping (engine-internal); change-data-capture (engine-adjacent); multi-DC topology (engine-aware)	Replication lag; reconstruction overhead in CDC; consistency-versus-availability configuration surface
Backup	Outside the program, in a dedicated archival pipeline	Agent-based copy; filesystem or engine snapshots; continuous log archival (PITR); object-store lifecycle	Recovery-time objective and recovery-point objective; storage-tier cost; periodic exercise of the recovery procedure
Offline operation	At the boundary between system and counterpart	Persistent queue with retry semantics; durable consumer offsets; convergence protocol over replicated stores	Buffer sizing; offset management; eventual-consistency reasoning at the application layer

Three observations follow from the table. The location of the complexity differs across rows: the deployment apparatus lives in the release pipeline, the replication apparatus lives inside or beside the engine, the backup apparatus lives in a separate archival pipeline, and the offline-operation apparatus lives at the system's boundary. The mechanism in each row was developed without reference to the others: the blue-green literature does not cite the CDC literature; the snapshot literature does not cite the consumer-offset literature. The four literatures evolved in parallel, each solving what it believed to be a different problem. And the cost in each row is paid separately: a system that wishes to be zero-downtime, cross-datacenter, recoverable, and tolerant of disconnection budgets independently for each of the four columns.

The canon, taken as a whole, presents these four as four different problems requiring four different apparatuses. §3 names the assumption that made this fragmentation appear necessary.

---

3. The assumption named

What §2 shows to be continuous across four separate bodies of literature can be stated as a single structural assumption:

> **Operational concerns of a system are addressed by apparatus around the program rather than by the program itself.**

Across the four disciplines, the response to each operational concern is the introduction of a mechanism external to the program. Blue-green deployment builds a parallel environment to absorb the cut-over. Change-data-capture inserts a translation layer between stores to carry replication. Snapshot agents observe the data store from outside and write captures to archival media. Message-queue brokers hold messages at the system boundary to absorb periods of disconnection. In each case, the mechanism is not a sentence of the program it serves. It observes the program from outside, maintains a parallel artefact, and is operated independently of the program's own execution.

The assumption embedded in these practices is that operational concerns require such external apparatuses, each developed independently to address a specific condition.

The alternative this assumption leaves unexplored is that the program might address these concerns by varying how it is read — by changing when or where the program is replayed, without leaving the program's own representation. Under this alternative, the four disciplines become four readings of a single primitive. The question becomes structural: under what condition would the program admit such readings?

§4 names that condition and the four readings it permits. The apparatuses surveyed in §2 remain appropriate responses in their original context; what this paper shows is that they arise from the absence of the substrate condition, which, if present, subsumes the need for each apparatus.

---

4. The substrate theorem

4.1 The substrate

The first two papers in this series established two structural properties of the journal that this paper takes as preconditions. Paper 1, §3 named *anti-porosity*: the journal records the operations of the program rather than the states they produce, and admits no representational sparsity at the boundary between domain code and persisted form. The journal is therefore *code-as-data* — entries are executable sentences in the same language the program is written in, in the narrow *script-form* sense Paper 1 establishes rather than Lisp's strong homoiconicity — and dense — every effect that crosses the boundary is named in that language, not summarised as state delta. Paper 2, §1.2 and §3 named *separability*: programs that are parametric in their arguments admit a compiled form in which the operation's body is cached under an identifier and the journal entry reduces to that identifier plus the call's arguments. Together the two conditions yield a journal whose entries are compact, named operations rather than serialised states.

Under these two conditions the journal is not a record kept by the program; it is the program written out over time. Each entry is a sentence the program has uttered; the journal is the corpus of those sentences in the order they were uttered. The naming move of this paper is to call this artifact *the substrate of the program*: the journal does not store the program's outputs, it constitutes the program's existence over time. The program and the journal are the same artifact under two readings — one synchronous, one diachronic. Under the synchronous reading, the runtime is the program. Under the diachronic reading, the journal is the program.

One further property, established in Paper 1 §4.2, is what makes the substrate *replayable without divergence*, and every equivalence of §4.2 rests on it: the arguments in an entry are *values captured at execution time, not expressions re-evaluated on read.* A nondeterministic or state-derived argument — a generated identifier, a clock reading — is resolved once, on the writing path, and the resolved value is frozen into the entry; a later read reuses that value rather than recomputing it. *(Verification: the capture-at-execution of the `Eval` modifier, Paper 1 §4.2 — on first execution the value is captured as a literal-assigning script `name = (type)(value);` persisted in the entry, and on replay that literal is re-assigned rather than recomputed; `Parameter.cs:33–36, 163–224, 228–247`.)* Were an argument re-evaluated on read, replaying it would diverge from the run that wrote it, and deployment, replication, backup, and offline catch-up would each reconstruct a different program than the one recorded.

4.2 The theorem statement

The theorem of this paper can be stated as a single proposition.

> *Given a program whose execution is recorded as a journal satisfying the anti-porosity condition of Paper 1 and the separability condition of Paper 2 — that is, a journal that is code-as-data, dense, and made of compact named operations rather than serialised states — the following four equivalences hold.*

E1 — Deployment is replay. A new version of the runtime, started from the substrate, reconstructs the program by reading the journal sequentially and applying each entry. Nothing about the new version's instantiation differs in kind from what the previous version did at every entry up to that point; the act of bringing a process to a state in which it can serve requests is the act of replaying the substrate up to its present head. Deployment therefore differs from steady-state operation only in the position of the read cursor over the same corpus.

E2 — Replication is sharing history. A second instance of the program, running on a different machine or in a different site, reconstructs the program by reading the same sequence of entries the first instance wrote. The act of replicating is the act of transmitting the substrate from one site to another and replaying it at the destination; no state extraction, no schema mapping, and no change-data inference enter the act. Replication therefore differs from deployment only in the site at which the same corpus is replayed.

E3 — Backup is copying the program. A consumer that reads entries from the substrate and persists them locally — without instantiating the runtime that would replay them — holds a copy of the program in the form in which it was written. The act of backing up is the act of duplicating the substrate; recovery is the act of replaying the duplicate. Backup therefore differs from replication only in whether the destination chooses to replay the same corpus or merely to hold it.

E4 — Offline operation is delayed replay. A consumer that is unreachable when entries are written, and that catches up by reading them later from a persistent buffer or directly from the source, applies the same replay to the same sequence as a consumer that received the entries in real time. The act of operating offline is the act of replaying the substrate after a delay; the delay does not enter the program, only the cursor of the consumer. Offline operation therefore differs from replication only in the elapsed time between the write and the read of the same corpus.

The four equivalences are not parallel restatements of a single mechanism. Each addresses one classical operation of distributed systems (§2) and shows that the operation, as the canon presents it, dissolves into a single primitive — replay

— once the substrate's properties are taken as given. E1 addresses the time axis (a new version replacing an old one in place); E2 addresses the space axis (a new site running the same program); E3 addresses the storage axis (a consumer holding the corpus without running it); E4 addresses the connectivity axis (a consumer reading later than the writer wrote). The four together exhaust the operational variations that the canonical literature in §2 treats as separate disciplines. The operational specifics of how each replay is realised — handoff, transmission, passive ingestion, catch-up — are taken up in §5; §4 confines itself to the structural claim.

4.3 Apparatus: the substrate and its four replay arrows

The shape of the theorem can be rendered as a simple spatial arrangement. No new content is introduced; the arrangement makes the four equivalences of §4.2 visible at a glance, with the substrate at the centre and the four conditions arrayed around it.

The substrate — the journal as program — sits at the centre. From it radiate four equivalences, each corresponding to one classical operational discipline:

- **E1 — Deployment**: replay to the present head.
- **E2 — Replication**: replay of the same corpus at another site.
- **E3 — Backup**: the corpus held without replay.
- **E4 — Offline operation**: replay after a delay.

The vertical pair (E1, E4) varies the temporal position of the replay — at the present head, or after a delay. The horizontal pair (E2, E3) varies the spatial position of the corpus — replayed at a different site, or merely held at one. The canonical literature in §2 names four disciplines by looking at the destinations; viewed from the substrate, only replay occurs, under four conditions on when and where.

---

5. Consequences: four equivalences as variants of replay

Each of the four sub-sections that follow develops one equivalence of §4.2. The sub-section opens with the operational realisation of the equivalence — the structural intervals through which the program is read under that variant — and follows with a measurement that exhibits the régime in which the equivalence holds. Each lab is designed to isolate the structural content of its equivalence, not to optimise throughput; performance figures are reported as evidence of the régime, not as benchmark claims.

5.1 E1 — Deployment is replay

§4.2 stated the equivalence: a new version of the runtime, started from the substrate, reconstructs the program by reading the journal sequentially and applying each entry. §5.1 develops the operational realization of this equivalence — the handover window during which an instance about to become authoritative reads the substrate up to the present head, and the precise sense in which that window is replay and nothing else.

The handover window

A new version of the runtime begins as a *follower*: a process that reads from the same substrate the live instance writes but does not yet accept requests. The follower's task is to reach the present head of the substrate. Once it has — and once a brief synchronization tail has cleared — the follower is admitted as authoritative and the previous version is released. Three structural intervals comprise the window:

1. **Bulk replay.** The follower opens the journal from entry zero (or from a checkpoint, if the substrate is partitioned with one) and applies each entry to its in-memory state in the order written. The state reconstructed at the end of bulk replay equals, entry-for-entry, the state the previous version reached by writing those same entries.
2. **Lock and synchronization tail.** The previous version is paused briefly so that no further entries are written. The follower reads the small residue of entries that arrived between the start of bulk replay and the pause — the *handover tail* — and applies them. The state at the end of the tail equals the state the previous version held at the moment it was paused.
3. **Gate flip.** The follower is admitted as the authoritative instance. From this point on the substrate's writer is the new version; the previous version is released.

The operational specifics of each interval are realized by a single primitive in the instantiation (§7.1 develops the gate mechanism); what §5.1 establishes is that the *content* of the window is replay. No state extraction, no schema migration, and no inference step enters the operation. The new version becomes authoritative by reading what the previous version wrote, in the order it was written, until the read cursor reaches the position the previous version had left it at.

What the substrate measures

L1 (lab `paper05-lab1-redblack-replay-time @ 99e8202`) times the handover window over four journal sizes (N $\in$ {1k, 10k, 100k, 1M} entries) under the compact-action régime named in Paper 2. The measurement brackets the inner bulk-replay interval and the handover tail separately with the public `Stopwatch` API — around `follower.Start(asFollower: true)` and the lock/synchronisation tail — so that the structural content of the window — the act of replay — is reported in its own right rather than aggregated with orchestration overhead.

Under the compact-action régime with `AlwaysCompiled` compilation, the substrate replays N entries at a sustained rate of **532 thousand entries per second** (p50, N = 100 000), completing the bulk replay in **187.8 ms** (p95 = 190.6 ms). The rate is reached as the runtime amortises between N = 10 000 and N = 100 000 entries and **continues to climb through the N = 1 000 000 anchor to 1.76 million entries per second** — fixed startup cost spread over more events. The handover tail is below 0.3 ms at every N measured. **Bulk replay accounts for more than 99.8 % of the deploy window in every cell.** The replay rate is sustained — indeed it improves — across a tenfold journal range; were it linear in expanded state rather than in compact-event count, the rate would fall as state grew. It does not.

Three properties of the measurement deserve naming. First, the substrate cost of deployment is bounded by compact-event count, not by expanded state — replaying ten times the journal takes no more than ten times the time, and here less: the per-event rate climbs as fixed startup cost amortises over more entries. Second, the orchestration tail is empirically negligible at every measured size; the deploy window *is* the replay window to within a fraction of one percent. Third, the régime under which this holds is the régime Paper 2 already named the canonical one: parametric verbs encoded as `ActionId` plus arguments, not literal scripts. A journal of literal scripts would replay slower in proportion to script length; this paper concerns the compact-action substrate named in Paper 2.

Apparatus

The handover window can be rendered as a sequence between four roles. The leader is the live instance accepting writes; the journal is the substrate; the follower is the new instance approaching the present head; the gate is the predicate that decides which instance is authoritative at any moment.

```
sequenceDiagram
    autonumber
    participant L as Leader (authoritative)
    participant J as Journal (substrate)
    participant F as Follower (new version)
    participant G as Gate

    Note over F,G: Follower starts; gate closed for follower
    F->>J: open from entry 0 (or checkpoint)
    loop bulk replay
        J-->>F: next entry
        F->>F: apply entry to in-memory state
    end
```

```
Note over F: follower at near-head
F->>L: request pause
L->>L: stop accepting writes
loop handover tail
    J-->>F: residual entry written before pause
    F->>F: apply
end
Note over F: follower at present head
F->>G: open for follower
G-->>L: close for leader
Note over L: leader released; follower authoritative
```

The diagram contains no operation other than reading from the journal and applying. The leader writes nothing between pause and release; the follower reads from the same artifact at the same offsets the leader was about to read had it continued. The gate's flip is bookkeeping: it nominates which instance is authoritative; it neither produces nor consumes substrate content.

One canonical exception

The equivalence developed here covers the in-process deployment of a single program. One canonical case lies outside its scope: a coordinated cut-over of a producer/consumer pair on an external message broker (typically Kafka) when the message format itself changes between versions. There the two endpoints must release together, because the substrate they share is the broker's topic — a contract between independent programs — rather than the journal of either. The construct of this paper does not deny this case; it narrows its scope. Within a journaled program, deployment is replay; across two journaled programs that share a broker topic with a changing schema, the cut-over remains a separate concern. Paper 3, which develops the partition primitive this paper's Reactions build on, named this exception in its claim 8 on in-actor continuity; it is preserved here without amendment.

To the next equivalence

§5.1 has shown that bringing a new version of a program to a state in which it can serve is replay of the substrate up to the present head, and that the orchestration around that replay is residual at every measured size. The equivalence varies the position of the read cursor in time — at the present head of the same substrate. §5.2 varies the position of the substrate in space: a second instance, at another site, reconstructs the same program by reading the same sequence of entries the first instance wrote.

5.2 E2 — Replication is sharing history

§4.2 stated the equivalence: a second instance of the program, running on a different machine or in a different site, reconstructs the program by reading the same sequence of entries the first instance wrote. §5.2 develops the operational realization in two parts. First, that the bytes streamed between sites are structurally dense — the substrate's wire form carries named operations, not serialized states, by construction. Second, that a consumer reading those bytes reaches the same program as the writer by replay alone, without recourse to a coordinator. The transport across which the bytes travel is treated as a witness — an off-the-shelf service that satisfies the substrate's requirement on consumer-pull delivery.

The compact wire

Paper 2, §2.3 (*dense journaling as reference*), established that under separability the journal entry of an invocation reduces to a reference to the parametric verb's definition plus the arguments of the call. The verb's body is not present in the entry; it lives once, named, in the corpus's vocabulary. Replication across datacenters is the transmission of those entries, not of states; the wire encodes the program as it was uttered, not as it was applied.

The entries so transmitted are the domain program: operations and their argument values. Operational metadata *about* the program — the identity, origin, or request context under which an operation was recorded — is, where a deployment maintains it, a distinct channel keyed to the same entries, not part of the entries themselves. It rides the same substrate mechanisms (replicated alongside the corpus, copied with the store), but it is neither a domain value nor an operation, and the equivalences of this paper concern the program the journal constitutes, not the audit envelope a deployment may keep around it. The distinction matters at the cross-actor boundary, where the two channels part ways (Paper 4, §8.2): a value the receiver's domain needs crosses as an argument; the sender's envelope does not cross at all.

L2 (lab `paper05-lab2-bytes-per-event` @ `99e8202`) instruments the codec on the substrate's wire surface, comparing the compact encoding (`ActionId` + parameters) against the literal encoding (script DSL + parameter assignments) at three tiers of verb richness, with and without an opportunistic `gzip` on the literal form.

Bytes-per-event in compact form is 33 at the trivial-arithmetic tier (68-byte script body), 33 at the branching-arithmetic tier (99-byte body), and 96.7 at the production-shaped tier (681-byte body). The literal form at the same three tiers is 118, 150, and 910.7 bytes. The ratio between literal and compact rises from **3.6× at tier 1** to **9.4× at tier 3**. Applying `gzip` to the literal form closes the gap only to 3.2× / 3.9× / **4.0× at tier 3**. The structural saving cannot be recovered by an opportunistic compressor, because the script body is not present in the compact form at all — it lives once in the definition record (909 bytes at tier 3, amortized to ≈0.91 bytes per invocation over 1 000 calls) and is referenced by a 4-byte `ActionId` thereafter.

The cross-validation against Paper 2 Lab 4 sharpens the claim. Tier 3's compact payload is 67.7 bytes here; Paper 2 Lab 4 measured 67 bytes for the production verb that the synthetic tier-3 stand-in calibrates against. The two numbers agree within rounding. At full production scale, Paper 2 Lab 4 reports the literal-to-compact ratio reaches 20× over a thousand invocations of a real ticket-purchase verb. The 9.4× headline here is a conservative lower bound under the lab's literal-projection; the production figure is reported in Paper 2 Lab 4.

Three properties of the measurement deserve naming. First, density is structural rather than algorithmic — compression operating after the fact cannot recover what naming achieved by construction. Second, the saving grows with verb richness; a domain whose operations are named transitions (Paper 2 §3) replicates per event at a width that does not scale with the operation's internal complexity. Third, replication across datacenters, in this régime, is event streaming over a substrate that does not carry state by construction — the wire is the program as uttered, not as expanded.

The push-to-pull inversion

The substrate is written by the producer and read by the consumer; the role of the transport between them is to hold the corpus durably and to deliver entries at the consumer's pace. The producer does not know the consumer's rate; the consumer's progress is measured against the substrate's own offsets, not against any signal from the producer. A consumer that lags absorbs the lag in its own cursor; the producer's write path is unaffected.

This inversion is not a property of any particular delivery service. It is a property of the substrate. The producer's journal is the source of record; any consumer downstream is a reader of that record. An off-the-shelf webhook delivery service satisfies the consumer-pull requirement without adding any structure the substrate did not already license: it accepts an inbound POST from the producer's local journal callback, holds it durably, and offers it to subscribers at the rate they fetch. The fact that an off-the-shelf service suffices is the relevant observation; the service is a witness to the substrate's structural sufficiency, not an extension of it.

This is the *passive-consumer* arrangement: the consumer reads from a durable buffer at its own pace, and no party coordinates the two sides. It is distinct from a synchronous replicated-actor topology — one instance authoritative, others following under an elected role — which is a different mechanism with different coupling and is not the arrangement measured here. E2 is the claim that the *passive* path needs no coordinator, not that every replication topology dispenses with one.

The shape can be rendered between five roles: the primary instance at site A, the primary's journal, the delivery service, the consumer at site B, and the consumer's journal.

```
sequenceDiagram
```

```
autonumber
participant P as Primary (site A)
participant Ja as Journal A (substrate)
participant T as Delivery (off-the-shelf, e.g. SVIX)
participant C as Consumer (site B)
participant Jb as Journal B (substrate)

P->>Ja: append entry n
Ja->>P: callback (entry n written)
P->>T: post entry n
Note over T: held durably; offered at subscriber's pace
C->>T: fetch next
T-->>C: entry n
C->>Jb: append entry n
C->>C: apply entry to in-memory state
```

The diagram contains no synchronous coupling between producer and consumer. The producer's append-and-post is local; the consumer's fetch-and-apply is local; the durable buffer between them is the delivery service. The substrate at A and the substrate at B contain the same entry at the same offset — the two journals are bit-equal under the measurement examined next.

The symmetric consumer

§4.2 stated that replication "differs from deployment only in the site at which the same corpus is replayed." The structural content of this claim is that the consumer at site B, running the same binary as the producer at site A, derives the same program from the corpus it reads as the producer derived from the corpus it wrote. No coordinator is required to synchronise the two sites; the substrate is the coordinator. This property removes the need for any external coordinator between sites: coordination is implicit in the shared corpus rather than enacted by a privileged runtime.

L3 (lab `paper05-lab3-inproc-symmetric` @ `99e8202`) measures this property at the level of Reactions — the partition primitive of Paper 3 — under two terminators, `Emit` and `MarkAsSkip` (Elide). Two instances of the same actor binary consume the same event stream; the lab compares, at the byte level, both the callback tuples each instance's Reactions produce and the journal segments each instance writes. Across 6 cells (N $\in$ {100, 500, 1 000} $\times$ K = 2 repetitions), **all 6 cells produce 0 callback byte diffs and 0 journal-segment byte diffs**. The instance at site B reconstructs the Reaction output entirely from the journal prefix it reads.

The discipline that makes this work without a coordinator is a property of the program, not of the runtime. A Reaction's guard refers only to data resident in the corpus — in L3, the arguments the program journaled when it invoked each operation, frozen at write time by the anti-porosity of Paper 1. Whatever a Reaction at B needs to evaluate a guard travels in the corpus the program writes; whatever the program did not write there, no remote evaluator can see. The substrate carries the program, and the program carries the contract on which its consumers depend.

Paper 4, §5.3 and claim 10, named the closely related property for cross-actor causation: the journal record of the act of speaking from one actor to another is what survives cross-datacenter replication, where canonical apparatuses (CDC, log shipping, message-queue replication) reconstruct state and lose the causal chain that produced it. §5.2 of the present paper completes the picture from the substrate side: the journal record is what the consumer reads, and the consumer's Reactions derive from it without needing the producer's runtime to be reachable.

Two caveats are worth naming, in the spirit of full disclosure. L3's transport is in-process — a channel between two instances in the same process — and its measurement caps at N = 1 000 entries (beyond which the harness becomes memory-bound). The symmetric property is structurally insensitive to the transport's distance and to N: a Reaction's match path is per-entry, and the journal-segment diff at N = 1 000 is identically zero across all measured cells. But the lab demonstrates the property in vitro; full cross-datacenter delivery is the operational mode realised in §7.2, and the higher-N regime is bounded by the consumer's append throughput rather than by anything the matcher does.

What §5.2 establishes, and what §5.3 develops next

§5.2 has shown that replication, in the substrate's terms, is the streaming of named operations over a wire whose density is by construction; that the transport between sites can be an off-the-shelf consumer-pull service; and that a consumer running the same binary reaches the same program as the producer by reading the corpus it shares. The equivalence varies the spatial position of the corpus while preserving replay as the means of reconstruction.

§5.3 varies the spatial position of the corpus without reconstructing the program at the destination. A consumer that reads the substrate and persists it locally — without instantiating the runtime that would replay it — holds the program in the form it was written. Backup, in this view, is replication arrested at the moment of holding rather than carried through to replay.

5.3 E3 — Backup is copying the program

§4.2 stated the equivalence: a consumer that reads entries from the substrate and persists them locally — without instantiating the runtime that would replay them — holds a copy of the program in the form in which it was written. The canonical literature on backup (§2.3) treats backup as a captured state, archived periodically by an external apparatus and replayed only on recovery. The equivalence stated here re-reads the operation: for a journaled program, backup is the program materializing in another substrate, and the destination's choice is whether to hold the corpus or replay it.

The structural argument

The equivalence follows in four steps from the substrate condition of §4.1:

1. If the journal is the program (Papers 1 and 2), the artifact whose duplication preserves the program is the journal.
2. Duplication of the journal is the streaming of named entries to a destination substrate — the program materializing there, sentence by sentence, in the order it was uttered.
3. The destination need not instantiate the runtime to receive the corpus; it need only persist what arrives. The consumer is a substrate, not a process.
4. When and if the destination chooses to replay, the corpus reconstructs the program. The choice between *holding* and *replaying* is independent of the act of materializing.

Three symmetries

Backup, in this view, is positioned by its relation to the three other equivalences of §4.2. The position is fixed by what the destination commits to, not by any new operation.

- **Backup and replication.** Replication (§5.2) is the destination streaming the corpus and replaying it; backup is the destination streaming the corpus and holding it. The same protocol governs both. Replication is backup carried through to replay; backup is replication arrested before replay.
- **Backup and deployment.** Deployment (§5.1) is replay at a different time of the same site, ending with the new instance admitted as authoritative; backup is replay-eligible material held at a different site, with no instance ever admitted as authoritative there. The same protocol reads the corpus forward; what differs is the gate flip at the end — present in deployment, absent in backup.
- **Backup and offline operation.** Offline operation (§5.4) is the consumer reading the corpus with a delay between write and read but eventually catching up and serving; backup is the consumer reading the corpus and never serving. The same protocol delivers entries; what differs is whether the consumer ever takes a turn at serving requests.

The three symmetries are not corollaries discovered after the fact. They follow from the substrate condition directly: each of the four equivalences is read off the substrate by varying one of two axes — when the read happens, and what the consumer commits to once it has read — and the four exhaust the combinations. §4.3 named this arrangement.

What the substrate measures

L4 (lab `paper05-lab4-passive-consumer` @ `99e8202`) instruments the materialize cycle on three storage backends under the compact-action régime of L1. The dataset spans 3 backends × 3 journal sizes × 2 protocol layers × K = 2 measurement repetitions = 36 cells, plus 3 catch-up cells.

Under régime N = 100 000 compact entries, the destination journal converges at ≈ **2 150 events/sec on FileSystem**, ≈ **952 events/sec on MySQL**, and ≈ **766 events/sec on SQL Server (SQL Edge stand-in)**. The wire round-trip itself accounts for **less than 0.6 % of the cycle** at every cell; the substrate cost is the destination's append rate, not the operation. A replica actor instantiated over the destination journal — a verification step, not part of the backup operation — reaches the same in-memory state as the primary in **18 of 18 measured cells across the three backends and three journal sizes**.

The independence from backend is the empirically significant property here. The equivalence is not a feature of any storage technology; it holds wherever the journal can be appended to.

The destination as cursor

The destination's three reading regimes — near the head of the substrate, paused after an interruption, or newly instantiated after the primary has run for some time — share a single protocol. Each is a cursor on the substrate, read forward from its present position to the present head. Catch-up after a retention loss is not a separate mode; it is the same read, started further back. L4's catch-up cells confirm this: the catch-up throughput matches the steady-state throughput within stochastic variation on each backend. The substrate does not contain a distinction between *current* and *recovering*; it contains a sequence of entries, and each consumer holds a cursor over it.

Caveats

Two caveats are worth naming, in the spirit of full disclosure. First, L4's transport between primary and destination is in-process — a local proxy, not a real cross-datacenter wrapper; the network latency × number of round-trips a cross-datacenter deployment adds is bounded but not measured here. Second, L4 measures the *passive* half of the equivalence — the destination consuming the corpus — not the act of promoting a destination to authoritative status; that promotion is the E1 case §5.1 already developed, and is structurally available to any destination that holds the corpus to a chosen EntryId.

The operational realisation of E3 — the modes under which a destination starts, how the program declares which entries materialize where, the transport that carries them, and the dimensions along which the construct scales (heterogeneous backends, multi-destination fanout, per-datacenter topology) — is developed in §7.3.

To the next equivalence

§5.3 has shown that backup is the program materializing in another substrate; that backup is positioned by its relation to the three other equivalences along two structural axes — when the read happens, and what the consumer commits to once it has read; and that the corpus convergence rate on heterogeneous backends is independent of the construct. The equivalence varies the consumer's commitments while preserving replay as the means by which the program is reconstructed if and when needed.

§5.4 varies a different axis: the delay between when the substrate is written and when it is read. Offline operation, in this view, is replay after a gap — the consumer reads the same corpus the producer wrote, but the read trails the write by an arbitrary interval.

5.4 E4 — Offline operation is delayed replay

§4.2 stated the equivalence: a consumer that is unreachable when entries are written, and that catches up by reading them later from a persistent buffer or directly from the source, applies the same replay to the same sequence as a consumer that received the entries in real time. The canonical literature on offline operation (§2.4) treats the buffer as an apparatus added at the system's boundary — a message queue, a durable offset, a convergence protocol — whose function is to absorb the period during which the system cannot reach its counterpart. In the regime described here, no such apparatus exists: the substrate itself is the durable buffer, and offline operation reduces mechanically to cursor advancement over the same corpus.

If the producer writes to a substrate that is local — local to the producer's process, locally durable — then there is no point at which the producer needs to know whether any consumer is reachable. The producer's write is to its own substrate. A consumer reading from that substrate, whether it reads in real time or after an interruption, holds a cursor over the same corpus; when the cursor advances, the consumer applies the entries between the old position and the new one. The buffer of the canonical literature is therefore not added; it is the substrate itself.

Offline as the limit of replication

§5.2 developed replication as the consumer reading the corpus the producer wrote, at a different site, with the transport between them holding entries durably until the consumer fetches them. §5.4 is the limit case: the consumer's read trails the write not by transport latency but by an arbitrary interval — minutes, hours, or longer. The protocol is identical. The act of catching up is the act of advancing the cursor over the entries that accumulated in the meantime.

The same mechanism applies in the reverse direction. A producer whose canonical remote store is unreachable continues writing to its local substrate; the remote store, when reachable again, is a consumer that fell behind. The role of *producer* and *downstream store* is asymmetric only in steady-state framing; structurally, both are cursors over the same corpus.

What the substrate measures

L5 (lab `paper05-lab5-offline` @ `99e8202`) measures the offline equivalence by configuring an actor's substrate as a local persistent journal with an asynchronous flush to a remote canonical backend (MySQL and Azure SQL Edge). The remote backend is then partitioned for a measured interval (~5 s of write volume); the partition is resolved; the backlog drains.

Three properties of the measurement deserve naming.

1. **Decoupling of write latency from the remote.** Under online operation, the primary appends at **516 µs p50 against MySQL** and **429 µs p50 against SQL Edge — 6.0× and 2.7× faster** than the unbuffered direct path. The producer's lock release time is bounded by the local substrate's append, not by the remote's commit latency.
2. **Partition latency is not worse than online latency.** During the partition, the buffered primary's per-append p50 is 333 µs on MySQL and 339 µs on SQL Edge — the same régime as online; the write lock never traverses the network. The substrate's local persistence is what makes this hold; the absence or presence of the remote does not enter the write path.
3. **Catch-up is linear in the backlog.** On reconnect, the accumulated entries drain at the remote backend's steady-state ingest rate — ≈ 296 events/sec on MySQL, ≈ 500 events/sec on SQL Edge — with **zero events lost** across the measured cells. Catch-up is not a separate mode; it is the remote's cursor advancing from its last position to the present head.

Three caveats are worth naming, in the spirit of full disclosure. First, L5 partitions the remote by stopping its container — a realistic partition mode but not the only one; half-open connections without container death would exercise the connection-timeout path differently and are not measured here. Second, the partition-phase latency being at or below the online phase is partly an artifact of single-host scheduling — the asynchronous flush thread is idle during partition, freeing CPU for the writer; in production with separate threads on separate cores, the artifact shrinks, but the structural property — the write path is independent of the remote's reachability — holds either way. Third, *what* drains on reconnect is journal entries to the canonical backend — the remote's cursor advancing to the present head; the measured path does not re-emit cross-actor *tells*. Tell re-delivery on a replay-style catch-up is a separate path belonging to the tell primitive (Paper 4), with its own delivery semantics and not exercised here; the *zero-events-lost* result is a property of journal drain to the backend, not a claim about tell re-delivery on replay.

The operational realisation of E4 — the configuration that enables the local substrate as the primary's source of truth, the asynchronous component that flushes to the canonical backend, and the catch-up path on reconnect — is developed in §7 alongside the other operational details.

To the next equivalence

§5.4 has shown that offline operation is replay after a gap, and that the gap is absorbed by the substrate's own local persistence rather than by a separate apparatus. The four equivalences of §4.2 are now developed.

§5.5 turns to the cost that underlies all four: replay against a local substrate is cheap because append against a local substrate is cheap. The latency budget that supports the four equivalences is examined as a single property of the substrate, not as a property of any one operation.

5.5 The latency budget

The four equivalences of §4.2 reduce four classical operational disciplines to a single primitive: replay. Replay is the act of reading the substrate forward from a cursor and applying each entry. Its cost is the cost of reading and applying entries. For a substrate that is read often and written often, the cost of *writing* entries — appending to the journal — is the cost basis that supports the family of replay-based operations. §5.5 examines that basis.

The structural claim that underlies E1–E4 is this: every replay-based operation reduces to advancing a cursor over N entries, and therefore inherits the substrate's per-entry append latency as its dominant cost term. The actor's apply cost is identical across all regimes because it operates on the same in-memory state. The only term that varies between regimes is the cost of appending entries to the substrate.

The relevant comparison is between two regimes for the substrate's persistence. Under the regime this paper develops, the substrate is a local journal whose persistence is one *fsync*-bounded append per entry. Under the canonical regime (§2), the substrate's role is played by an RDBMS whose persistence is one commit-bounded *INSERT* per entry. The latency gap between the two regimes propagates linearly to the cost of deployment, replication, backup, and offline catch-up. The four operational disciplines therefore share a single cost basis: the substrate's append latency.

What the substrate measures

L6 (lab `paper05-lab6-latency-budget` @ `99e8202`) measures per-entry append latency on three local backends under a one-event = one-durable-commit régime: a local FileSystem journal, a co-located SQL Server (Azure SQL Edge stand-in), and a co-located MySQL. The dataset spans 3 backends × K = 5 runs × N = 100 000 entries per backend = 1.5 million samples, with the first 1 000 appends per cell discarded as warm-up.

Backend	Append p50 (µs)	Append p95 (µs)	Append p99 (µs)	Ratio to local FS (p50)
Local FileSystem journal	**328**	540	3 434	1.0×
Co-located SQL Server	1 170	1 916	2 479	**3.56×**
Co-located MySQL	1 005	1 536	1 861	**3.06×**

Durability is parameterised identically across the three backends: the FileSystem backend invokes a per-entry flush-to-disk; SQL Server runs with default per-commit log flush; MySQL runs with `innodb_flush_log_at_trx_commit=1` and `sync_binlog=1`. The measurement therefore compares the same physical durability guarantee across all three — one entry, one durable commit.

The journal-append regime favours the substrate by ≈ 3× over both RDBMS engines at the median, and the ordering holds at p95 (the FileSystem journal remains the fastest). The advantage is *structural*: both RDBMS engines land in the same ≈ 3× band, so the gap is a property of the substrate's append being one *fsync* per record versus the RDBMS being one *fsync* per redo-log entry plus the protocol overhead of a transaction. At p99 the ordering inverts — the FileSystem journal's tail (3 434 µs) exceeds both engines' — because a direct `fsync` is exposed to occasional host-level flush and GC stalls that the RDBMS engines' buffered log-writers absorb; the median and p95 advantage is the operative figure, and the p99 tail is a durability-flush artifact, not a throughput property.

The corollary to E1–E4 follows. Every replay-based operation costs ≈ 3× less per entry against the local-journal substrate than against a co-located RDBMS in the durable-commit regime. Deployment at N entries (E1, §5.1), replication at N entries (E2, §5.2), backup ingestion at N entries (E3, §5.3), and catch-up after a partition at N entries (E4, §5.4) inherit the same factor.

Caveats

Two caveats are worth naming, in the spirit of full disclosure.

First, the comparison is *conservative against the substrate.* The RDBMS engines in L6 run on ephemeral container storage — MySQL on a tmpfs mount, SQL Edge on a Docker named volume — whereas the local journal goes through the host's NVMe controller. On a production RDBMS deployment with real disk, the RDBMS side would slow further; the 3× ratio reported here is therefore a lower bound, not an upper one. Adding network latency for a remote RDBMS — a typical production topology — adds a flat additive term per commit, pushing the ratio further still.

Second, the comparison can be narrowed by relaxing durability on the RDBMS side. Group commit, batched inserts, and asynchronous-durability modes — SQL Server's `DELAYED_DURABILITY` and MySQL's `innodb_flush_log_at_trx_commit=2` — all close part of the gap by trading durability for throughput. These are honest engineering choices; L6 measures the durable-commit baseline because the substrate's local-journal regime keeps full durability and the apples-to-apples comparison demands the same of the RDBMS side. A reader who deploys an RDBMS-backed substrate with relaxed durability will see a smaller gap; the structural claim — that replay cost is bounded by append latency — does not change, only the constant.

To the next sub-section

§5.5 has shown that the cost basis of the four equivalences is one quantity — the substrate's per-entry append latency — and that against a co-located RDBMS in the durable-commit regime, the local-journal substrate runs at $\approx 3\times$ lower latency per entry. The result is structural rather than engine-specific: two RDBMS engines, two implementations, the same ratio band. §5.6 closes §5 by naming two further consequences of the substrate — observability and debug — that fall out of the same property without requiring separate apparatuses.

5.6 Forensic operations as substrate consequences

Two further operations follow from the substrate without requiring separate apparatuses. The substrate is a total, ordered record of every named operation performed by the program. Conventional observability stacks reconstruct this record indirectly by correlating logs, metrics, and traces emitted outside the program. Under the substrate condition, the record already exists in primary form; any observability layer becomes a reader of the substrate rather than a reconstructor of program history. Sagas do not appear as patterns to be inferred post-hoc from logs; they are named at write time by the program's partition primitive (Paper 3) and therefore exist in the substrate as first-class entries.

Replay is deterministic — by the argument-capture precondition of §4.1, the same corpus, applied to the same binary, produces the same in-memory state at the same cursor position, because every argument in the corpus is a value frozen at write time rather than an expression re-evaluated on read. A runtime issue traceable to an entry is therefore reproducible by replaying the substrate to that entry's position; a conditional breakpoint over the entry's identifier is trivial because the identifier is a position on the cursor, and the cursor advances one entry at a time. Debugging reduces from forensic analysis over logs to advancing the cursor to a chosen position in the substrate. Both observations exist on the same axis as §5.1–§5.5: they require no additional mechanism beyond the substrate itself. §6 turns to the cost in operational complexity that the canonical disciplines pay to achieve, by separate apparatuses, what the substrate already provides.

---

6. Why existing approaches duplicate work the substrate already does

The four disciplines surveyed in §2 are not unrecognised by their respective traditions. Each has developed, over decades, a refined apparatus to address the operational concern it names — blue-green and rolling-update for deployment, change-data-capture and log shipping for replication, snapshot agents and point-in-time recovery for backup, message-queue offsets and convergence protocols for offline operation. The maturity of these apparatuses is not in question. What this section examines is whether any of them dissolves the operational concern it addresses, or whether each reconstructs, from outside the program, work that the substrate condition of §4.1 already does.

This section is not a critique of existing practices but a structural comparison between what those practices reconstruct and what the substrate condition provides by construction. The four sub-sections below examine one canonical apparatus per discipline. The pattern of each examination is the same: name what the apparatus does, identify the structural property it relies on, and observe that the substrate condition provides that property by construction. The point is not that the apparatuses are wrong but that they are responses to the absence of the substrate condition. Once the condition holds, the work each apparatus performs is already done by the program's own act of writing the journal.

6.1 Blue-green vs substrate

Blue-green deployment (Fowler, 2010) provisions two parallel production environments and switches traffic between them at release time. The apparatus has three components: a duplicate environment, a router that decides which environment is authoritative at any moment, and a procedure for bringing the standby environment to the state from which it can accept traffic. The first two are operational infrastructure; the third — bringing the standby to a serving state — is what the apparatus does that the substrate condition recasts.

Under the substrate condition (§4.1), the act of bringing a process to a state in which it can serve is the act of replaying the journal up to the present head (E1, §4.2). The new version's instantiation differs from the old version's continued operation only in the position of its read cursor. The duplicate environment is therefore not a parallel system to which state must be propagated; it is a process whose cursor on the same corpus has not yet reached the present head. The replay that the apparatus performs implicitly — by re-running migration scripts, by exporting and re-importing state, by re-priming caches — is performed explicitly by the substrate's reading discipline.

The structural redundancy is not that blue-green is wrong; it is that blue-green's hardest step, the act of synchronising the standby with the live state, is replay reconstructed manually. The reconstruction is manual because the live state, in the canonical regime, is not the journal of the program; it is a database whose entries record what is, not what was said. Under the substrate condition the manual reconstruction has nothing to do, because the corpus from which the standby reads is the same corpus the live instance writes. The router and the duplicate environment remain; what dissolves is the apparatus that bridges the two.

6.2 Change-data-capture vs event streaming

Change-data-capture (Debezium, n.d.; Maxwell, n.d.) reads the binary log of a relational store and re-emits each row-level change as an event on an external transport. The apparatus exists because the originating system does not emit events; it emits state changes, and CDC observes those changes and translates them back into the event form that downstream consumers require. The translation step is the apparatus.

The translation is informationally lossy in one direction and inferential in the other. State changes do not carry the operation that produced them; CDC infers, from a sequence of column-level deltas, what the application program intended. The inference is necessarily heuristic — two distinct programmatic operations can produce identical column-level deltas, and the CDC consumer cannot recover the distinction. The apparatus is therefore not only a transport; it is a reconstruction layer that approximates events from state, with the loss inherent in that approximation.

Under the substrate condition the reconstruction has no work to do. The originating system emits events — they are the entries of the journal, written in the language of the program (Paper 1, anti-porosity) and reduced to compact named operations (Paper 2, separability). The wire across which replication travels carries those entries directly (§5.2); no inference from state, no column-level delta extraction, no schema mapping enters the act. The CDC apparatus is structurally duplicative: the substrate emits exactly the artifact CDC reconstructs — minus the reconstruction loss, and minus the apparatus that produces it.

6.3 Snapshot backup vs passive consumer

Snapshot backup (Hitz, Lau, & Malcolm, 1994; PostgreSQL, n.d.; Veeam, n.d.) captures the state of a data store at a chosen moment and writes that capture to an archival medium. Recovery instantiates the captured state and resumes operation from it. The apparatus has two components: the snapshot mechanism that produces the capture, and the recovery procedure that consumes it. Both operate on state — the captured artifact is a frozen image of what the program had stored, not a record of what the program had done.

The structural property the apparatus relies on is that the program's state, if captured atomically at a moment, suffices to characterise the program at that moment. The property holds for programs that are reducible to their state. It does not hold for programs that are reducible to their history: a snapshot of the state at moment t does not record the operations that produced it, only their cumulative effect, and recovery from the snapshot loses the operational record.

Under the substrate condition the program is not reducible to its state; it is constituted by the sequence of operations in its journal (§4.1). A consumer that holds the corpus — without instantiating the runtime that would replay it — holds the program in the form in which it was written (E3, §4.2; §5.3). The choice between holding and replaying is a property of the destination, not of the captured artifact. The snapshot apparatus produces a state image and discards the operational record; the substrate emits the operational record continuously and lets the destination decide whether and when to replay.

The redundancy is structural in two directions. First, the snapshot apparatus is unnecessary: the substrate is already the program, copied continuously. Second, the snapshot apparatus is also insufficient: it captures state, which is less than the program — recovery from a snapshot reconstructs *where* the program was, not *what it did to get there*. The passive consumer of §5.3 holds the program, and any moment in the program's history is replayable from the corpus the consumer holds.

6.4 Message queue buffer vs persistent local journal

The message-queue pattern places a durable buffer between a producer and a consumer that may not be reachable in real time. RabbitMQ's dead-letter exchanges, Kafka's consumer offsets, and the eventually-consistent stores derived from Dynamo (DeCandia et al., 2007) each place an apparatus at the boundary between the system and its counterpart. The apparatus is external to the producer; the producer writes to its store, and a separate mechanism — the queue, the broker, the convergence protocol — handles the case in which the consumer is unreachable.

The structural property the apparatus relies on is that the producer's store and the consumer's store are different artifacts, with a transport between them that must be made durable. Under that structural premise, the queue is the right apparatus: it absorbs the period during which the consumer cannot reach the producer, and it persists the messages that would otherwise be lost.

Under the substrate condition the producer's store and the consumer's store are not different artifacts at the level of representation. The producer writes to its substrate; the consumer reads from a copy of the same substrate, or from the substrate directly. The substrate is itself the durable artifact, and offline operation reduces to the consumer's cursor trailing the producer's write head by an arbitrary interval (E4, §4.2; §5.4). No queue is added between the two parties because the substrate is already the queue — it persists every operation in the order it was uttered, and a consumer that has been unreachable for an arbitrary period catches up by reading the entries that accumulated.

The redundancy here is the most structural of the four. Where the queue, the broker, and the convergence protocol exist as boundary infrastructure, the substrate is itself the boundary — it is what holds the program between writer and reader, between connected and disconnected, between now and later. The infrastructure that the canonical literature places between two stores is, under the substrate condition, the property of the single store both parties share.

6.5 The substrate match table

The four examinations of §6.1–§6.4 share a structure. Each canonical pattern relies on a property the substrate condition supplies by construction; each apparatus performs work that the journal, written as the program, already does. The pattern is summarised in the table below, in the form of Paper 4 §6.4: a single question — *does the apparatus address a property the substrate already provides?* — answered for each pattern.

Pattern	Property the apparatus relies on	Does the substrate condition already provide it?
Blue-green deployment	The standby must be brought to the same state as the live instance before traffic switches	Yes — replay of the same corpus to the present head (E1, §5.1) brings the standby to that state by construction; no external state-propagation step is required

Pattern	Property the apparatus relies on	Does the substrate condition already provide it?
Change-data-capture	The originating store emits state changes; events must be reconstructed from them	Yes — the substrate emits the operations themselves (Papers 1 and 2; §5.2); no reconstruction step is required and no inferential loss is incurred
Snapshot backup	The program is characterised by its state at a moment; recovery instantiates that state	Yes — the program is constituted by the sequence of operations in the journal (§4.1, §5.3); the corpus held by a passive consumer is the program, not a state capture of it
Message queue buffer	A durable apparatus is required between the producer's store and the consumer's store	Yes — the substrate is the durable artifact both parties share (§5.4); a consumer reading later than the producer wrote reduces to cursor advancement over the same corpus

The four "Yes"s share a structure. In every case, an apparatus that the canon developed to bridge an absence of the substrate condition becomes structurally subsumed once the condition holds. The pattern remains usable — none of the four is wrong as engineering — but the work it performs duplicates work the substrate already does. The maturity, sophistication, and widespread adoption of the four patterns are not evidence that the substrate condition is unnecessary. They are evidence that the absence of the substrate condition is costly enough to justify four separate apparatuses to compensate for it, each developed independently by a different tradition.

The diagnosis of §2–§6 is now complete. §7 turns from the construct to the instantiation: a system in production whose runtime already satisfies the substrate condition, whose four operational disciplines fall out of the substrate without dedicated apparatus, and whose existence is the evidence that the construct names a realisable property and not only a conceptual one.

7. Instantiation: realization in Puppeteer

The construct of §§3–6 makes claims about a class of systems characterised by the substrate condition. The present section exhibits one such system. The voice from this point forward is concentrated and authorial: the runtime described below is *Puppeteer*, the framework the present author maintains, and the operational decisions named are decisions taken in deployments that run today. The framework is presented here as an existence proof for the construct rather than as the subject of the paper; the four equivalences hold in any runtime that meets the substrate condition, of which Puppeteer is one.

7.0 Origins

Paper 3, §6 introduces the runtime decomposition this section relies on: *Performance* is the local hosting of a single actor over a configured storage, *Theater* is the host for one or many Performances on a node, *Ensemble* is a pool of actors with routing, and *StageManager* is the peer-to-peer replicated state machine that coordinates topology across nodes. The full vocabulary is presented in Paper 3 §6 and is not re-introduced here. §7 names only the runtime surfaces on which the four equivalences of §4.2 land: the *aliveGate* (E1, §7.1), the per-event push callback wired to a webhook delivery service (E2, §7.2), and the *Materialize* construct that declares which entries materialise in which destinations (E3, §7.3). The offline equivalence (E4) lands on the same write path the local-journal régime of §5.4 already exhibited, and is referenced rather than re-developed here.

7.1 Theater.aliveGate: the red-black mechanism

The handover window of §5.1 is realised in the runtime by a single boolean predicate: the *aliveGate*, a manual-reset event held on every `Performance` instance. The gate is the locus of the deployment apparatus, and its life-cycle is the operational realisation of the structural intervals named in §5.1.

A `Performance` is started in one of two modes. A primary starts with `Start()` (default `asFollower: false`): the runtime hydrates the actor from its substrate, raises the `OnFirstHydration` and `OnHydrated` callbacks, and sets the gate. A follower starts with `Start(asFollower: true)`: the runtime hydrates the actor from the same substrate, but the gate remains reset and the public `IsAlive` predicate reports `false`. External callers — request dispatchers, write coordinators — block on `WaitUntilAlive` until the gate is set.

The handover is a single sequence over the gate. The follower invokes `LockWhileNotSyncronized()`, which pauses writes on the live primary and returns the EntryId at which the pause occurred. The follower then catches up over the residual tail with `CatchUpFromJournal(targetEntryId)`, which replays pending events under the actor's write lock until the follower's cursor reaches the targeted EntryId. `UnlockAndRunAlive()` flips the follower's gate and releases the primary. From the perspective of any external caller, the system transitions from "primary alive" to "follower alive" in a single atomic moment — the gate flip — preceded by replay of the substrate and nothing else.

The structural content of the apparatus is therefore minimal. The gate is bookkeeping over the substrate's read cursor; the runtime contains no separate deployment subsystem, no state-extraction layer, and no schema-mediated handoff. The orchestration tail measured in §5.1 — below 0.3 ms at every measured N — is the cost of the gate flip and the residual-tail replay, not of any apparatus. Appendix A lists the exact source locations of each named surface.

7.2 Event streaming: push-to-pull inversion

The substrate's push-to-pull inversion (§5.2) is realised in Puppeteer by composing three primitives that already live in the runtime. None of them was introduced for §7.2; each existed for its own structural reason, and together they satisfy the consumer-pull requirement.

First, the journal exposes a per-record callback. `Diary.OnRecordWritten` is invoked by the storage layer for every entry written, with the entry's `entryId` and its raw byte representation. The setter accepts a single subscriber; the companion `AddRecordWrittenCallback` chains additional subscribers without disturbing the existing one, so that multiple downstream consumers can attach to the same primary without the primary knowing how many.

Second, the partition primitive of Paper 3 exposes a *Cue* mode for Reactions. A `Cue()` Reaction runs continuously: it batches over the substrate from the current cursor to the present head and then enters *push mode*, subscribing to the same `OnRecordWritten` callback above. The push loop applies each entry as it arrives, matches it against the Reaction's pattern, and dispatches the Reaction's terminator if the pattern matches.

Third, the substrate's externalisation point is the terminator of a `Cue` Reaction. The terminator's body — DSL the program writes — is the place at which the runtime delegates to whatever external delivery service the deployment has elected. We observe that an off-the-shelf webhook delivery service (SVIX, in the deployments referenced here; equivalent products would suffice) satisfies the substrate's consumer-pull requirement: the Reaction's terminator posts the entry to the service from a per-entry callback; the service holds the entry durably; downstream subscribers fetch from the service at their own pace.

No code in `Puppeteer/` or `Choreography/` references SVIX. The runtime exposes the callback and the partition primitive; the wiring to a particular webhook service is a property of the deployment, not of the framework. The substrate's claim is that any service satisfying the consumer-pull contract is sufficient; the off-the-shelf adoption is the empirical witness to that sufficiency.

7.3 Passive consumer: replicate without an actor running

The passive consumer of §5.3 is the operational realisation that, in earlier drafts of this paper, was named as an honest gap — a property derivable from the substrate condition but not yet exhibited as a system in production. The gap closed on the day this section was drafted, by composition of patterns the runtime already had. The composition has since been formalised under the construct *Materialize*. The remainder of this sub-section walks the composition brick by brick, in the spirit of the andragogical commitment of Paper 2: a reader who has followed §§4–6 is at risk of dismissing E3 as "stated, not exhibited"; the present sub-section makes the assembly visible so that the dismissal has nothing to dismiss.

The instantiation has four bricks. The reader is asked to hold them separately and observe how they compose.

Brick 1 — Same binary, alternate startup mode. A destination is a process running the same actor binary as the primary, started in an alternate mode. The mode admits no external requests: the `aliveGate` of §7.1 is not opened for the destination's lifetime. Two such modes exist. `Performance.Start(asFollower: true)` starts the actor as a *follower* — a process whose `Job()` Reactions run against a shared substrate; the follower is a worker, not a backup, and its Reactions contribute markers back to shared auxiliary tables. The second mode, `/materialize`, runs no Reactions at all — it consumes precomputed markers from the primary via the wire protocol described under Brick 3. The mode is selected by the host process at startup; the runtime is the same in either case.

Brick 2 — Per-event marker in the DSL. The program declares which entries materialise in which destinations through a marker on the Reaction's metadata plane:

```
actor.Reactions.DefineReaction("...")
    .Job().Company()
    .Seek("...")
        .OnMatch("...")
    .Metadata.Materialize("DC-B");
```

`Metadata.Materialize("DC-B")` records, for every entry that matches the pattern, a row in the `EventMaterialization` table naming the destination. The row is the program's declaration that this entry is part of the corpus the named destination is to receive. Reading the program reveals which entries are intended for which destinations, in the same DSL the program is written in — the construct is, in the sense of Paper 1, an instance of the code-as-data recording the substrate condition requires.

Brick 3 — The wire protocol between primary and destination. The destination, when it wakes up, asks the primary for the entries it has not yet seen. The exchange is realised by four wire verbs on the primary's `actor.Materialization` sub-namespace. `ReadRecordsAfter(destination, fromEntryId)` returns the raw substrate records the destination is missing — Layer 1, the program as the primary wrote it. `ConfirmUntil(destination, entryId)` records the destination's acknowledgment on the primary side as a Max-monotonic watermark — the primary now knows the destination has the corpus up to that EntryId. `ReadReactions(destination)` and `ReadElidedRange(destination, from, to)` return Layer 2 derived state — the Reaction registry's atomic snapshot and the elision markers the primary computed in the same range — so that a destination electing to replay can reach the same in-memory state the primary reached without re-running the Reactions itself.

The destination side is the fluent client `MaterializeMirror`. The contract is two-layered:

```
mirror.Sync();                        // Layer 1 — orchestrates (a) + (b).
mirror.AsProgramMirror().Sync();                       //    Layer    2    —
orchestrates (a) + (c) + (d) + (b).
```

`Sync()` fetches records and confirms the watermark; `AsProgramMirror().Sync()` additionally fetches the Reactions snapshot and elision markers, so that the destination's program state is reconstructible without instantiating the Reaction matcher. The destination decides what to do with the result — a passive backup persists the records and discards Layer 2; a replicated read-replica persists both and applies the elision; a snapshot-time mirror runs Layer 1 only.

Brick 4 — Recovery primitive. When a destination has been offline for an arbitrary period — or when an intermediate delivery service has exhausted its retention — the destination resumes by issuing `Sync()` against its current watermark. The primary serves the missing entries from its substrate; the destination's watermark advances; the round-trip is the same as steady-state. There is no separate recovery mode in the runtime, because there is no separate steady-state mode either: every fetch is `Sync()` against the watermark, and every catch-up is `Sync()` over a wider range. The recovery primitive is the steady-state primitive.

The four bricks compose into a single operation. The program declares which entries materialise in which destinations (Brick 2). A destination process — running the same binary in `/materialize` mode (Brick 1) — fetches records from the primary's Materialization surface (Brick 3) at its own pace, confirms what it has received, and catches up after any interruption by the same call (Brick 4). The runtime contains no replicator, no separate backup subsystem, and no synchronisation layer between primary and destination beyond the four wire verbs and the per-Reaction marker. The operation falls out of the substrate; the construct names the operation that the program already declares.

Three dimensions of the construct deserve naming, because each is structurally present without requiring a separate apparatus.

Heterogeneous backends. `Diary` selects its storage implementation polymorphically from `(DatabaseType, connectionString)` at construction time. A primary running on the local FileSystem can materialise to a destination running on MySQL or SQL Server, and the wire protocol carries the same raw records across the heterogeneity — `ReadRecordsAfter` returns Layer 1 records regardless of which backend the primary stores them in. The four wire verbs are implemented on each of the four backends with the same contract; L4 (§5.3) exhibits this independence with **18 of 18 cells reaching equal in-memory state across three backends and three journal sizes**.

Multi-destination fanout. `AddRecordWrittenCallback` chains additional subscribers without displacing the existing one. Multiple destinations may register against the same primary; the primary's write path is unaware of how many. Each destination holds its own watermark on the primary side; each progresses at its own rate; the primary serves them independently. Multi-backup of one actor — and the per-destination retention policies that go with it — falls out of the chain primitive.

Per-datacenter topology. A datacenter is an operational placement of `Performance` instances; it is not a construct the actor knows about. A primary at datacenter A and a destination at datacenter B is a topology decision realised by the operator: a local SVIX at each site for the Cue feed of §7.2, and `/materialize` processes at each site whose mirror clients fetch from the local SVIX or directly from the primary. The substrate's locality property (the journal is the actor's, not the system's; cross-ref §6.2 of Paper 3) extends laterally: each datacenter's destinations are wired to the local instance of the delivery service, and cross-site bandwidth is consumed only by the SVIX-to-SVIX replication the off-the-shelf service performs.

A gap is worth naming here, in the same spirit of full disclosure. The `Diary.OnRecordWritten` setter is wired today for primary FileSystem and for the buffered régime; for primary MySQL or SQL Server the setter is a silent no-op. The canonical case of the substrate (a FileSystem-primary deployment) is unaffected, and the §7.2 push-to-pull path runs against the FileSystem callback as designed. A future deployment with primary MySQL would require the callback to be wired in the MySQL storage path — a localised patch, not a structural change.

7.4 Numbers

The operational régime of the four bricks above runs under the latency budget §5.5 already developed. The substrate cost of materialisation is the cost of appending the program at the primary (which §5.5 measures) plus the cost of appending at the destination (which §5.3's L4 measures). No new numbers are introduced here; the relevant ones live in §5.1 (deployment window), §5.2 (wire density), §5.3 (destination convergence), §5.4 (offline catch-up), and §5.5 (per-entry append latency). Together they characterise the régime in which the four bricks above operate; together they materialise the construct in production-grade numbers.

---

8. Relation to previous work in this paper series

The substrate condition stated in §4.1 — that the journal is code-as-data, dense, and made of compact named operations — is not asserted ex nihilo by the present paper. Each of its components is the conclusion of prior structural analysis in this series, and each entered §4.1 as a precondition rather than as a claim of this paper. Without those preconditions the four equivalences of §4.2 would not follow; with them, the four equivalences are corollaries of the substrate's properties under variations of when and where the read happens. §8 makes the chain of dependence explicit.

Paper 1 introduces *anti-porosity* as a design principle for the boundary between domain code and persisted form. The journal records what the program said, not the data structures the program manipulated; entries are sentences in the same language the program is written in, and every effect that crosses the boundary is named in that language rather than summarised as a state delta. Anti-porosity is the precondition under which the journal can be the program at all: a porous journal records states whose operational origin is lost, and the substrate condition reduces to recording a derivative of the program rather than the program itself. §4.1 takes anti-porosity as given; without it, E2 (replication as sharing history) and E3 (backup as copying the program) collapse into the canonical regimes of §2.

Paper 2 introduces *separability* as the structural property that makes the journal entry of a parametric verb a compact reference plus its arguments. The verb's body is not present in the entry; it lives once, named, in the corpus's vocabulary. Separability is the precondition under which the wire of §5.2 carries operations at a density that does not scale with the operation's internal complexity, and it is the precondition under which the latency budget of §5.5 holds — replay is linear in compact-entry count, not in expanded state, only because separability has reduced the entry to its reference form. §5.5's three-times advantage against co-located RDBMS engines is, in part, the operational expression of the structural property Paper 2 names.

Paper 3 introduces *the partition* between immediate work, done at the actor's writer-lock cursor, and deferred work, done by Reactions that read the journal forward and produce their own derived entries. Paper 3 §6 establishes the runtime decomposition — *Performance*, *Theater*, *Ensemble* — that frames the operational realisation of §7. The mechanism that admits a new instance as authoritative (§5.1's gate flip) is named in Paper 3 claim 8 as a property derivable from the partition; the same paper names the cross-actor extension as a forward reference closed by Paper 4. §5.2's symmetric-consumer property — that two instances of the same binary, reading the same corpus, produce identical Reaction output (L3) — depends on a Reaction reading only corpus-resident data: in L3, the arguments the program journaled when it invoked each operation — frozen at write time by Paper 1's anti-porosity — so whatever the remote consumer needs to evaluate the guard travels in the corpus the program writes.

Paper 4 introduces *tell* as the primitive under which cross-actor causation is recorded as program in each participant's local journal. Paper 4 claim 10 names cross-datacenter replication as the canonical case in which tell's program-level recording survives where the conventional apparatus (CDC, log shipping) reconstructs state and loses the causal chain. §5.2 of the present paper completes the picture from the substrate side: the journal record is what the consumer reads, and the consumer reconstructs not only the local actor's state but, when tell entries are present, the cross-actor causal chain those entries record. The substrate condition makes Paper 4's cross-DC property hold without an external mechanism that observes the producer's runtime.

The dependence is summarised in the table below.

Paper	What it establishes	What §4.1 takes as given	Where it is load-bearing in this paper
Paper 1	Anti-porosity: the journal records operations, not state	The journal is code-as-data and dense	E2 wire density (§5.2); E3 the corpus is the program (§5.3); §6.3 snapshot is informationally less
Paper 2	Separability: the entry of a parametric verb is `ActionId` + arguments	The journal entry is compact, named, parameter-referenced	Wire density at production scale (§5.2; L2); replay rate linear in compact count, not state (§5.1; L1); latency budget (§5.5; L6)
Paper 3	Partition: immediate vs deferred; Reactions; runtime decomposition (*Performance*, *Theater*, *Ensemble*)	Reactions and the partition primitive; the runtime apparatus referenced in §7	Gate flip apparatus (§5.1; §7.1); symmetric consumer via corpus-resident reads (§5.2; L3); apparatus vocabulary (§7.0)

Paper	What it establishes	What §4.1 takes as given	Where it is load-bearing in this paper
Paper 4	*tell* primitive: cross-actor causation as program in sender's journal	The cross-actor recording the substrate carries across sites	Cross-DC causal chain travels with replication (§5.2; cross-ref claim 10)

The four prior papers can be read as the structural derivation that the present paper takes as substrate. Each named one property that the journal had to possess for the program to live in it; the present paper takes the four properties as given and reads off the four operational equivalences that follow. Where each prior paper went from one structural commitment to its consequences, the present paper goes from four structural commitments to a single primitive — replay — under which the four canonical operational disciplines collapse. The relation is therefore not parallel: §4.1's substrate is what the prior four established, and §4.2's theorem is what follows once it is in hand.

---

9. Counter-arguments

The substrate condition and its four equivalences invert vocabulary the canonical literature has used productively for decades. A reader familiar with that literature is likely to raise objections that are not addressed by §§2–8. Four are taken up here. The treatment of each follows the same shape: name the valid aspect of the objection, then identify the structural premise on which the invalid aspect depends and observe that the substrate condition removes that premise. The refutations are made from the construct, not from any framework that instantiates it.

9.1 "Loading a large journal takes time"

Canonical assumption. Bringing a system to a serving state requires reconstructing its persisted history end-to-end, and the cost of that reconstruction grows without bound with the system's operational age.

The objection. A journal that accumulates entries over the operational life of a program grows without bound. Bringing a new instance to a serving state by replaying the journal therefore takes longer the older the program becomes. A canonical anecdote names a relational system whose initial load — from a binary log accumulated over a year — required roughly an hour before the system could accept traffic. If deployment is replay (§5.1), then deployment is bounded below by that load time, and the substrate's account of deployment makes the deploy window worse, not better.

What is valid. Replay is linear in the entries it reads. A journal twice as long takes twice as long to replay. The objection correctly observes that the absolute deploy time grows with the program's history under any substrate-based account of deployment.

What does not follow. The hour-long load named in the anecdote is not replay over the substrate condition; it is reconstruction of state from a log whose entries are state-change deltas. The two régimes have different cost basis. Under the substrate condition, the entry is a compact named operation — `ActionId` plus arguments — and replay is the application of those operations in order (Papers 1 and 2). The per-entry cost is the in-memory apply of one operation, not the disk-bound write or schema-mediated commit that state-change replay incurs.

§5.1's measurement materialises the structural distinction. L1 reports a sustained replay rate of **532 thousand entries per second** at the median (N = 100 000), rising to **1.76 million entries per second** at N = 1 000 000 — the rate is bounded by neither N nor the per-entry apply cost, and the orchestration tail around bulk replay is below 0.3 ms at every measured N. A journal of ten million compact entries replays in the order of ten seconds at this rate, not an hour. The objection generalises a measurement made in one régime — state replay over a binary log — to a régime in which the entries are operations, and the generalisation does not hold. What grows linearly is the number of operations the program has uttered; what does not grow is the cost of uttering each one again in memory.

One canonical case lies outside this argument and is preserved as an honest limit. Where the cost-dominant operation at a single entry is itself expensive — a network round-trip to an external service, a complex domain calculation — replay carries that cost per entry just as live operation did. The substrate does not make any one operation cheaper; it makes the bookkeeping around the operation negligible.

9.2 "Change-data-capture already solves replication"

Canonical assumption. Cross-datacenter replication is properly addressed by reconstructing events from the originating store's state-change deltas, mediated by a CDC pipeline between primary and replica.

The objection. Cross-datacenter replication is a mature engineering concern. Change-data-capture pipelines, log shipping between database engines, and managed replication services route writes from primary to replica with bounded lag and well-understood failure modes. The substrate's account of replication (§5.2) appears to re-derive a problem that the canon has solved repeatedly. What additional structural claim does the substrate condition support that the existing apparatus does not?

What is valid. CDC pipelines work. Production systems replicate across datacenters with CDC every day; the apparatus is reliable, vendor-supported, and well-tooled. As an engineering solution, it solves the operational concern of moving state between sites.

What does not follow. The structural premise of CDC is that the originating store records state changes, not operations, and that events must be reconstructed from those state changes by inferential reading of the binary log (§2.2, §6.2). The reconstruction is lossy: two distinct programmatic operations that produce identical state changes are indistinguishable at the CDC layer. The wire across which CDC traffic travels carries a derived artifact — column-level deltas — at a width that scales with the size of the affected rows rather than with the operation that caused them.

The substrate condition replaces the reconstruction with direct emission. The journal entry is the program's operation, in the language of the program (Paper 1), reduced to a reference to a parametric verb plus its arguments (Paper 2). The wire carries the operation as uttered, not the state changes it produced. §5.2's measurement (L2) reports a compact wire of **33 to 96.7 bytes per event** depending on verb tier, against literal-form payloads of **118 to 910.7 bytes**; the literal-to-compact ratio reaches **9.4× at the production-shaped tier and 20× at full production scale** (Paper 2 Lab 4). Applying `gzip` to the literal form closes the gap only to **4.0× at tier 3** — the structural saving is not algorithmic but representational, and a compressor operating after the fact cannot recover what naming achieved by construction.

The objection is therefore precise in scope. CDC solves replication as state propagation. The substrate condition does not deny that solution; it shows that under the construct's conditions the problem CDC solves does not arise, and the wire carries a different artifact. The two are not in competition for the same engineering decision; they are at different layers of the design.

9.3 "Snapshot backup is simpler"

Canonical assumption. Backup is a captured image of state at a chosen moment, archived by an external apparatus and re-instantiated at recovery time.

The objection. Backup as a periodic state snapshot is a forty-year-old discipline with well-understood operational properties: recovery time objective (RTO), recovery point objective (RPO), storage tiering, lifecycle policies. The substrate's account of backup (§5.3) requires a continuously consuming destination — a process that holds the corpus as it grows — which appears operationally more complex than periodic state capture. Why prefer continuous to periodic?

What is valid. Periodic snapshots have an operational vocabulary the field knows. The discipline is mature, vendor-supported, and integrates with object-store lifecycle infrastructure. As an engineering choice for systems whose program is reducible to state, the snapshot apparatus is a sound default.

What does not follow. The structural premise of snapshot backup is that the program is reducible to its state — that a captured image at moment t suffices to characterise the program at that moment (§6.3). The premise holds for programs whose journal is, in effect, a derivative of state changes. It does not hold for programs whose journal is the program itself.

Under the substrate condition, the artifact that characterises the program is the journal, not any state derived from it. A snapshot at moment t captures *where* the program had arrived; the corpus to t captures *how it got there*. The two are not equivalent informational artifacts: from the snapshot, the operational history is lost; from the corpus, the snapshot at any past moment is recoverable by replay. The substrate-based account of backup is therefore not simply a different operational pattern with the same content; it is the strictly more informative choice when the construct's conditions hold.

The cost is honest. A continuously consuming destination, even one whose role is to hold the corpus and never replay it, must be reachable, must persist what arrives, and must catch up when it falls behind. §5.3's measurement (L4) reports steady-state convergence of ≈ **2 150 events/sec on FileSystem, ≈ 952 events/sec on MySQL, and ≈ 766 events/sec on SQL Server**, with per-instance verification reaching equal in-memory state in **18 of 18 cells across three backends and three journal sizes**. The numbers establish that the operational obligation is bounded — a destination that ingests at the rates above is not a degenerate case — and that the corpus arrives identically across heterogeneous backends. The objection's premise that simplicity favours snapshot is honest engineering; the construct's reply is that the simpler artifact carries less information, and the choice between holding state and holding program is the choice the substrate condition makes available.

9.4 "Append does not scale beyond N writes/sec"

Canonical assumption. A journaled append is bounded by the per-store write throughput of the underlying medium, and that ceiling constrains any substrate-based account to per-actor write rates the medium can absorb.

The objection. Every persistent store has a write-throughput ceiling. A substrate that records every operation as a journal append inherits whatever ceiling the underlying store imposes; production systems that exceed that ceiling are constrained either to batched-write modes or to relaxed-durability commits. The substrate condition therefore does not generalise to high-throughput workloads, and the four equivalences hold only for systems that fit within the per-actor write rate the journal can absorb.

What is valid. A local-journal substrate, like any persistent store, has a per-entry append latency that bounds its write throughput. §5.5's measurement (L6) reports **328 µs p50 on the local FileSystem backend** under one-entry one-durable-commit semantics, corresponding to roughly three thousand entries per second of single-writer append rate. A workload that exceeds that rate on a single actor cannot be absorbed by single-writer append at the same durability guarantee.

What does not follow. The single-actor ceiling is not a ceiling on the construct; it is a property of the configuration in which one actor handles one event stream. The substrate condition is preserved under sharding (one actor per shard, each with its own journal — the per-actor invariant of Paper 3, §6.2 and §6.8) and under local buffering with asynchronous replication to a canonical backend (§5.4's offline equivalence applies to the steady-state régime, not only to outage absorption). Neither sharding nor buffering is foreign to the construct; both fall out of the substrate's locality property — the journal is the actor's, not the system's.

The honest scope is therefore narrower than the objection suggests. The construct does not claim that a single actor with one journal absorbs unbounded write rates; it claims that within the rate the journal can absorb at a given durability guarantee, the four equivalences hold. §5.5's table records that the local-journal régime favours the substrate by $\approx$ **3× over both co-located RDBMS engines at p50 and at the long tail**, and that the advantage is structural (both RDBMS engines land in the same band). The trade-off is not journal-vs-no-journal; it is local-append-vs-network-commit, and the substrate's ceiling is the higher of the two before any sharding is considered.

Relaxing durability changes the constant. Group commit, batched inserts, and asynchronous-durability modes on the RDBMS side close part of the gap by trading durability for throughput; the substrate's local-append rate can be raised similarly by group-flush semantics. §5.5's caveats name both choices as honest engineering. What the comparison preserves under all durability régimes is the structural claim: replay-based operations inherit append latency as their dominant cost term, and a local append against a substrate is, in every measured régime, at least as fast as a comparable commit against a co-located transactional store.

---

10. Conclusion

The canonical literature on distributed systems has long treated deployment, cross-datacenter replication, backup, and offline operation as four operational disciplines, each supported by its own apparatus. This treatment proved productive: blue-green environments, change-data-capture pipelines, snapshot agents, and message-queue buffers matured into reliable engineering practice. From that productivity grew the fragmentation analysed in §2 and §6 — four parallel literatures, four parallel apparatuses, and four parallel cost models, borne independently by systems that must be zero-downtime, cross-datacenter, recoverable, and tolerant of disconnection.

This paper observes that the fragmentation is not entailed by the operations themselves. It follows from the assumption named in §3: that operational concerns are addressed by apparatus around the program rather than by the program itself. Under the substrate condition established by prior papers in this series, that assumption no longer holds. The journal is the program written out over time, and four equivalences follow from reading it under variations of when and where (§4.2): deployment is replay at the present head; replication is the same replay at another site; backup is the corpus held without replay; offline operation is replay after a delay.

Viewed from this condition, the apparatuses surveyed in §6 appear as mechanisms that reconstruct, from outside the program, work that the substrate already performs by construction. Blue-green deployment reconstructs replay manually; change-data-capture reconstructs operations from state; snapshot backup captures a derivative of the program's history; message-queue buffers introduce a durable artefact to absorb disconnection that the substrate's own persistence already absorbs.

The contribution of this paper is conceptual. The instantiation in production serves as an existence proof; the measurements in §5 exhibit the régime in which the four equivalences are observable. One artefact — the substrate — supports four operational readings without requiring separate mechanisms.

For a journaled program, deployment is replay, replication is sharing history, backup is copying the program, and offline operation is delayed replay. The four operational disciplines named in §2 remain meaningful; what changes is the structural locus of operational concerns: from externally-added apparatus to internal readings of the program's substrate.

Appendix A. Source-code verification

The constructs named in §7 are publicly available in the Puppeteer codebase. The line references below are pinned against the master branch as of the drafting of this section. Three tables organise the surface by sub-section: the *aliveGate* mechanism (§7.1), the push-to-pull primitives (§7.2), and the *Materialize* construct (§7.3).

§7.1 — Theater.aliveGate

Surface	File	Line(s)	What it shows
`aliveGate` manual-reset event	`Choreography/Theater/Performance.cs`	34	Reset while follower or during handover; Set when alive

Surface	File	Line(s)	What it shows
`Start(bool asFollower = false)`	`Choreography/Theater/Performance.cs`	100	Two-mode start; sets gate iff primary
`WaitUntilAlive(CancellationToken)`	`Choreography/Theater/Performance.cs`	136	External callers block here
`IsAlive` predicate	`Choreography/Theater/Performance.cs`	167	False while follower; true after handover
`LockWhileNotSyncronized()`	`Choreography/Theater/Performance.cs`	177	Pause writes on primary; return EntryId
`UnlockAndRunAlive()`	`Choreography/Theater/Performance.cs`	183	Flip gate; release primary
`CatchUpFromJournal(long targetEntryId)`	`Puppeteer/EventSourcing/ActorHandler.cs`	2159	Replay residual tail under write lock
`RehydrateFromEvent`	`Puppeteer/EventSourcing/DB/Diary.cs`	228	Diary-side bulk replay primitive

§7.2 — Push-to-pull primitives

Surface	File	Line(s)	What it shows
`Diary.OnRecordWritten` setter	`Puppeteer/EventSourcing/DB/Diary.cs`	145	Per-record callback (single subscriber)
`AddRecordWrittenCallback`	`Puppeteer/EventSourcing/DB/Diary.cs`	160	Chain N subscribers without displacing existing
`ReactionMode.Cue`	`Puppeteer/EventSourcing/Follower/Reaction.cs`	19	Continuous push-mode Reaction
Cue batch → push loop wiring	`Puppeteer/EventSourcing/Follower/Reaction.cs`	563-579	Catch-up batch then subscribe to callback

Surface	File	Line(s)	What it shows
SVIX integration	—	—	Not present in `Puppeteer/` or `Choreography/`; wired at the per-API host process

§7.3 — Materialize construct

Surface	File	Line(s)	What it shows
`Performance.Start(asFollower: true)`	`Choreography/Theater/Performance.cs`	100, 118	Alternate startup mode; `SuppressReactionJournaling` flag
`Reaction.MaterializeFromMetadata(destination)`	`Puppeteer/EventSourcing/Follower/Reaction.cs`	669	DSL marker plumbing: records destination on action
`actor.Materialization` sub-namespace	`Puppeteer/Materialization.cs`	(whole file)	Register / Deregister / List / ReadRecordsAfter / ConfirmUntil / ReadReactions / ReadElidedRange
`MaterializeMirror` fluent client	`Puppeteer/MaterializeMirror.cs`	(whole file)	`Sync()` (Layer 1); `AsProgramMirror().Sync()` (Layer 2)
`MirrorSyncResult` immutable struct	`Puppeteer/MaterializeMirror.cs`	170-202	Records / ReactionsSnapshot / ElisionMarkers + watermark advance
Diary backend polymorphism	`Puppeteer/EventSourcing/DB/Diary.cs`	53-78	`(DatabaseType, connectionString)` → InMemory / FileSystem / MySQL / SQLServer

Surface	File	Line(s)	What it shows
Local-buffer (offline régime)	`Puppeteer/EventSourcing/DB/Diary.cs`	28, 34, 80-135	`IsBuffered`; `ReplicationAgent` async drain to canonical storage

---

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `99e8202` (2026-07-13). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:c55af9b6466302ed402cb579440ceec870e449c8;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:99e8202c808c3d79ca7b1dfbdae0ffb7872c940e
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

The six laboratories' source and datasets accompany this paper as `paper05-data.zip`. The aggregated results each lab reports — the headline and summary tables this paper cites — are included in full. To keep the archive light, Lab 6's raw per-event sample log (`samples.csv`, ≈ 60 MB / ~500 000 rows) is omitted and replaced by a note giving the lab source and the runtime commit needed to regenerate it.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

Appendix B. Bibliography

Ambler, S. W., & Sadalage, P. J. (2006). *Refactoring databases: Evolutionary database design*. Addison-Wesley.

AWS. (n.d.). Managing your storage lifecycle. Retrieved from https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html

Azure. (n.d.). Overview of operational backup for Azure Blobs. Retrieved from https://learn.microsoft.com/en-us/azure/backup/blob-backup-overview

Bacula. (n.d.). Bacula backup software. Retrieved from https://www.bacula.org/

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, *59*(5), 50–57.

Debezium. (n.d.). Debezium. Retrieved from https://debezium.io/

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)* (pp. 205–220).

Flyway. (n.d.). Flyway database migrations. Retrieved from https://flywaydb.org/

Fowler, M. (2010). BlueGreenDeployment. *martinfowler.com*. Retrieved from https://martinfowler.com/bliki/BlueGreenDeployment.html

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, *30*(3), 611–642.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

Hitz, D., Lau, J., & Malcolm, M. (1994). File system design for an NFS file server appliance. In *Proceedings of the USENIX Winter Technical Conference*.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop, 6th International Workshop on Networking Meets Databases* (co-located with SIGMOD 2011).

Kubernetes. (n.d.). Deployments: Rolling update deployment strategy. Retrieved from https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-update-deployment

Lakshman, A., & Malik, P. (2010). Cassandra: A decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, *44*(2), 35–40.

Liquibase. (n.d.). Liquibase. Retrieved from https://www.liquibase.org/

Maxwell. (n.d.). Maxwell's daemon. Retrieved from https://maxwells-daemon.io/

MySQL. (n.d.). MySQL 8.0 reference manual: Replication (Chapter 17). Retrieved from https://dev.mysql.com/doc/refman/8.0/en/replication.html

PostgreSQL. (n.d.). Continuous archiving and point-in-time recovery (PITR). Retrieved from https://www.postgresql.org/docs/current/continuous-archiving.html

RabbitMQ. (n.d.). Dead letter exchanges. Retrieved from https://www.rabbitmq.com/dlx.html

Redis. (n.d.). Redis replication. Retrieved from https://redis.io/docs/management/replication/

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20404863

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20740697

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20792156

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.21207062

Sato, D. (2014). CanaryRelease. *martinfowler.com*. Retrieved from https://martinfowler.com/bliki/CanaryRelease.html

Veeam. (n.d.). Veeam backup & replication user guide. Retrieved from https://helpcenter.veeam.com/docs/backup/vsphere/overview.html

ewpage

Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks

TL;DR

Many layers taken for granted in production systems — caches, ORMs, message queues, distributed locks, application servers, orchestration clusters — do not exist because the problem requires them, but because the underlying persistence model does.

This paper names that property *infrastructural symptom*, defines the two diagnostic indicators that distinguish it from a structural requirement, provides an auditor workflow to read any layer under those indicators, and shows a journaled actor-native system in which the canonical symptoms disappear.

The construct is diagnostic, not prescriptive: it makes visible which parts of a stack compensate for the model rather than serve the problem.

Claims this paper makes

1. Some infrastructural layers in production deployments exist to compensate for structural deficiencies of the underlying persistence model rather than for structural requirements of the problem being solved (§3).
2. The property naming this distinction — *infrastructural symptom* — is read through two diagnostic indicators: compensation (the layer compensates for an identifiable model defect) and dissolution (the layer loses justification when the defect is removed) (§3).

3. An auditor workflow guides the application of the two indicators and produces a reading — model-dependent, structural, or unclassified — per (layer, use-case) pair (§5).
4. A journaled actor-native model instantiates a system in which four mechanisms — locality, journal density, journal-as-causal-substrate, self-contained substrate — dissolve the seven canonical categories of compensating infrastructure (§4.1).
5. The Redis case admits the workflow with per-use-case reading: six use-cases dissolve as symptoms; two remain structural (§4.2).
6. The ladder of compensated layers is open-ended; the workflow extends to backup tooling, service mesh, distributed-tracing/APM, and future emergent layers (§6).
7. The model permits appliance-class deployment as a limit case in which categories of business software become physically shippable — an architectural permission, not an operational proof (§7).

1. Introduction

This paper makes an analytic contribution: a diagnostic typology for reading existing stacks. It identifies and names a structural pattern that prior literature has documented case by case without articulating as one (the typology); derives the indicators under which the pattern reads as model-dependent (the criteria); and presents an instantiation in which those indicators read positively and the pattern is absent (the existence proof). The genre is closer to *theory for analyzing* in the sense of Gregor (2006) than to design theory: evidence is presented in the form of a working artifact, but the artifact illustrates the lens's reading rather than validating it. The contribution is conceptual; the instantiation confirms that the pattern is contingent rather than structurally necessary, not that any specific system is the only way to dissolve it.

Production software systems are routinely deployed alongside layers of supporting infrastructure: in-memory caches, object-relational mappers, message queues, distributed locks, application server frameworks, orchestration clusters, full operating systems. These layers are treated as standard components of any non-trivial deployment. Architectural discussions assume their presence by default; tutorials and reference architectures position them as foundational; engineering teams budget for them, staff them, and monitor them. Their absence is treated as a property of toy systems.

This paper names a property that some — not all — of these layers share. We call this property *infrastructural symptom*: a layer is an infrastructural symptom when it exists to compensate for a structural deficiency in the underlying persistence model rather than for a structural requirement of the problem being solved. The distinction is routinely elided; the two cases are treated alike in architecture, in tooling, and in engineering culture. Once the property is named, each layer in a given stack becomes auditable under the question: is this compensating for something, and if so, for what?

Section 2 situates the paper among prior work. Section 3 defines the construct and derives the two diagnostic indicators that distinguish infrastructural symptom from structural requirement. Section 4 presents the instantiation: a journaled actor-native system in which the canonical layers — caches, glue queues, locks, application server scaffolding — are absent not by omission but by structural irrelevance. Section 5 specifies the auditor workflow for reading a stack under the typology. Section 6 generalizes from the canonical case to a chain of compensated layers, from in-memory cache up through full operating system. Section 7 examines the limit case, where the chain collapses to a domain artifact deliverable as an appliance. Section 8 examines what the lens makes visible and what it does not.

2. Related work

The genre of this paper is closer to *theory for analyzing* (Gregor, 2006) than to design theory. An analytic paper of this kind identifies a typology, derives the indicators under which the typology reads, and presents an instantiation that demonstrates the reading produces non-trivial discrimination. The contribution is conceptual; the instantiation is the existence proof, not the substance of the claim.

Critique of DBMS-centric architecture as the dominant paradigm for production systems has accumulated in the literature without naming the construct presented here. Stonebraker and Çetintemel (2005) observe that the relational database has become a one-size-fits-all hammer applied to use-cases for which it is structurally ill-suited, and predict the proliferation of specialized stores. Kleppmann (2017) catalogs the patterns by which contemporary deployments combine caches, queues, ORMs, and search indices around a relational core, documenting their pairwise interactions and operational costs. Helland's *Life beyond Distributed Transactions* (2007) makes a parallel observation about the proliferation of coordination machinery under distributed-transaction semantics. His *Immutability Changes Everything* (2016) extends a related argument to the substrate itself: append-only representation dissolves much of that machinery by construction. The present paper names what these critiques document piecewise — the unifying property of compensation — and renders the discrimination available as a diagnostic lens.

The instantiation presented in §4 is constructed from three lines of prior work. The actor model originates with Hewitt (1973), articulating computation as message passing between isolated entities. Event sourcing as a persistence pattern is associated with Fowler (2005), which characterizes the program's history as the durable artifact rather than a snapshot of current state. Command Query Responsibility Segregation, articulated by Young (2010), separates the read and write paths so they no longer compete for the same model. Vernon (2015) brings these three together as practical patterns in *Reactive Messaging Patterns with the Actor Model*; that synthesis is the immediate ancestor of the present instantiation. The journaled actor-native system used as the existence proof here combines these threads; it does not invent them.

This paper is the sixth in a series. The series develops the construct's foundations across five preceding papers. Paper 1 introduces porosity as the representational defect that this paper's construct identifies in its compensation form. Paper 2 develops program-value separability and the externalized-parameters precondition that supports journal density as a §4.1 mechanism. Paper 3 introduces Reactions and the pragmatic partition between work-due-before-responding and deferred work, which §4.1 invokes as the journal-as-causal-substrate mechanism. Paper 4 introduces the Tell primitive and cross-actor causal continuity, used in §4.1 for the same mechanism. Paper 5 develops the journal as substrate for deployment, replication, backup, and offline operation, which §6 references as the substrate-mediated lifecycle that dissolves application-server scaffolding. Paper 7 carries the same discrimination upward from infrastructural layers to architectural roles; §9 introduces this extension.

The contribution unique to this paper is conceptual. The construct *infrastructural symptom* names a property that prior work documents in pieces but does not unify. The two diagnostic indicators — compensation and dissolution — give the property its shape. The auditor workflow of §5 guides its application without requiring adoption of any particular alternative. The ladder of §6 and the limit case of §7 extend the typology's reach. The instantiation of §4 demonstrates that the typology reads cleanly under the indicators; the demonstration is not the contribution.

3. The construct: infrastructural symptom

We state the construct before naming its two diagnostic indicators and the workflow that guides their application.

An infrastructural symptom is a layer of infrastructure that exists to compensate for a structural deficiency in the underlying persistence model rather than for a structural requirement of the problem being solved.

The construct operates at the level of purpose, not at the level of product. A given physical layer in a deployment may serve multiple purposes; the construct discriminates among those purposes one at a time. The unit of analysis is the use-case, not the layer.

A layer reads as an infrastructural symptom for a given use-case when both of two diagnostic indicators are positive.

The compensation indicator. The layer's purpose for that use-case is traceable to a specific deficiency of the underlying persistence model. The deficiency must be identifiable: not "we use this layer because it is fast," but "we use this layer because the persistence model exhibits property P, and this layer compensates for P." When the compensation indicator is positive, asking what would happen if P were not present? yields a non-trivial answer.

The dissolution indicator. When that deficiency is removed — by adopting a persistence model in which P does not arise — the layer loses justification for that use-case. The deficiency's absence is sufficient to make the layer redundant for that purpose; nothing else about the problem domain, workload, or operational context needs to change.

The dissolution indicator requires that M' — the counterfactual model in which P does not arise — be *constructible*: a persistence model with operationally defined semantics, viable under the workload, hardware, and programming-model constraints of the use-case under examination. A counterfactual whose viability is unspecified — an imagined absence of P without a model that exhibits the absence in operational form — does not count as a model in which D does not arise; it only restates the wish that D were absent. The lens does not admit such counterfactuals. The instantiation of §4 is offered precisely as one such constructible M', sufficient to make the dissolution indicator non-vacuous for the seven canonical categories.

Both indicators must be positive. Compensation without dissolution describes a genuine patch whose value survives model evolution. Dissolution without compensation is not constructible: a layer cannot dissolve under a model change unless its purpose was tied to the model in the first place.

The construct is not a universal accusation against infrastructure. Many layers in production systems exist as structural requirements — they serve purposes rooted in the problem domain, independent of any persistence model. Three categories are typical:

- **External-system integration.** Layers mediating communication with systems outside the boundary — payment gateways, partner APIs, sensor networks — compensate for no model deficiency. The integration problem remains under any persistence model.
- **Cryptographic and protocol enforcement.** TLS termination, request signing, OAuth verification address security or protocol requirements rooted in the problem domain. No persistence-model change makes encryption optional.
- **Genuinely expensive computation.** Analytics engines, machine-learning inference services, video encoders exist because the computation itself is intrinsically expensive. That cost does not change under a different persistence model.

The boundary between symptom and structural requirement is not a property of the layer as a product but of the purpose for which it is used. The same caching system, deployed to absorb read-vs-write contention at the persistence layer, instantiates an infrastructural symptom. Deployed instead to memoize a deliberately expensive computation, it instantiates a structural requirement. The reading is produced per use-case.

Seven categories of layer are encountered regularly in production deployments and admit reading under the two indicators. The third column describes, in model-property terms rather than system terms, what a persistence model under which both indicators read positively would offer instead.

These seven are not a privileged set but an entrenched one: the layers most consistently assumed by default in conventional production stacks, taught as foundational, and budgeted as standing operational concerns. Other commonly-deployed layers — logging frameworks, schedulers, service discovery, secret managers, feature-flag servers — admit reading under the same workflow of §5 and produce more varied results. Some dissolve (in-process logging, internal scheduling and discovery); some are structural (secret managers, whose purpose is rooted in access control rather than persistence; the A/B-testing facet of feature-flag servers, rooted in product methodology); some split per use-case (logging at internal vs. external boundaries; service discovery at internal vs. external). The lens discriminates; Table 1 catalogs the architectural baseline against which the discrimination is most consequential.

Table 1. Diagnostic reading: seven canonical categories of compensating infrastructure under the two indicators of §3.

Layer	Underlying defect compensated	What a model with both indicators positive offers instead
In-memory cache	Reads contend with writes at the persistence layer; reconstructing state from storage is expensive	State local to the unit of computation; no contention between read and write paths
Object-relational mapper	Domain objects do not align with relational rows; mapping is opaque and lossy	Domain objects are the unit of persistence; no translation layer
Message queue used as glue	Components cannot directly invoke deferred work without losing causal record	Deferred work is journal-recorded and replayable; queues are not the substrate of causality
Distributed lock	Multiple writers contend for the same logical entity across processes	A single point of authority per logical entity; serialization is intrinsic, not externalized

Layer	Underlying defect compensated	What a model with both indicators positive offers instead
Application server framework	The persistence model leaks into application code; lifecycle plumbing must be added	The domain is the application; lifecycle is a property of the substrate, not added scaffolding
Orchestration cluster as coordination substrate	Stateful coordination across instances requires external choreography	State has location; coordination is between actors, not between containers
Full operating system	The runtime, the framework, and the application require disparate substrates	A minimal substrate suffices to run the domain artifact

The seven rows are not exhaustive. Backup tooling, service mesh, and distributed-tracing frameworks admit similar analysis and are discussed in Section 6. Whether such a persistence model exists, and at what cost, is the subject of Section 4.

The construct is diagnostic, not prescriptive. It enables auditing existing stacks under the two indicators; it does not specify how to build a model in which infrastructural symptoms do not arise. Construction is the work of Section 4, which exhibits one such model, and of Section 5, which specifies the auditor workflow that produced the diagnostic table.

The value of the typology is diagnostic. A practitioner who does not adopt the instantiation of Section 4 can nevertheless use it to read which layers in their stack are tied to their persistence model and which are tied to their problem domain. That visibility is independent of the choice to act on it.

4. Instantiation

4.0 Genealogy

Puppeteer's earliest layer dates to 2005, when a persistence library internally called *autopersistencia* adopted a single design rule: the classes representing the domain would carry no persistence concerns. Persistence would be discovered by reflection over the class hierarchy; storage details lived elsewhere. The rule was modest in scope and pragmatic in motivation. Its consequence was structural: the domain became a clean substrate, unobstructed by the technical aspects that conventional architectures embed inside it.

At no point in this evolution was there an intention to eliminate caches, queues, locks, or application scaffolding; those layers were assumed to belong in any serious system. Their later absence was not a design objective but an empirical outcome. This distinction matters for the argument of this paper: the construct is not dissolved here by architectural ideology but as a byproduct of unrelated design constraints that happened to satisfy the indicators derived in §3.

The natural follow-up question was: if domain classes carry no persistence concerns, where does state live? The answer that emerged across the following years became the framework's first generation. State lives in a journal, and the journal records the program itself — not serialized events, not row deltas, but the DSL script that produced the state, replayable in order. The narrative of execution, not the photograph of memory, became the unit of persistence — a property whose absence, in the terms of §3, is the deficiency that caches, ORMs, and queue-glue exist to compensate for in DBMS-centric architectures.

The framework's second generation added four properties, each developed in preceding papers of this series. Parameters were externalized, allowing programs to be compiled, cached, and recorded as references rather than literal scripts (Paper 2). Reactions were introduced as the developer-controlled partition between work due before responding and work deferred to placement (Paper 3). The Tell primitive established cross-actor causality recorded in the journal rather than orchestrated externally (Paper 4). The journal itself became the catalog of the actor's declarative vocabulary, dissolving the lateral storage of action definitions that the framework had carried since V1.

The framework was not designed to dissolve the construct named in this paper. The construct was identified retrospectively, after the chain of architectural decisions described above had converged on a system in which most of the canonical layers had become unnecessary. The work of this paper is, in that sense, a forensic reading of an existing artifact, not an engineering plan derived from a principle. The principle was a property of the artifact before it was a property of the literature.

4.1 The instantiation under the two indicators

Four mechanisms in the journaled actor-native model account for the dissolution of the seven canonical categories of §3. A single mechanism may dissolve more than one category, and the in-memory cache dissolves under two mechanisms because §3 records two distinct defects compensated by that layer. Each mechanism decomposes into several sub-mechanisms — properties of the model that, taken together, exhibit the umbrella behavior. Table 2 correlates each mechanism with its sub-mechanisms, the §3 defects it dissolves, and the canonical layers that become symptoms under it.

Table 2. Mechanisms in the journaled actor-native model mapped to the §3 defects they dissolve and the canonical layers that become symptoms.

Mechanism	§3 defect compensated	Canonical layer that becomes symptom
Locality • privacy of state • actor-as-serializer • placement	Reads contend with writes at the persistence layerMultiple writers contend for the same logical entity	In-memory cache (contention defect)Distributed lock
Journal density • journal-as-program (homoiconic) • replay-as-reconstruction • domain-class-as-unit-of-persistence	Reconstructing state from storage is expensiveDomain objects do not align with relational rows	In-memory cache (reconstruction defect)Object-relational mapper
Journal as causal substrate • Reactions recorded in journal • Tell recorded in journal • causality boundary is the journal	Components cannot directly invoke deferred work without losing causal recordStateful coordination across instances requires external choreography	Message queue used as glueOrchestration cluster
Self-contained substrate • domain artifact is the application • substrate carries lifecycle, persistence, dispatch • journal-mediated release handoff • minimal OS surface	The persistence model leaks into application codeThe runtime, framework, and application require disparate substrates	Application server frameworkFull operating system

Locality. State is private to the actor that owns it; the actor processes commands and queries serially against its own memory. There is no shared store from which concurrent reads must be isolated from concurrent writes. Two of the §3 defects vanish mechanically under this property: the read-versus-write contention that motivates in-memory caches, and the inter-process contention that motivates distributed locks. The compensation indicator is vacuous for both. Placement is the developer-controlled partition treated in Paper 3. This mechanism reads positively under the actor-native preconditions established across the preceding papers in this series: the actor's state fits in memory and is processed serially over its private storage (Papers 1 and 3), and cross-actor continuity is mediated through the journal by Reactions and Tell rather than through shared queries (Papers 3 and 4). Aggregates that exceed the in-memory precondition — extremely hot single entities, datasets larger than a single host's working set

— fall outside this reading and require actor splitting or partitioning, which is a design technique distinct from the lens itself.

Journal density. The journal does not record serialized snapshots or row-by-row deltas; it records the program — the DSL script — that produced the state. Reconstruction is replay, not deserialization; replay of a compact script is cheap. The domain class is the unit of persistence, with no intermediate row to map. Both §3 defects compensated by caches and ORMs — reconstruction expense and object-row impedance — are dissolved mechanically. The mechanism is treated as compilation in Paper 2 and as homoiconic representation in Paper 1. Implementation: `Puppeteer/EventSourcing/DB/Diary.cs`. Replay cost for long-lived actors is bounded by complementary substrate primitives: Distill compacts the journal in place (`Puppeteer/EventSourcing/DB/Diary.cs:264`); elision markers and Materialize watermarks enable offload-and-trim through a materialization destination (Paper 5, §7.3). Together these primitives fill the role that Rolling Snapshots (Young, 2010) play in conventional event-sourced systems, without re-introducing snapshot semantics.

Journal as causal substrate. Pragmatic deferral, developed in Paper 3 as Reactions, records work the verb does not perform before responding as a journal entry, with the causal link to the originating event preserved. Cross-actor continuity, developed in Paper 4 as Tell, records dispatch from one actor to another in the journal as well. The journal becomes the boundary of causality. In DBMS-centric systems, causality crosses the persistence boundary and must be reconstructed with message queues used as glue and orchestration clusters used as coordination substrates; here, causality never leaves the journal. Implementation: the `reactions` field in `Puppeteer/EventSourcing/ActorHandler.cs:38`.

Self-contained substrate. The runtime is a small loader; the application is the domain artifact. Lifecycle, persistence, and dispatch are properties of the substrate, so no external process is required to host or coordinate the application artifact. Release handoff and rolling deployment, handled in DBMS-centric deployments by application server orchestration, are journal-mediated here and are the subject of Paper 5. The application server framework as a category has no equivalent role; the operating system surface is reduced to the minimum required, a limit case treated in §7.

Together, these four mechanisms account for the dissolution of the seven canonical categories of §3 mechanically, not by intent. None was added to the model with the construct of this paper in mind; each was developed for the reasons enumerated in §4.0.

4.2 The Redis case in detail

Redis occupies a distinctive position in the contemporary infrastructure landscape: a single product whose presence is taken for granted across deployments that are otherwise architecturally dissimilar. The diversity of its typical use-cases — cache, session store, pub/sub, queue, distributed lock, leaderboard, rate limiter — does not reflect the diversity of a single problem domain. It reflects the diversity of the pressures that DBMS-centric architectures present to application designers, each of which Redis conveniently absorbs in a familiar form. This makes Redis the canonical case for inspection under the construct of §3. The analysis that follows is not, however, about Redis as a product; it is about the defects of the underlying persistence model that Redis exists to absorb — defects that other compensating layers also address in variant forms, treated briefly after the canonical table.

Table 3 applies the two indicators of §3 to the most common Redis use-cases, tracing each to the specific defect it compensates and the mechanism in §4.1 that dissolves it.

Table 3. Application of the diagnostic typology to canonical Redis use-cases.

Redis use-case	§3 defect compensated	Native response in journaled actor-native
In-process cache for database results	Reads contend with writes at the persistence layer; reconstructing state from storage is expensive	Actor state is the cache (§4.1 Locality + Journal density)
Session storage	Stateless application servers cannot retain user state across requests	The user actor retains session naturally as part of its state (§4.1 Locality + Self-contained substrate)
Pub/sub among internal components	No causal record of cross-component events; coordination must be reconstructed	Reactions record deferred work in the journal with causal links preserved (§4.1 Journal as causal substrate)
Message queue for deferred work	Components cannot directly invoke deferred work without losing causal record	Reactions and Tell record causation in the journal (§4.1 Journal as causal substrate)
Distributed lock for shared entity	Multiple processes write the same logical entity across boundaries	The actor owns its entity; serialization is intrinsic, not externalized (§4.1 Locality)

Redis use-case	§3 defect compensated	Native response in journaled actor-native
Leaderboard / sorted set	Real-time aggregation against the persistence layer is expensive	Query against actor state or a follower; replay is cheap (§4.1 Locality + Journal density)

Each row reads positively under both the compensation and dissolution indicators of §3. The pub/sub row covers internal-component coordination; the external-systems variant is structural and is treated below.

Two Redis use-cases do not read positively under the dissolution indicator and are therefore structural under the typology. The first is pub/sub directed at external systems: webhooks to third-party consumers, event distribution to partners, broadcast to subscribers outside the operator's boundary. The external system does not dissolve under any persistence-model change; the distribution problem persists. The second is rate limiting applied to external API consumption: per-customer quotas, third-party API throttling, abuse mitigation against external traffic. The rate limit is a property of the external relationship, not of the internal model. Redis, or any comparable substrate, remains the appropriate tool for both.

Other compensating layers in the DBMS-centric ecosystem exhibit variants of the same pattern. Memcached shares the cache analysis: locality and journal density make in-process caches mechanically redundant. Distributed coordinators such as ZooKeeper share the lock and coordination analysis: the actor model makes the underlying contention vacuous. Message brokers such as RabbitMQ share the queue analysis: Reactions and Tell record causation in the journal.

Event-storage products such as EventStore DB and Kafka deserve a separate observation. Both adopt a journaled posture — they record events rather than serialized snapshots, and they offer replay. But they instantiate the journal as an external infrastructure layer that the application consumes through a client API, not as the substrate of the application itself. Causality is recorded but remains external to the unit of computation; coordination between event production, event consumption, and application state still requires application-side glue. The product captures the right idea and externalizes it, leaving the application code in the compensating position. Under the construct, these products do not dissolve the symptom; they relocate it.

A second observation about these compensating layers is that they do not absorb their compensation freely. Each requires the application to carry the boilerplate of its use: cache invalidation logic, lock acquisition and release with retry and fencing semantics, serialization between domain objects and external representations, key naming and TTL discipline, consumer offset management and rebalancing handlers, schema registry coordination, idempotency keys for at-least-once delivery. The application code carries the porosity imposed by RDBMS representations (Paper 1) as its baseline; the compensating layer adds a second porosity, this time imposed by the layer itself: its own API surface, lifecycle, and consistency contracts. Domain logic interleaves with both. Under the instantiation, the compensating layer and the boilerplate that the application carries to use it are both absent. The actor's code addresses the problem domain; the substrate addresses the rest.

Redis as a product is not what the typology identifies; nor is any single compensating layer treated above. What the typology reads is the architectural pattern in which a persistence model exposes defects and successive layers — caches, brokers, coordinators, even externalized event stores — accumulate to absorb them. Replacing one such layer while retaining the model that produces the others does not change how the typology reads. The unit of change is the persistence model itself, not any individual layer.

5. Auditor workflow

The typology of §3 enables reading per use-case, but practitioners face stacks containing dozens of compensating candidates. Without an explicit workflow for application, the typology risks remaining theoretical: invoked at the level of architecture review, but absent from the operating discipline that distinguishes one layer from the next. The workflow that follows guides the application of the two indicators of §3, producing a reading in a finite number of steps for any (layer, use-case) pair under examination. It can be applied without adopting the instantiation of §4.

```
Input:  layer L deployed for use-case U.
Output: reading in {model-dependent, structural, unclassified};
        defect D identified when model-dependent.

Step 1 — Compensation indicator
  Identify the defect D of the underlying persistence model
  for which L compensates in use-case U.
  If a defect D is identifiable, proceed to Step 2.
  If no defect is identifiable AND L's purpose is rooted in the
    problem domain (external-system integration, cryptographic
    or protocol enforcement, intrinsically expensive computation),
    return structural.
  If no defect is identifiable AND L's purpose is opaque to the
    auditor (organizational, regulatory, historical, team-skill),
    return unclassified.
```

```
Step 2 — Dissolution indicator
  Consider a counterfactual model M' in which D does not arise.
  M' must be constructible: a persistence model with operationally
    defined semantics, viable under the workload, hardware, and
    programming-model constraints of U.
  If no such M' is constructible, return unclassified.
  If L retains a role in M' for U, return structural.
  Otherwise return model-dependent with D.
```

Four notes on application. First, the unit of analysis is the pair (layer, use-case), not the layer alone. The same product may produce different readings for different uses; the workflow must be run once per use-case. Second, identifying the defect in Step 1 requires explicit attribution to a property of the persistence model. *"We use this layer because it is faster"* does not name a defect; *"we use this layer because the model serializes writes against a shared store"* does. The workflow depends on the discipline of this attribution. Third, the *unclassified* reading exists precisely so that the workflow does not silently absorb opaque or non-model-rooted layers as structural by default; an honest auditor records what the lens cannot see rather than forcing a binary outcome. Fourth, the diagnostic table of §3 and the Redis case of §4.2 are worked examples of the workflow applied to canonical and product-specific layers respectively. Each row in those tables is the reading produced by the workflow for one (layer, use-case) pair.

The workflow renders readings; it does not prescribe action. A model-dependent reading does not require that the layer be removed in the current system. It indicates that the layer's continued presence is structurally tied to retaining the persistence model that produces the defect. Practitioners may rationally retain a model-dependent layer — for legacy preservation, contractual constraint, team familiarity, migration cost — but they do so knowing that the layer carries a debt that another model would not impose. The workflow's product is visibility, not directive.

6. The ladder of compensated layers

The seven canonical categories enumerated in §3 are not a list. Read in order, they form a ladder of increasing scale: in-memory cache (per-process), object-relational mapper (within-application), message queue and distributed lock (between processes), application server framework (the application stack), orchestration cluster (across machines), and full operating system (the hosting substrate itself). The order is not arbitrary; it follows the expanding radius of the defect the layer compensates for. Each rung compensates for a deeper limitation of the model; each rung dissolves under the alternative for the same reason.

Three additional categories that §3 forward-referenced extend the ladder.

Backup and disaster-recovery tooling — point-in-time restore, write-ahead log archiving, snapshot orchestration — exists because state in DBMS-centric architectures lives in mutable rows whose history is not the primary artifact. Under the model, the journal is the backup, and rehydration is the recovery operation; both reduce to a single substrate property treated more fully in Paper 5.

The service mesh — inter-service routing, discovery, traffic shaping, mTLS, sidecar instrumentation — exists because stateless application servers, by design, do not retain coordination state internally. Under the model, much of this coordination is internal: Reactions and Tell handle deferred work and cross-actor dispatch as journal entries. The parts that cross the operator's boundary (TLS to external partners, traffic shaping for external clients) remain structural and continue to belong in a mesh or comparable substrate.

The distributed-tracing and APM stack — span propagation, trace ingestion, request-flow visualization, latency aggregation — requires discrimination per use-case, treated in Table 4 below.

Table 4. Use-case-level reading for the distributed-tracing and APM stack.

Observability use-case	Reading	Reason
Distributed tracing to reconstruct what happened inside the system	Model-dependent	The journal already records the trace; replay is deterministic
Distributed tracing across boundaries with external systems	Structural	The external system does not dissolve under any persistence-model change
APM as *"find the bug in this incident"*	Model-dependent (predominantly)	Replay localizes the bug without requiring external instrumentation
APM as continuous capacity, latency, and error-rate telemetry in production	Structural	Resource consumption is a real-world property, not a model artifact

The first and third rows read as model-dependent; the second and fourth read as structural requirements. Debug as a one-shot operation dissolves more cleanly than continuous telemetry because the journal substitutes directly for the debugger. Continuous telemetry is mixed: the internal-trace component dissolves, but the capacity-and-cross-system component remains. §8 treats this as a worked example of the typology's reading at the boundary of its applicability.

The typology's readings do not eliminate the products from a deployment. The framework does not dissolve APM and OTel; it dissolves the internal-tracing use-case while leaving APM and OTel plugged in at the points where they instantiate a structural requirement. The same observation applies to mesh products at external boundaries, and to backup tooling for legacy systems retained alongside a journaled core.

The ladder is not closed. New layers will emerge to compensate for new defects of DBMS-centric architectures — feature-flag servers, schema registries, secret stores, sidecars for auth and identity, configuration-as-a-service products. Each new layer can be audited under the workflow of §5 and read under the typology of §3. The ladder is open upward, sideways, and into the future because the underlying persistence model continues to generate compensations; the workflow walks each new rung as it appears.

7. At the limit: from datacenter footprint to a deliverable appliance

The workflow of §5 walks each rung of the ladder as it appears; §6 showed the ladder extends. The natural question is what remains when the workflow of §5 marks every rung as model-dependent. §7 names that limit.

What remains is the domain artifact, the journal that records its execution, and the minimal substrate that runs them. The substrate retains what is irreducibly needed: a runtime, journal storage, network I/O, and a scheduling loop. Not required are the cluster orchestrator, the application server framework, the object-relational mapper, the external cache, the distributed coordinator, and the general-purpose operating system in its hosting form. The domain artifact is the application; the substrate is what is required to run it; everything else has been audited and removed under the typology.

The artifact's architectural requirements are independent of scale. The same architecture sits behind a multi-datacenter cluster with cross-region replication (where the journal and Tell handle topology), behind a Kubernetes red-black rolling deployment (where the journal mediates the release handoff), behind a single server (where the artifact runs against local storage), and behind an on-premises appliance hosting an accounting system, a clinic management system, a hardware-store inventory and point-of-sale system, or a professional-services back office. The substrate's storage requirement reduces to two operations — appending entries and advancing a cursor over them — which together compose the substrate's single primitive of replay (Paper 5); both are executed by commodity flash at marginal cost. The RAM requirement scales with the domain's working state, not with the deployment topology. The difference between these shapes is operational variation over the same architecture. What bounds the appliance case is the customer's domain — its data volume, user count, workload heaviness — not anything the architecture introduces.

This is an engineering-compatibility claim, not an operational demonstration. The deployments of the framework's home site run in conventional datacenter infrastructure across eight domain installations; no appliance-class on-premises shipment is currently among them. What §7 argues is that the path to that shape is bounded by ordinary engineering parameters — runtime footprint, journal latency on flash storage, fleet management for updates — and not by an architectural obstacle that the lens has not already named. The barriers that conventional architectures encounter at the appliance limit (a DBMS that does not fit the box and its administration overhead, an application server that demands a separate tier, an orchestrator with no peer to coordinate with, a cache for a model that does not need it) are precisely the layers the workflow of §5 reads as symptoms. The model's permission for the appliance shape is the structural counterpart of those symptoms' dissolution.

This bears restating because of what it implies. Whole categories of small-and-medium-business software — accounting, clinic management, retail inventory, professional services — are today delivered on infrastructure scaled to a fraction of their actual workload, because the conventional stack obliges a datacenter footprint for a database, an application server, and the orchestration to keep them running. The barrier is not only to delivery but to development: building software for a single hardware store or a single clinic requires the infrastructural expertise of running a datacenter, which is the structural reason such products often do not exist at all. The lens reads each of those layers as a symptom. What §7 names is the limit case made available by that reading: software that is shippable as hardware to the customer, hosted in a single appliance, operated by the customer or a local service provider.

At the structural limit, the architecture permits software to be delivered as a physical artifact: an appliance the customer owns, not a datacenter the vendor leases on their behalf. The barrier to that shape is not in the application or in the customer's needs — it is in the infrastructure that conventional architectures require. The barrier dissolves with the symptoms.

8. What the lens makes visible and what it does not

Sections 3 through 7 showed the typology in action: canonical layers in §3, the Redis case in §4, the workflow in §5, the extended ladder in §6, and the limit case in §7. Each of those sections assumed conditions — that the persistence model is identifiable, that use-cases are decomposable, that a counterfactual model is constructible. §8 treats what the lens makes visible and what it does not. The typology still applies; the readings become coarser and the translation to action grows more complicated.

The canonical boundary case is observability. Table 4 (§6) shows that the APM stack contains four use-cases with mixed readings: two model-dependent (internal distributed tracing, one-shot debug) and two structural (cross-system tracing, continuous capacity telemetry). Each reading is correctly assigned per use-case. But a single OTel deployment in a production system serves all four use-cases simultaneously without surgical separation. Removing the instrumentation that serves the internal-tracing use-case affects the instrumentation that serves cross-system tracing; the two share span propagation, collector pipelines, and storage backends. The typology reads the use-case, not the deployment. Operational reality does not admit the same separation, and the practitioner retains the deployment intact and carries the model-dependent use-cases with it.

Four boundaries describe what the lens makes visible and what it does not.

Quantification. The workflow returns a reading per use-case; it does not predict cost saved, performance gained, or operational complexity reduced. Quantification is the work of instrumentation, not of a diagnostic typology — distinct from a limitation, this is a property of the genre.

Stack prescription. A stack populated by model-dependent layers can be identified as suboptimal, but the typology does not prescribe an alternative. The instantiation of §4 demonstrates that one exists; it does not specify the unique alternative for any given system.

Adoption prediction. A model-dependent reading does not inform the difficulty of migration: organizational costs, contractual constraints, vendor lock-in, and engineering skills carry independent weight.

Opaque models. The workflow requires identifying the defect of the underlying persistence model in Step 1 of §5. Systems whose persistence model is opaque — third-party SaaS exposing only an API, proprietary platforms without published internals — fall outside its applicability and are recorded as *unclassified* under the workflow's third reading.

Section 5's closing observation extends here: the visibility that the lens renders is independent of the action the practitioner takes. Model-dependent layers may remain in deployments for legitimate reasons — legacy preservation, contractual obligation, organizational constraint, migration cost. The typology documents the debt; it does not resolve it.

The lens reads cleanly where the persistence model is explicit and characterizable, where use-cases are decomposable, and where a counterfactual model is constructible. Where one of these conditions fails, the typology still applies but its readings grow coarser, or the workflow falls back to *unclassified*. Where it reads cleanly, what the lens surfaces is not a smaller stack but a domain that the model permits to be modeled, libraried, and evolved without being interleaved with compensating concerns. The utility of the typology is proportional to the decomposability of the system under examination.

Persistence is one axis along which representational pressure deforms a domain — the axis this paper has traced. There is at least one more. When interaction is modeled before the domain — when aggregates take the shape of the screen rather than the screen rendering the actor's output — the same compensation pattern reappears in a different register: UI components, view-models, request DTOs, and step-state objects accumulate around a domain that no longer fits its own substrate.

The lens of accidental category reads there with the same discrimination it produced for infrastructural layers and for the server-role. The actor's existing output boundary — the primitives by which it emits to its invoker, agnostic of who consumes — already factors transport out of the domain. This axis is not developed here.

9. Extension: from layers to roles

The typology of §3 reads among *layers* of infrastructure — components that occupy positions in a stack and compensate for the persistence model from those positions. Architectural *roles* — the entity that accepts authoritative writes, the master in replication, the bootstrap issuer, the orchestrator — admit the same reading under the same two indicators. A role is an accidental category when its necessity is traceable to a specific representational choice (what nodes replicate to share state) and the role loses justification when that choice is replaced.

The same lens therefore applies beyond layers. Under a substrate in which nodes replicate programs rather than data, state, or operations, the server-role does not survive as a structural requirement. Its dissolution is not an architectural objective; it is a structural consequence of the same representational choice that dissolves the layers described in §6.

The two analyses operate at different levels of abstraction — first on layers, then on the roles that those layers exist to serve — without modifying the typology itself. The reading scales upward unchanged.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `2f31f96` (2026-05-18). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Fowler, M. (2005). Event sourcing. Retrieved from https://martinfowler.com/eaaDev/EventSourcing.html

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Helland, P. (2007). Life beyond distributed transactions: An apostate's opinion. *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 132–141.

Helland, P. (2016). Immutability changes everything. *Communications of the ACM*, 59(1), 64–70.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1. Zenodo. https://doi.org/10.5281/zenodo.20404863

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/02-program-value-separability.md

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/03-reactions-and-partition.md

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/04-cross-actor-continuity.md

Rivera, A. (2026e). The journal as substrate: unifying deployment, replication, backup, and offline operation in distributed systems. *Puppeteer Papers Series*, Paper 5. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/05-substrate-operations.md

Rivera, A. (2026g). The server is not a structural requirement: identifying accidental architectural roles under journaled programs. *Puppeteer Papers Series*, Paper 7. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/07-server-as-accidental-category.md

Stonebraker, M., & Çetintemel, U. (2005). "One size fits all": An idea whose time has come and gone. *Proceedings of the 21st International Conference on Data Engineering (ICDE)*, 2–11.

Vernon, V. (2015). *Reactive messaging patterns with the actor model: Applications and integration in Scala and Akka*. Addison-Wesley.

Young, G. (2010). CQRS documents. Retrieved from https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

ewpage

TL;DR

Building software for a small clinic, a hardware-store inventory system, or a regional logistics back office requires the operational expertise of running a datacenter — not because the domain demands it, but because the conventional cloud-microservice architecture does. The barrier is not the problem being solved; it is the stack that the problem is conventionally solved on.

This paper observes that under a journaled-program substrate — developed piece by piece across the prior six papers of this series — the datacenter ceases to be a structural requirement of running production software. Cloud and microservice ecosystems, when accessed from a journaled program, become libraries the program invokes rather than habitats the program must inhabit. Software can be shipped in shapes the conventional stack does not permit: a single on-premises appliance the customer owns, a peer mesh of devices that cooperate without a central server, a hybrid that invokes cloud services as libraries. The paper establishes the structural permission for these deployment shapes; their end-to-end operational demonstrations are engineering work the paper does not undertake.

A working instantiation makes the observation operationally inspectable: a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application to a journaled-program substrate, replicated across three nodes that bootstrap each other through paper QR codes. After bootstrap, the cluster operates against its own journals; none of the services that originally constituted the Ordering microservice are reachable from it; the cluster continues to function under each one's absence.

A side observation, stated honestly: the role that prior literature names *server* becomes ephemeral under this substrate — discharged once per peer during the analog bootstrap, then absent from the running cluster. The construct *accidental category*, applied here at the role layer, is offered as a diagnostic lens the reader may apply to their own architecture; it is not the contribution. The contribution is the practical observation that journaled-program substrates make datacenter-less software construction operationally inspectable, in one working case.

Thesis

Under a journaled-program substrate, the datacenter ceases to be a structural requirement of running production software. The components that the conventional cloud-microservice stack treats as habitats — database, application server, orchestration cluster, observability platform — become libraries the program invokes from its own journaled context, or fall away entirely. Software can be deployed in shapes the conventional stack does not permit: a single on-premises appliance the customer owns, a peer mesh of devices (workstations, phones) that cooperate without a central server, a hybrid that invokes cloud services as libraries. *Structurally compatible* and *operationally shipped in production* are distinct: the paper establishes the former through one working case; the latter is engineering work the paper does not undertake.

The role that prior literature names *server* — the entity that accepts authoritative writes, owns coordination, mediates between clients, and provides the address against which other parties direct their traffic — becomes ephemeral under this substrate. It is discharged once per peer during a bootstrap that can be carried by analog means and decided by humans, then absent from the running topology. This side observation connects the paper to Paper 6's lens: the server-role reads as an *accidental category* — the role-level analog of *infrastructural symptom* — once we change what nodes replicate.

Evidence. Three operating nodes derived from a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application to a journaled-program substrate, bootstrapped by paper QR codes and authorized by human decisions. After bootstrap, the cluster operates against its own journals; none of the services that originally constituted the Ordering microservice are reachable from it; the cluster continues to function under each one's absence. The original deployment topology required a datacenter; the ported topology runs on three modest hosts and one paper QR code per peer.

Claims this paper makes

1. Under a journaled-program substrate, cloud and microservice ecosystems become libraries the program invokes rather than habitats the program inhabits; this relation is achieved through the substrate's first-class deferred-

work mechanism (Reactions, Tells) plus the actor-lock's structural cost on raw I/O in the verb body, rather than through application-level resilience discipline (§2).
2. The substrate's three properties — locality, journal as substrate of the program, and self-contained runtime — combined with the actor's write-lock semantics, make the library relation the structurally cheaper path for deferred external work rather than a per-call developer discipline (§4).
3. A working instantiation, constructed by porting the Ordering bounded context of `dotnet/eShop` to a journaled-program substrate and deploying it across three nodes that bootstrap each other through paper QR codes, makes the observation operationally inspectable: after bootstrap, the cluster operates against its own journals, none of the services that originally constituted the Ordering microservice are reachable from it, and the cluster continues to function under each one's absence (§5).
4. The bootstrap of a new node — the operation that prior literature treats as the residual case requiring a persistent server — admits a presentation in which the information transferred crosses by analog means (F2) and the authorization is exercised by humans (F4), and the software portion that remains (F1, F3, F5) lasts only as long as the handshake, after which the issuer process exits (§5.3).
5. The substrate of §4 permits whole categories of small-and-medium-business software — accounting, clinic management, retail inventory, professional services — to run on hardware sized to the customer's domain, rather than on infrastructure scaled to the operator's stack. This is a structural permission, not an operational demonstration; the present paper does not exhibit an end-to-end customer-delivered appliance (§6).
6. The role that prior literature names *server* becomes ephemeral under the substrate — discharged once per peer during bootstrap, absent from the running topology thereafter — as the structural counterpart of claims 1–5. The construct *accidental category*, applied at the role layer, extends Paper 6's *infrastructural symptom* to roles; the extension is a diagnostic lens, offered as a side benefit rather than as the headline (§7).
7. The contribution is the practical observation that journaled-program substrates make datacenter-less software construction operationally inspectable in one working case; the accidental-category construct is the analytic tool through which the observation may be applied to other architectures (§7, §9).

1. Introduction

This paper extends the *infrastructural symptom* construct of Paper 6 from infrastructural layers to architectural roles, an extension that Paper 6 §9 anticipates. The genre is closer to *theory for analyzing* in the sense of Gregor (2006) than to design theory: the construct itself is Paper 6's, applied at a different level of abstraction. The novel material of this paper is the role-level demonstration —

a working instantiation in which the server-role dissolves — and the operational observation that, under the same conditions, cloud ecosystems pass from architectural dependencies to libraries invoked by the program. The contribution is conceptual at the extension level and operational at the demonstration level.

Software for a small clinic, a hardware-store inventory system, a regional logistics back office: these are familiar domains. They are not technically deep — the data models are bounded, the workflows are tractable, the user counts are modest. Yet building any of them today as production-grade software requires assembling a stack — databases, application servers, orchestration clusters, observability platforms, deploy pipelines — that the domain does not demand. The barrier is not the domain. It is the operational expertise required to run the conventional stack.

This paper observes what happens to that stack under a journaled-program substrate. Across the prior six papers of this series the substrate has been developed piece by piece: anti-porous representation (Paper 1), program-value separability (Paper 2), the pragmatic partition for deferred work (Paper 3), cross-actor causal continuity (Paper 4), the journal as substrate of the program (Paper 5), and the diagnostic lens for reading which infrastructural layers compensate for the persistence model rather than serve the problem (Paper 6). Each was argued on its own terms. This paper observes their joint operational consequence.

The consequence is that the datacenter ceases to be a structural requirement of running production software. Cloud and microservice ecosystems, when accessed from a journaled program (in the sense established in Paper 2 §1.2: the pair of domain library and journal of invocations), become libraries the program invokes rather than habitats the program must inhabit. Software can be deployed in shapes the conventional stack does not permit: a single on-premises appliance the customer owns; a peer mesh of devices (workstations, phones) that cooperate without a central server; a hybrid that invokes cloud services as libraries. The paper establishes this consequence as structural permission demonstrated through one working case (§5); end-to-end production deployment in any of these shapes is engineering work it does not undertake (§6).

The paper's argument runs in three movements. §2 names the operational relation: when external services are reached from a journaled program, what was a dependency (whose absence is an outage) becomes a library invocation (whose absence is a journaled failed call). §3 situates the paper among prior work on local-first, peer-to-peer, agent-centric, federated, and actor-model distributed systems. §4 sketches the substrate properties that make the library relation default rather than disciplined. §5 makes the observation operationally inspectable: a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application — a widely-used example of cloud-microservice architecture in the .NET ecosystem — to a journaled-program substrate, deployed across three nodes that bootstrap each other through paper QR codes. The cluster operates against its own journals; none of the services that originally constituted the Ordering microservice are reachable from it; the cluster continues under each one's absence.

§6 examines what the observation implies for software construction in practice as a *structural permission*: software becomes deployable in shapes the conventional stack does not permit — a single appliance the customer owns, a peer mesh of devices that cooperates without a central server, a hybrid that invokes cloud services as libraries — and the barrier of operational expertise dissolves with the architectural requirement that produced it. §6 also marks the boundary between structural permission and end-to-end operational demonstration; the present paper establishes the former, not the latter.

§7 introduces, briefly, a diagnostic lens that the reader may apply to their own architecture: the *accidental category* — a role whose necessity is traceable to a representational choice and that loses justification when the choice is replaced. The server-role of the conventional cloud-microservice stack reads as accidental under the substrate; what was a permanent fixture of the topology becomes ephemeral, discharged once per peer during analog bootstrap and absent from the running cluster thereafter. The construct extends to the role layer what Paper 6 introduced for the layer of infrastructure; it is a tool the reader may use, not the contribution of the paper.

§8 examines limitations honestly, including what the present demonstration does and does not establish. §9 concludes.

The contribution is the observation about datacenter optionality, made operationally inspectable in one working case. The construct is offered as a side benefit — the lens through which the observation can be applied to other architectures — not as the central claim. The choice of `dotnet/eShop` for the instantiation is rhetorical: it is a widely-used reference application for cloud-microservice patterns in the .NET ecosystem, whose original deployment topology makes the contrast visible. The framework underlying the instantiation was not designed to support this paper's claim; the property is identified retrospectively, after a chain of unrelated architectural decisions converged on a system in which the conventional infrastructure requirements had quietly stopped being requirements. The work is, in that sense, a forensic reading of a class of artifacts in which the property happens to hold — consistent with the analytic genre Gregor (2006) describes as *theory for analyzing.*

2. Cloud as library, not as habitat

A cluster of journaled-program nodes accesses cloud and microservice ecosystems as libraries the program invokes, not as habitats the program must inhabit. The structural difference is not in what happens when an external service is unreachable — resilience patterns developed for microservices already achieve graceful degradation under that condition — but in the substrate's relation to external invocation by default. Resilience patterns (Hystrix at Netflix; Polly in .NET; sidecar-based circuit breakers in Istio; the catalog in Nygard, 2018) achieve library-like consumption at the application level: each external call is wrapped in an opt-in policy, each fallback handler is explicitly written, and the discipline of doing this consistently is a property of the development team, not of the runtime. Under the journaled substrate, library-like consumption is the substrate's idiomatic path: external invocations made through Reactions (Paper 3) or Tells (Paper 4) are recorded as journal entries, the outcome (including failure) is part of the program's durable state, and the journal is replayable as part of the program's history. The substrate makes this path structurally favored rather than purely disciplinary: raw I/O inside the verb body is permitted as domain code but carries a lock-cost penalty as the substrate's structural disincentive; external invocations made through Reactions and Tells are journaled by mediation through those constructs. The boundaries of this property — the lock-cost penalty on raw I/O in the verb body, the developer convention of placing slow reads in the caller's code before the verb is invoked, and the deferred-work mediation through Reactions and Tells — are stated explicitly as falsifiability conditions in §8. The journal-recorded failure is not merely observability metadata but program state available to subsequent Reactions and Tells.

The demonstration in §5 makes this concrete. After the port described in §5.1 and the bootstrap of §5.3, none of the services that previously had to be running for the Ordering microservice to exist are reachable from the cluster. The domain code runs entirely inside each node; what previously required an API host, a database, and an event bus is now a library invoked by journaled programs. The cluster operates against its own journals; the cloud topology of the original microservice has no point of contact with the cluster after the bootstrap completes.

The same relation generalizes to external services that the cluster invokes during normal operation. If the identity provider is unreachable at the moment of invocation, the journal records the failed invocation and the cluster continues. If a payment gateway is unreachable, the journal records the attempt and its outcome. The external service's absence is recorded as data, not experienced as outage. The cluster does not inhabit any of these services; it invokes them at chosen moments through journaled Reactions (Paper 3) and proceeds with their result, including the result of their absence.

This operational relation inverts the formulation that has organized the local-first software literature:

Table 1. The inversion of Kleppmann's formulation, captured as a six-word pair.

Kleppmann (Local-first)	The present observation
Own your data in spite of the cloud.	*Record the cloud; don't inhabit it.*

Local-first software treats the cloud as something the software must survive without. A journaled cluster neither survives without the cloud nor depends on it; it invokes cloud services when useful and proceeds without them when not. The two relations differ in what the absence of a cloud service means for the cluster's continued existence.

The observation shows that cloud services, when used from a journaled program, are consumed as invocable capabilities rather than as the substrate in which the program must reside.

3. Related work

Local-first software (Kleppmann et al., 2019) articulates seven ideals for software that operates primarily on devices owned by their users rather than on remote services. Replication is mediated by Conflict-free Replicated Data Types (CRDTs), which allow concurrent edits to merge without coordination. The position with respect to cloud infrastructure is precise: cloud services are admissible, but the software must continue to function in their absence. The title of the paper — *Local-First Software: You Own Your Data, in spite of the Cloud* — captures the stance: the cloud is neither rejected nor required. The server-role is reduced to one of several optional services, and its elimination is the explicit objective of the architectural pattern.

Peer-to-peer append-only logs constitute a second line. Secure Scuttlebutt (Tarr et al., 2019) replicates per-identity signed logs across a gossip network without a central coordinator; the log, not the data it carries, is the unit of synchronization. Hypercore (Holepunch, n.d.) and the Dat protocol (Ogden et al., 2017) generalize the same pattern, treating the append-only feed as a primitive replicated by a distributed hash table. In each case the unit of replication is a log of data — events, document edits, file fragments — and the application logic that interprets the log lives in the consuming application. The server's elimination is the design target of the network layer.

Holochain (Harris-Braun et al., 2018) makes the agent the source of truth. Each agent maintains a local source chain — an immutable, signed sequence of its own transactions — and contributes to a validating distributed hash table. The architecture inverts the data-centric assumption of blockchain rather than addressing the cloud directly; the elimination of the server-role is a consequence of the agent-centric inversion. The replicated artifacts are the agent's chain and the validated DHT; application logic, written as zomes, is interpreted locally by each agent.

Federated systems form a fourth line. Matrix federates an event graph across cooperating homeservers; ActivityPub federates streams of activities; the AT Protocol of Bluesky splits hosting across a Personal Data Server, a Relay, and an AppView, with account portability as the displacement mechanism. The server-role is preserved but pluralized: every participating host fulfills the role for some subset of users. The cloud is partitioned across cooperating providers rather than centralized or rejected.

Adjacent work admits brief mention. Solid (Sambra et al., 2016) proposes Personal Online Datastores that decouple identity from application providers; the server-role moves from application owner to data host without being eliminated. Earthstar (Earthstar Project, n.d.) pursues local-first synchronization without a distributed hash table, sitting between Kleppmann's CRDT model and Hypercore's gossip topology. Neither treats the server-role as a category whose referent is contingent.

Each of these lines treats the server-role as a *design target* — an architectural feature to redesign, redistribute, replace, or pluralize. None of them names the property that the present paper identifies: that the role is contingent on a choice about what nodes replicate, and that under a different choice the role has no referent at all. Table 2 summarizes the four families and the position of the present construct.

Table 2. Five families of prior work and how each treats the server-role. The bottom row identifies the construct of the present paper; the columns describe what each family replicates between nodes, where its application logic resides, why a privileged server is absent (or pluralized), and how each family situates itself with respect to cloud infrastructure.

Family	What is replicated	Where the logic lives	How the server-role is treated	Relation to the cloud
Local-first (Kleppmann)	CRDTs / data	In the application	Eliminated as a design goal	Survival without it
Hypercore / Dat / SSB	Append-only logs	In the application	Eliminated as a network goal	Replaced by peers
Holochain	Source chain + DHT	In the agent	Eliminated by agent-centric inversion	Treated as an alternative architecture
Federated (Matrix / ActivityPub / AT Protocol)	Events / activities	On federated servers	Pluralized across hosts	Fragmented across providers
The present construct	**Programs**	**In the journal of each node**	**Not a pre-requisite (accidental category)**	**Subsumed as a library**

The discriminator is the second column. What each family replicates between nodes — CRDTs and data deltas, append-only logs of data, agent source-chains, federated events and activities — determines what residual role survives. The present construct replicates neither data, deltas, logs, nor events: it replicates *programs*, in the substrate-level sense of Paper 2 §1.2 (the pair of domain library and journal of invocations). The fourth column is downstream of this choice. Under a representational choice in which what travels between nodes is the program itself, the server-role's necessity does not survive. The fifth column is downstream of the fourth: under the present construct the cloud is neither evaded, replaced, nor pluralized, but reused as a library within the program.

Beyond these four families that treat the server-role as a design target, a fifth body of work shares direct lineage with the substrate of §4 without making the server-role its design objective. Erlang/OTP (Armstrong, 2003) is the longest-running production actor system; its supervision trees and gen_server abstraction shape the ecosystem from which the substrate descends, though OTP nodes carry explicit names and gen_server processes carry explicit server-role at the level of the diagnostic lens (§7). Microsoft Orleans (Bernstein et al., 2014) introduces virtual actors with location-abstracted activation: an actor's host is determined by silo-cluster membership, not by application code. Orleans dissolves the location-of-the-actor but retains the silo cluster's coordination layer — the server-role is internalized in the membership protocol rather than externalized to the application, but it is not dissolved at the lens's reading. Akka Persistence (Lightbend, n.d.) and EventStoreDB (Event Store Ltd., n.d.) are the most direct comparators on the actor-and-event-sourcing axis: both persist and replicate the *events* an actor emits, with the actor's behavior recovered by replaying those events through code that lives outside the journal — the unit of replication is the event stream, and the program that interprets the stream is deployed alongside the runtime as application code. The present substrate inverts this relation: the unit of replication is the program itself, and what was previously called the event is recovered as the trace of the program's execution. Under the lens of §7 this difference is not a feature comparison but a representational one: events are what the program produces, not what the program *is*. Amazon Dynamo (DeCandia et al., 2007) operates as eventually-consistent without master, with conflict resolution per-key; it is the canonical AP-without-master system within the CAP positioning (Brewer, 2000; Gilbert & Lynch, 2002). Datomic (Cognitect, n.d.), a commercial immutable database, treats its journal as the source of truth but routes all writes through a single transactor — under the lens of §7, the transactor itself is an accidental category that the present substrate dissolves.

Two theoretical results bracket the structural claim. The CALM theorem (Hellerstein, 2010; Ameloot et al., 2013) establishes a formal correspondence between monotonic computation and coordination-freeness — a program admits coordination-free distributed execution iff its computation is monotonic. The substrate of §4 does not appeal to monotonicity directly, but the dissolution it documents at the role layer is consistent with CALM's prediction: when state propagation is by replay of a deterministic program over a journal rather than by reconciliation of competing writes, the conditions under which coordination becomes structurally necessary are restricted. Conflict-free Replicated Data Types (Shapiro et al., 2011), the mechanism underlying Kleppmann's local-first model, achieve coordination-free convergence at the data layer; the present substrate achieves the same property at the program layer, replicating script invocations rather than data deltas.

The substrate of §4 combines the actor model (Hewitt, 1973), event sourcing (Fowler, 2005), and CQRS (Young, 2010), synthesized as practical patterns by Vernon (2015). An additional move — domain classes carrying no persistence concerns, discovered by reflection — antedates the synthesis by roughly a decade in the framework underlying the instantiation; that genealogy is described in Paper 6 §4.0.

This paper is the seventh in a series. Papers 1–6 develop the substrate-level foundations on which the present observation rests, culminating in Paper 6's *infrastructural symptom*, of which the diagnostic lens of §7 applies the same two indicators to architectural roles — a move Paper 6 §9 anticipates explicitly.

The contribution unique to this paper is the observation that under a journaled-program substrate, cloud and microservice ecosystems pass from architectural dependencies to library invocations (§2), with the practical consequence of making the datacenter optional for production software construction (§6). The instantiation of §5 makes the observation operationally inspectable in one working case. The diagnostic lens of §7 — *accidental category*, applied at the role layer as an extension of Paper 6's *infrastructural symptom* — is offered as a side benefit: the analytic tool through which the observation can be applied to architectures beyond the one demonstrated here.

4. The substrate that makes it possible

The substrate underlying this paper's observation is the journaled actor-native framework developed across the prior six papers of this series. Its architectural lineage is documented in Paper 6 §4.0 and is not retraced here. The substrate is described in the abstract throughout this section; its concrete instantiation, which the demonstration in §5 builds upon, is the framework whose source is publicly available (see Appendix A).

The term *program* throughout this section is used in the substrate-level sense of Paper 2 §1.2: the pair (domain library, journal of invocations) that nodes replicate to share state. The substrate's representational choice — replicating programs in this sense — is what makes it distinct from prior actor-and-event-sourcing systems. CRDT systems replicate state deltas; pub-sub and message-queue systems replicate messages; event-sourced systems (Akka Persistence, EventStoreDB, conventional CQRS+ES stacks) replicate events. The substrate of this paper replicates programs. The three properties of the substrate that follow are entailments of this choice, not three independent design decisions; together they make the cloud-as-library relation of §2 default rather than disciplined.

First, each node owns a local journal of invocations — DSL scripts (in the unit sense of Paper 2) that, on replay, reconstruct the node's state deterministically. The journal is the cumulative state-producing artifact; the state is its replay, not a captured snapshot. No external store mediates between the node and its state.

Second, the domain library — the classes and verbs that the journal scripts invoke — is loaded by reflection at startup; no node hosts the domain as a service for others. Every node carries the same library; the substrate is symmetric across the cluster.

Third, replication is journal-to-journal: nodes exchange script references and parameters, not state diffs, and reach convergence by replaying the same sequence of scripts against the same library. The substrate's primitive is *replay* — the single primitive to which Paper 5 reduces deployment, replication, backup, and offline operation.

None of these properties requires a node to fulfill a privileged role. Each is intrinsic to the substrate, not externalized. When the cluster of §5 reaches external services, it does so against this substrate's storage: the journal records the invocation and its outcome as part of the program's history, and the cluster continues by replaying the journal. That is what makes cloud-as-library the default — the library relation is structural, not a property of how the developer wrote the call site.

The implementation source is publicly available at the commit pinned in Code Provenance; reproduction details are consolidated in Appendix A.

5. The demonstration

The observation of §2 — that under a journaled-program substrate the cloud becomes a library and the datacenter becomes optional — is made operationally inspectable by the demonstration in this section: a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application to the substrate of §4, deployed across three nodes that bootstrap each other through paper QR codes.

5.1 The port

The instantiation ports the *Ordering* bounded context of Microsoft's `dotnet/eShop` reference application — a widely-used example of microservice-oriented cloud architecture in the .NET ecosystem — onto the substrate of §4.1. The diagnostic value of the port is what it makes visible: the lines of the original codebase that the substrate cannot host without modification are a small minority, concentrated in persistence-and-deployment glue. The aggregate, the value objects, the lifecycle verbs, and the invariants move into the journaled substrate without modification. What was previously named "the Ordering microservice" turns out to be a clean domain library wrapped in infrastructure compensating for a different substrate; the port separates the two.

5.2 The three-node deployment

Three identical processes — each running the same substrate with the ported domain library loaded by reflection — deploy across three Docker containers. The coordination mesh is fully connected: three pairwise channels forming the complete `N(N−1)/2` graph at `N = 3`. Three is the minimum peer count at which the dissolution claim is unambiguous: with two peers, any rotation can be read as one serving the other; with three, no subset of two dominates the third.

```
┌─────────────┐     coord pair      ┌─────────────┐
│ ordering-a  │◀───────────────────▶│ ordering-b  │
└──────┬──────┘                     └──────┬──────┘
       │ coord pair                        │
       └──────────┐              ┌─────────┘
                  │              │

             ┌─────────────┐
             │ ordering-c  │
             └─────────────┘
```

Figure 1. The three-node coordination mesh. Each node is a peer; none is privileged.

No node carries the write-acceptance privilege permanently; the role rotates symmetrically across peers. After the rotation completes, the three journals are byte-coincident.

Director rotation is the specific peer-coordination mechanism chosen for the demonstration. It is part of the StageManager pattern — a peer-to-peer coordination layer built on top of the actor substrate, located in `Choreography/StageManager/Stage.cs:470` — and is not a property of the actor model itself. The rotation is not consensus-based in the Raft (Ongaro & Ousterhout, 2014) or Paxos sense; the substrate does not elect Directors internally, run voting, maintain term or epoch numbers, or provide fencing tokens. Promotion is an external decision; the new Director's identity propagates via best-effort announce. The substrate's contribution is journal-determinism between peers (Paper 5), composable with various coordination patterns and transports; StageManager with Director rotation, over Kestrel/HTTPS, is the combination chosen for §5's Docker-container demonstration. What it does not remove, and does not claim to remove, is the broader consensus problem — a different question with different machinery.

5.3 The analog bootstrap

The bootstrap of a new peer proceeds in five phases. **F1** opens an invitation on the issuer side. **F2** transmits the invitation to the joining peer by any means, including analog ones — printed paper, a verbal dictation, an email. **F3** opens an authenticated connection back to the issuer. **F4** pauses for a human decision to approve or reject. **F5** seals a credential to the joining peer's public key and closes.

Two of the five phases are sub-software: F2 (the information transfer) and F4 (the authorization decision). Neither requires a service to be running anywhere; the carrier of F2 is whatever the operator chooses, and the decision-maker of F4 is human. The other three phases (F1, F3, F5) are themselves server-role-like discharge: the issuer listens on an address (F1), accepts a connection (F3), and seals a credential (F5). The discharge is genuine, not concealed. What §5 documents is that this discharge is ephemeral, time-bounded, and terminating: the three software-mediated phases are local to the issuer process, which exits when the bootstrap is complete. The transport stack that carries F3 (HTTPS/TLS via Kestrel; see Appendix A.4) is a property of the underlying transport, not of the journaled substrate — the substrate's contract is that whatever the transport delivers, the journal records and replays. **The issuer process does not act as a server, coordinator, authority, registry, or rendezvous point after bootstrap; its only role is to transfer a credential during the brief lifetime of the handshake, after which it ceases to exist.**

After all three containers complete the handshake — each with its own QR code carried by analog means and approved by a human — the issuer process exits. The containers continue to operate.

```
ordering-a   Up 11 seconds   0.0.0.0:6443->6443/tcp
ordering-b   Up 11 seconds   0.0.0.0:6444->6443/tcp
ordering-c   Up 11 seconds   0.0.0.0:6445->6443/tcp
```

Output of `docker compose ps` after the issuer exits. No surviving network path connects the containers to the issuer; the bootstrap papers are in a drawer.

The information that constituted the bootstrap traveled by paper. The decision that authorized it was made by a human. The cryptographic protection of the credential happened during the brief lifetime of the issuer process. The artifact that survives is the three running containers.

5.4 The empirical matrix

The demo runs across a two-by-two matrix: *in-process* versus *cross-container*, and *inline* versus *parametric*. Each cell is a complete execution of the workload across the three nodes; the in-process row reduces the bootstrap to direct invocation, while the cross-container row runs the full §5.3 protocol.

Table 3. Empirical matrix of the existence proof. Each cell reports the final

journal entry count, identical on all three peers and byte-coincident on disk.

Workload regime	In-process (3 Stages, single OS process)	Cross-container (3 Dockers, real TLS + analog bootstrap)
Inline (literal scripts)	22 entries	22 entries
Parametric (Define + bind)	7 entries	7 entries

The four cells produce identical final entry counts per workload regime, and the three peers in each cell are byte-coincident on disk. The cross-container row confirms that the operational properties of the substrate survive the addition of the analog bootstrap and the real cryptographic stack; the in-process row is the structural rehearsal that isolates the substrate's intrinsic properties from the network. The compaction from inline to parametric is real and measured; its precise ratio and the conditions under which it varies are treated in Paper 2.

Byte-coincidence is the structural witness of the headline observation: it shows that no peer is privileged — the cluster operates symmetrically, with no node fulfilling the role of authoritative writer or central coordinator. Operational metrics — write throughput, append latency, cross-DC convergence — live elsewhere in the series: Paper 5 lab L1 reports 451k events/sec on the substrate, lab L6 reports 347µs p50 append on the FS backend, and lab L4 demonstrates bit-exact cross-DC convergence under sustained workload. The present experiment is not a benchmark; it is the existence proof for the observation of §2, alongside the operational floor that Paper 5 establishes.

In the parametric regime, the journal records seven entries rather than five: each rotation Director emits its own Define + Invocation pair, because the Define cache is local to the Stage that emits it, not replicated. The journal is the catalog of each peer's declarative vocabulary; the cache's compactness lives on the Stage axis, not on the journal axis.

5.5 Partition tolerance and re-join

The same artifact additionally exhibits partition tolerance and re-join from disk. A peer can exit the cluster while the others continue operating; a fresh process can mount the absent peer's data directory and rehydrate its journal-relative state from disk alone, without contacting the issuer or the surviving peers; and the rehydrated peer integrates into the still-running cluster via peer-to-peer catch-up.

Five phases describe the sequence:

- **R1.** The three peers converge to a shared journal state.

- **R2.** One peer exits; the others continue. The cluster retains a Director and accepts new work.
- **R3.** The surviving Director issues additional work; the two surviving journals advance; the absent peer's on-disk journal is unchanged.
- **R4.** A fresh process — distinct in-memory identity, same data directory — reads the disk and reconstructs the journal-relative state of the absent peer, without contacting the issuer and without contacting the surviving peers.
- **R5.** The rehydrated peer integrates into the still-running cluster via peer-to-peer catch-up; the gap is replayed from a surviving peer; subsequent work brings all three peers back to a shared state.

The properties exercised by this scenario are not separable. The cluster's continued operation in the absent peer's absence is only possible if no node is the cluster's source of authority. The rehydration without consulting the issuer is only possible if the disk is a self-contained witness of the peer's prior participation. The catch-up between peers is only possible if any surviving peer is capable of replaying the gap; no specialized recovery node is invoked.

Operationally, the scenario shows that a peer can leave the cluster, the cluster can continue accepting work in its absence, and the peer can re-join by reading the journal that survived its disappearance on its own disk — no central authority is consulted at any point in this loop. The role of the issuer was discharged once, at the original onboarding, and is not invoked again.

The issuer is invoked exactly once per peer. The credential sealed during the bootstrap of §5.3 is preserved across restarts, and the peer's continued participation is verified by the journal's cryptographic integrity, not by reference to a running service. If a peer loses its disk, it re-onboards as a new peer; if it preserves its disk, it returns as itself, regardless of how long it was absent.

6. What it means in practice (a structural permission)

The observation of §2 — cloud as library, not as habitat — and the demonstration of §5 together state that under a journaled-program substrate, the datacenter ceases to be a structural requirement of running production software. This section examines what that implies for software construction in practice, and what it does not.

The existing landscape. Software delivery without a datacenter is not a new category. Appliance-delivered business software (Odoo Community, Sage 50, QuickBooks Desktop, SAP Business One, Splunk Enterprise) and self-hosted applications (Nextcloud, Plex, Home Assistant, Synology DSM) deliver workloads without a datacenter, in some cases for decades. In-memory databases (VoltDB Community, SQLite, DuckDB) run application data outside cloud infrastructure. Edge-computing platforms (Cloudflare Workers, AWS IoT Greengrass) distribute logic outside the datacenter. Embedded business systems — PLCs, point-of-sale terminals, ATMs — have run software on modest hardware for forty years. The substrate of §4 does not invent any of these deployment shapes; it does not compete with the systems above; it does not claim that the categories they occupy do not exist.

What the substrate adds. The reframing is narrower than the categorical "datacenter optional" headline would suggest in isolation. The systems above each deliver software outside the datacenter, but each pays a price for that delivery: bundled database administration (Nextcloud, Sage, Odoo); no native event sourcing in most self-hosted applications; in-memory databases as storage layers without application-level semantics for replay; no journal as a unified substrate for deployment, replication, backup, and offline operation (per Paper 5). The journaled-program substrate brings the joint operational profile of cloud-native event-sourced microservices — event sourcing native (Paper 1), program-value separability for compiled cached invocations (Paper 2), the pragmatic partition between work-due-before-responding and deferred work (Paper 3), cross-actor causal continuity through Tell (Paper 4), the journal as substrate of the four operational disciplines (Paper 5), and the diagnostic lens for reading which infrastructural layers compensate for the persistence model (Paper 6) — to hardware modest enough to fit any of the deployment shapes the systems above already inhabit. Operating an on-premises business system in the conventional stack composes several layers of operational expertise — database administration, application hosting, backup, updates, coordination between components — at the customer site or via the reseller; the journaled substrate collapses these into the substrate itself, so storage, backup, updates, and dispatch become properties of the runtime rather than concerns the operator composes. The substrate is not necessarily faster or cheaper than the existing systems — that is a separate empirical question — but its *operational surface* is structurally smaller. The unique contribution is this bundle on modest hardware; the categories of datacenter-less delivery themselves predate the substrate.

The contribution can be considered illustratively (not as evidence) through a concrete scenario. A developer or small team encounters a customer too small to be addressed economically by existing mid-market or enterprise systems and not adequately served by generic spreadsheets, single-purpose SaaS, or bespoke development. Under the conventional stack, the developer faces a ladder of operational concerns before they reach the domain: *I need a server — at the customer or in a cloud — and either way I need to provision a database, configure an application tier, set up backup, arrange monitoring, and handle updates before I can start writing the domain.* The toy alternative — a spreadsheet, a script, a one-off utility — does not survive contact with the customer's actual usage. The ladder does not begin at the domain; it begins below it. The journaled-program substrate flattens the ladder: the developer can write the domain as their first concern (a library of classes and verbs in the language they already know), while the runtime substrate handles persistence, backup, updates, and dispatch. The operational surface that the conventional stack ten-steps to is, under the substrate, one or two — and those one or two do not demand operational specializations the developer does not have. This scenario is offered for illustration; the paper makes no controlled-study claim about which developer populations are or are not currently served by existing systems, nor about which barriers (operational, marketing, support, sales, regulatory) are binding in particular cases.

New deployment shapes. The substrate is compatible with deployment shapes the conventional cloud-microservice stack does not permit. Beyond the single-appliance shape — software shippable as a deliverable artifact the customer owns, sized to the domain — the same substrate enables a peer mesh: devices that cooperate without a central server, where each peer participates as a node in the §5 sense and the cluster operates from the peers' joint awakeness rather than from a server's uptime.

The §5 demonstration is precisely this mesh at N=3 on Docker containers, composing three layers: the substrate (the actor model with journaled programs replayed against a shared domain library), a peer-coordination pattern (the StageManager pattern with its Director rotation), and a transport (Kestrel/HTTPS). The substrate's core does not prescribe a particular coordination pattern or transport; StageManager and Kestrel/HTTPS are demo-specific choices appropriate to Docker. The transport-layer pluggability is exercised in the published artifact: alongside the Kestrel/HTTPS used in §5, a second transport implementation based on the SimpleX messaging protocol has been implemented and verified to run on Android, and — when substituted for Kestrel/HTTPS as the peer transport in the §5 three-node Docker scenario (an `smp-server` container providing the SMP relay; the three `ordering-*` containers connecting to it via `ConfigureTransport(TransportType.SimpleX, ...)` instead of HTTPS) — produces the same convergence outcome with bit-equivalent final journal entries to the HTTPS variant (`Choreography/Transport/SimpleX/` in the public repository at commit `b42d0f7`). The layered framing — substrate / coordination pattern / transport — is therefore not hypothetical but exercised across at least two transport implementations. Other deployment shapes compose the substrate with different layer combinations. Workstations on a corporate LAN can reuse the demo's Kestrel-based transport directly — a clinic with three workstations is the closest direct lift of the demonstration. Mobile peer meshes compose the substrate with the SimpleX-based transport (its Android viability verified at the transport layer) and a coordination pattern appropriate to opportunistic synchronization rather than continuous Kestrel sessions; the operational engineering of a full mobile peer-mesh deployment — battery management, NAT traversal between home networks, OS suspension handling, cellular reconnection patterns — lives at the operational layer downstream of transport choice and is not undertaken by the present paper. The substrate's contribution is the actor-journal-replay model; transport and coordination are downstream choices the deployment makes.

The contrast with specialized peer-to-peer systems — blockchain consensus, distributed hash tables, mesh-routing protocols — is that the substrate's mechanism (journaled program, replayed deterministically against a shared domain library) is not specialized to a particular peer-mesh problem; the same substrate composes with whatever transport-and-coordination layer the deployment requires, serving a delivery mesh of riders coordinating inventory, a small clinic where nurses share a patient roster during their shifts, a classroom of students cooperating on a shared exercise, a community game where players' devices form the world's persistent state — each with the transport and coordination layers chosen appropriately for that deployment substrate.

What remains. §2 and §5 demonstrate operationally that a journaled cluster operates without a datacenter footprint in one case — the Ordering bounded context of `dotnet/eShop` — at one scale — N=3 — under one transport — HTTPS/TLS, on Docker containers. From that demonstration, this section derives by structural argument that appliance delivery, peer-mesh deployment, and accessibility-of-development are architecturally permitted. The paper does not demonstrate an end-to-end appliance delivery to a customer in production, a peer mesh of mobile devices in field use, a developer-to-customer flow with operational expertise lower than the conventional stack requires, or the long-tail viability of journaled software for any of the application categories listed. The structural permission is what the paper establishes; the end-to-end demonstration of each deployment shape is engineering work the paper acknowledges as outside its scope. Beyond the architectural permission, the substrate does not address sales channels (how the customer learns this software exists), distribution (how it reaches the customer), support models (who answers the phone when something fails), regulation (where data resides and whose privacy laws apply), or pricing (license vs subscription vs free). These are downstream of architecture and depend on choices the architect does not control. The shapes the developer might choose (single appliance at the customer site, peer mesh across devices, cluster across machines, hybrid with cloud invocations) are not new categories; they are deployment topologies for which polished products often do not exist or are poorly fitted at the small end of the market, and the obstacle to building those products has multiple components. This paper addresses one of those components — the architectural one. The others remain. The conditions under which empirical evidence would refute the structural permission are stated in §8.

7. A diagnostic lens, briefly: when the server-role is accidental

The substrate's three properties of §4 and the demonstration of §5 make a side observation possible: the role that prior literature names *server* — the entity that accepts authoritative writes, owns coordination, mediates between clients, provides the address against which other parties direct their traffic — becomes ephemeral rather than permanent. In the demonstration of §5, no node carries the four functions of the server-role permanently. Acceptance of authoritative writes and ownership of coordination redistribute under Director rotation; mediation and addressing distribute across the peer-to-peer mesh; partition tolerance and re-join distribute across surviving peers via catch-up (§5.5); the bootstrap issuer (§5.3) participates once per peer and exits. **The server is not a permanent identity. It is a transferable coordinator role any peer can assume in turn — and any peer can leave the cluster and re-join without reference to a coordinator outside it.**

At the level of construct, this is Paper 6's *infrastructural symptom* lens applied at the role layer: the server-role reads as an *accidental category* — the role-level expression of infrastructural symptom — when its necessity is traceable to a specific representational choice (Paper 6's *contingency* indicator at the role layer) and disappears when that choice is replaced (Paper 6's *dissolution* indicator). The reading operates per role-in-use-case; many roles remain structural under any substrate — identity providers backed by cryptographic root authority, gateways mediating external systems, computationally expensive services — and no substrate change makes them otherwise. The full apparatus of the lens — formal indicators, the constructibility bound on counterfactual substrates, per-use-case discrimination — is in Paper 6 §3 and is not retraced here. Appendix B catalogs seven canonical architectural roles under the two indicators, as the role-level analog of Paper 6's Table 1, for readers who want to apply the lens to systems beyond the one demonstrated in §5.

The lens is offered as the analytic tool through which the consequence of §2 and §6 can be applied to architectures beyond the demonstration of §5 — not as the contribution of the paper.

8. Limitations and counter-arguments

The demonstration of §5 carries specific limitations that follow from the choices made for the demo. They are reported here honestly; none of them affects the observation of §2 or its practical consequence in §6, but each affects what the present experiment alone establishes.

Identity in the port is local. The original `dotnet/eShop` deployment uses an external identity provider; the port substitutes a local string-typed identifier. The lens of §7 does not read identity providers as dissolving — identity providers backed by cryptographic root authority are structural requirements, as Appendix B notes — and the demo does not exhibit one. A second experiment that integrates an external identity provider as a Reaction-invoked library would extend the empirical surface without changing the lens's reading.

Cross-context dependencies are not portrayed. The Ordering bounded context of `dotnet/eShop` carries references resolved by the Catalog and Basket bounded contexts; the port treats those references as opaque identifiers. The same port mechanism applies to Catalog and Basket; the present experiment exhibits Ordering only.

The two limitations above warrant clarification, because they are easy to read as selection bias when they are in fact bounded-context isolation in the Domain-Driven Design sense (Evans, 2003; Vernon, 2013). The Ordering bounded context, in any architecture — microservices, monolith, or journaled-program substrate — references Catalog and Basket entities by identifier rather than by full object, because cross-context object resolution is itself an integration boundary, not a property of Ordering's domain. Identity, similarly, lives in its own bounded context (the Identity microservice in the original eShop); Ordering does not authenticate users, it accepts a `UserId`. Removing those from the port is not eliminating components that "need a server"; it is respecting the same boundaries DDD prescribes. Appendix B itself lists identity providers backed by cryptographic root authority as structural requirements — the lens never reads identity as dissolving, so its exclusion is not adverse evidence.

The empirical matrix is at N = 3. The mesh in §5.2 connects three peers — the minimum non-trivial fully-connected mesh — using a specific peer-coordination mechanism (Director rotation) chosen for the demonstration. The substrate's structural claim is independent of node count and of the specific coordination mechanism: it is that journaled programs replicate without a privileged coordinator at any scale at which the journal-determinism property holds between peers. Whether the specific Director rotation mechanism demonstrated at N=3 scales operationally to larger N is engineering work the paper does not undertake; other coordination mechanisms (gossip, hierarchical overlays, sharding) are well-understood in distributed systems and could be used in conjunction with the substrate's journal-determinism at larger scales, but the present paper does not demonstrate any such combination. The substrate's structural permission holds at any N where journal-determinism between peers is preserved; the operational scaling characteristics at large N are not demonstrated and are explicitly out of scope.

The cross-container partition-tolerance scenario is captured in a parallel artefact, not in the matrix of §5.4. The natural cross-container equivalent of the in-process scenario in §5.5 — stopping a container, allowing the running cluster to continue accepting work in its absence, restarting the container, and watching it rehydrate from its mounted volume and rejoin via catch-up against a surviving peer — is implemented in the same orchestrator that produced the matrix (the `--rehydrate-demo` flag of the demo script) and depends on no substrate feature that is not already exercised by the in-process test in §5.5. The decision to keep §5.4 to the four cells of the matrix, rather than expanding to a fifth cell, was rhetorical: the in-process exercise establishes the property structurally. The cross-container capture documents the same property under a different process boundary; it does not add a new claim.

The compaction ratio between inline and parametric workloads depends on Director rotation. Each new Director journals a fresh Define +

Invocation pair, so the Define cache compacts on the Stage axis rather than on the journal axis. The compaction is real and measured at three rounds; its precise value at larger workloads is treated in Paper 2.

What would falsify the structural permission. The structural permission framing of §6 absorbs many objections by deferring to engineering work downstream of architecture — operational difficulties at scale, transport-specific failure modes, deployment-environment quirks. None of these would refute the structural permission; they would calibrate the cost of exercising it. The structural permission is refuted only by failure modes that show the substrate cannot host the deployment shape even in principle. This subsection names four such conditions.

- **A runtime central component is required for intra-cluster operation at N > 3.** If extending the §5 demonstration to N=4 or higher required introducing a process that must remain alive — a coordinator, a directory server, a bootstrap re-issuer — beyond the peers themselves, the cluster's peer-symmetric character would not generalize from N=3, and the appliance and peer-mesh shapes would not be structurally compatible with cooperation beyond the demonstration's scale.
- **Journal-determinism breaks under transport change.** The substrate's claim of transport pluggability requires that the journal records (and replays) the same operations regardless of which transport carried them. If swapping Kestrel/HTTPS for an alternative transport — SimpleX, analog transfer, or any other — produced journals with different content for the same operations, or state not recoverable from replay, the transport-pluggability claim of §6 would be refuted.
- **Raw I/O in verb bodies systematically breaks journal replay.** §2 describes cloud-as-library as the substrate's idiomatic path, with raw I/O inside the verb body permitted as domain code under a known lock-cost penalty as the substrate's structural disincentive. If such I/O could systematically produce journal entries that the substrate could not replay deterministically — beyond the lock-cost penalty and the convention boundary of using Reactions or Tells for external invocation — the substrate's contribution to cloud-as-library would reduce from substrate-supported to purely convention-mediated, and Paper 2's "journal is the program" claim would be similarly narrowed.
- **Peer-mesh requires global identity for cluster-to-cluster operation.** If two journaled clusters established independently — each through analog bootstrap (§5.3) — could not be reconciled without consulting a shared identity authority, the peer-mesh's independence from central authority would not extend to inter-cluster operation, and the deployment-shape claims of §6 would narrow.

Each condition is testable. The paper does not undertake these tests; it states the structural permission and the conditions under which empirical evidence would refute it.

The lens of §7 extends to other architectural roles that admit the same analysis. Service discovery, queue-as-glue between bounded contexts, configuration-management roles, and distributed-tracing collectors are auditable under the two indicators; the present experiment does not exercise them, and the substrate's relation to each is left as a forward question. The lens admits this open extension by design — its diagnostic value is in reading which categories in any given system are contingent, not in enumerating them exhaustively.

9. Conclusion

This paper observed what happens to the conventional cloud-microservice stack under a journaled-program substrate developed across the prior six papers of this series: the datacenter ceases to be a structural requirement of running production software. Cloud and microservice ecosystems, when accessed from a journaled program, become libraries the program invokes rather than habitats the program must inhabit. Software becomes shippable as a physical artifact — an appliance the customer owns, sized to the customer's domain, not a datacenter the vendor leases on the customer's behalf.

The observation was made operationally inspectable by porting the Ordering bounded context of `dotnet/eShop` to the substrate of §4, deploying it across three nodes that bootstrap each other through paper QR codes, and observing that after bootstrap the cluster operates against its own journals — none of the services that originally constituted the Ordering microservice are reachable from it, and the cluster continues under each one's absence. The demonstration does not argue uniqueness of the substrate; it establishes that the observation is inspectable in one working case.

The practical consequence (§6) is, as structural permission, an opening in the space of deployment shapes the conventional stack does not permit. Software becomes structurally compatible with delivery as an on-premises appliance, with cooperation across a peer mesh of devices (workstations, phones) without a central server, and with hybrid topologies that invoke cloud services as libraries. The shapes are not new categories — existing systems (Odoo Community, Sage 50, QuickBooks Desktop, Splunk Enterprise, Nextcloud) already deliver business software without a datacenter — but the journaled substrate offers a different operational profile that does not require the operator to compose specialized expertise (database administration, application hosting, backup, updates, orchestration) on top of the architecture. The paper establishes the structural permission; whether the permission becomes operational practice in any of these shapes is engineering work the paper does not undertake.

A side observation, stated honestly, is that the role prior literature names *server* becomes ephemeral under this substrate: discharged once per peer during analog bootstrap, then absent from the running topology. The construct *accidental* *category* (§7), applied at the role layer as an extension of Paper 6's *infrastructural symptom*, is offered as a diagnostic lens the reader may apply to their own architecture. The construct is the tool; the practical observation is the contribution.

The lens does not promise universal dissolution. Identity authorities, payment networks, sensor integrations, and computationally intensive services remain structural under any substrate. The reading is per role-in-use-case, not per system. What the lens offers is examinability: any production architecture can be examined role-by-role, asking whether each role is tied to the problem being solved or to the persistence model chosen.

Paper 6 applied the lens at the layer level, identifying many infrastructural layers — caches, queues, locks, application servers, orchestration clusters — as compensations for the underlying persistence model. The present paper observes the joint operational consequence: with those layers absent, and the architectural roles they serve revealed as accidental, the datacenter itself becomes optional, and the construction of production software passes back into the hands of those who model the domain. From this point on, treating the datacenter as a structural requirement is no longer an assumption but a choice that can be made explicit and evaluated.

Appendix A — Reproducibility

The artifact described in §5 is publicly available. The references in this appendix to file paths, test method names, and orchestrator scripts resolve against the released code; the artifact comprises the substrate framework, the runtime, the ported `Ordering` domain library and its test suite, the demo orchestrator, the supporting reports, and the asciinema recordings, hosted across two repositories described in §A.1.

A.1 Source and suite

The artifact is hosted in two public repositories under https://github.com/alvaroNCubo. The substrate framework (`Choreography/`), the runtime (`Puppeteer/`), the ported `Ordering` domain library and its test suite (`tests-local/UnitTestPaper7EShop/`), the demo orchestrator (`docker/`), and the supporting reports referenced in §A.7 (`notes/paper7_phase1_results.md`, `notes/paper7_phase2_results.md`) are in https://github.com/alvaroNCubo/puppeteer. The asciinema recordings and GIFs of §A.8 are in the `paper7-assets/` directory of https://github.com/alvaroNCubo/puppeteer-papers. Inline code citations of the form `file.cs:NN` resolve against the commit pinned in Code Provenance.

A.2 Demo orchestrator

The orchestrator script `docker/run-demo.sh` provisions three Docker containers, runs the analog bootstrap handshake against each, and exercises one cell of the matrix per invocation. Three flags select the scenario:

- `bash docker/run-demo.sh` — cross-container inline (22 entries final).
- `bash docker/run-demo.sh --parametric` — cross-container parametric (7 entries final).
- `bash docker/run-demo.sh --rehydrate-demo` — captures the cross-container leave-and-rejoin scenario referenced in §8: stops one container, allows the cluster to advance, restarts the container, and observes its re-hydration and catch-up.

Each invocation runs in approximately 60–90 seconds on Docker Desktop.

A.3 Tests cited

The empirical claims in the body are exercised by specific tests in the suite:

Body section	Property	Test method	Project
§5.3	F1–F5 bootstrap handshake under real cryptography	`EndToEnd_RealCryptoOverRealTls_RoundsTripIdentity`	`UnitTestChoreography/UsherOnboardingTests.cs`
§5 (single peer operates alone)	Force-promoted Stage operates without peers	`SingleStage_OperatesAlone_WhenPromotedForce`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`
§5.4 (in-process, inline)	Three-stage Director rotation, inline regime	`ThreeStages_DirectorRotation_AllConverge_HappyPath`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`
§5.4 (in-process, parametric)	Three-stage Director rotation, parametric regime	`ThreeStages_DirectorRotation_AllConverge_HappyPath_Parametric`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`
§5 (rotation under cancellation)	Cancellation branch replicates across three stages	`CancellationBranch_Replicates_AcrossThreeStages`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`
§5.4 (instrumentation)	DLL load + per-phase convergence + journal bytes snapshot	`Metrics_F1_Snapshot`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`

Body section	Property	Test method	Project
§5.5 (partition tolerance)	Leave-and-rejoin from disk, peer-to-peer catch-up	`OneStage_Offline_OthersAdvance_Rehydrate_CatchUp_Parametric`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`
§5.5 (rehydration without onboarding)	Rehydrated Stage restores identity from disk alone	`RehydratedStage_RestoresIdentityAndJournal_WithoutOnboarding`	`UnitTestPaper7EShop/ThreeNodeOrderingTests.cs`

A.4 Cryptographic stack

The cryptographic primitives underlying the F1–F5 handshake of §5.3 are documented in `Choreography/Usher/Crypto/*.cs`. The implementations use BouncyCastle 2.6.2:

- Ed25519 keypair generation, signing, verification: `Ed25519StageKeyGenerator.cs`, `Ed25519Signer.cs`, `Ed25519SignatureVerifier.cs`.
- Ed25519-to-X25519 derivation (Edwards-to-Montgomery, $u = (1 + y) / (1 - y) \bmod p$): `Ed25519ToX25519.cs`.
- Sealed-box AEAD using X25519 ECDH and ChaCha20-Poly1305, with the participating public keys bound as additional authenticated data: `SealedBoxPayloadSealer.cs`.
- Kestrel TLS listener and self-signed certificate generation with SAN: `HttpsTransportListener.cs`, `SelfSignedCert.cs`.
- TOFU certificate fingerprint pin via `SocketsHttpHandler.SslOptions.RemoteCertificateValidationCallback`: `HttpsClientFactory.cs`.

A.5 Share-link URI

The share-link emitted by the issuer takes the form `puppeteer-usher://v1/{base64url(json)}`. The JSON payload carries the transport address of the issuer endpoint and a SHA-256 fingerprint (`fp` field) of its TLS certificate. The encoder/decoder is defined by `IUsherShareLinkEncoder` in `Choreography/Usher/`. A representative share-link under load is 449 characters in length, within the alphanumeric capacity of a QR code at version 14 with error correction level M (approximately 535 characters).

A.6 Wire format

The HTTPS transport between containers carries a `ChannelPurpose` byte after the sender identifier in each frame. This wire format (v2) prevents Define and

Invocation channels from collapsing into the same channel-dictionary key when Director rotation runs the parametric workload across multiple containers; the in-process regime is unaffected. The wire format is implemented in the transport handlers under `Choreography/Transport/Https/`.

A.7 Supporting reports

Two markdown reports in the `notes/` directory of the same branch document the experiment in detail:

- `notes/paper7_phase1_results.md` — the in-process row of the matrix (eleven sections plus three addenda).
- `notes/paper7_phase2_results.md` — the cross-container row, the hardening cycle that converged on the demo, the three-Docker mesh, the rotation protocol, and the parametric cell.

A.8 Recordings

Eleven cast recordings (asciinema format) and eleven GIF renders capture the four cells of the matrix of §5.4 and the rehydration property of §5.5, one recording per Docker container per cell. The recordings are published alongside this paper as supplementary material, hosted in the `paper7-assets/` directory of the `puppeteer-papers` repository.

Appendix B — Role-level diagnostic table

The lens of §7 — Paper 6's *infrastructural symptom* applied at the role layer — produces a reading per role-in-use-case. The same node, identified in a deployment as "API server," reads as an accidental category when deployed to accept authoritative writes against a shared database, and as a structural requirement when deployed to enforce cryptographic identity claims. The discriminator is the function, not the name. Seven canonical categories of architectural role admit reading under the two indicators of the lens; their substrate-property counterparts are summarized below as the role-level analog of Paper 6's Table 1.

Table B.1. Diagnostic reading: seven canonical categories of architectural role under the two indicators of Paper 6's lens, re-expressed at the role layer.

Architectural role	Representational choice that necessitates it	What a substrate satisfying both indicators offers instead
Server as write authority	Nodes replicate data; consistency requires a single point of acceptance	Each node accepts writes locally; the journal records the program, which replays deterministically on every node
Master / primary in replication	Nodes replicate state diffs; a primary must order them	Each node owns its journal; cross-node causality is recorded by reference, not orchestrated
Coordinator in distributed transactions	Multiple writers contend for a logical entity across processes	Each logical entity has a single point of authority intrinsic to the substrate; serialization is not externalized
Stateful gateway	Clients cannot directly invoke deferred work without losing causal record	Deferred work is journal-recorded; the substrate carries causality, not the gateway
Application server / runtime container	The runtime, the framework, and the application require disparate substrates	A minimal substrate suffices to run the domain artifact
Bootstrap service	New nodes require an issuer of credentials and addresses	Bootstrap is information transfer that crosses by any means, including analog artifacts; the issuer process is ephemeral
Orchestrator as coordination substrate	Stateful coordination across instances requires external choreography	Coordination is between actors; the substrate provides location and continuity intrinsically

The seven rows are not exhaustive; service discovery, queue-as-glue between bounded contexts, and configuration management admit similar analysis. The lens is diagnostic, not prescriptive: it enables auditing existing architectures under the two indicators of Paper 6, independently of any decision to adopt the substrate of §4.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `b42d0f7` (`2026-05-26`). This commit includes the substrate code described in §4-§5, the reproducibility artifacts of Appendix A (`docker/`, `notes/`), and the SimpleX-based transport implementation referenced in §6 (`Choreography/Transport/SimpleX/`). Earlier commits in the repository preserve the substrate's prior states: `91118fc` (2026-05-22) added the reproducibility artifacts; `2f31f96` (2026-05-18) is the earlier substrate snapshot before the SimpleX transport fixes (C2S envelope decryption, batch parsing, catch-up piggyback routing, StageManager Director-rotation race, Usher F5 invitation-consumed race) of commit `b42d0f7`. The earlier substrate snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against `b42d0f7`. A reader can construct a per-file SWHID by adding the qualifiers ;path=<path>;lines=<NN> to the directory SWHID above for the `2f31f96` snapshot; for the post-`2f31f96` state (which includes the SimpleX transport fixes of `b42d0f7`), the GitHub `b42d0f7` reference is canonical. Future commits to the repository may renumber lines; the SWHID preserves the cited `2f31f96` state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Ameloot, T. J., Neven, F., & Van den Bussche, J. (2013). Relational transducers for declarative networking. *Journal of the ACM*, 60(2), 1–38.

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors* [Doctoral thesis]. KTH Royal Institute of Technology.

Bernstein, P. A., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Tech. Rep. MSR-TR-2014-41). Microsoft Research.

Brewer, E. A. (2000). Towards robust distributed systems [Keynote address]. *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '00).*

Cognitect. (n.d.). *Datomic: A transactional database with a flexible data model, queryable as a log of changes* [Software project]. Retrieved May 22, 2026, from https://www.datomic.com/

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*, 205–220.

Earthstar Project. (n.d.). *Earthstar: simple personal data sync* [Software project]. Retrieved May 22, 2026, from https://earthstar-project.org/

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Event Store Ltd. (n.d.). *EventStoreDB: The database for event sourcing* [Software project]. Retrieved May 23, 2026, from https://www.eventstore.com/eventstoredb

Fowler, M. (2005). Event sourcing. Retrieved from https://martinfowler.com/eaaDev/EventSourcing.html

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Harris-Braun, E., Luck, N., & Brock, A. (2018). *Holochain: Scalable agent-centric distributed computing* (Alpha 1 white paper). Holo. https://www.holochain.org/documents/holochain-white-paper-alpha.pdf

Hellerstein, J. M. (2010). The declarative imperative: Experiences and conjectures in distributed logic. *ACM SIGMOD Record*, 39(1), 5–19.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.

Holepunch. (n.d.). *Hypercore: append-only logs for the peer-to-peer web* [Software project]. Retrieved May 22, 2026, from https://docs.pears.com/building-blocks/hypercore

Kleppmann, M., Frazee, P., Gold, J., Graber, J., Holmgren, D., Ivy, D., Johnson, J., Newbold, B., & Volpert, J. (2024). Bluesky and the AT Protocol: Usable decentralized social media. In *Proceedings of the ACM CoNEXT-2024 Workshop* *on the Decentralization of the Internet*. ACM. https://doi.org/10.1145/3694809.3700740

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In *Onward! 2019: Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (pp. 154–178). ACM. https://doi.org/10.1145/3359591.3359737

Lemmer-Webber, C., Tallon, J., Shepherd, E., Guy, A., & Prodromou, E. (2018). *ActivityPub* (W3C Recommendation, 23 January 2018). World Wide Web Consortium. https://www.w3.org/TR/activitypub/

Lightbend. (n.d.). *Akka Persistence* [Software documentation]. Retrieved May 23, 2026, from https://doc.akka.io/libraries/akka-core/current/typed/persistence.html

Matrix.org Foundation. (n.d.). *Matrix specification* [Online document]. Retrieved May 22, 2026, from https://spec.matrix.org/

Microsoft. (2026). *dotnet/eShop: A reference .NET application implementing an eCommerce site* [Software repository, commit 9b4f943, 2026-04-21]. Retrieved May 22, 2026, from https://github.com/dotnet/eShop

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

Ogden, M., McKelvey, K., & Madsen, M. B. (2017). *Dat: Distributed dataset synchronization and versioning* [White paper]. Code for Science. https://github.com/datprotocol/whitepaper

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, 305–319.

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1. Zenodo. https://doi.org/10.5281/zenodo.20404863

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/02-program-value-separability.md

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/03-reactions-and-partition.md

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/04-cross-actor-continuity.md

Rivera, A. (2026e). The journal as substrate: unifying deployment, replication, backup, and offline operation in distributed systems. *Puppeteer Papers Series*, Paper 5. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/05-substrate-operations.md

Rivera, A. (2026f). Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks. *Puppeteer Papers Series*, Paper 6. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/06-infrastructural-symptom.md

Sambra, A. V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., & Berners-Lee, T. (2016). *Solid: A platform for decentralized social applications based on linked data* (Technical Report). MIT CSAIL & Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). *Conflict-free replicated data types* (Research Report RR-7687). INRIA.

Tarr, D., Lavoie, E., Meyer, A., & Tschudin, C. (2019). Secure Scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN '19)* (pp. 1–11). ACM. https://doi.org/10.1145/3357150.3357396

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Vernon, V. (2015). *Reactive messaging patterns with the actor model: Applications and integration in Scala and Akka*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. Retrieved from https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf